



Computer Science 9618

Paper 2 · Fundamental Problem-solving and Programming Skills

Part-level Topical Workbook

with Mark Scheme

- Every question part filed under its own syllabus topic (units 9 to 12 of the 2027-29 syllabus)
- Each item carries the official context it needs: stem, code, tables, diagrams, earlier parts
- Answer lines removed; tables, trace tables and code to complete kept
- Newest papers first; mark scheme in an Answers Section after each unit
- Index of every part with its unit, marks and page; pseudocode insert in the Appendix

Coverage

30 papers: M/J 21/P21 to M/J 26/P23 (newest first in each unit)

366 items, 2225 marks across 4 units

Syllabus codes: 9618

Contents

UNIT 9	Algorithm Design and Problem-solving	3
	Answers Section	65
UNIT 10	Data Types and Structures	115
	Answers Section	231
UNIT 11	Programming	350
	Answers Section	450
UNIT 12	Software Development	524
	Answers Section	624
INDEX	Topic index	678
APPENDIX	Insert: pseudocode functions and operators	684

Unit 9

ALGORITHM DESIGN AND PROBLEM-SOLVING

73 items · 382 marks

Syllabus sections in this unit

- 9.1 Computational Thinking Skills
- 9.2 Algorithms

Unit 9: Algorithm Design and Problem-solving

1. M/J 26/P21/Q2/a(ii,iii)

2 A washing machine contains:

- two trays for the washing powder
- a user interface display
- five sensors.

The washing machine control program includes the following variables:

variable	description
MainsOn	TRUE when the mains power is on
WaterOn	TRUE when the water supply to the machine is turned on
DisplayOn	TRUE when the user interface display is active
Tray1	TRUE when it detects washing powder in tray 1
Tray2	TRUE when it detects washing powder in tray 2

- (ii) The display is only active when both the mains power is on and the water supply is turned on.

Complete the pseudocode statement.

DisplayOn ←

[1]

- (iii) The washing machine control program will start when a new variable `WashStart` is assigned the value `TRUE`

`WashStart` is assigned the value `TRUE` when the mains power is switched on and there is washing powder in tray 1 or tray 2 or both trays.

Write the pseudocode statement for the given description.

[3]

2. M/J 26/P21/Q4

4 An algorithm is expressed as:

- A user inputs a sequence of integer values.
- The process will calculate the number of positive integers.
- The user will input value 0 to indicate there are no more numbers to be input.
- Output the number of positive integers after the terminating 0 is input.

Write a structured English description for this algorithm as a sequence of steps.

Do **not** include pseudocode statements in your description.

Your answer might **not** require all **six** steps shown.

step 1

step 2

step 3

step 4

step 5

step 6

[4]

3. M/J 26/P22/Q2/a,b,c

2 A new computer system is being developed to help manage a tool hire business. A customer can specify if they want to hire a tool for either half a day or a full day.

An abstract model for the new system is required.

(a) Explain the purpose of abstraction.

[1]

(b) State **four** items of data that must be stored each time a tool is hired.

item 1

item 2

item 3

item 4

[2]

(c) The computer system will have to process the tool hire data. One process is to cancel a tool hire booking.

State **two** other processes that would be required.

process 1

process 2

[2]

4. M/J 26/P22/Q3

- 3** A computer game uses a text file named `WordList.txt` containing 10 000 words, all in lower case. Each word is unique and is stored on a separate line of the text file.

An efficient algorithm is needed to check that a word input by the user is stored in `WordList.txt`. If the word is found, the line number where the word is stored must be output. If the word is **not** found, the message "Not Found" must be output.

Complete the steps needed to describe the algorithm.

Do **not** include pseudocode statements in your answer.

The first **three** steps have been completed for you.

step 1: set line counter to 0

step 2: input a word

step 3: convert the word to lower case

[6]

5. M/J 26/P23/Q2/b,c

- 2** A control program is being developed for a robot floor vacuum cleaner. The settings for the robot can be changed and include:

- the strength of the suction (numerical values: 0, 1, 2, 3, 4 or 5)
- the total vacuum cleaning time (a whole number of minutes).

The robot has a number of sensors.

The control program includes the variables:

variable	description
MainsOn	TRUE when the mains power is on
DisplayOn	TRUE when the user display is shown
Suction	TRUE when the program detects that the user has selected a non-zero value for the strength of the suction

- (b)** Write a pseudocode statement to set the `DisplayOn` variable to TRUE when the mains power is on.

[1]

- (c)** When the robot is ready to move, the program assigns the value TRUE to a variable, `Move`

The control program contains the pseudocode logic expression:

`Move ← MainsOn AND Suction`

Explain the pseudocode statement.

[2]

6. M/J 26/P23/Q4

4 An algorithm is expressed as:

- A user inputs a sequence of integers.
- The user will input value -1 to indicate there are no more integers to be input.
- The program will calculate the total number of integers greater than 120.
- Output the total number of integers greater than 120 after the -1 terminator is input.

Write a structured English description for this algorithm as a sequence of steps.

Do **not** include pseudocode statements in your description.

Your answer might **not** require all **six** steps shown.

step 1

step 2

step 3

step 4

step 5

step 6

[4]

7. O/N 25/P21/Q2

2 `Data` is a global 1D array containing 30 elements of type `STRING`

An algorithm will output:

- all non-blank elements (elements that do **not** contain an empty string)
- the final total of the number of elements output.

Complete the program flowchart to represent the algorithm:

START

END

[5]

8. O/N 25/P22/Q1/a

1 (a) The table contains pseudocode examples.

Each example may contain statements that relate to one or more of:

- selection
- iteration (repetition)
- subroutines (procedures or functions).

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Subroutine
IF Status = FALSE THEN FOR Count ← 1 TO 20 CALL Reset (Count) NEXT Count ENDIF			
OTHERWISE : Status ← TRUE			
WHILE AllDone() = TRUE			
30 : NextChar ← 'X'			

[4]

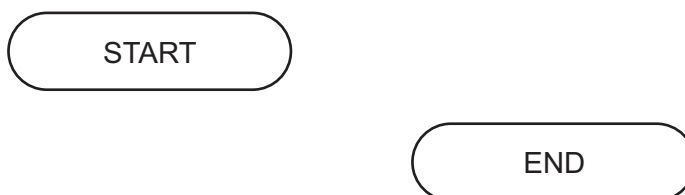
9. O/N 25/P22/Q2

2 Data is a global 1D array containing 20 elements of type REAL

An algorithm will:

- input a sequence of real values, one at a time
- assign each value to consecutive array elements, starting from index 1
- end when the value 99.9 is input, or all 20 elements have been assigned (the value 99.9 must **not** be stored in the array).

Complete the program flowchart to represent the algorithm:



[5]

10. O/N 25/P22/Q3/a

- 3 A text file `OldFile.txt` contains IDs, names and email addresses for members of a club. Three information items are stored for each member and each item is stored on a separate line.

The example shows the information for the first two members in the first six lines of the file:

Line in file	Information item	Example data
1	member 1 ID	"AB1234"
2	member 1 name	"Freddie Jones"
3	member 1 email address	"FreddieJ909@Cambridge.org"
4	member 2 ID	"BC2345"
5	member 2 name	"Sue Smith"
6	member 2 email address	"Sue1024@Cambridge.org"

The member ID string is always two letters followed by four digits.

The file design is to be changed so that information for each user is stored as a single line of the file, with the character '`\`' used as a separator between data items.

For example, the single line for member 1 will be:

"AB1234\Freddie Jones\FreddieJ909@Cambridge.org"

An algorithm will produce a new file `NewFile.txt` from the contents of `OldFile.txt`

Assume:

- `LineX`, `LineY` and `LineZ` are of type `STRING` and are used to store the three items of information for each member
- `NewString` is a temporary variable of type `STRING`
- `OldFile.txt` exists and contains valid data.

- (a) The algorithm to create the new file is expressed in steps.

Complete the following numbered steps:

- Step 1 : open the file in
- Step 2 : open the file in
- Step 3 : read a line from and store in
- Step 4 : read a line from and store in
- Step 5 : read a line from and store in
- Step 6 : set `NewString` to
- Step 7 : write `NewString` to
- Step 8 : repeat from step until
- Step 9 : close both files.

11. O/N 25/P22/Q4/c

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	TRUE	valid total score (no correct answers)
1	FALSE	invalid total score
2	FALSE	invalid total score
3	TRUE	valid total score (one 3-point question correct)
4	FALSE	invalid total score
5	TRUE	valid total score (one 5-point question correct)
6	TRUE	valid total score (two 3-point questions correct)
7	FALSE	invalid total score
8	TRUE	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(c) The array elements have been assigned the required values.

A module `ValidateScore()` will take an integer value representing a total score as a parameter. It will return `TRUE` if the total score is valid, or `FALSE` if it is **not** valid.

Describe the algorithm for `ValidateScore()` using **four** steps.

Do **not** use pseudocode in your answer.

Step 1

Step 2

Step 3

Step 4

[4]

12. O/N 25/P23/Q2/a,b

- 2 (a)** A program contains a global 1D array of type `STRING` containing 65 elements.

An existing text file is used to store the data in the array. Only non-blank elements (those that do **not** contain an empty string) are written to the text file.

An algorithm will:

- write the first element of the array as a new line in the text file
- continue until all elements have been written, each to a new line of the text file.

Stepwise refinement is applied to the algorithm.

Describe up to **six** steps for this algorithm that could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

Step 6

[6]

- (b)** Iteration is one programming construct.

Identify **one** other programming construct that will be required when the algorithm from part **(a)** is converted into pseudocode and explain how it is used.

Construct

Use

[2]

13. M/J 25/P21/Q1/a,b,c

1 A program is being developed to help the manager of a shop control the stock.

(a) An identifier table has been used during the design stage.

Complete the identifier table:

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop		
20/02/2025	when an item was sold		
12.67	the cost of an item		
TRUE	to indicate if an item is in stock		

[4]

(b) A module `Sales()` is part of the stock control program.

The table contains pseudocode extracts from the module `Sales()`

Each extract may include all or part of:

- assignment
- selection
- iteration (repetition).

Complete the table by placing one or more ticks (✓) in each row:

Pseudocode extract	Assignment	Selection	Iteration
<code>Result ← CalculateTotal()</code>			
<code>WHILE IsClosed</code>			
<code>REPEAT INPUT Value UNTIL Sales[4] > Value</code>			
<code>IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF</code>			
<code>CASE OF Option</code>			

[5]

(c) Decomposition has been used to design the program to help the shop manager control the stock.

Describe decomposition.

[3]

14. M/J 25/P21/Q3

- 3** A student has been asked to create a simple guessing game program. This program will generate a random integer value between 1 and 100. It will then repeatedly prompt the user to input an integer value until they input the randomly generated value.

The student has written a structured English description:

step 1 – randomly generate an integer value between 1 and 100 inclusive

step 2 – prompt the user to input an integer value

step 3 – output an appropriate message if the value input was too high; then repeat from **step 2**

step 4 – output an appropriate message if the value input was too low; then repeat from **step 2**

step 5 – output an appropriate message if the value input was the same value that was randomly generated; then end the program.

Write a pseudocode algorithm from this structured English description.

Assume no input validation is needed.

[8]

15. M/J 25/P22/Q2/a

- 2** A program to calculate the pay of employees working for a company is being designed.

(a) Stepwise refinement has been used in the design of the program.

Describe stepwise refinement.

[2]

16. M/J 25/P22/Q3

- 3** An algorithm is designed to generate and output two unique random integers. Each integer value is between –10 and 10 inclusive.

If both integers output are negative, a third random integer between 30 and 35 inclusive will be generated and output.

Write pseudocode for this algorithm.

[8]

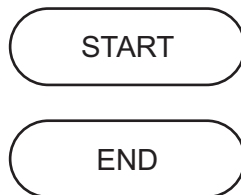
17. M/J 25/P22/Q4

4 An algorithm will:

- input 100 integer values, one value at a time
- store the first value input into the first location of the array `Number`
- store the next input value in the next unused location of the array `Number`
- output the contents of `Number` array in the opposite sequence to that in which the values were input.

Complete the program flowchart to represent the algorithm.

Variable declarations are **not** required.



[6]

18. M/J 25/P23/Q3

3 A programmer has been asked to create a module `RollDice()` to simulate multiple rolls of a dice. This module will be used as part of a program for a game.

The module will:

- step 1 – take a positive integer parameter representing the number of times the dice will be rolled
- step 2 – simulate **one** roll of the dice by generating a random integer between 1 and 6 inclusive
- step 3 – output the integer generated
- step 4 – repeat, as required from **step 2**
- step 5 – calculate the average value of the random integers generated
- step 6 – return the average value.

Write pseudocode for the module `RollDice()`

[7]

19. M/J 25/P23/Q4/a

- 4** A programmer is developing a new stock control program for a shop owner to produce sales reports.

(a) A text file `Sales.txt` stores strings representing the number of items sold.

The file contains 52 lines, each representing the number of items sold in each week of the year.

For example:

File line number	File data
1	"350"
2	"502"
3	"434"
	
49	"492"
50	"872"
51	"772"
52	"201"

The example shows that 502 items were sold in week 2.

The owner requires a module to output all week numbers where the number of items sold is greater than 500.

Describe an algorithm that produces the required module.

Do **not** include pseudocode statements in your answer.

[5]

20. O/N 24/P21/Q3/a,b

- 3** An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

- (a)** The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file
2. Loop until
3. and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop,
8. Output `LineX`, `LineY`, `LineZ`

[4]

- (b)** Explain the purpose of steps 4, 5 and 6 in the algorithm from part **(a)**.

[1]

21. O/N 24/P22/Q7/a,b

- 7** A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

- (a)** Identify **three** items of customer information that will be used by the new module **and** justify your choices.

Item 1

Justification

Item 2

Justification

Item 3

Justification

[3]

- (b)** Identify the computational thinking skill that you needed to use to answer part **(a)**.

[1]

22. O/N 24/P23/Q1/a

- 1 (a)** The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- subroutine (procedure or function).

Complete the table by placing **one or more** ticks ('✓') in each row.

Pseudocode example	Selection	Iteration	Subroutine
FOR Index ← 1 TO 3 IF Safe[Index] = TRUE THEN Flap[Index] ← 0 ENDIF NEXT Index			
CASE OF Compound(3)			
REPEAT UNTIL AllDone() = TRUE			
WHILE Result[3] <> FALSE			

[4]

23. O/N 24/P23/Q2/a,b

2 An algorithm will:

1. prompt and input a sequence of 100 integer values, one at a time
2. sum the positive integers
3. output the result of the sum.

(a) Write pseudocode for the algorithm.

Assume the value zero is neither positive nor negative.

You must declare all variables used in the algorithm.

[5]

(b) The algorithm requires the use of basic constructs. One of these is sequence.

Identify **one other** basic construct required by the algorithm **and** describe how it is used.

Construct

Use

[2]

24. O/N 24/P23/Q3/b,c

- 3** The implementation of a linked list uses an integer variable and a 1D array `List` of type `Node`.

Record type `Node` is declared in pseudocode as follows:

```
TYPE Node
  DECLARE Data : STRING
  DECLARE Pointer : INTEGER
ENDTYPE
```

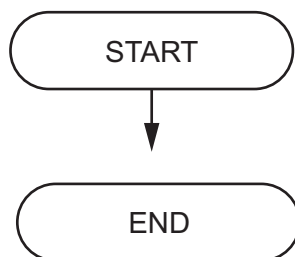
The array `List` is declared in pseudocode as follows:

```
DECLARE List : ARRAY[1:200] OF Node
```

The linked list will operate as follows:

- Integer variable `HeadPointer` will store the array index for the first node in the linked list.
 - The `Pointer` field of a node contains the index value of the next array element (the next node) in the linked list.
 - The value 0 is used as a null pointer.
- (b)** The array `List` will be initialised so that each node points to the following node. The last node will contain a null pointer.

Complete the program flowchart to represent the algorithm for this operation.



[4]

- (c)** An algorithm outputs the `Data` field from all nodes in the array `List`. The order the `Data` is output should be the same order it is stored in the linked list.

Describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

[4]

25. O/N 24/P23/Q7/b

- 7** A coffee shop owner wants to introduce a computerised loyalty card system.

A programmer discusses the details of the system with the shop owner.

- (b)** The shop will give each customer a loyalty card that displays a unique customer ID as a bar code. A customer will be able to present their card each time they make a purchase. The system will scan the bar code, calculate points, and add them to the customer's total. When the customer next makes a purchase and presents their card, they will have the option to exchange points for a discount.

The designer decides that this activity will be handled by a new module. Decomposition will be used to break the problem of designing the new module down into sub-problems (sub-modules).

Identify **four** sub-modules that could be used in the design of the new module **and** describe their use.

Sub-module 1

Use

Sub-module 2

Use

Sub-module 3

Use

Sub-module 4

Use

[4]

26. M/J 24/P21/Q1/c

- 1 An algorithm is developed in pseudocode before being coded in a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables A, B, C and D.

Assignment statement	Data type
A ← LEFT(MyName, 1)	
B ← Total * 2	
C ← INT(ItemCost) / 3	
D ← "Odd OR Even"	

[4]

(c) The variable names A, B, C and D in part (a) are **not** good programming practice.

(i) State why these variable names are **not** suitable.

[1]

(ii) Identify **one** problem that these variable names might cause.

[1]

(iii) The choice of suitable variable names is one example of good programming practice.

Give **one other** example.

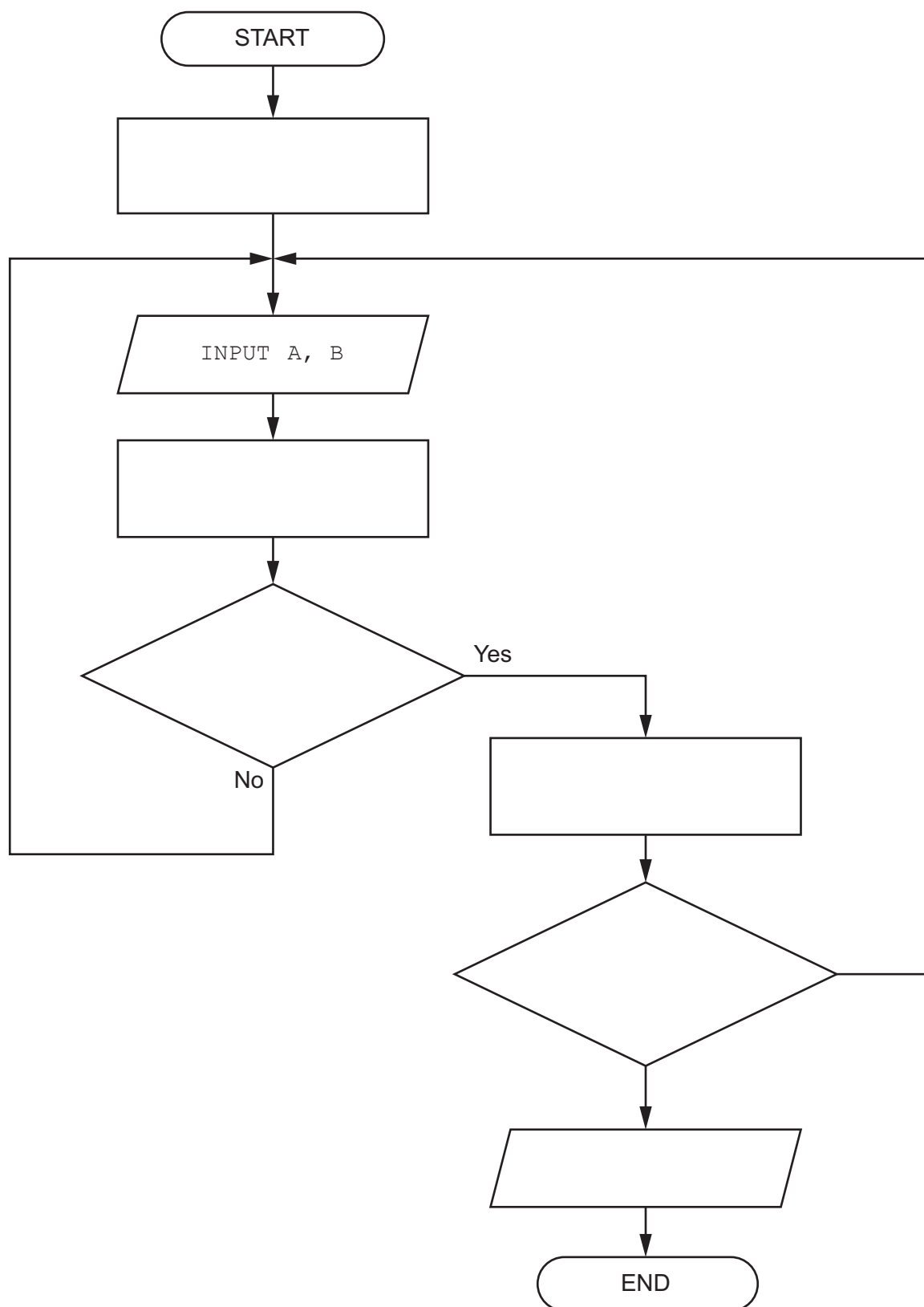
[1]

27. M/J 24/P21/Q2/a

- 2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values A and B
2. count the number of pairs that are input until A has been greater than B 10 times
3. output the number of pairs that were input.

(a) Complete the program flowchart.



28. M/J 24/P21/Q7/a

- 7 A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

- (a) When an exercise class is planned, a new module will send personalised text messages to each member who has expressed an interest in that exercise. Members wishing to join the class send a text message back. Members may decide **not** to receive future text messages by replying with the message 'STOP'.

The process of abstraction is used to filter out unnecessary information.

- (i) State **one** advantage of applying abstraction to this problem.

[1]

- (ii) Identify **three** items of information that will be required by the new module. Justify your choices with reference to the given scenario.

Item 1 required

Justification

Item 2 required

Justification

Item 3 required

Justification

[3]

- (iii) Identify **two** operations that would be required to process data when the new module receives a text message back from a member.

Operation 1

Operation 2

[2]

29. M/J 24/P22/Q1/a,c

- 1 (a) The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- input/output.

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Input/Output
FOR Index $\leftarrow$ 1 TO 10 Data[Index] $\leftarrow$ 0 NEXT Index			
WRITEFILE ThisFile, "*****"			
UNTIL Level > 25			
IF Mark > 74 THEN READFILE OldFile, Data ENDIF			

[4]

(b) Program variables have data types as follows:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER

Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
MyInt $\leftarrow$ (3.1415926)
MyChar $\leftarrow$ ("Elwood", 3, 1)
MyString $\leftarrow$ (..... (27.509))
MyInt $\leftarrow$ (..... ("ABC123", 3))

[4]

(c) The variables given in part (b) are chosen during the design stage of the program development life cycle.

The choices are to be documented to simplify program maintenance.

State a suitable way of documenting the variables **and** give **one** piece of information that should be recorded, in addition to the data type.

[2]

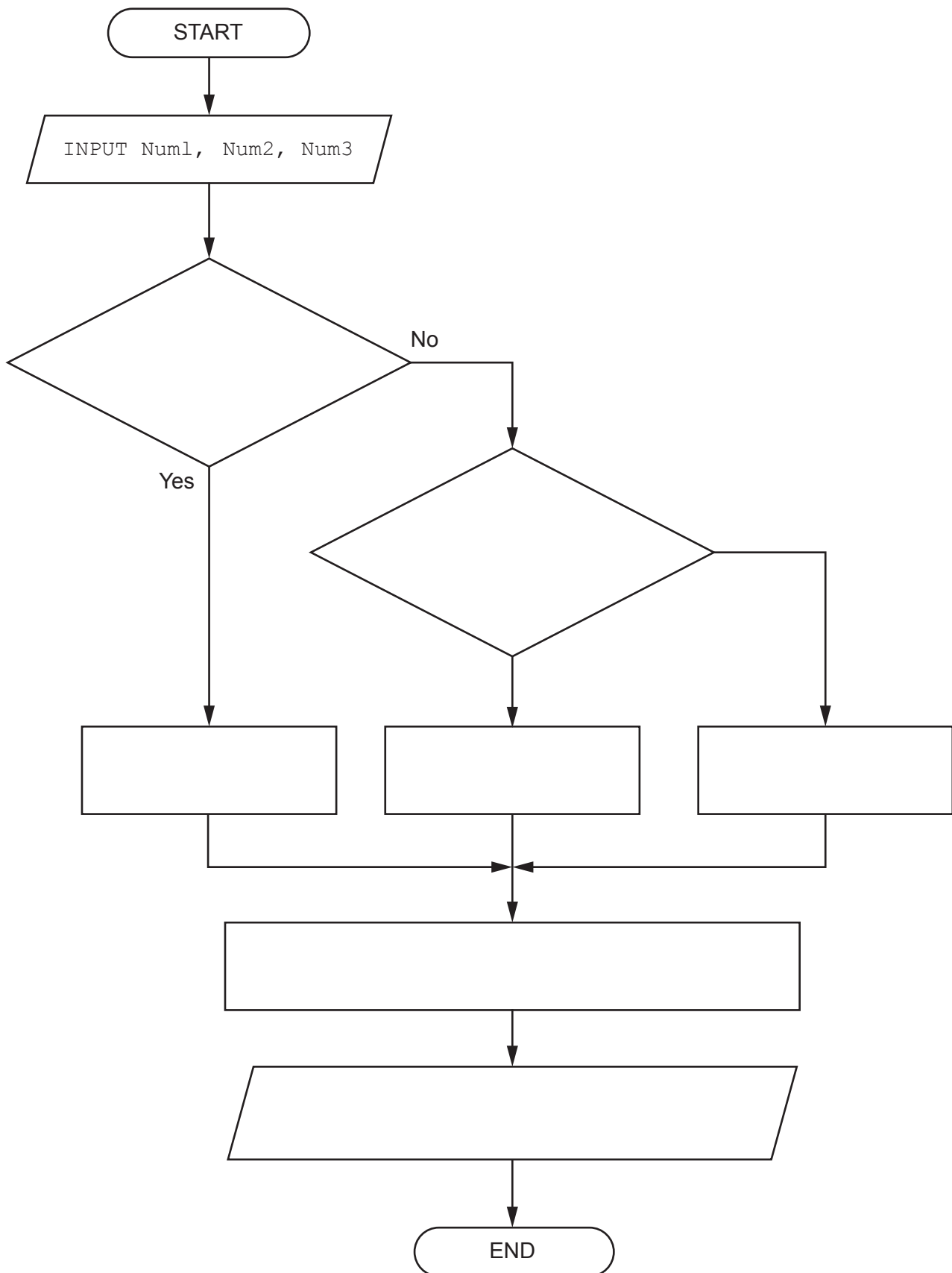
30. M/J 24/P22/Q2/a,b

2 A program is being developed.

- (a) An algorithm for part of the program will:
- input three numeric values and assign them to identifiers `Num1`, `Num2` and `Num3`
 - assign the largest value to variable `Ans`
 - output a message giving the largest value and the average of the three numeric values.

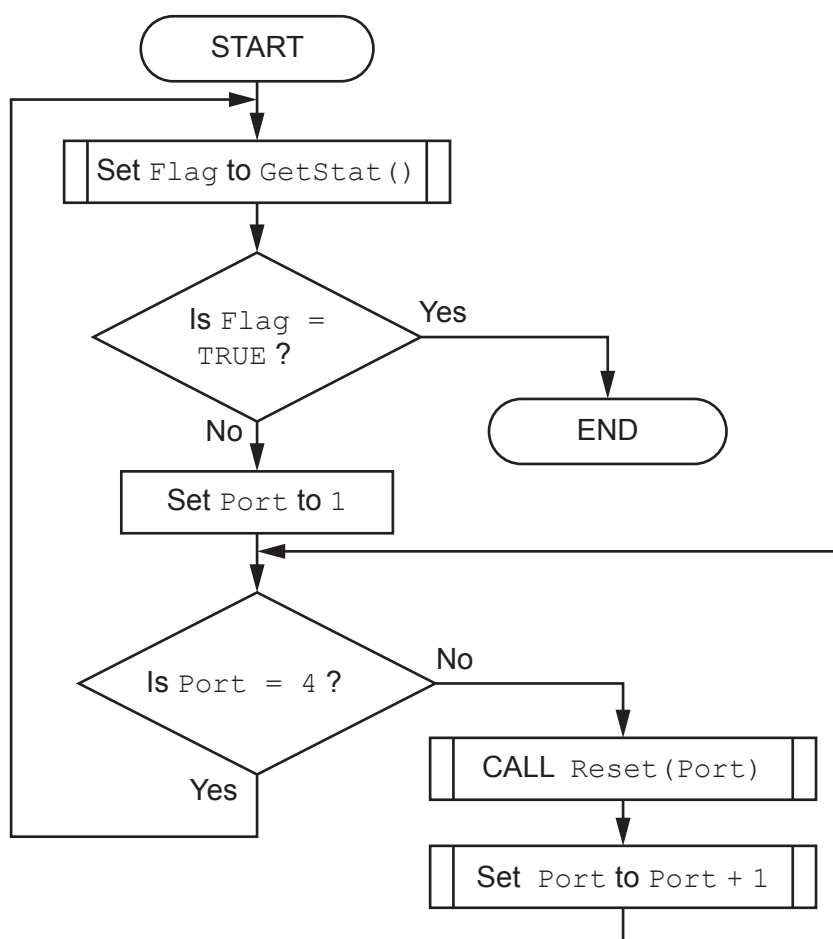
Assume the values are all different and are input in no particular order.

Complete the program flowchart on page 5 to represent the algorithm.



[5]

(b) A different part of the program contains an algorithm represented by the following program flowchart:



Write pseudocode for the algorithm.

[5]

31. M/J 24/P22/Q7/a

- 7** A fitness club has a computerised membership system.

The system stores information for each club member: name, home address, email address, mobile phone number, date of birth and exercise preferences.

Many classes are full, and the club creates a waiting list for each class. The club adds details of members who want to join a class that is full to the waiting list for that class.

When the system identifies that a space is available in one of the classes, a new module will send a text message to each member who is on the waiting list.

- (a)** Decomposition will be used to break the new module into sub-modules (sub-problems).

Identify **three** sub-modules that could be used in the design **and** describe their use.

Sub-module 1

Use

Sub-module 2

Use

Sub-module 3

Use

[3]

32. M/J 24/P22/Q8/a

- 8** A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

- (a)** Suggest a reason why the teacher's program has been split into a number of stages **and** give the benefit of producing a different file from each stage.

Reason

Benefit

[2]

33. M/J 24/P23/Q2/a,b

- 2 (a) A program uses a global 1D array of type string and a text file.

An algorithm that forms part of the program is expressed as follows:

- copy the first line from the file into the first element of the array
- copy the second line from the file into the second element of the array
- continue until all lines in the file have been copied into the array.

Stepwise refinement is applied to the algorithm.

Outline **five** steps for this algorithm that could be used to produce pseudocode.

Assume there are more elements in the array than lines in the file.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

- (b) Sequence is one programming construct.

Identify **one other** programming construct that will be required when the algorithm from part (a) is converted into pseudocode **and** explain its use.

Construct

Use

[2]

**34. M/J 24/P23/Q3/b**

- 3 A record structure is declared to hold data relating to components being produced in a factory:

```
TYPE Component
  DECLARE Item_ID : STRING
  DECLARE Reject : BOOLEAN
  DECLARE Weight : REAL
ENDTYPE
```

The factory normally produces a batch (or set) of 1000 components at a time. A global array is declared to store 1000 records for a batch:

```
DECLARE Batch : ARRAY [1:1000] OF Component
```

Two global variables contain the minimum and maximum acceptable weight for each component. The values represent an inclusive range and are declared as:

```
DECLARE Min, Max : REAL
```

(b) A module `InRange()` will:

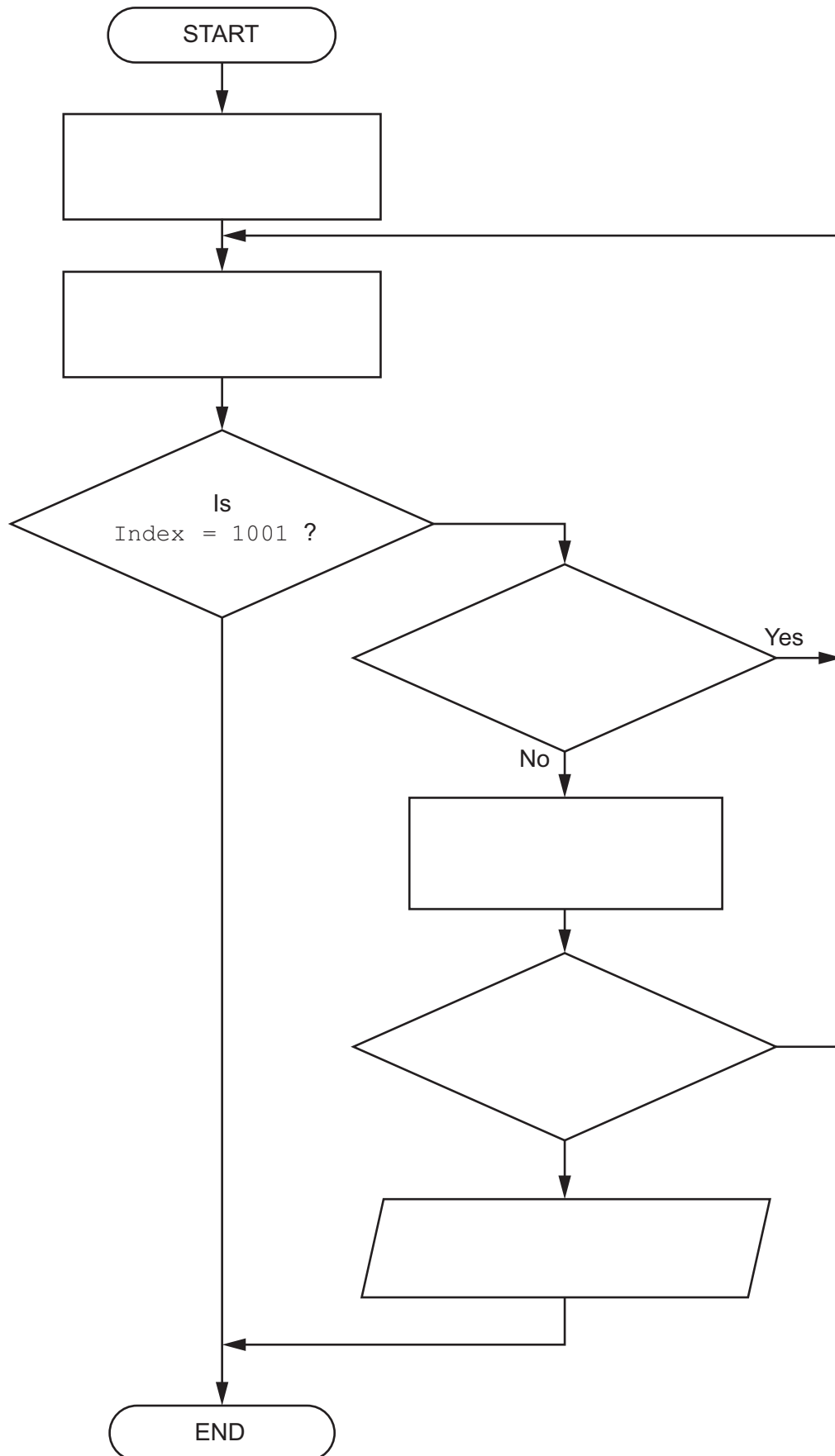
- be called with an integer parameter representing an index value of a record in the `Batch` array
- check if the weight of the indexed component is within the acceptable range
- return `TRUE` if the weight is in the range and `FALSE` if it is **not**.

A module `BatchCheck()` will:

- iterate through a batch of 1000 component records
- call module `InRange()` to check each individual component record
- keep a count of the number of components that fail
- output a suitable warning message and immediately stop if the number of failed components exceeds 5.



Complete the program flowchart to represent the algorithm for module `BatchCheck()`.



35. O/N 23/P21/Q2/a,b

2 (a) An algorithm is expressed as follows:

- input 100 numbers, one at a time
- keep a total of all numbers input that have a value between 30 and 70 inclusive and output this total after the last number has been input.

Outline, using stepwise refinement, the five steps for this algorithm which could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

(b) Sequence is one programming construct.

Identify **two other** programming constructs that will be required when the algorithm is converted into pseudocode.

Construct 1

Construct 2

[2]

36. O/N 23/P22/Q1/a

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer		
3	The number of items in the order		
TRUE	To indicate whether this is a new customer		
15.75	The deposit paid by the customer		

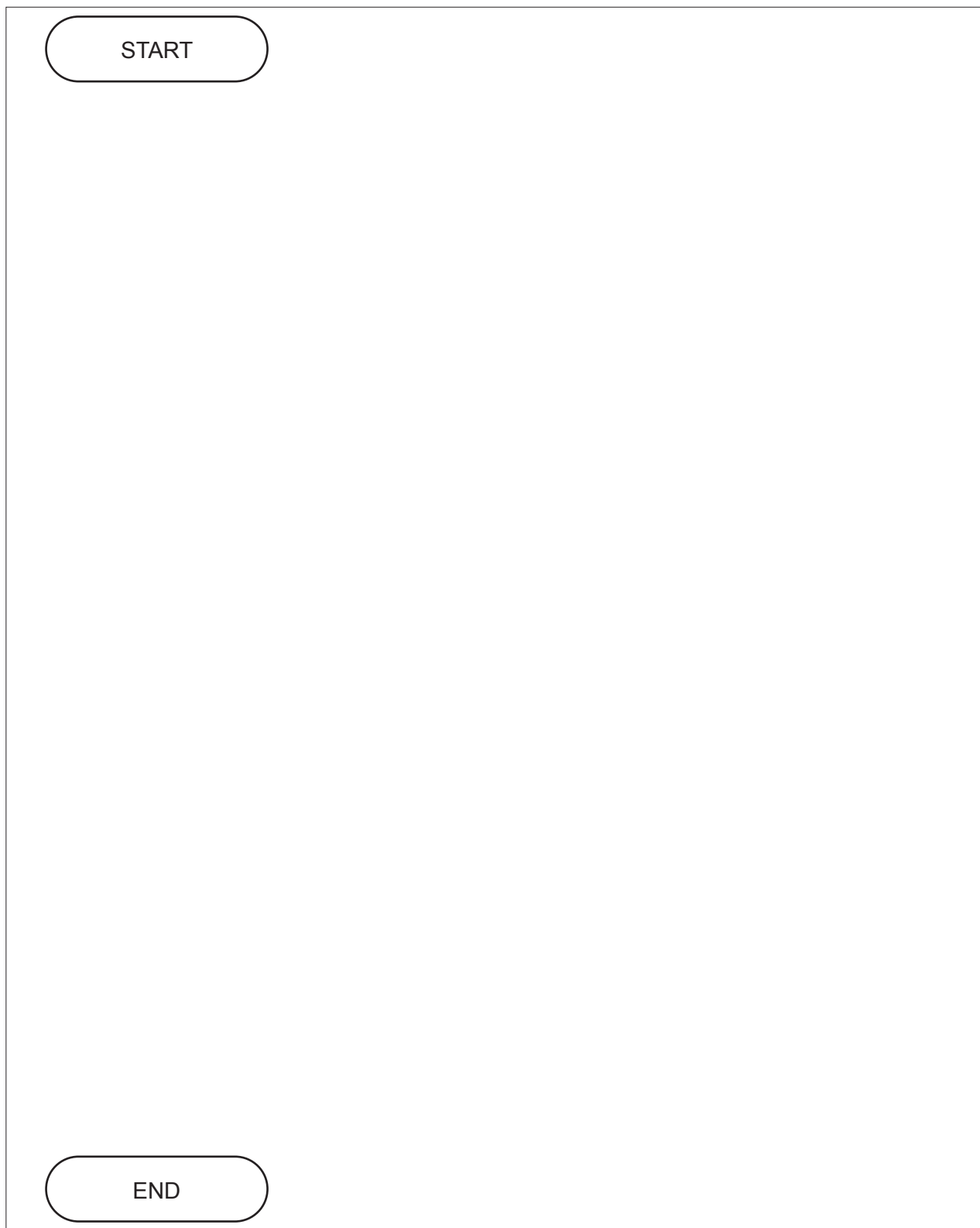
[4]

37. O/N 23/P22/Q2/a

- 2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(a) Draw a program flowchart to represent the algorithm.



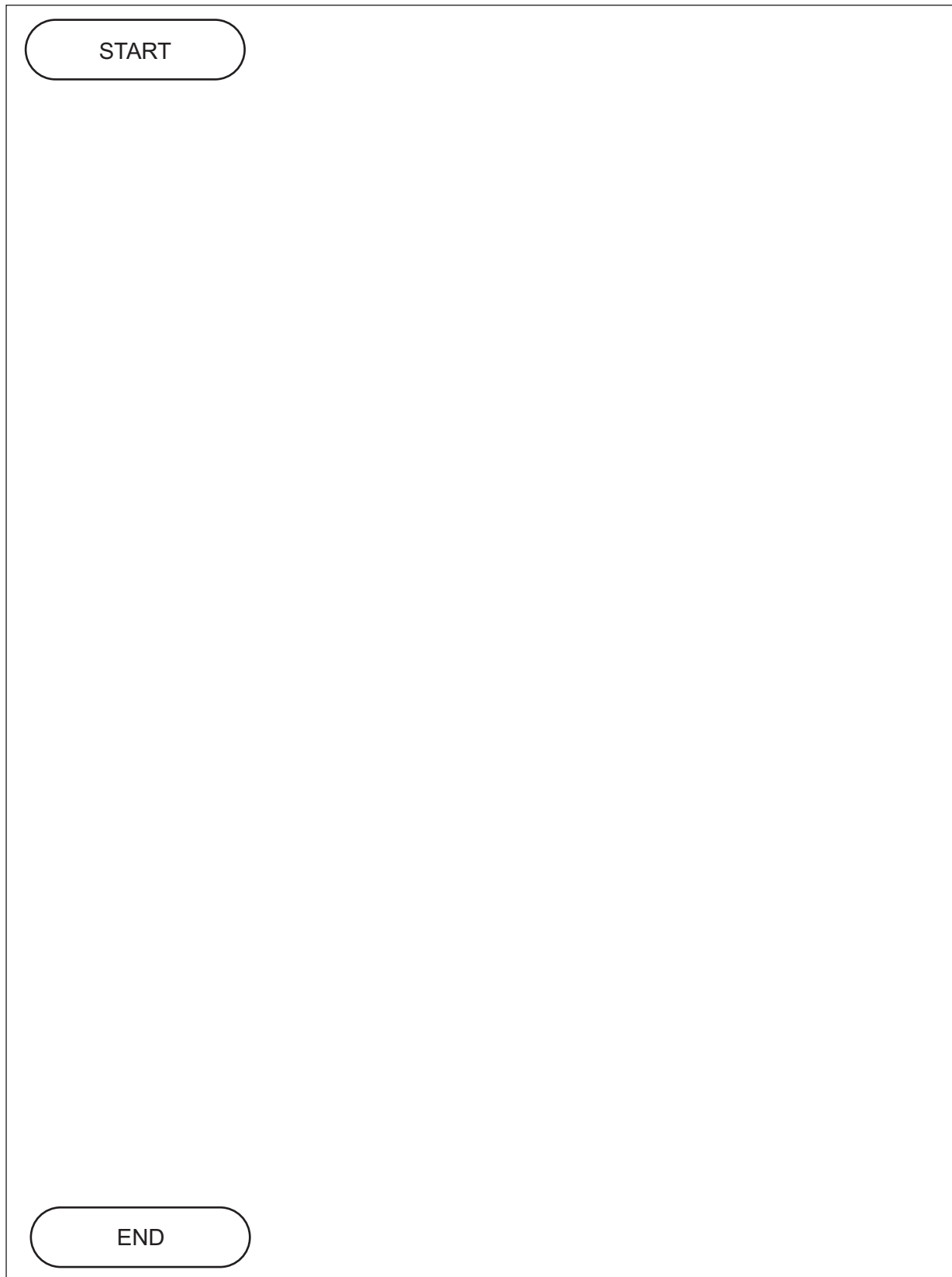
[5]

38. O/N 23/P23/Q2/a

2 Data is a 1D array of integers, containing 30 elements. All element values are unique.

(a) An algorithm will output the index of the element with the smallest value.

Draw a program flowchart to represent the algorithm.



[5]

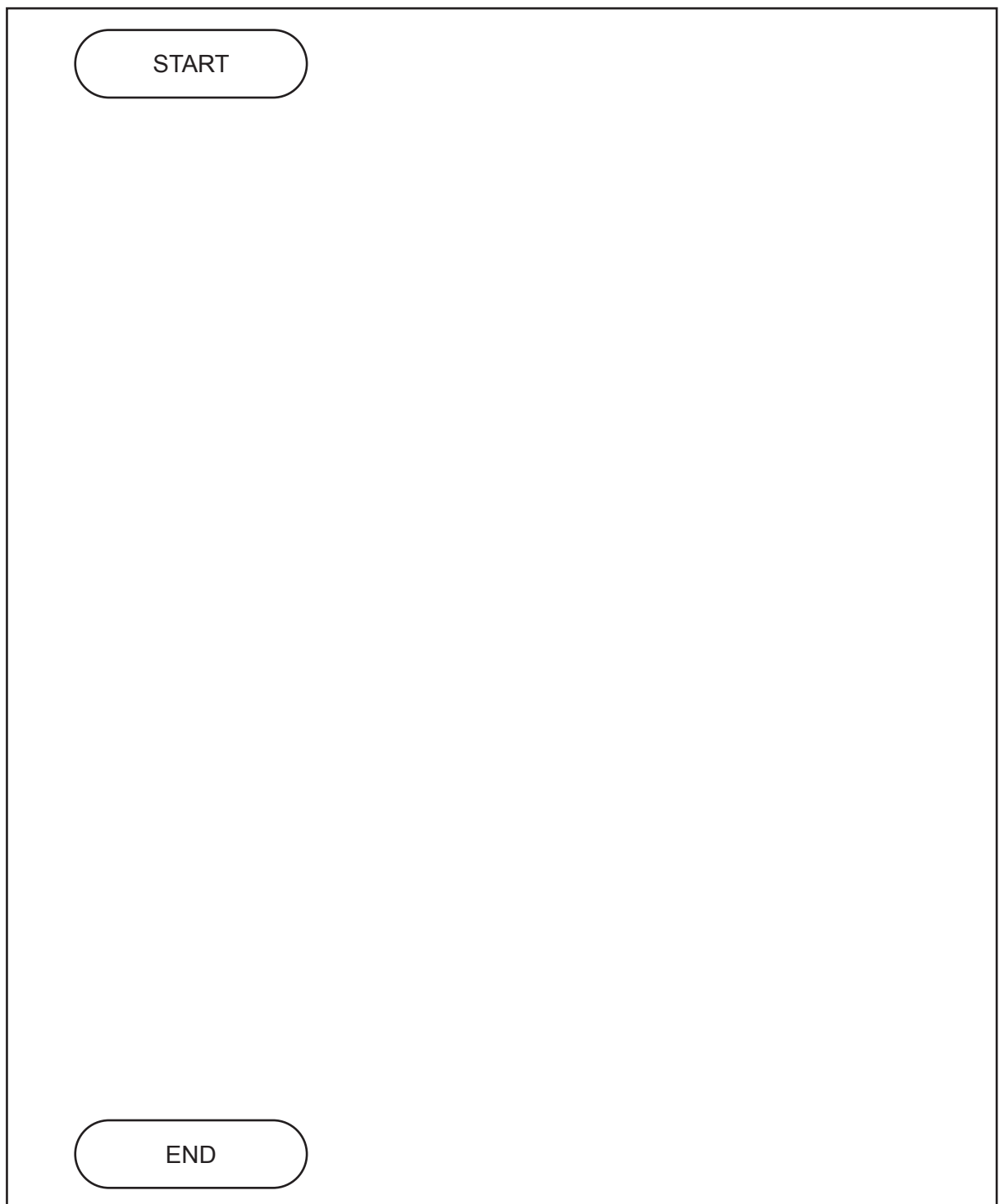
39. M/J 23/P21/Q2/a

- 2 A program contains an algorithm to output a string of a specified length containing identical characters.

(a) The algorithm is described as follows:

1. prompt and input a character and store in `MyChar`
2. prompt and input an integer and store in `MyCount`
3. generate a string consisting of `MyChar` repeated `MyCount` times
4. output the string.

Draw a program flowchart to represent the algorithm.



40. M/J 23/P21/Q3/b

- 3** Customers collect points every time they make a purchase at a store.

A program is used to manage the points system and the table lists some of the information stored for one customer.

Information	Data type required
Name	String
Number of points collected	Integer
Date of birth	Date

- (b)** Customers receive points depending on the amount they spend. The number of points depends on the band that the amount falls into:

Band	Amount	Points
1	Less than \$10	5 per whole dollar (\$)
2	Between \$10 and \$100 inclusive	7 per whole dollar (\$)
3	Over \$100	10 per whole dollar (\$)

For example, if the amount is \$99.77, this amount is in band 2 and therefore the number of points is 7×99 , which is 693 points.

The algorithm to calculate the points from a given amount is expressed as follows:

- work out the appropriate band
- calculate and output the number of points.

Apply the process of stepwise refinement to increase the detail of the algorithm. Structure your algorithm into a sequence of five steps that could be used to produce pseudocode.

Write the **five** steps.

- 1
- 2
- 3
- 4
- 5

41. M/J 23/P21/Q7/a

- 7 A school has a computerised library system that allows students to borrow books for a fixed length of time. The system uses text files to store details of students, books and loans.

A new module is to be written which will generate emails to each student who has an overdue book.

- (a) Decomposition will be used to break down the problem of designing the new module into sub-problems.

Identify **three** program modules that could be used in the design **and** describe their use.

Module 1

Use

Module 2

Use

Module 3

Use

[3]

42. M/J 23/P22/Q1/b

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (b) The lines of pseudocode contain features that make them easier to understand.

State **three** of these features.

1

2

3

[3]

43. M/J 23/P22/Q2/c

2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

(c) The program will output the day of the week corresponding to `MyDOB`.

For example, given the date 22/06/2023, the program will output "Thursday".

An algorithm is required. An array will be used to store the names of the days of the week.

Define the array **and** describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Array definition

Step 1

Step 2

Step 3

Step 4

[6]

44. M/J 23/P22/Q3/b

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

(b) The design of the queue is completed and the number of locations is increased.

A function `AddToQueue()` has been written. It takes a string as a parameter and adds this to the queue. The function will return `TRUE` if the string was added successfully.

A procedure `FileToQueue()` will add each line from the file to the queue. This procedure will end when all lines have been added or when the queue is full.

Describe the algorithm for procedure `FileToQueue()`.

Do **not** use pseudocode in your answer.

[5]

45. M/J 23/P22/Q7/a

- 7** A computer system for a shop stores information about each customer. The items of information include name and address (both postal and email) together with payment details and order history. The system also stores the product categories they are interested in and how they would like to be contacted.

- (a)** The shop wants to add a program module that will generate emails to be sent to customers who may be interested in receiving details of new products.

- (i)** State **three** items of information that the new module would need. Justify your choice in each case.

Information

Justification

Information

Justification

Information

Justification

[3]

- (ii)** Identify **two** items of customer information that would **not** be required by the new module. Justify your choice in each case.

Information

Justification

Information

Justification

[2]

46. M/J 23/P23/Q1/d

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```

10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100 FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103 NEXT PCount

```

- (d) The pseudocode statements given in the following table are used in other parts of the algorithm.

Complete the table by placing **one or more** ticks (✓) in each row.

The first row has already been completed.

Pseudocode statement	Input	Process	Output
INPUT MyChoice	✓		
OUTPUT FirstName & LastName			
WRITEFILE YourFile, TextLine			
READFILE MyFile, TextLine			
Result ← SQRT (NextNum)			

[4]

47. M/J 23/P23/Q3/b(ii)

- 3 A program processes data using a stack. The data is copied to a text file before the program ends.

- (b) The data is copied to a text file before the program ends.

- (ii) A module `SaveStack()` will write the data from the stack to a text file.

Express an algorithm for `SaveStack()` as five steps that could be used to produce pseudocode.

Write the **five** steps.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

48. M/J 23/P23/Q7/a

- 7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

A program helps the staff to manage the borrowing of books.

- (a) A new module needs to be written to generate emails to send to students who have an overdue book. Students who are sent an email are prevented from borrowing any more books until the overdue book is returned.

The process of abstraction has been used when designing the new module.

- (i) State the purpose of applying abstraction to this problem.

[1]

- (ii) Identify **one** item of information that is required and **one** item that is **not** required in the new module. Justify your choices.

Item required

Justification

Item not required

Justification

[2]

- (iii) Identify **two** operations that would be required to process data when an overdue book is returned.

Operation 1

Operation 2

[2]

49. O/N 22/P21/Q2/a,b

- 2** A system is being developed to help manage a car hire business. A customer may hire a car for a number of days.

An abstract model needs to be produced.

- (a)** Explain the process of abstraction **and** state **four** items of data that should be stored each time a car is hired.

Explanation

Item 1

Item 2

Item 3

Item 4

[3]

- (b)** Identify **two** operations that would be required to process the car hire data.

Operation 1

Operation 2

[2]

50. O/N 22/P21/Q3

- 3** A 1D array `Data` of type integer contains 200 elements. Each element has a unique value.

An algorithm is required to search for the largest value and output it.

Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

[5]

51. O/N 22/P22/Q2/a

also Unit 10 (2 marks); this unit 5 marks

- 2 (a)** An algorithm will process data from a test taken by a group of students. The algorithm will prompt and input the name and test mark for each of the 35 students.

The algorithm will add the names of all the students with a test mark of less than 20 to an existing text file `Support_List.txt`, which already contains data from other group tests.

- (i) Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

[5]

- (ii) Explain why it may be better to store the names of the students in a file rather than in an array.

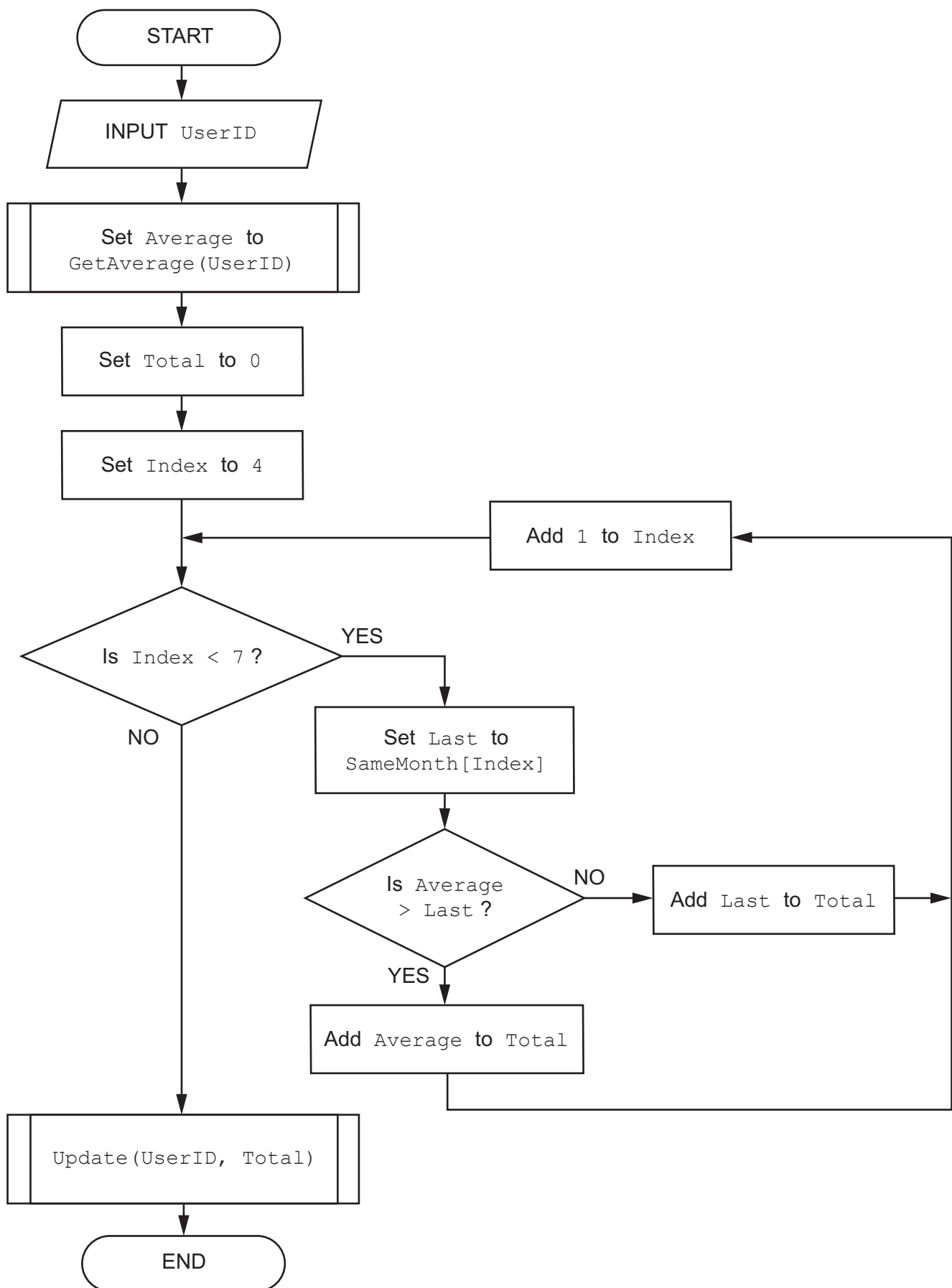
[1]

- (iii) Explain why `WRITE` mode cannot be used in the answer to **part 2(a)(i)**.

[1]

52. O/N 22/P22/Q4/a

- 4** The program flowchart represents a simple algorithm.



(a) Write the equivalent pseudocode for the algorithm represented by the flowchart.

53. O/N 22/P22/Q5

5 Examine the following pseudocode.

```
IF A = TRUE THEN
  IF B = TRUE THEN
    IF C = TRUE THEN
      CALL Sub1 ()
    ELSE
      CALL Sub2 ()
    ENDIF
  ENDIF
ELSE
  IF B = TRUE THEN
    IF C = TRUE THEN
      CALL Sub4 ()
    ELSE
      CALL Sub3 ()
    ENDIF
  ELSE
    IF C = FALSE THEN
      CALL Sub3 ()
    ELSE
      CALL Sub4 ()
    ENDIF
  ENDIF
ENDIF
```

A programmer wants to re-write the pseudocode as **four** separate IF...THEN...ENDIF statements, each containing a single CALL statement. This involves writing a single, simplified logic expression as the condition in each statement.

Write the amended pseudocode.

- 1
- 2
- 3
- 4

[4]

54. O/N 22/P23/Q2

- 2 A program is being designed for a smartphone to allow users to send money to the charity of their choice.

Decomposition will be used to break the problem down into sub-problems.

Identify **three** program modules that could be used in the design **and** describe their use.

Module 1

Use

Module 2

Use

Module 3

Use

[3]

55. O/N 22/P23/Q3

- 3 An algorithm is needed to process a sequence of numbers. Numbers may be positive or negative and may be integer or decimal.

The algorithm will:

- prompt and input one number at a time until the value zero is input
- sum the negative numbers
- sum the positive numbers
- when zero is input, output the two sum values and then end.

Describe the algorithm needed. Do **not** include pseudocode statements in your answer.

[5]

56. O/N 22/P23/Q5/c

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (c) `Unpack()` is part of a program made up of several modules. During the design stage, it is important to follow good programming practice. One example of good practice is the use of meaningful identifier names.

Give the reason why this is good practice. Give **two other** examples of good practice.

Reason

Example 1

Example 2

[3]

57. O/N 22/P23/Q7/b(ii)

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	""
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description
<code>OutputRange()</code>	• Prompts for input of two error numbers • Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description • Outputs a warning message when the error description is missing as for error number 50 in the example • Outputs a suitable header and a final count of error numbers found Output based on the example array data above: <pre>List of error numbers from 1 to 60 10 : Invalid identifier name 20 : Bracket mismatch 50 : Error Text Missing 60 : Type mismatch in assignment 4 error numbers output</pre>

- (ii) A new Module `RemoveError()` will remove a given error number from the array.

Describe the algorithm that would be required. Do **not** include pseudocode statements in your answer.

58. O/N 21/P21/Q1/c

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.
- (c) An airline wants to provide passengers with information about individual flights and allow them to book their flight using an online booking system.
- (i) Tick (✓) **one** box in each row of the table to indicate whether each item of information would be essential for the customer when making the booking.

Information	Essential	Not essential
Departure time		
Flight number		
Departure airport		
Aircraft type		
Ticket price		
Number of seats in aircraft		

[3]

- (ii) Identify the technique used to filter out information that is not essential when designing the booking system **and** state one benefit of this technique.

Technique

Benefit

[2]

- (iii) Identify **two additional** pieces of essential information that a passenger might need when booking a flight.

1

2

[2]

59. O/N 21/P21/Q2/a,b

- 2 (a) An algorithm to sort a 1D array into ascending order is described as follows:

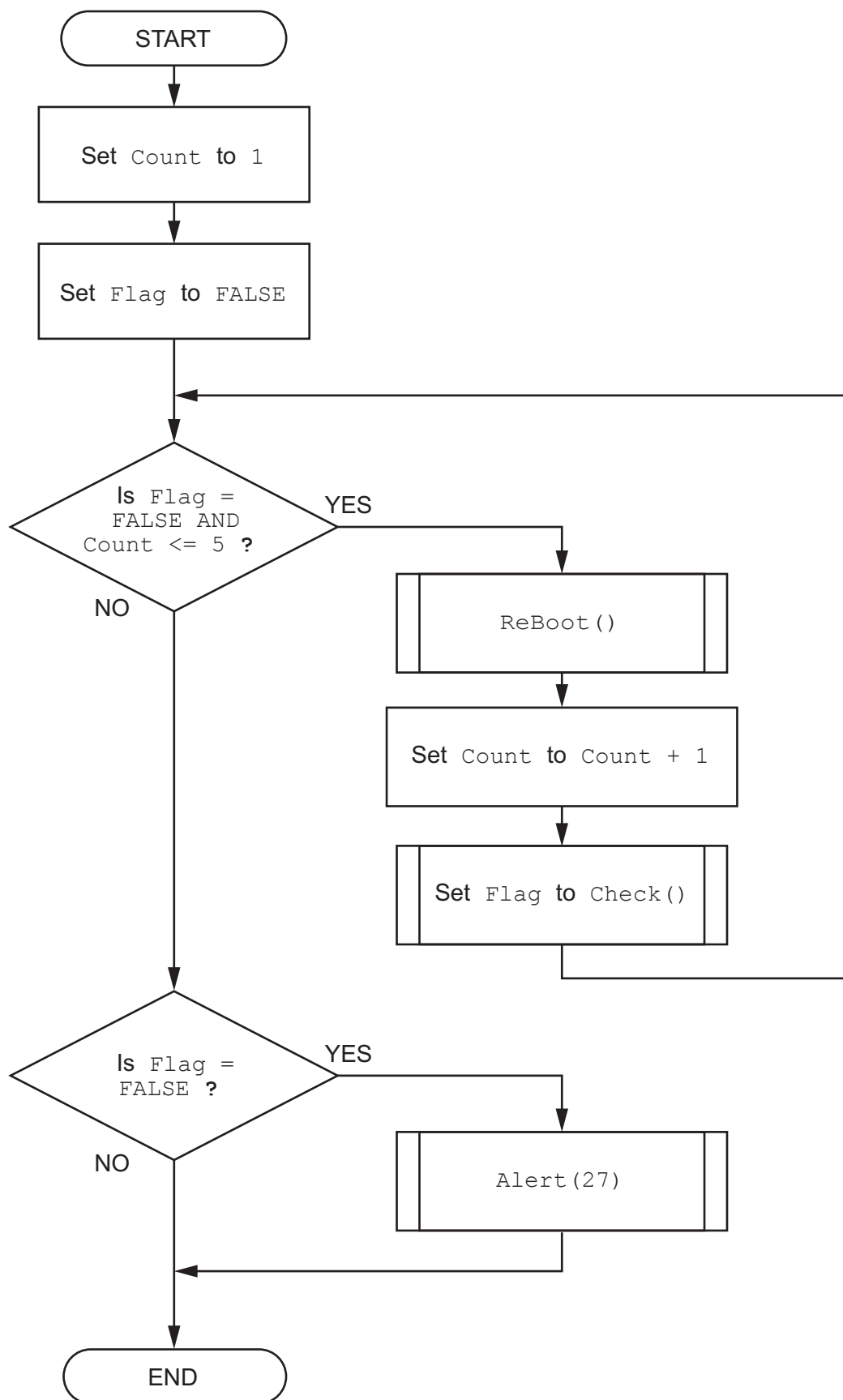
- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

[6]

- (b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

60. O/N 21/P22/Q1/a

- 1 (a) A programmer applies decomposition to a problem that she has been asked to solve.

Describe decomposition.

[2]

61. O/N 21/P22/Q2/a

- 2 (a) An algorithm will:

1. input an integer value
2. jump to step 6 if the value is zero
3. sum and count the positive values
4. sum and count the negative values
5. repeat from step 1
6. output the two sum values and the two count values.

Draw a program flowchart on the following page to represent the algorithm.

Note that variable declarations are not required in program flowcharts.

62. O/N 21/P22/Q4/b

4 The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```
10  PROCEDURE Check(InString : STRING)
11      DECLARE Odds, Evens, Index : INTEGER
12
13      Odds ← 0
14      Evens ← 0
15      Index ← 1
16
17      WHILE Index <= LENGTH(InString)
18          IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19              Odds ← Odds + 1
20          ELSE
21              Evens ← Evens + 1
22          ENDIF
23          Index ← Index + 1
24      ENDWHILE
25
26      CALL Result(Odds, Evens)
27  ENDPROCEDURE
```

(b) The pseudocode includes several features that make it easier to read and understand.

Identify **three** of these features.

1

2

3

[3]

63. O/N 21/P23/Q1/c

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.
- (c) An airline wants to provide passengers with information about individual flights and allow them to book their flight using an online booking system.
- (i) Tick (✓) **one** box in each row of the table to indicate whether each item of information would be essential for the customer when making the booking.

Information	Essential	Not essential
Departure time		
Flight number		
Departure airport		
Aircraft type		
Ticket price		
Number of seats in aircraft		

[3]

- (ii) Identify the technique used to filter out information that is not essential when designing the booking system **and** state one benefit of this technique.

Technique

Benefit

[2]

- (iii) Identify **two additional** pieces of essential information that a passenger might need when booking a flight.

1

2

[2]

64. O/N 21/P23/Q2/a,b

- 2 (a) An algorithm to sort a 1D array into ascending order is described as follows:

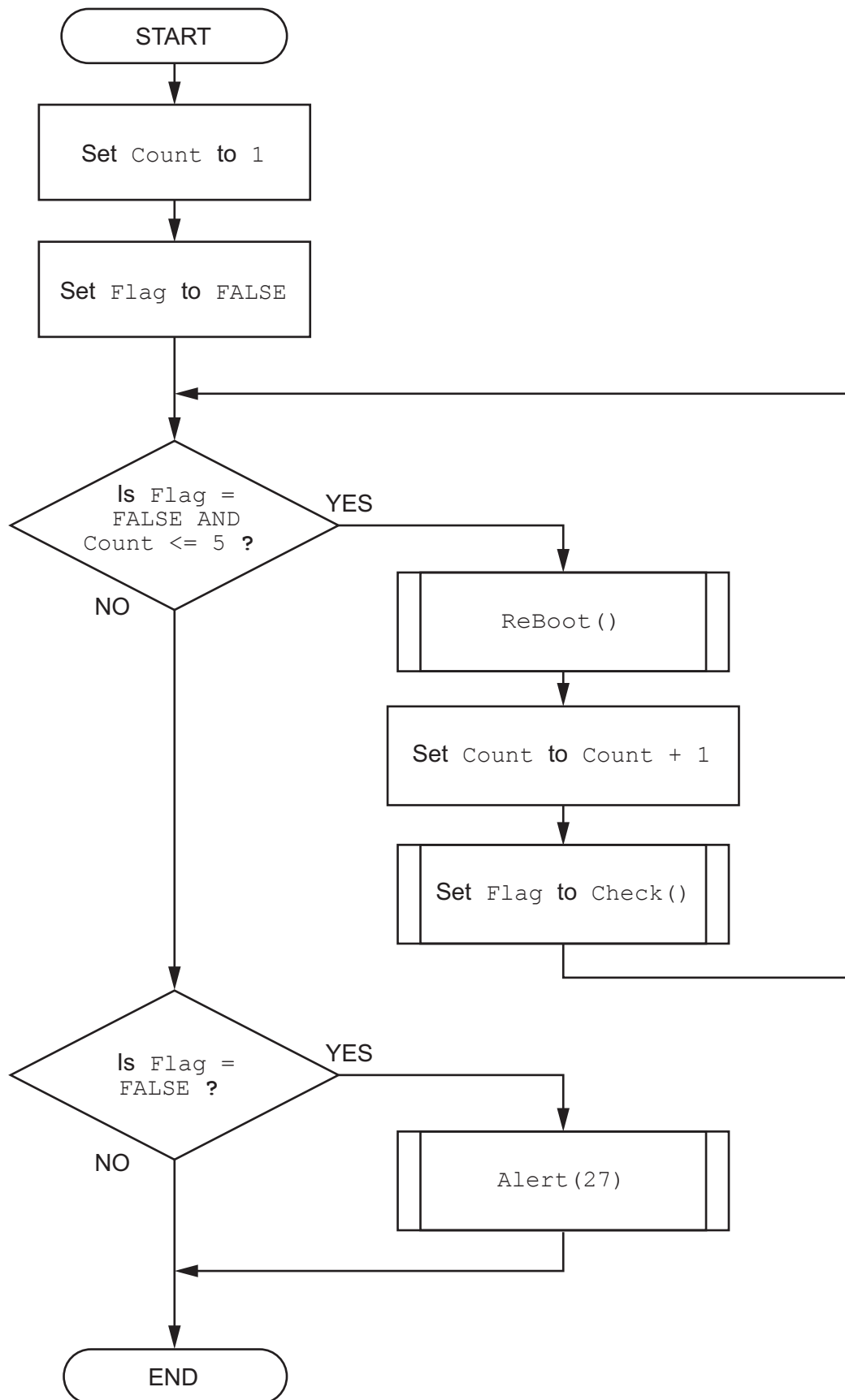
- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

[6]

- (b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

65. M/J 21/P21/Q1/a

- 1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club		
FALSE	A value to indicate whether an existing member of the club lives at the same address		
19/02/1983	When the member joined the football club		
1345	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

66. M/J 21/P21/Q2/c

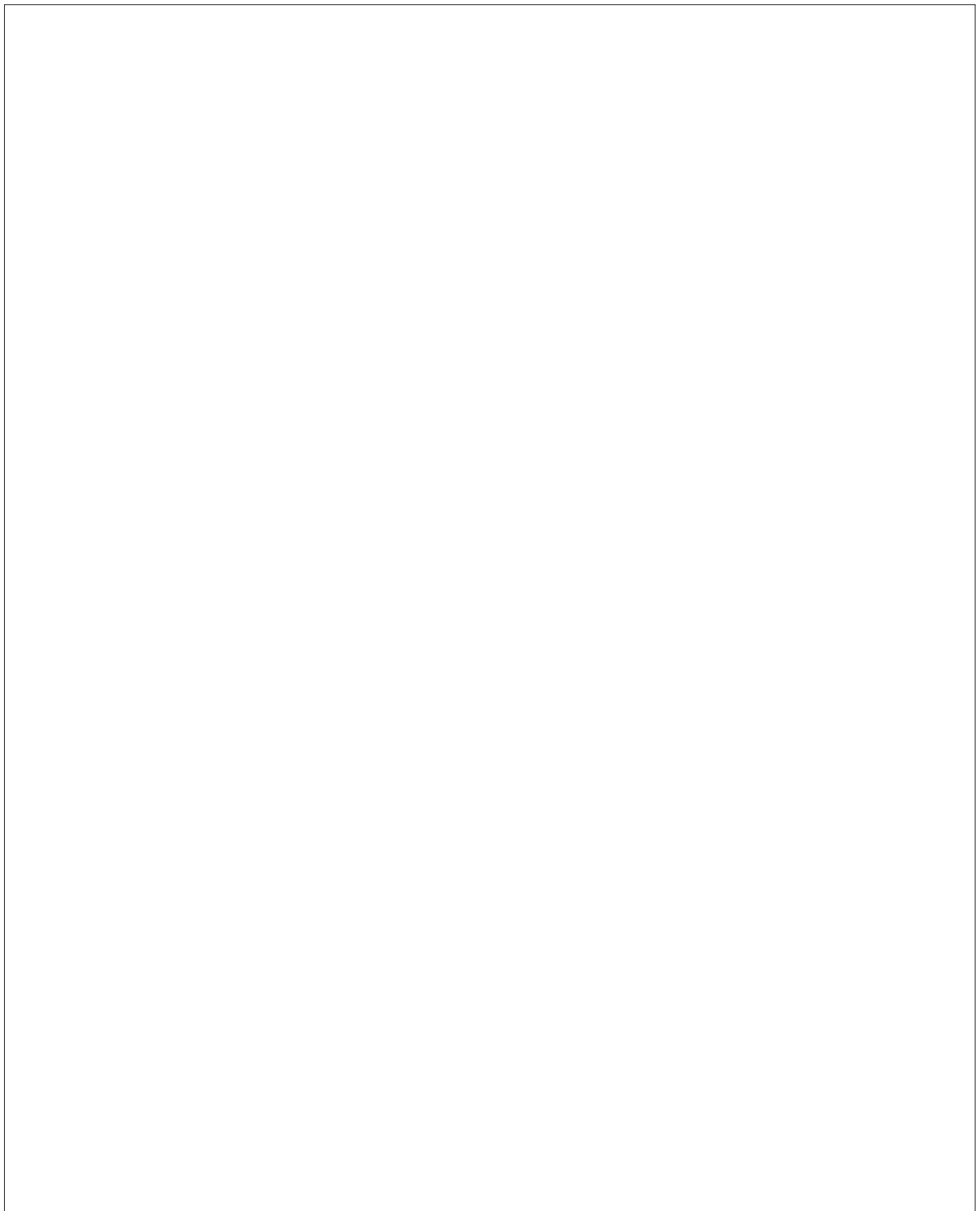
- (c) A program will:

- input 50 unique integer values
- output the largest value
- output the average of the values **excluding** the largest value.

Draw a program flowchart to represent the algorithm.

Variable declarations are **not** required.

It is not necessary to check that each input value is unique.



[6]

67. M/J 21/P21/Q7/b

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

- (b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string ("") and may occur anywhere in the array.

A new module `AddWord()` is needed as follows:

Module	Description
<code>AddWord()</code>	• Called with a parameter representing a word • Stores the word in an unused element of the <code>IgnoreList</code> array and returns <code>TRUE</code> • Returns <code>FALSE</code> if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

68. M/J 21/P22/Q1/b

- 1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Name	"Catherine"	
Index	100	
Modified	FALSE	
Holiday	25/12/2020	

[4]

- (ii) Evaluate each expression in the following table by using the initial data values shown in part (a)(i).

Expression	Evaluates to
Modified OR Index > 100	
LENGTH("Student: " & Name)	
INT(Index + 2.9)	
MID(Name, 1, 3)	

[4]

- (b) Each pseudocode statement in the following table contains an example of selection, assignment or iteration.

Put **one** tick (✓) in the appropriate column for each statement.

Statement	Selection	Assignment	Iteration
Index ← Index + 1			
IF Modified = TRUE THEN			
ENDWHILE			

[3]

69. M/J 21/P22/Q2/b

(b) A system is being developed to help manage book loans in a library.

Registered users may borrow books from the library for a period of time.

(i) State **three** items of data that must be stored for each loan.

1

2

3

[2]

(ii) State **one** item of data that will be required in the library system but does not need to be stored for each loan.

[1]

(iii) One operation that manipulates the data stored for each loan, would produce a list of all overdue books.

Identify **two other** operations.

Operation 1

Operation 2

[2]

70. M/J 21/P22/Q4

4 A teacher uses a paper-based system to store marks for a class test. The teacher requires a program to assign grades based on these results.

The program will output the grades together with the average mark.

Write a detailed description of the algorithm that will be needed.

[6]

71. M/J 21/P23/Q1/a

- 1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club		
FALSE	A value to indicate whether an existing member of the club lives at the same address		
19/02/1983	When the member joined the football club		
1345	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

72. M/J 21/P23/Q2/c

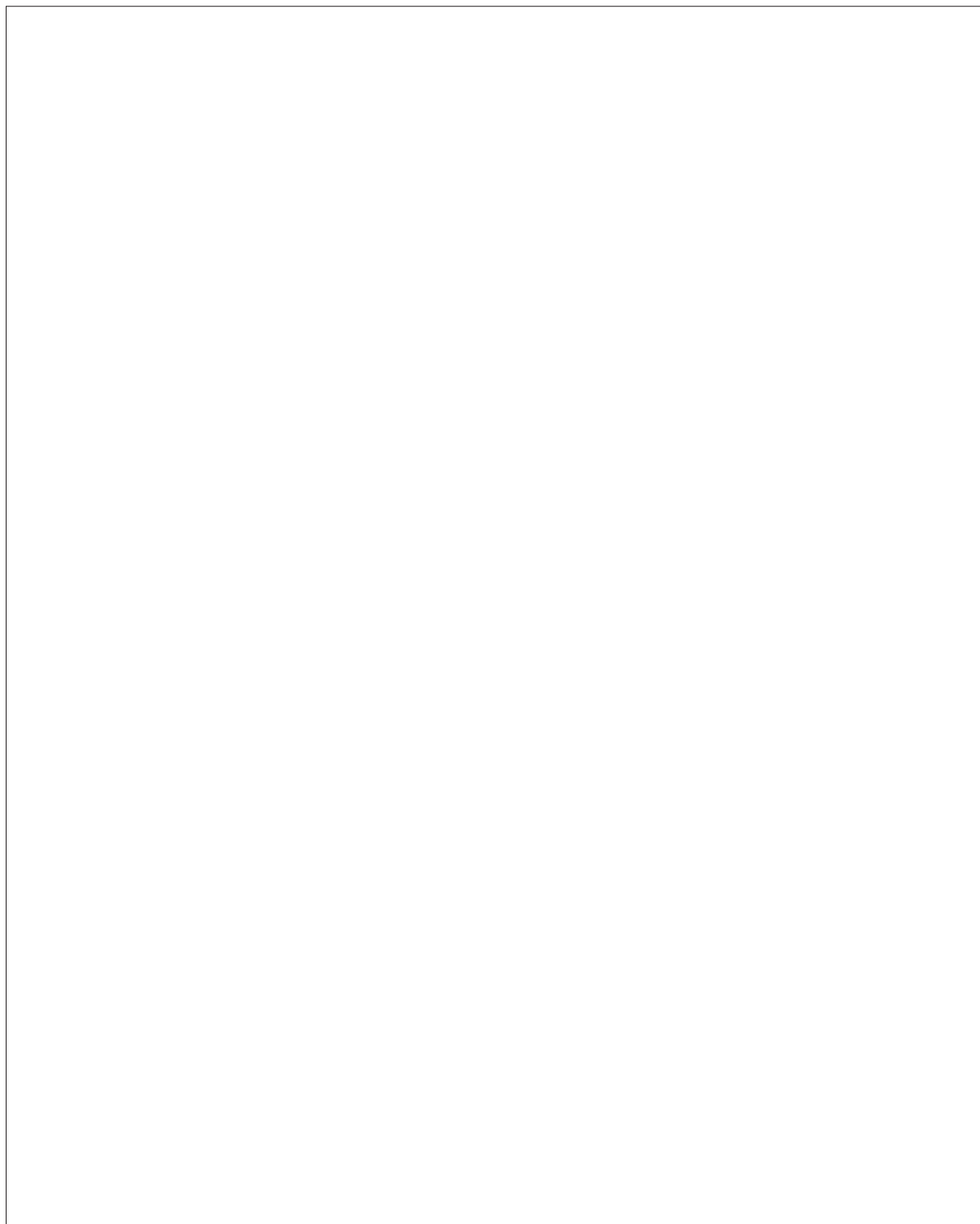
- (c) A program will:

- input 50 unique integer values
- output the largest value
- output the average of the values **excluding** the largest value.

Draw a program flowchart to represent the algorithm.

Variable declarations are **not** required.

It is not necessary to check that each input value is unique.



[6]

73. M/J 21/P23/Q7/b

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

- (b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string ("") and may occur anywhere in the array.

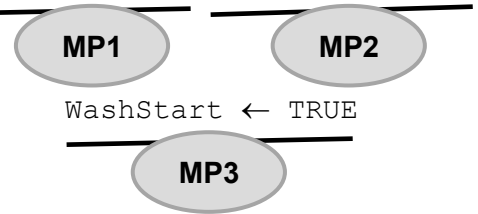
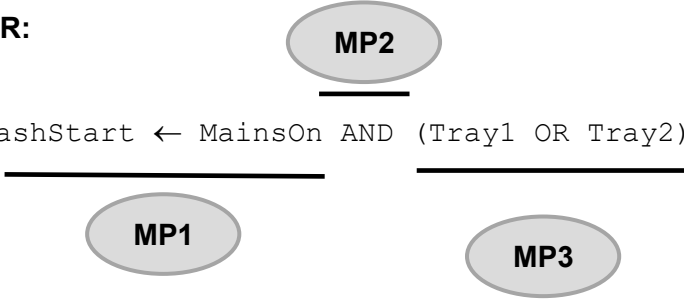
A new module `AddWord()` is needed as follows:

Module	Description
<code>AddWord()</code>	• Called with a parameter representing a word • Stores the word in an unused element of the <code>IgnoreList</code> array and returns <code>TRUE</code> • Returns <code>FALSE</code> if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

Answers Section

1. M/J 26/P21/Q2/a(ii,iii)

2(a)(ii)	MainsOn (= TRUE) AND WaterOn (= TRUE) Or reverse order ...	1
2(a)(iii)	IF MainsOn AND (Tray1 OR Tray2) THEN  WashStart ← TRUE ENDIF OR:  WashStart ← MainsOn AND (Tray1 OR Tray2)	3

2. M/J 26/P21/Q4

4	<i>Example solution:</i> Initialise Count to zero REPEAT INPUT integer number IF integer number > 0 then increment the count UNTIL integer number equals 0 OUTPUT Count <i>Mark as follows:</i> MP1 Initialise Count to zero and later evidence of use of counter MP2 Conditional loop and terminated when 0 is input MP3 Test for number > 0 and increment count inside the loop MP4 Output the count (either 'sum' or 'count') Max 4	4
---	---	----------

3. M/J 26/P22/Q2/a,b,c

2(a)	One mark for either explanation: - To filter out <u>information / data</u> that is not necessary for the task - To keep only <u>information / data</u> that is necessary for the task Max 1 mark	1	
2(b)	One mark for each TWO data items Example answers 1 Tool ID // Tool name // Type of tool 2 Customer ID // Customer name // Customer address // Account reference 3 Hire ID 4 (Hire) Date 5 Hire time 6 Length of hire: Full or Half day 7 Cost of hire 8 Payment details 9 Tool status Max 2 marks	2	Reject Quantity Reject Return date if Hire date already given MP 2 Reject just 'Customer'
2(c)	One mark for each process. Examples include: 1 Input customer details / customer ID / payment details / Tool ID 2 Search for required tool // Checking if tool returned // Checking if tool available // Searching for dates when tool is available 3 Record tool booking // Amend Tool booking // Create tool booking // Record when tool returned // Confirm booking 4 Calculating time when tool has to be returned 5 Change / check tool status e.g. hired / available / damaged // Prevent same tool being hired twice 6 Process payment / calculate hire cost Max 2 marks	2	

4. M/J 26/P22/Q3

3	Example solution Mark points: MP 1 (Step 4) Open <u>WordList.txt</u> / File in read mode (and subsequently close) MP 2 (Step 5) Read the next line from the file in a loop MP 3 (Step 6) Increment line counter in a loop MP 4 (Step 7) Check if the line read equals the word (input) in a loop MP 5 (Step 8) ...If it does output line counter MP 6 (Step 9) If word not found in file then output "Not Found" after all iterations MP 7 (Step 10) Otherwise repeat from step 5 Max 6 marks from 7 possible marks	6 MP 1 cannot be in a loop structure Do not accept once if MP 2, 3, 4 is clearly not in a loop Award only one mark for MP 5 and MP 6 if both outputs are possible MP 7 Can be any loop but it must be clear where it starts and ends and must include at least one of the end conditions.
---	--	--

5. M/J 26/P23/Q2/b,c

2(b)	<pre>IF MainsOn (=TRUE) THEN DisplayOn ← TRUE ENDIF</pre> Accept: <pre>DisplayOn ← MainsOn</pre> Accept '=' in place of the assignment arrow	1	
2(c)	MP1 The robot is only ready to move if... the mains power is on and MP2 The strength of the (suction) value selected is non-zero	2	Reject – Zero for a 'repeat' of the pseudocode statement.

6. M/J 26/P23/Q4

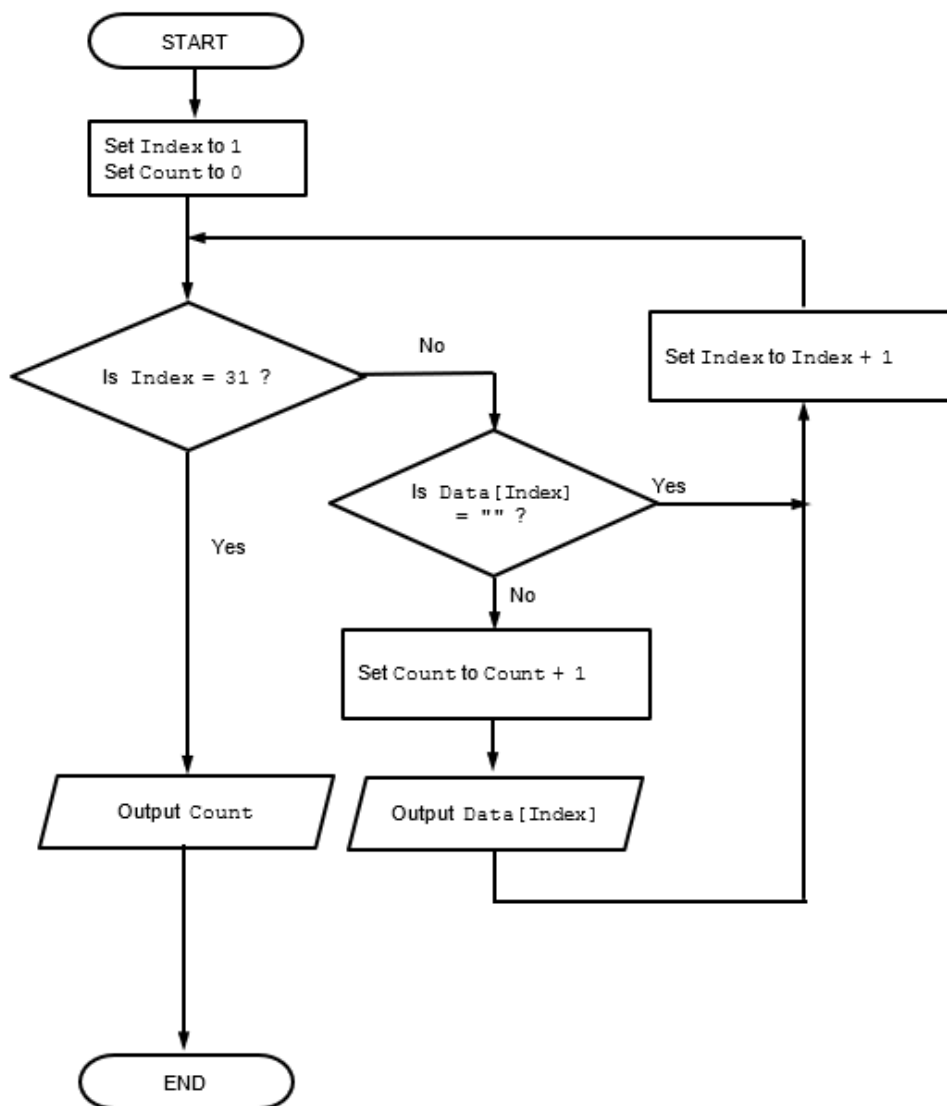
4	<i>Example solution:</i> • Initialise count to zero • REPEAT • Input next number • If number > 120 then increment the count • Until number = -1 • Output the count <i>Mark as follows:</i> MP1 Initialise count and later evidence of use of counter (not 'sum') MP2 Conditional loop and terminated when -1 is input MP3 Test for number > 120 and increment the count (not 'sum') inside the loop MP4 Output the count (Accept either 'sum' or 'count')	4	A 'sum' solution will have a Max 2.
---	---	---	--

7. O/N 25/P21/Q2

2

Example solution

5



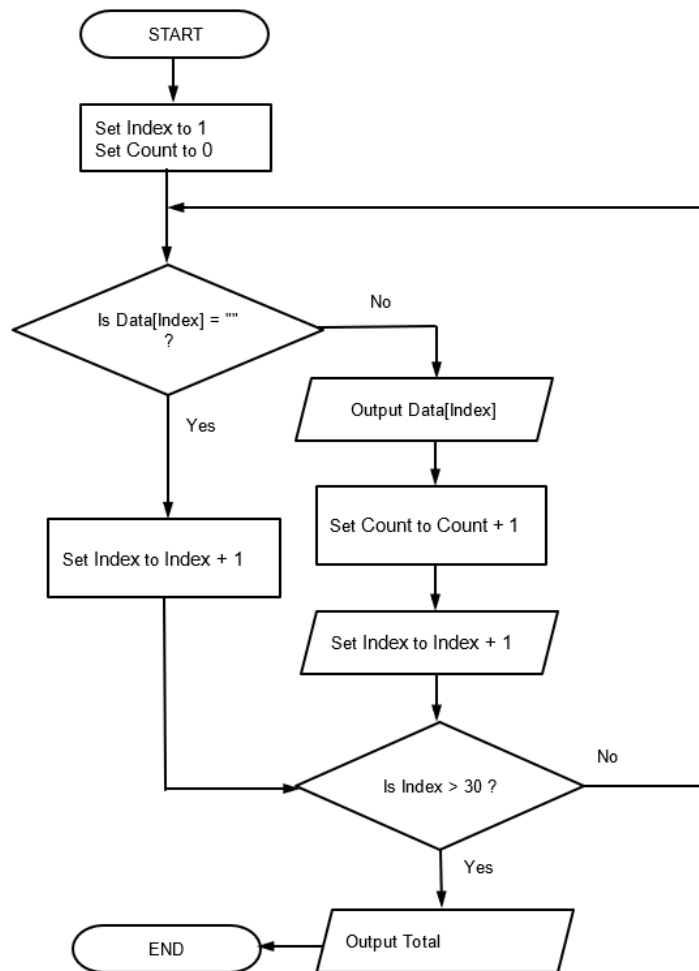
Mark as follows:

- 1 Initialisation of variables used as **array index** and **count**
- 2 Loop through exactly 30 elements in the array
- 3 Test for empty string in current array element // Test for non-empty string
- 4 Increment count if appropriate **in a loop**
- 5 Both required outputs under correct conditions with correct outputs

2

Alternative solution

Checking for empty string first



8. O/N 25/P22/Q1/a

1(a)

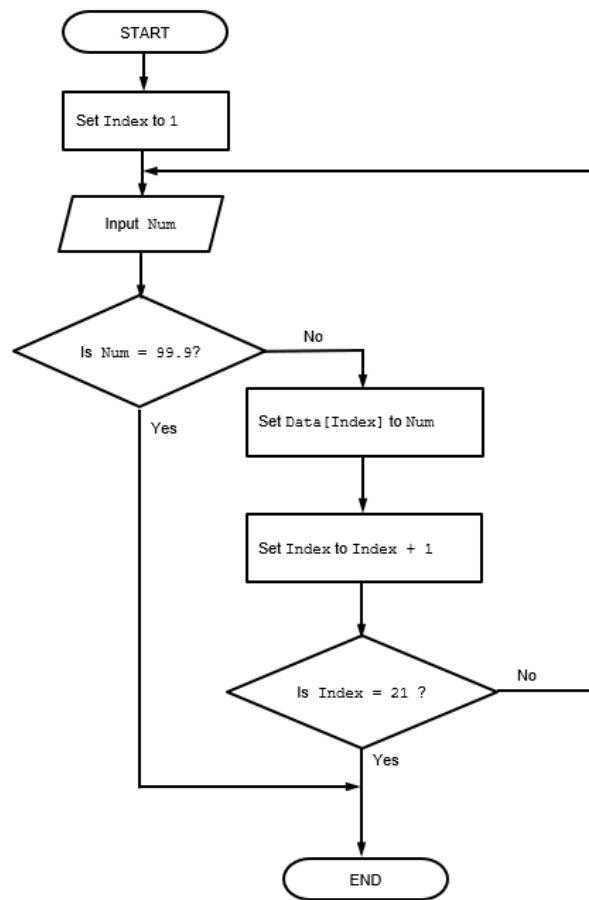
Pseudocode example	Selection	Iteration	Subroutine
IF Status = FALSE THEN FOR Count ← 1 TO 20 CALL Reset(Count) NEXT Count ENDIF	✓	✓	✓
OTHERWISE : Status ← TRUE	✓		
WHILE AllDone() = TRUE		✓	✓
30 : NextChar ← 'X'	✓		

One mark per row

4

9. O/N 25/P22/Q2

2



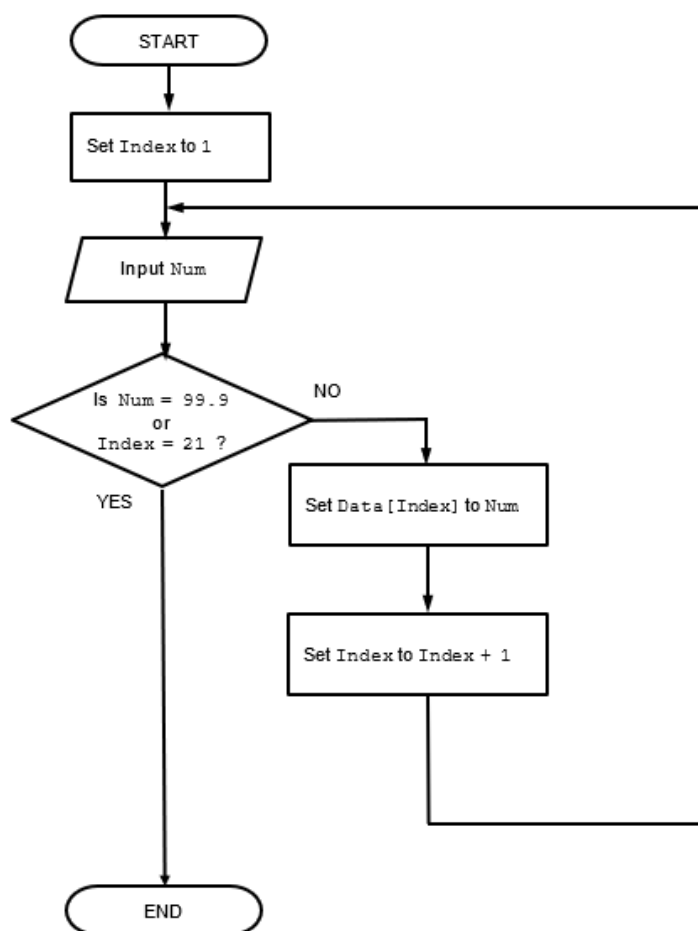
5

Mark as follows:

MP1 Initialisation of the array index**MP2** Input number **in a loop****MP3** Test for termination value (99.9) **and** correct ending**MP4** Test for array full / maximum 20 iterations **and** correct ending**MP5** Assign number to Data array element **and** increment index

2

Alternative solution:



10. O/N 25/P22/Q3/a

3(a)

Mark as follows:

6

MP1	1		open the file OldFile.txt in read (mode)
MP2	2		open the file NewFile.txt in write (mode)
	3		read a line from OldFile.txt and store in LineX
MP3	4		read a line from OldFile.txt and store in LineY
	5		read a line from OldFile.txt and store in LineZ
MP4	6		set NewString to LineX & '\ ' & LineY & '\ ' & LineZ
MP5	7		write NewString to NewFile.txt
MP6	8		repeat from step 3 until end of OldFile.txt
	9		close both files

11. O/N 25/P22/Q4/c

4(c)	MP1 Check that the parameter value is in the range 0 to 35 MP2 ... and if not, return FALSE MP3 Use the parameter value as the index to array <code>CheckTotal</code> MP4 Return the value from given index position	4
------	---	----------

12. O/N 25/P23/Q2/a,b

2(a)	One mark per point: 1 Open the file in append mode (and subsequently close) before loop 2 Initialise a variable to 1 / 0 before loop 3 Use the variable as an index to the array to access an element in a loop 4 Test if element value is blank / empty in a loop 5 If not blank then write the value to the file in a loop 6 Increment the variable in a loop 7 Repeat from MP3 until variable value is 66 / 65 / all elements (in the array) have been checked Max 6	6
2(b)	One mark per point: Construct: Selection Use: To test whether the (array) element is blank Alternative: Construct: Sequence Use: To specify the order in which the steps of the algorithm have to be followed so that the task is completed correctly	2

13. M/J 25/P21/Q1/a,b,c

1(a)

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop	Category	STRING
20/02/2025	when an item was sold	DateSold	DATE
12.67	the cost of an item	(Item) Cost/Price	REAL
TRUE	to indicate if an item is in stock	(Item) InStock	BOOLEAN

Mark as follows:
1 mark per row for each variable name **and** data type

4

1(b)	<table><tr><th>Pseudocode extract</th><th>Assignment</th><th>Selection</th><th>Iteration</th></tr><tr><td>Result ← CalculateTotal()</td><td>✓</td><td></td><td></td></tr><tr><td>WHILE IsClosed</td><td></td><td></td><td>✓</td></tr><tr><td>REPEAT INPUT Value UNTIL Sales[4] > Value</td><td>✓</td><td></td><td>✓</td></tr><tr><td>IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF</td><td>✓</td><td>✓</td><td></td></tr><tr><td>CASE OF Option</td><td></td><td>✓</td><td></td></tr></table>	Pseudocode extract	Assignment	Selection	Iteration	Result ← CalculateTotal()	✓			WHILE IsClosed			✓	REPEAT INPUT Value UNTIL Sales[4] > Value	✓		✓	IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF	✓	✓		CASE OF Option		✓		5
	Pseudocode extract	Assignment	Selection	Iteration																						
	Result ← CalculateTotal()	✓																								
	WHILE IsClosed			✓																						
	REPEAT INPUT Value UNTIL Sales[4] > Value	✓		✓																						
	IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF	✓	✓																							
	CASE OF Option		✓																							
Mark as follows: One mark for each correct row																										
1(c)	Decomposition involves: MP1 Breaking down a problem into sub problems MP2 In order to explain / understand the process/task of how a (stock control) can be managed in more detail MP3 Leading to the concept of the program designed as a series of modules / procedures / functions // or by example of different stock control modules MP4 Makes the program easier to test / debug Max 3 Max 2 if no mention of stock control	3																								

3	Example solution: <pre> DECLARE GeneratedValue : INTEGER DECLARE Guess : INTEGER GeneratedValue ← INT(RAND(100)) + 1 REPEAT OUTPUT "Enter a guess between 1 and 100: " INPUT Guess IF Guess > GeneratedValue THEN OUTPUT "Guess was too high" ENDIF IF Guess < GeneratedValue THEN OUTPUT "Guess was too low" ENDIF UNTIL Guess = GeneratedValue OUTPUT "Number guessed correctly" MP1 Declare all variables used MP2 Uses RAND() function MP3 Uses INT() function MP4 Correct syntax for random number between 1 and 100 MP5 Conditional loop until correct number guessed MP6 Prompt and input a guess in a loop MP7 Check if guess is too high in a loop MP8 Check if guess is too low in a loop MP9 Output appropriate message (x2) when the number is not guessed correctly (in the loop) and output correct guess message Max 8 </pre>	8
---	---	---

15. M/J 25/P22/Q2/a

2(a)	Mark as follows: 1. To increase the level of detail of the algorithm // To break the problem / task into smaller steps ... 2. ... until steps can be directly translated into lines of code // from which it can be programmed	2
------	---	---

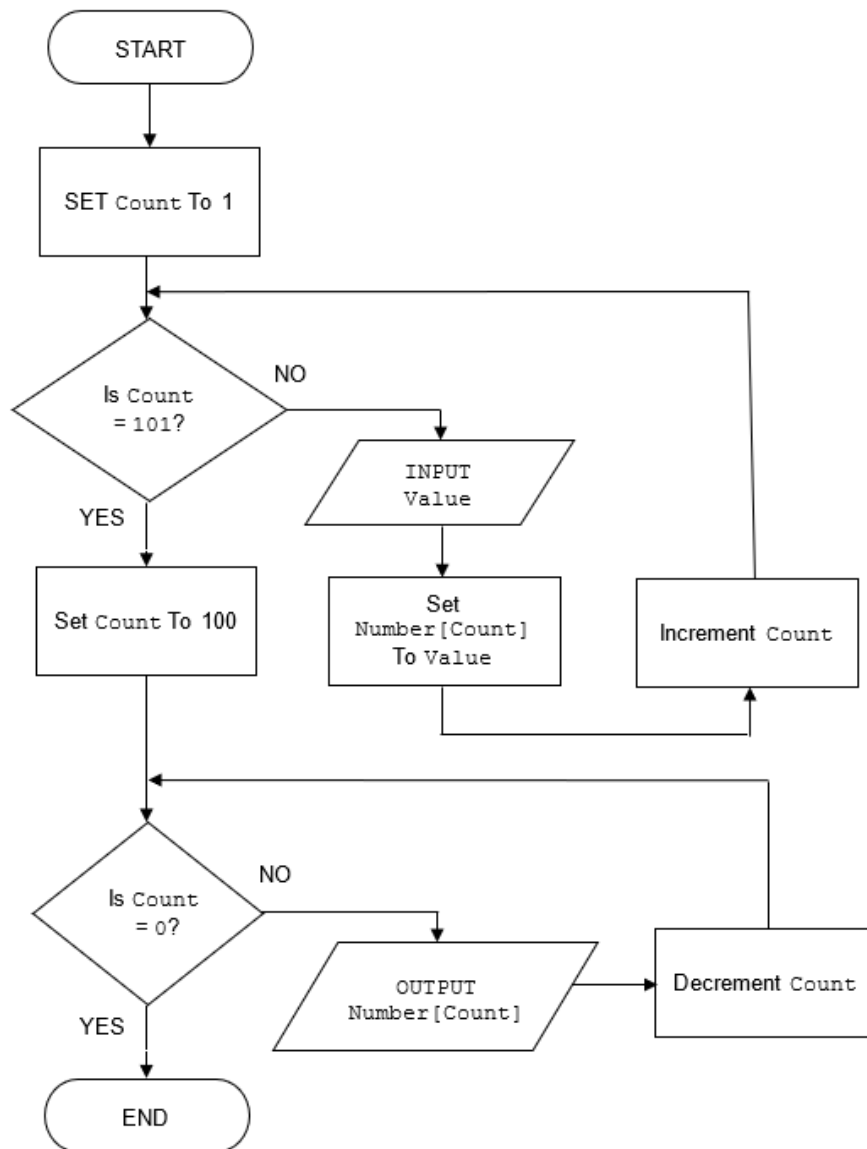
16. M/J 25/P22/Q3

3	Example solution <pre> DECLARE FirstNumber : INTEGER DECLARE SecondNumber : INTEGER DECLARE ThirdNumber : INTEGER FirstNumber ← INT(RAND(21)) - 10 OUTPUT FirstNumber REPEAT SecondNumber ← INT(RAND(21)) - 10 UNTIL FirstNumber <> SecondNumber OUTPUT SecondNumber IF FirstNumber < 0 AND SecondNumber < 0 THEN ThirdNumber ← INT(RAND(6)) + 30 OUTPUT ThirdNumber ENDIF </pre> Mark points: 1. Declare all variables used 2. Use <code>RAND()</code> function with any integer parameter 3. Use <code>INT()</code> function with any numeric parameter 4. Conditional loop until two different random numbers are generated 5. Use <code>INT()</code> function using random number generated between –10 and 10 inclusive in a loop 6. Output two different random integers / numbers 7. Check if both random integers / numbers are negative 8. then output a (third) random integer / number between 30 and 35 inclusive	8
---	--	---

17. M/J 25/P22/Q4

4

6



One mark for each functional group as listed:

1. Initialise Index used for `Number` to either 1 or 0 before first loop
2. Loop 100 times
3. Input value
4. Increment array index **and** store value in `Number` array at element referenced by array index **in a loop**
5. Output loop starts at last element in `Number` array
6. Output `Number` array element referenced by Index **in a loop**
7. Output all elements of `Number` array **once** in reverse order

Max 6

18. M/J 25/P23/Q3

3	Example solution: <pre> FUNCTION RollDice (NoOfTimes : INTEGER) RETURNS REAL DECLARE RollValue : INTEGER DECLARE Average : REAL DECLARE Count : INTEGER DECLARE Sum : INTEGER Sum ← 0 FOR Count ← 1 TO NoOfTimes RollValue ← INT(RAND(6)) + 1 OUTPUT RollValue Sum ← Sum + RollValue NEXT Count Average ← Sum / NoOfTimes RETURN Average ENDFUNCTION </pre> MP1 Declare all variables used and initialise Sum to 0 MP2 Loop for 'number of times' parameter MP3 Use RAND() function with Integer parameter in a loop MP4 Use INT() function in a loop MP5 Generate a random integer between 1 and 6 in a loop MP6 Output each value generated in a loop MP7 Calculate Sum and if Average used check declared as REAL and return Average and check function header returns REAL	7
---	---	---

19. M/J 25/P23/Q4/a

4(a)	MP1 Open the file (Sales.txt) in read mode and subsequently close MP2 Read a line (from the text file) MP3 Convert the line read (string) to a number MP4 If the number is greater than 500 and then output week number MP5 Increment week number (must have been Initialised ...) MP6 Repeat from step 2 for 52 times // Repeat from step 2 until end of file Max 5	5
------	---	---

20. O/N 24/P21/Q3/a,b

3(a)	One mark per emboldened part: 1 Open the file (Result.txt) in <u>read</u> mode 2 Loop until EOF(Result.txt) // EOF // end of file 3 Read a / the / next line (from the file) and store in ThisLine 4 Assign LineY to LineX 5 Assign LineZ to LineY 6 Assign ThisLine to LineZ 7 After the loop, close the file (Result.txt) 8 Output LineX, LineY, LineZ	4
3(b)	Example answer: So that the last three lines of the file are output in the correct order A mark for mentioning one of: • (Ensuring) the lines are output in correct order • Ordering the last <u>three</u> lines • Ordering the lines so they are in the same order as (they occur) <u>in the file</u> Max 1 marks	1

21. O/N 24/P22/Q7/a,b

7(a)	• Customer ID – to reference the other customer details • Email address – to send the email • Name – to personalise the email • Date of last visit – to select which customers should receive an email • Unique voucher code (or method of code generation) – to include in the email Mark as follows: One mark per item and justification Max 3 marks	3
7(b)	Abstraction	1

22. O/N 24/P23/Q1/a

1(a)	Pseudocode example	Selection	Iteration	Subroutine	4
	FOR Index ← 1 TO 3 IF Safe[Index] = TRUE THEN Flap[Index] ← 0 ENDIF NEXT Index	✓	✓		
	CASE OF Compound(3)	✓		✓	
	REPEAT UNTIL AllDone() = TRUE		✓	✓	
	WHILE Result[3] <> FALSE		✓		
One mark per row					

23. O/N 24/P23/Q2/a,b

2(a)	DECLARE Count, Total, NextNumber : INTEGER Total ← 0 FOR Count ← 1 TO 100 OUTPUT "Input an integer value" INPUT NextNumber IF NextNumber > 0 THEN Total ← Total + NextNumber ENDIF NEXT Count OUTPUT Total Mark as follows: MP1 Declarations of all variables used MP2 Loop for 100 iterations MP3 Prompt and input a value in a loop and MP4 Test for value > 0 // >=1 in a loop MP5 Sum the Total in a loop MP6 Output of Total after the loop Max 5 marks	5
2(b)	MP1 Construct: Iteration / Repetition MP2 Use: To loop through all <u>100</u> inputs // To loop <u>100</u> times ALTERNATIVE: MP1 Construct: Selection MP2 Use: To test whether the value input is positive	2

24. O/N 24/P23/Q3/b,c

3(b)	<pre>graph TD START([START]) --> SET1[SET Index TO 1] SET1 --> IS200{IS Index = 200 ?} IS200 -- NO --> SETPTR[SET List[Index].Pointer TO Index + 1] SETPTR --> SETINC[SET Index TO Index + 1] SETINC --> IS200 IS200 -- YES --> SET0[SET List[Index].Pointer TO 0] SET0 --> END([END])</pre> Mark as follows: MP1 Use of variable as array index and initialisation to 1 / to first element of array MP2 Loop for 199 / 200 iterations MP3 Assign Index+1 to each pointer field in a loop MP4 Assign 0 to 200th pointer field	4
------	---	----------

3(c)	One mark per step: MP1 Assign <code>HeadPointer</code> to <code>ThisPointer</code> / Current Pointer // Identify first node using <u>headpointer</u> MP2 Loop the following until <code>ThisPointer</code> value is 0 (zero) / null <u>pointer</u> reached MP3 ... Output data (field) from array element at index <code>ThisPointer</code> // Output data (field) of the current node / array element MP4 ... Assign pointer field from current array element / index / to <code>ThisPointer</code> / Current Pointer // Assign pointer field from current node to <code>ThisPointer</code> / Current Pointer	4
------	---	----------

25. O/N 24/P23/Q7/b

7(b)	One mark for name and use Mark as follows: Examples include: Sub-module: <code>ScanCard()</code> / <code>GetID()</code> Use: Read the barcode from the loyalty card / Get the customer ID from the barcode Sub-module: <code>GetPoints()</code> Use: Get the number of points the customer has Sub-module: <code>ExchangePoints()</code> / <code>UpdateCard()</code> Use: Reduce the number of loyalty points Sub-module: <code>GetDiscount()</code> / <code>CalculateDiscount()</code> Use: Calculate the discount Sub-module: <code>CalculatePoints()</code> Use: Calculate points (following a purchase) Sub-module: <code>UpdatePoints()</code> Use: Update total points a customer has (following a purchase) Sub-module: <code>ShowDiscount()</code> Use: Display the discount Max 4 marks	4
------	--	----------

26. M/J 24/P21/Q1/c

1(c)(i)	The names do not reflect / indicate the purpose of the variable // the names are not meaningful	1
1(c)(ii)	They make the program more difficult to understand / debug / maintain	1

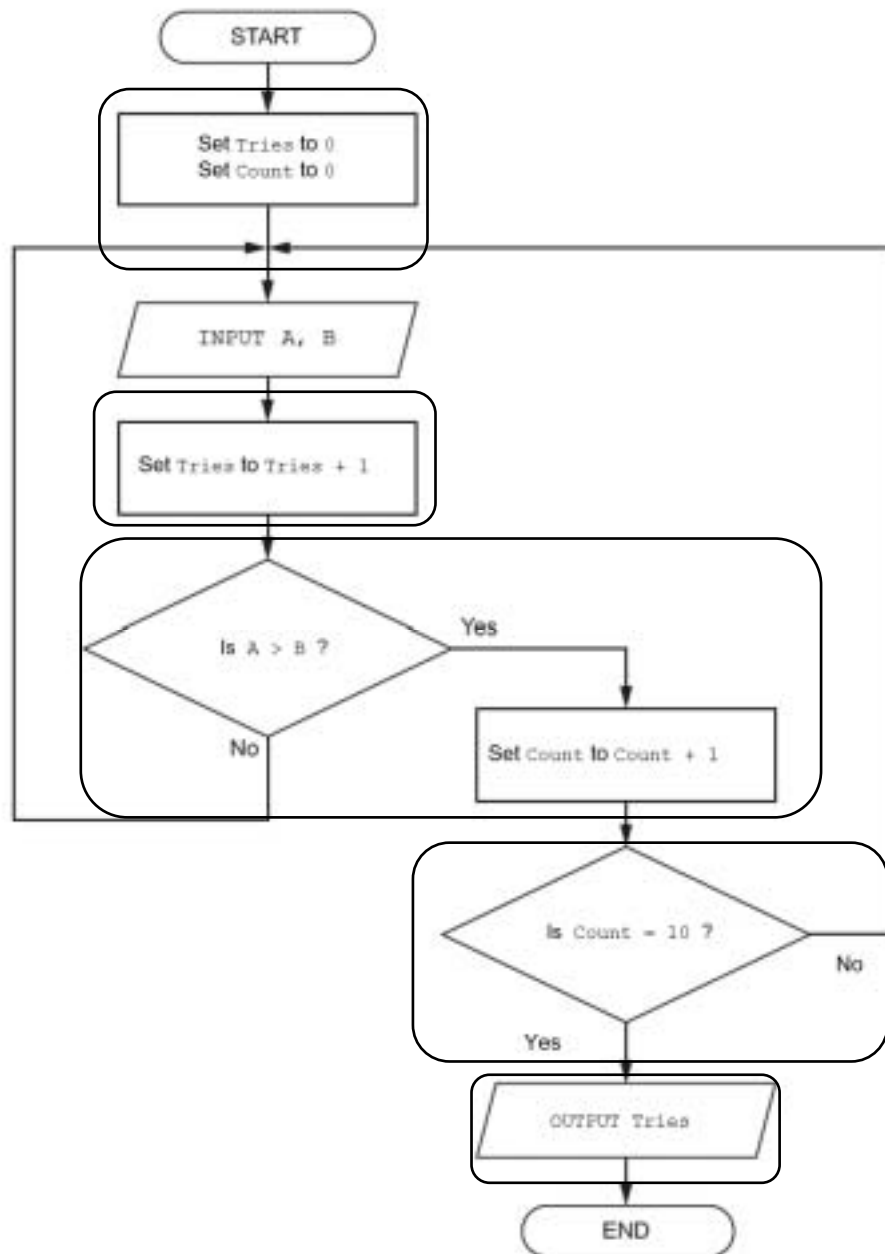
1(c)(iii)	One mark from: • Indentation / use of white space • Capitalisation of keywords • Use of comments • Use of modular programming • Use of local variables Note: max 1 mark	1
-----------	--	----------

1(a)	Assignment statement	Data type	4
	A ← LEFT(MyName, 1)	CHAR / STRING	
	B ← Total * 2	INTEGER / REAL	
	C ← INT(ItemCost) / 3	REAL	
	D ← "Odd OR Even"	STRING	
	One mark per row		

27. M/J 24/P21/Q2/a

2(a)

5



One mark per outlined region:

- 1 Initialise both counts
- 2 Increment `Tries` every time a pair is input
- 3 Compare `A > B` **and** increment `Count` if TRUE
- 4 Test for `Count = 10` (10th time `A > B`) – MUST include Yes / No labels
- 5 If so output `Tries`, otherwise loop

28. M/J 24/P21/Q7/a

7(a)(i)	To make the solution easier to design / implement / solve	1
---------	---	---

7(a)(ii)	One mark per item and justification Item: mobile phone number Justification: to send the text message Item: name Justification: to personalise the text message Item: exercise interest Justification: to determine whether this member would be interested	3
7(a)(iii)	Examples include: • Add a member to a list of those interested in the new class • Remove the member from future SMS messages • Read/process Message • Identify who from One mark for each. Max 2 marks	2

29. M/J 24/P22/Q1/a,c

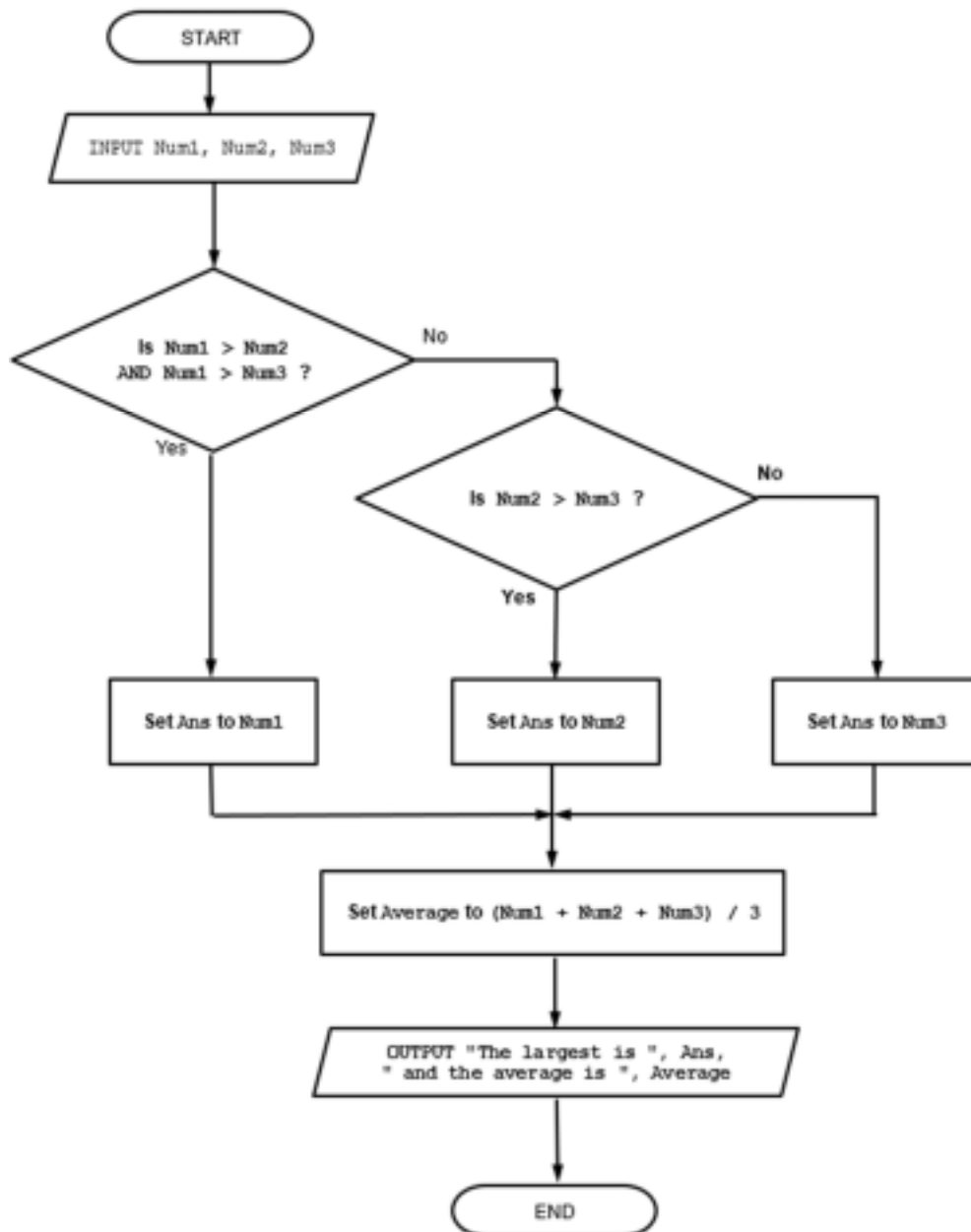
1(a)	<table><tr><th>Pseudocode example</th><th>Selection</th><th>Iteration</th><th>Input/Output</th></tr><tr><td>FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index</td><td></td><td>✓</td><td></td></tr><tr><td>WRITEFILE ThisFile, "****"</td><td></td><td></td><td>✓</td></tr><tr><td>UNTIL Level > 25</td><td></td><td>✓</td><td></td></tr><tr><td>IF Mark > 74 THEN READFILE OldFile, Data ENDIF</td><td>✓</td><td></td><td>✓</td></tr></table> One mark per row.	Pseudocode example	Selection	Iteration	Input/Output	FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index		✓		WRITEFILE ThisFile, "****"			✓	UNTIL Level > 25		✓		IF Mark > 74 THEN READFILE OldFile, Data ENDIF	✓		✓	4
Pseudocode example	Selection	Iteration	Input/Output																			
FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index		✓																				
WRITEFILE ThisFile, "****"			✓																			
UNTIL Level > 25		✓																				
IF Mark > 74 THEN READFILE OldFile, Data ENDIF	✓		✓																			
1(c)	1 mark for stating a suitable way of documenting: • <u>Identifier table</u> 1 mark for giving one piece of information that should be recorded. examples include: Explanation of what (each) variable is used for The purpose of (each) variable An example of data values stored // Initialisation value	2																				

1(b)		

30. M/J 24/P22/Q2/a,b

2(a)

5



Mark points

- 1 Condition for selecting one of the input numbers as largest value
- 2 ... and assign to Ans
- 3 Condition for selecting largest number for all three number input and assigning to Ans
- 4 Average calculated using Num1, Num2 and Num3 and stored in a variable. Reject use of DIV
- 5 Output of Ans and average in output symbol / parallelogram

2(b)	Example solutions: <pre> Flag ← GetStat() WHILE Flag <> TRUE FOR Port ← 1 TO 3 CALL Reset(Port) NEXT Port Flag ← GetStat() ENDWHILE </pre> Alternative: <pre> REPEAT Flag ← GetStat() IF Flag <> TRUE THEN FOR Port ← 1 TO 3 CALL Reset(Port) NEXT Port ENDIF UNTIL Flag = TRUE </pre> One mark per point: 1 (Outer) conditional loop testing <code>Flag</code> 2 Correct assignment of <code>Flag</code> from <code>GetStat()</code> in a loop 3 (Inner) loop checking / counting port // Check if <code>Port</code> is different to 4 4 ... loop for 3 iterations 5 a call to <code>Reset()</code> in a loop	5
------	--	----------

31. M/J 24/P22/Q7/a

7(a)	Examples include: Module: <code>IdentifyMember()</code> Use: Identifies a club member who has expressed an interest in a given class Module: <code>GetMemberPhoneNumber()</code> Use: Gets the mobile phone number of a member Module: <code>CreateMessage()</code> Use: Generates a text message to a member Module: <code>SendMessage()</code> Use: Sends a text message to a member in the waiting list One mark for name and use Note: max 3 marks	3
------	--	----------

32. M/J 24/P22/Q8/a

8(a)	One mark for reason, one for benefit Reason: (Program is) easier to design / implement / test / debug / modify Benefit: Easier to check that each stage works as expected	2
------	--	----------

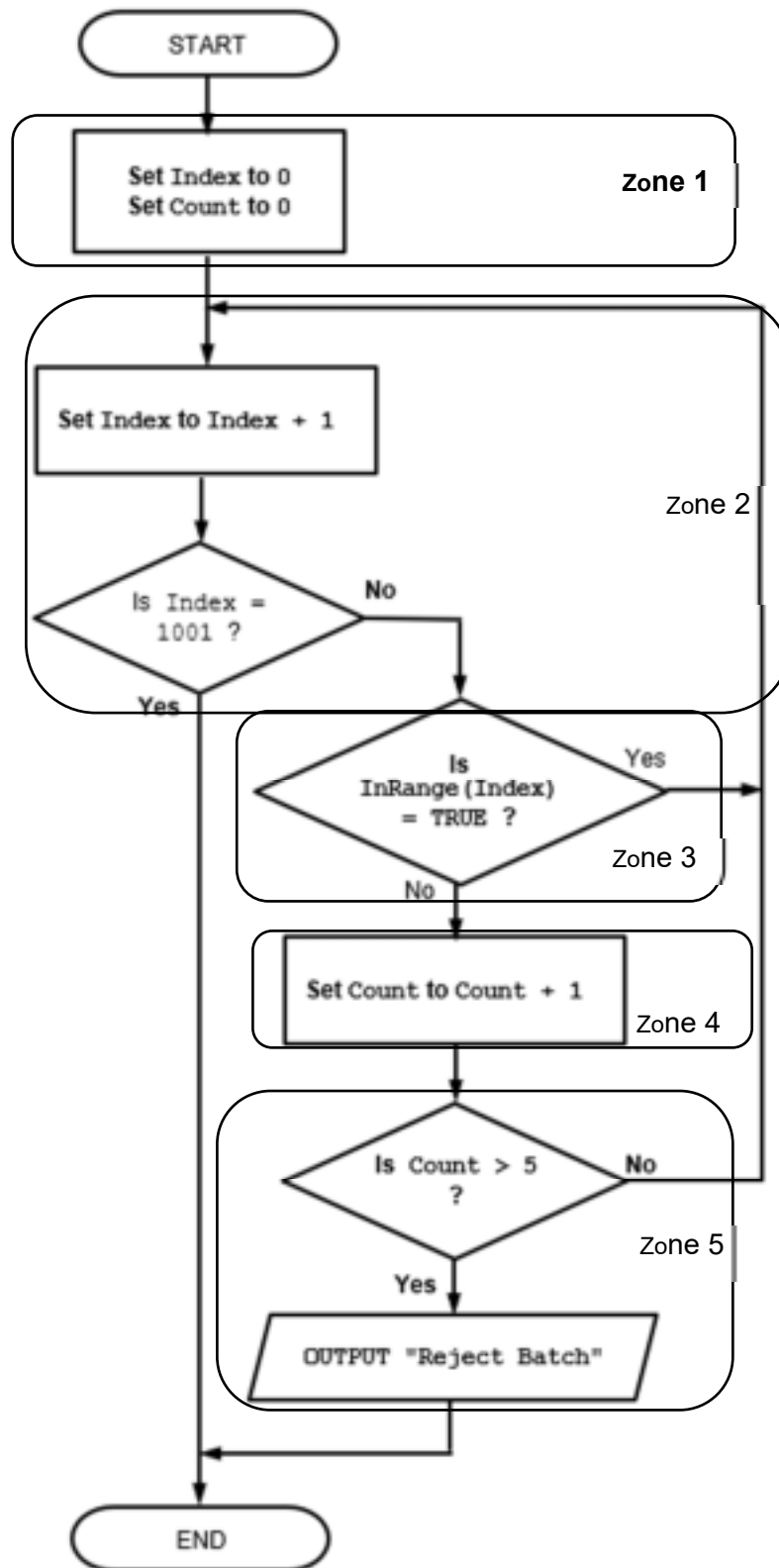
33. M/J 24/P23/Q2/a,b

2(a)	One mark per point: 1. Open the file (in read mode and subsequently close) 2. Initialise an index variable (to 1 // 0) 3. Repeat (the next three steps) until the end of file is reached 4. Read a line from the file (into a string variable) 5. Store the line / variable in the array at the index 6. Increment the index in a loop Note: max 5 marks	5
2(b)	One mark per point: Construct: a <u>conditional</u> loop Use: To keep repeating until the end of the file is reached ALTERNATIVE: Construct: a <u>selection</u> statement Use: To test / check the value returned by the EOF () function	2

34. M/J 24/P23/Q3/b

3(b)

5



One mark per zone

35. O/N 23/P21/Q2/a,b

2(a)	Max 5 marks MP1 Set total to <u>zero</u> MP2 Input a number MP3 Check if number greater than 29 and less than 71 MP4 ... if check is true - add number to total MP5 Repeat from step 2 99 times // for a total of 100 iterations MP6 Output the total	5
2(b)	MP1 An iterative construct // a (count-controlled) loop MP2 A selection construct // an IF statement	2

36. O/N 23/P22/Q1/a

1(a)

4

One mark for each row with appropriate variable name and data type

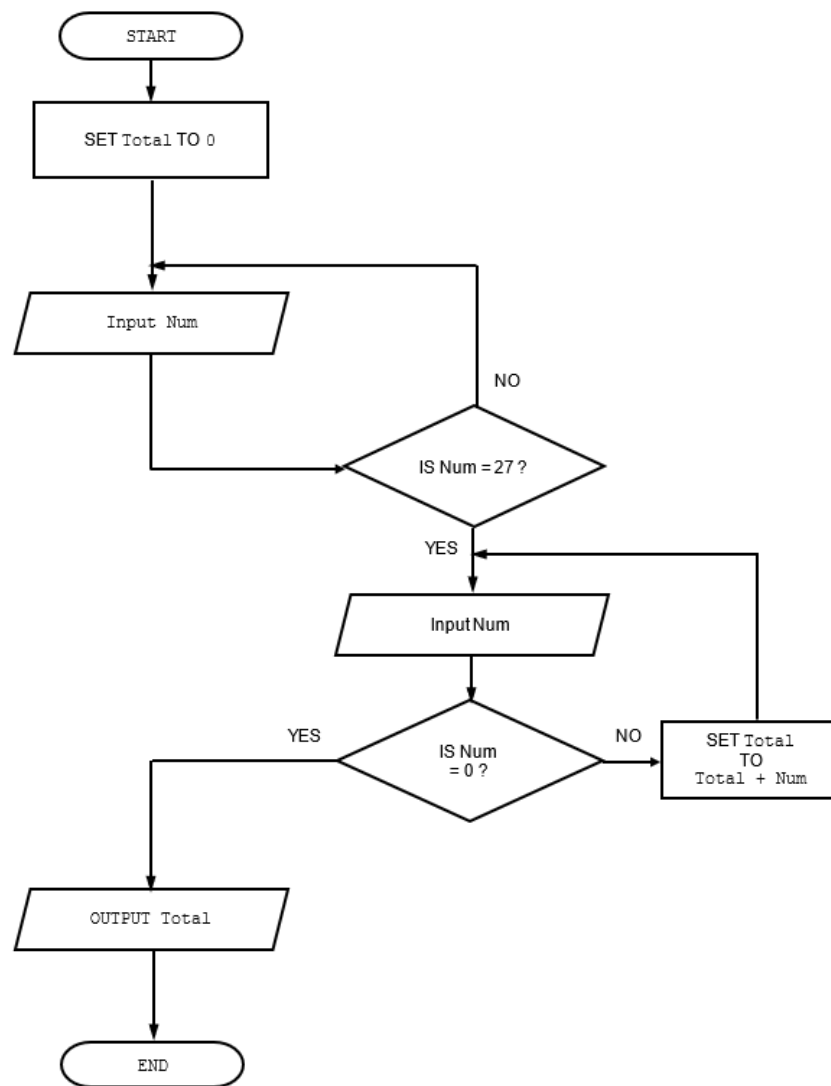
Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer	CustomerName	STRING
3	The number of items in the order	NumItems	INTEGER
TRUE	A value to indicate whether this is a new customer	NewCustomer	BOOLEAN
15.75	The deposit paid by the customer	Deposit	REAL

37. O/N 23/P22/Q2/a

2(a)

Example Solution

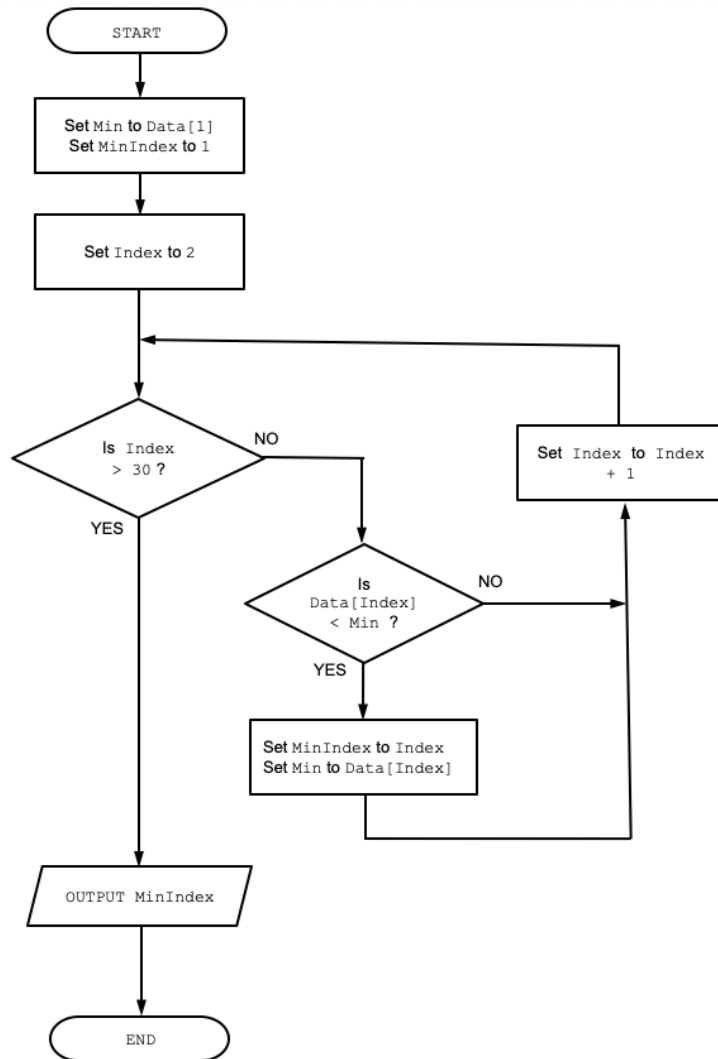
5

**One mark per point:**

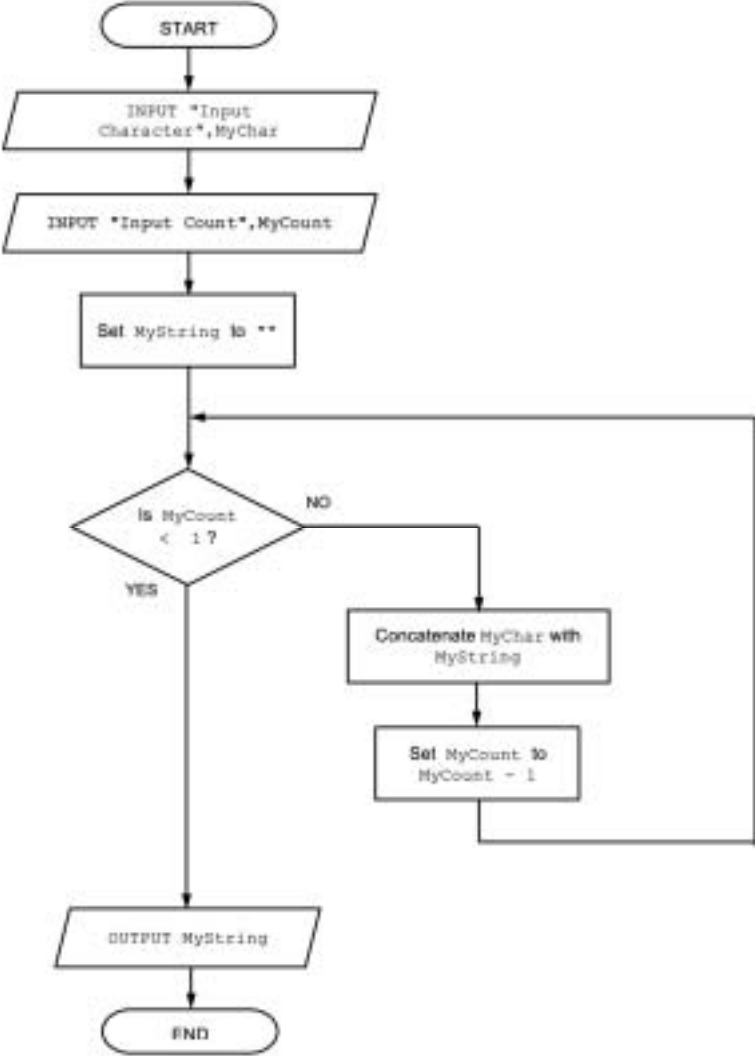
- 1 Initialise `Total` to zero
- 2 Check for only **first** input of 27 **in a loop** then attempt to sum values
- 3 Loop until 0 input
- 4 Sum values **input** (after input of first 27) **in a loop**
- 5 Output `Total` after a reasonable attempt

38. O/N 23/P23/Q2/a

2(a)

5**MP1** Initialise Min to first value in Data **and** MinIndex to 1**MP2** Loop through 29 more values (or 30 values in total)**MP3** Compare element from Data[] with Min**MP4** Set new Min **AND** save MinIndex when element value < Min **in a loop****MP5** Output MinIndex

39. M/J 23/P21/Q2/a

2(a)	 <pre> graph TD Start([START]) --> InputChar[/INPUT "Input Character", MyChar/] InputChar --> InputCount[/INPUT "Input Count", MyCount/] InputCount --> SetString[Set MyString to ""] SetString --> Decision{Is MyCount < 1?} Decision -- YES --> Output[/OUTPUT MyString/] Output --> End([END]) Decision -- NO --> Concat[Concatenate MyChar with MyString] Concat --> DecCount[Set MyCount to MyCount - 1] DecCount --> Decision </pre> One mark per point: - Both prompts - Both inputs using correct identifiers as given in question, <code>MyChar</code> and <code>MyCount</code> - Initialise <code>MyString</code> to empty string and subsequent output after loop - Loop <code>MyCount</code> iterations including decrement <code>MyCount</code> - Use of Concatenate or equivalent pseudocode statement inside loop Max 4 Marks	4
------	--	---

40. M/J 23/P21/Q3/b

3(b)	One mark for reference to each: 1 Reference to the use of constants or variables for the two threshold values of 10 and 100 // Input amount spent (by customer and store in a numeric variable) 2 Work out one band that amount maps to 3 Work out all bands that amount maps to 4 Calculate rounded value of amount / whole number part of amount 5 Calculate the points by multiplying the (rounded) amount by the appropriate value for appropriate band /all bands 6 Output the number of points Note: Max 5 from available points	5
------	--	----------

41. M/J 23/P21/Q7/a

7(a)	Examples include: Module: <code>GetOverdueLoan()</code> Use: Identifies an overdue book Module: <code>IdentifyStudent()</code> Use: Identifies a student (with an overdue book)¹ Module: <code>GetStudentEmail()</code> Use: Gets the email address of a student with an overdue book Module: <code>CreateEmail()</code> Use: Generates an email to a student with an overdue book Module: <code>SendEmail()</code> Use: Sends an email to a student with an overdue book One mark for name and use Note: Max 3 marks	3
------	--	----------

42. M/J 23/P22/Q1/b

1(b)	One mark per bullet point: • Indentation • White space • Comments • Sensible / meaningful variable names // use of Camel Case • Capitalised keywords Note: Max 3 marks	3
------	---	----------

43. M/J 23/P22/Q2/c

2(c)	One mark per array definition bullet: • A (1D) array containing 7 elements • of type STRING One mark per Step: Step1: Assign value "Sunday" to first element, "Monday" to second element etc. Step2: Use the function <code>DAYINDEX()</code> to return / find the day number from <code>MyDoB</code> Step3: Use the returned value as the array index / to access the element that contains the name / string Step4: Output the element / name / string Note: Max 2 for Array definition, Max 4 for steps	6
------	---	----------

44. M/J 23/P22/Q3/b

3(b)	One mark per bullet: 1 Open file in <code>READ</code> mode 2 Loop to <code>EOF()</code> // read / process all the lines in file 3 Loop will end when return value from <code>AddToQueue()</code> is <code>FALSE</code> / queue is full 4 Read a line from the file in a loop 5 Pass string to <code>AddToQueue()</code> // <code>AddToQueue()</code> is executed with line as parameter	5
------	--	----------

45. M/J 23/P22/Q7/a

7(a)(i)	Correct answers include: • Information: customer name Justification: to personalise / address the email • Information: email address Justification: so that the email can be delivered correctly • Information: product category preference Justification: to check that the customer would be interested in the product • Information: contact preference Justification: to check that the customer will accept contact via email • Information: order history Justification: to send details of product similar to that already bought // to identify frequent shoppers • Information: new product information Justification: to include information about the new product in the email One mark for each piece of information and matching justification Note: Max 3 marks	3
7(a)(ii)	One mark for each piece of information and matching justification: • postal address Justification: sending an email, not a letter • payment details Justification: Nothing being bought / sold at this stage • order history Justification: Customer preference used to decide if new product is relevant Note: Max 2 marks	2

46. M/J 23/P23/Q1/d

1(d)	One mark for each of rows 2 - 5				4
Pseudocode statement		Input	Process	Output	
INPUT MyChoice		✓			
OUTPUT FirstName & LastName			✓	✓	
WRITEFILE OutputFile, TextLine				✓	
READFILE MyFile, TextLine		✓			
Result ← SQRT (NextNum)			✓		

47. M/J 23/P23/Q3/b(ii)

3(b)(ii)	Max 5 marks MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 POP value MP4 Write value to the text file MP5 Repeat from Step 2 // loop referencing the stack items Alternative solution: Not using POP primitive MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 Read value from ToS location MP4 Write the value to the text file – Must some attempt at ‘the value’ NOT ‘<u>all</u> the values’ MP5 Decrement ToS MP6 Repeat from step 2 // loop referencing the stack items	5
----------	--	----------

48. M/J 23/P23/Q7/a

7(a)(i)	To filter out information (that is not necessary to solve the problem) // to include only essential information	1
7(a)(ii)	Required: Student : Student name / email (address) Loan: Return/Issue date Book: Book title Not Required: Student: Home address / DoB / tutor / subject choices Book: Library location / category / author / book title	2

7(a)(iii)	Max 2 marks Examples could include: • Clear the loan // indicate that the book has been returned // update loan history • Take the student off 'block' // allow the student to borrow further books • Send acknowledgement to the student when book is returned	2
-----------	--	----------

49. O/N 22/P21/Q2/a,b

2(a)	One mark for Explanation: • Abstraction is used to filter out information / data that is not necessary for the task Or the opposite: • To keep only information / data that is necessary for the task One mark for each TWO data items (not dependent on 'Explanation'): Items include: • Car details: ID, Car Registration, car type etc • Customer details: ID, name, address, licence details etc • Start date (of hire) • Return date / Number of days (of hire) • Cost of hire	3
2(b)	One mark for each (Max 2) Examples include: 1 Input customer details 2 Input car details 3 Input payment details 4 Create hire / start hire 5 Return car / end hire 6 Change / check car status (hired / available / written-off) 7 Cancel hire 8 Process payment / calculate hire cost	2

50. O/N 22/P21/Q3

3	One mark per point (Max 5): 1 Declare a variable / an integer <code>Max</code> 2 Assign value of first / any element to <code>Max</code> 3 Set up a loop to repeat 200 times / from start to end of array 4 Use the loop counter as the array index 5 If value of current element is greater than <code>Max</code>... 6 ...then assign value to <code>Max</code> 7 After the loop, Output <code>Max</code>	5
---	--	----------

51. O/N 22/P22/Q2/a

2(a)(i)	One mark per step (or equivalent): 1 Open file in <u>APPEND</u> mode (and subsequent Close) 2 Prompt and Input a student name and mark 3 If mark greater than or equal to 20 jump to step 5 4 Write only the name to the file 5 Repeat from Step 2 for 35 times / the number of students	5
2(a)(ii)	Data in a file is saved after the computer is switched off / stored permanently // no need to re-enter the data when the program is re-run	1
2(a)(iii)	Example answer: So that existing file data is not overwritten.	1

52. O/N 22/P22/Q4/a

4(a)	One mark per point: 1 Input UserID and use of GetAverage() and assignment 2 Initialisation of Total to zero and Index to 4 3 Conditional loop with Index from 4 to 6 4 Assignment of Last from element SameMonth[Index] in a loop 5 Structure: IF...THEN...ELSE...ENDIF in a loop 6 Correct assignments and final call to Update() after the loop <pre> INPUT UserID Average ← GetAverage(UserID) Total ← 0 Index ← 4 WHILE Index < 7 // REPEAT Last ← SameMonth[Index] IF Average > Last THEN Total ← Total + Average ELSE Total ← Total + Last ENDIF Index ← Index + 1 ENDWHILE // UNTIL Index = 7 CALL Update(UserID, Total) </pre> <u>Alternative solution using FOR loop:</u> One mark per point FOR loop solution: 1 Input UserID and use of GetAverage() and assignment 2 Initialisation of Total to zero 3 loop Index from 4 to 6 4 Assignment of Last from array SameMonth in a loop 5 Comparison IF...THEN...ELSE...ENDIF in a loop 6 Appropriate assignments in a loop AND final call to Update() after the loop <pre> INPUT UserID Average ← GetAverage(UserID) Total ← 0 FOR Index ← 4 TO 6 Last ← SameMonth[Index] IF Average > Last THEN Total ← Total + Average ELSE Total ← Total + Last ENDIF NEXT Index CALL Update(UserID, Total) </pre>	6
------	--	---

53. O/N 22/P22/Q5

5	One mark per IF...THEN...ENDIF clause: 1 IF A AND B AND C THEN CALL Sub1() ENDIF 2 IF (A AND B) AND NOT C THEN CALL Sub2() ENDIF 3 IF (NOT A) AND (NOT C) THEN CALL Sub3() ENDIF 4 IF (NOT A) AND C THEN CALL Sub4() ENDIF	4
---	---	---

54. O/N 22/P23/Q2

2	One mark for name and two marks for use (Max 3 in total): Examples include: Module: SelectCharity() Use: Allows the user to choose a particular charity Module: SpecifyAmountAndType() Use: Allows the user to specify a single or regular payment Module: MakePayment() Use: Make payment to the charity Module: ValidatePayment() Use: Validate payment details (by accessing bank computer) Module: AddBankAccountDetails() / AddPaymentDetails() Use: Allows the user to add bank account information that donation to be taken from Module: AddDonorDetails() Use: Allows user to add details such as name and contact details	3
---	---	---

55. O/N 22/P23/Q3

3	One mark per point, for example: 1 Declare two (REAL) variables for the two sum values AND initialise both to zero 2 Prompt AND Input a number 3 If number greater than zero add to positive sum and If number less than zero add to negative sum 4 Repeat from step 2 if number not zero 5 After loop the Output SumPos and SumNeg	5
---	--	---

56. O/N 22/P23/Q5/c

5(c)	One mark for reason One mark for each example (Max 2) Reason: • Makes the code easier to understand // Describes the purpose of the identifier // Makes the code easier to debug/test/maintain Further examples include: • White space • Indentation • Keywords in capitals • Comments • Local variables // parameters	3
------	--	---

57. O/N 22/P23/Q7/b(ii)

7(b)(ii)	One mark per point (Max 3): 1. Loop through 500 elements (while error number not found) 2. Compare ErrCode for current element with the error number 3. If same, set element value to 999 (and terminate loop) 4. ... and call SortArrays() (to move 999 to the end) – once only	3
----------	---	---

58. O/N 21/P21/Q1/c

1(c)(i)	<table border="1"> <thead> <tr> <th>Information</th><th>Essential</th><th>Not essential</th></tr> </thead> <tbody> <tr> <td>Departure time</td><td>✓</td><td></td></tr> <tr> <td>Flight Number</td><td></td><td>✓</td></tr> <tr> <td>Departure airport</td><td>✓</td><td></td></tr> <tr> <td>Aircraft type</td><td></td><td>✓</td></tr> <tr> <td>Ticket price</td><td>✓</td><td></td></tr> <tr> <td>Number of seats in aircraft</td><td></td><td>✓</td></tr> </tbody> </table> One mark for two rows correct Two mark for four rows correct Three mark for all rows correct	Information	Essential	Not essential	Departure time	✓		Flight Number		✓	Departure airport	✓		Aircraft type		✓	Ticket price	✓		Number of seats in aircraft		✓	3
Information	Essential	Not essential																					
Departure time	✓																						
Flight Number		✓																					
Departure airport	✓																						
Aircraft type		✓																					
Ticket price	✓																						
Number of seats in aircraft		✓																					

1(c)(ii)	One mark for technique and one for benefit, Max 1 mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	2
1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability Max 2 marks	2

59. O/N 21/P21/Q2/a,b

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm Max 6 marks	6
2(b)	<pre> Count ← 1 Flag ← FALSE WHILE Flag = FALSE AND Count <= 5 CALL ReBoot() Count ← Count + 1 Flag ← Check() ENDWHILE IF Flag = FALSE THEN CALL Alert(27) ENDIF </pre> One mark per point: 1 Initialisation of Count AND Flag 2 WHILE ... ENDWHILE // REPEAT ... UNTIL loop 3 ... including both conditions 4 Call ReBoot() AND increment Count inside the loop 5 Assign return value from Check() to Flag inside the loop 6 Final test of Flag AND call to Alert(27) not in a loop	6

60. O/N 21/P22/Q1/a

1(a)	The process involves: 1 Breaking down a problem / task into sub problems / steps / smaller parts 2 In order to explain / understand // easier to solve the problem 3 Leading to the concept of program modules // assigning problem parts to teams Max 2	2
------	---	----------

61. O/N 21/P22/Q2/a

2(a)	<pre>graph TD; START([START]) --> Init[Set PosCount, NegCount to 0]; Init --> Init2[Set PosTotal, NegTotal to 0]; Init2 --> Input[/INPUT NextNum/]; Input --> IsZero{Is NextNum = 0?}; IsZero -- YES --> Output1[/OUTPUT PosCount, NegCount/]; IsZero -- YES --> Output2[/OUTPUT PosTotal, NegTotal/]; Output2 --> END([END]); IsZero -- NO --> IsPos{Is NextNum > 0?}; IsPos -- YES --> IncPos[Increment PosCount]; IncPos --> SetPosTotal[Set PosTotal to PosTotal + NextNum]; IsPos -- NO --> IncNeg[Increment NegCount]; IncNeg --> SetNegTotal[Set NegTotal to NegTotal + NextNum]; SetPosTotal --> Input; SetNegTotal --> Input;</pre> One mark for each outlined group. Note: - Sum and increment steps (bottom-right rectangles) may be in reverse order in which case sum group will have two output lines - Max 4 for non-working solution	5
------	--	---

62. O/N 21/P22/Q4/b

4(b)	One mark per point: - Meaningful variable names - Indentation / white space / blank lines - Capitalisation of keywords	3
------	---	---

63. O/N 21/P23/Q1/c

1(c)(i)	<table><tr><th>Information</th><th>Essential</th><th>Not essential</th></tr><tr><td>Departure time</td><td>✓</td><td></td></tr><tr><td>Flight Number</td><td></td><td>✓</td></tr><tr><td>Departure airport</td><td>✓</td><td></td></tr><tr><td>Aircraft type</td><td></td><td>✓</td></tr><tr><td>Ticket price</td><td>✓</td><td></td></tr><tr><td>Number of seats in aircraft</td><td></td><td>✓</td></tr></table>	Information	Essential	Not essential	Departure time	✓		Flight Number		✓	Departure airport	✓		Aircraft type		✓	Ticket price	✓		Number of seats in aircraft		✓	3
	Information	Essential	Not essential																				
	Departure time	✓																					
	Flight Number		✓																				
	Departure airport	✓																					
	Aircraft type		✓																				
	Ticket price	✓																					
	Number of seats in aircraft		✓																				
One mark for two rows correct Two mark for four rows correct Three mark for all rows correct																							

1(c)(ii)	One mark for technique and one for benefit, Max 1 mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	2
----------	---	---

1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability Max 2 marks	2
-----------	--	---

64. O/N 21/P23/Q2/a,b

2(a)	One mark for reference to:	6
	1 The use a variable as an index to the array	
	2 A loop to iterate through the array	
	3 An Inner loop (with a reducing range)	
	4 Test if current element is greater than next element	
	5 if so then swap elements	
	6 Description of swap	
	7 Attempt at efficient algorithm	
	Max 6 marks	

2(b)	<pre> Count ← 1 Flag ← FALSE WHILE Flag = FALSE AND Count ≤ 5 CALL ReBoot() Count ← Count + 1 Flag ← Check() ENDWHILE IF Flag = FALSE THEN CALL Alert(27) ENDIF </pre> One mark per point: 1 Initialisation of Count AND Flag 2 WHILE ... ENDWHILE // REPEAT ... UNTIL loop 3 ... including both conditions 4 Call ReBoot() AND increment Count inside the loop 5 Assign return value from Check() to Flag inside the loop 6 Final test of Flag AND call to Alert(27) not in a loop	6
------	--	---

65. M/J 21/P21/Q1/a

1(a)

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club	MemberName	STRING
FALSE	A value to indicate whether an existing member of the club lives at the same address	FamilyMember	BOOLEAN
19/02/1983	When the member joined the football club	StartDate	DATE
1345	The number of points a member has earned. Members of the club earn points for different activities.	Points	INTEGER

One mark for each appropriate variable name plus data type

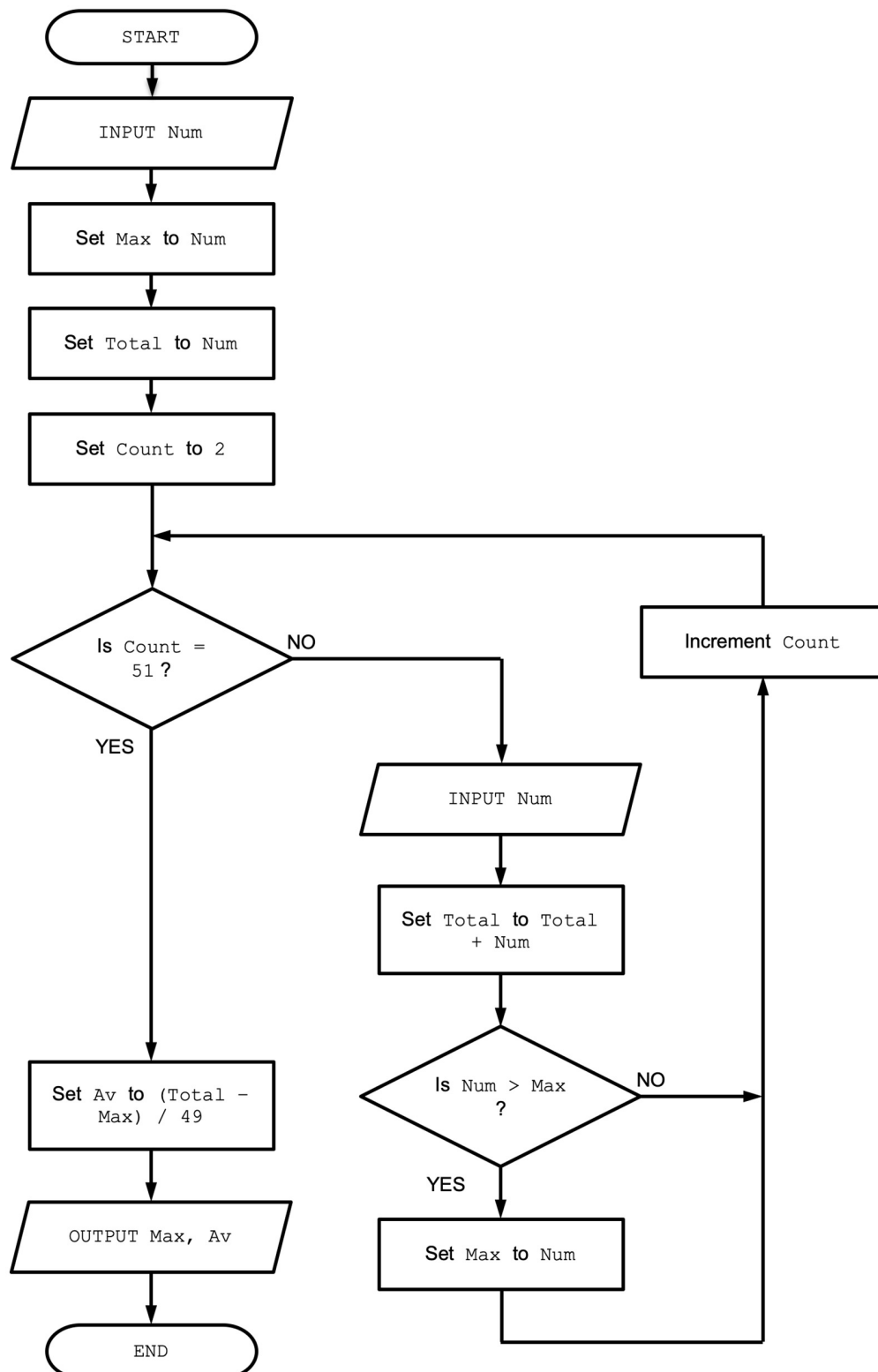
4

66. M/J 21/P21/Q2/c

2(c)

Example solution:

6



One mark for each functional group as listed below:

2(c)	Explanation of mark points: 1 Initialise MAX to first value input 2 Set Total to zero 3 Input 49 more values (or 50 values in total) 4 Sum all values input 5 Set new MAX when Input value > MAX in a loop 6 Sum all but largest (or subtract MAX from total), calculate and output average	
------	---	--

67. M/J 21/P21/Q7/b

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return FALSE / set flag 4 If word not already in array, loop to find unused element (<i>second loop</i>) 5 Store word in unused element and return TRUE, otherwise return FALSE VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return FALSE / set flag 5 If word not already in array and unused element available, store word in unused element and return TRUE otherwise return FALSE Note: Max 4 marks	4
------	--	----------

68. M/J 21/P22/Q1/b

1(b)					3
	Statement	Selection	Assignment	Iteration	
	Index ← Index + 1		✓		
	IF Modified = TRUE THEN	✓			
	ENDWHILE			✓	
One mark per row					

1(a)(i)	Variable	Example data value	Data type	4
	Name	"Catherine"	STRING	
	Index	100	INTEGER	
	Modified	FALSE	BOOLEAN	
	Holiday	25/12/2020	DATE	
One mark per data type				

69. M/J 21/P22/Q2/b

2(b)(i)	Answers include: • User ID / Username • Book ID • Date of loan / return date One mark for 1 correct Two marks for all 3 correct Note: Max 2 marks	2
2(b)(ii)	Many examples but must be data that is NOT required for a loan, but which COULD be required somewhere by the library system. Note: must be data relating to users, books or loans Answers include: • Users name / address / phone number / DOB • Book title / author / publisher / library rack number / ISBN number / price • Date of loan / return date (if not already given in part (i)) • The length of the loan (assumed to be the same for all books)	1
2(b)(iii)	Many examples including: • Create loan / borrow book • Return book • Send letter / email / contact a user ref an overdue book • View the loan history for a given book • View the loan history for a given user One mark for each Note: Max 2 marks	2

70. M/J 21/P22/Q4

4	Marks awarded for a description of each of the following steps of the algorithm: 1 Reference variables for <code>Count</code> of students and <code>Total</code> marks 2 Loop through all students (<code>Count</code>) 3 Input individual mark (in loop) 4 Compare mark with threshold / boundary values to determine grade (in loop) 5 Output the grade for a student (in loop) 6 Maintain a <code>Total</code> (and <code>Count</code> if required) (in loop) 7 Calculate average by dividing <code>Total</code> by <code>Count</code> and Output (after loop) Note: Max 6 marks	6
----------	--	----------

71. M/J 21/P23/Q1/a

1(a)

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club	MemberName	STRING
FALSE	A value to indicate whether an existing member of the club lives at the same address	FamilyMember	BOOLEAN
19/02/1983	When the member joined the football club	StartDate	DATE
1345	The number of points a member has earned. Members of the club earn points for different activities.	Points	INTEGER

One mark for each appropriate variable name plus data type

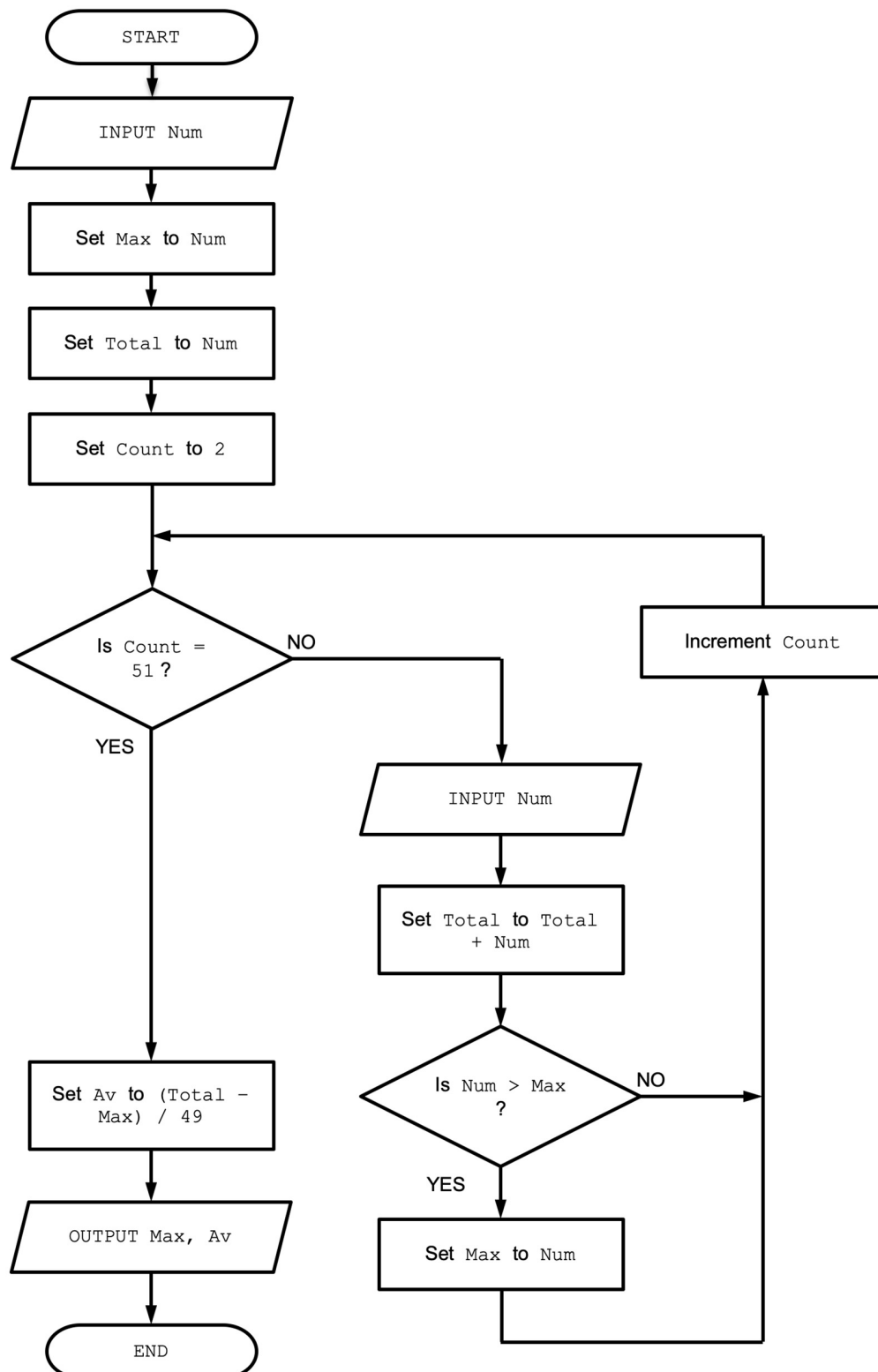
4

72. M/J 21/P23/Q2/c

2(c)

Example solution:

6



One mark for each functional group as listed below:

2(c)	Explanation of mark points: 1 Initialise MAX to first value input 2 Set Total to zero 3 Input 49 more values (or 50 values in total) 4 Sum all values input 5 Set new MAX when Input value > MAX in a loop 6 Sum all but largest (or subtract MAX from total), calculate and output average	
------	---	--

73. M/J 21/P23/Q7/b

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return FALSE / set flag 4 If word not already in array, loop to find unused element (<i>second loop</i>) 5 Store word in unused element and return TRUE, otherwise return FALSE VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return FALSE / set flag 5 If word not already in array and unused element available, store word in unused element and return TRUE otherwise return FALSE Note: Max 4 marks	4
------	--	----------

Unit 10

DATA TYPES AND STRUCTURES

112 items · 876 marks

Syllabus sections in this unit

- 10.1 Data Types and Records
- 10.2 Arrays
- 10.3 Files
- 10.4 Introduction to Abstract Data Types (ADT)

Unit 10: Data Types and Structures

1. M/J 26/P21/Q2/a(i)

2 A washing machine contains:

- two trays for the washing powder
- a user interface display
- five sensors.

The washing machine control program includes the following variables:

variable	description
MainsOn	TRUE when the mains power is on
WaterOn	TRUE when the water supply to the machine is turned on
DisplayOn	TRUE when the user interface display is active
Tray1	TRUE when it detects washing powder in tray 1
Tray2	TRUE when it detects washing powder in tray 2

(a) (i) Identify the data type used for the **five** variables.

[1]

2. M/J 26/P21/Q6/c

- 6** Fence parts are manufactured by a number of similar machines. The length of the part must fall within a certain range. Parts **not** within the length range are rejected.

The data for each rejected part forms a data packet made up of three data items, each separated by the '/' character delimiter. The format of a data packet is:

<length>/<machine ID>/<description of the fault>

- The length is 3 digits (in millimetres).
- The machine ID is a single character ('A', 'B', or 'C').
- The description of the fault is a variable length string.

A pseudocode program will process the data for several data packets.

The program contains the statement:

DataPacket ← "563/B/the fence length is too long"

- (a)** Complete the **two** pseudocode expressions which extract the appropriate data item.

PartLength ←

MachineID ←

[2]

- (b)** The program will create a module `RejectDescription()` to return the fault description.

Write the pseudocode for the module `RejectDescription()`

[4]

- (c)** The data packets for all rejected parts are saved in this format in a text file `Rejects.txt`. The file stores one data packet per line.

Write pseudocode to:

- find all the rejected parts with a length greater than 560 millimetres and that were produced on machine 'B' or machine 'C'
- output the description for each matching rejected part.

Your pseudocode will include a module call to `RejectDescription()` and the answers given for **6(a)**.

[6]

3. M/J 26/P21/Q7/a,b,c

- 7** Different data structures will be suitable for different computing applications.

- (a) (i)** Identify a computing application where a queue data structure is suitable.

[1]

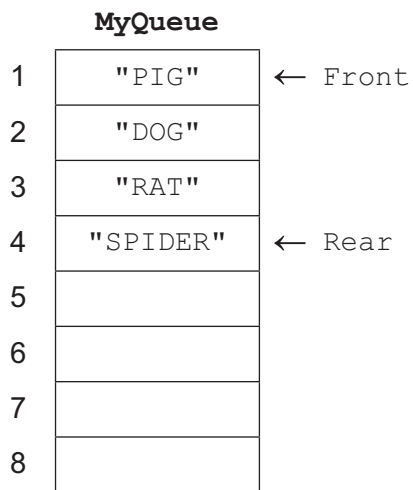
- (ii)** Justify why a queue is suitable for your application identified in **7(a)(i)**.

[2]

- (b)** A programmer designs a circular queue.

- The data items are stored in a 1D array, `MyQueue`
- The queue is managed using two pointers.

The diagram shows the circular queue populated with four items.



(i) Write **three** pseudocode declaration statements to implement this circular queue.

- 1
- 2
- 3

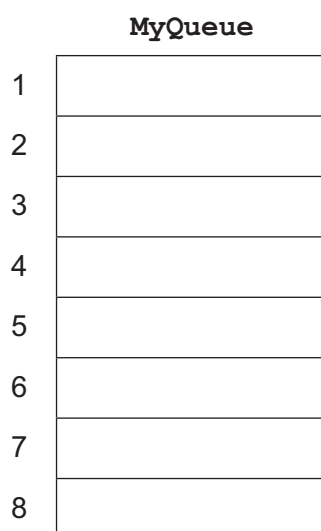
[2]

(ii) Two program modules are available to add and remove items to and from the queue.

module	description
EnQueue (<value>)	when <value> is added to the queue
DeQueue ()	when an item is removed from the queue

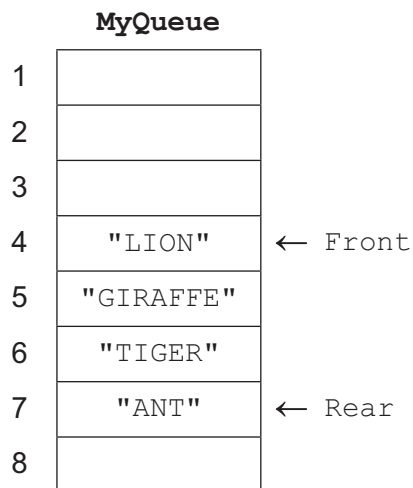
Complete the state of `MyQueue` after the following sequence of changes:

- DeQueue ()
- EnQueue ("HORSE")
- EnQueue ("CAT")



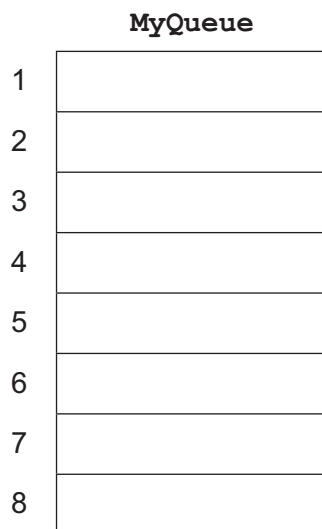
[2]

(iii) On a different occasion, the state of `MyQueue` is:



Complete the state of `MyQueue` after the following sequence of changes:

- `EnQueue ("GERBIL")`
- `EnQueue ("MONKEY")`
- `EnQueue ("SNAKE")`
- `DeQueue ()`



[2]

(c) `MyQueue` will be initially empty.

Write a pseudocode procedure `InitialiseQueue()` to set up the empty `MyQueue`, using a loop.

The initial array contents will be set to empty strings.

You are **not** required to repeat the declaration statements in 7(b)(i).

```
PROCEDURE InitialiseQueue()
ENDPROCEDURE
```

[3]

4. M/J 26/P21/Q9/a,b,c

- 9 A hotel has three floors numbered 0 to 2 (floor 0 is the ground floor). There are 15 rooms on each floor.

The hotel gets many enquiries from customers to check if a room is available that day.

The availability data for that day is stored in a 2D array `Version1`

- The first index is the floor number.
- The second index is the room number.
- On each of the three floors, the room numbers are 1 to 15.

Each cell of the array stores either:

- 'A' – the room is available
- 'B' – the room is already booked.

(a) Write the declaration statement for the 2D array `Version1`

[2]

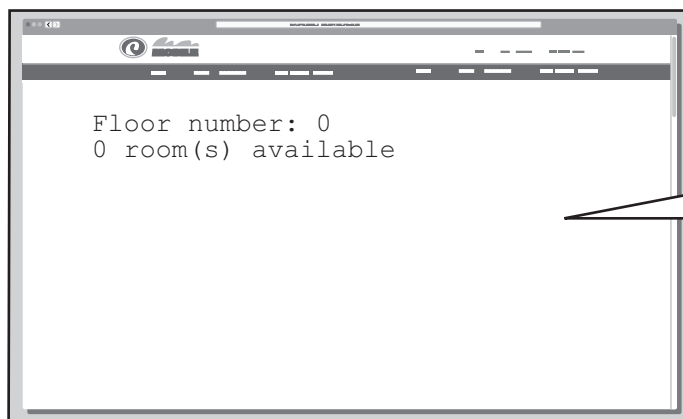
(b) An enquiry program will:

- input a floor number
- output all room numbers that are available on that floor
- output the number of rooms that are available.

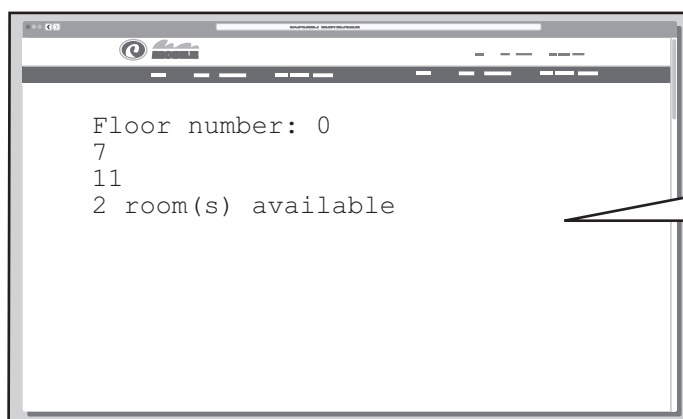
The output must be in the format shown in the two examples.

A wheelchair user makes an enquiry. She requires a room on the ground floor only.

Example program outputs:



There are no available rooms on the ground floor.



Shows two rooms available on the ground floor:

- rooms 7 and 11.

Write pseudocode for this enquiry program.

[6]

(c) Customers are becoming more selective with their enquiries when searching for an available room.

A new program design is required. It will create a record to define the data for a room.

Each room:

- may have a sea view
- is classified as: single room ('S'), double room ('D') or family room ('F')
- is either standard ('ST') or deluxe ('DL').

(i) Complete the pseudocode.

```
TYPE RoomData
```

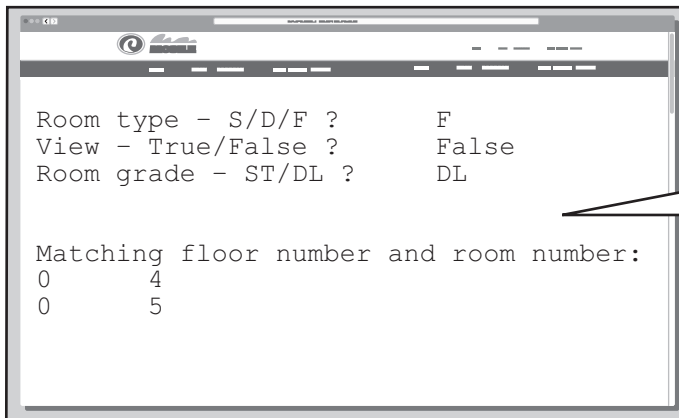
[3]

(ii) The room data is stored in a 2D array `Version2` of type `RoomData`

The new enquiry program uses this new data design and will:

- input the **three** customer requirements
- output the floor number(s) and room number(s) for all matching available rooms.

Example program output:



The customer asks for a deluxe family room; no need for a sea view.

The new enquiry program has found two matching rooms on the ground floor.

Write pseudocode for the new enquiry program.

[7]

5. M/J 26/P22/Q1/a(i)

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (i) Complete the pseudocode to declare the variable to be used to hold the password.

```
DECLARE Pass : .....
```

[1]

6. M/J 26/P22/Q4/a,b

- 4 A desktop web browser stores the history of web addresses visited.

The web browser uses a back button and forward button to move between web pages recently visited.

Two stacks are used when each of these buttons are clicked:

- back stack
- forward stack.

The back button is clicked to return to a previously visited web page.

The back button operates as follows:

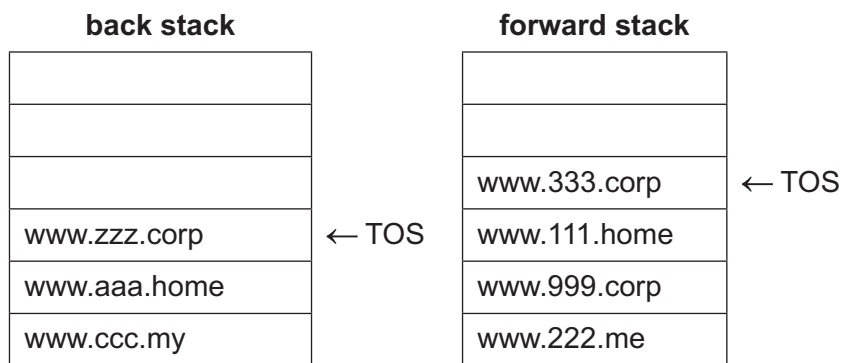
- The web address currently being displayed by the browser is pushed onto the forward stack.
- The web address from the top of the back stack is popped.
- The web address popped is loaded by the web browser.

The forward button is clicked to revisit a web page previously visited.

The forward button operates as follows:

- The web address currently being displayed by the browser is pushed onto the back stack.
- The web address from the top of the forward stack is popped.
- The web address popped is loaded by the web browser.

The current state of the two stacks:



The top of stack pointer is represented by TOS.

The current web address being displayed by the web browser is: www.bbb.home

The browsing session continues; the buttons are clicked in sequence in the three groups:

group 1:

back button

group 2:

forward button

group 3:

forward button

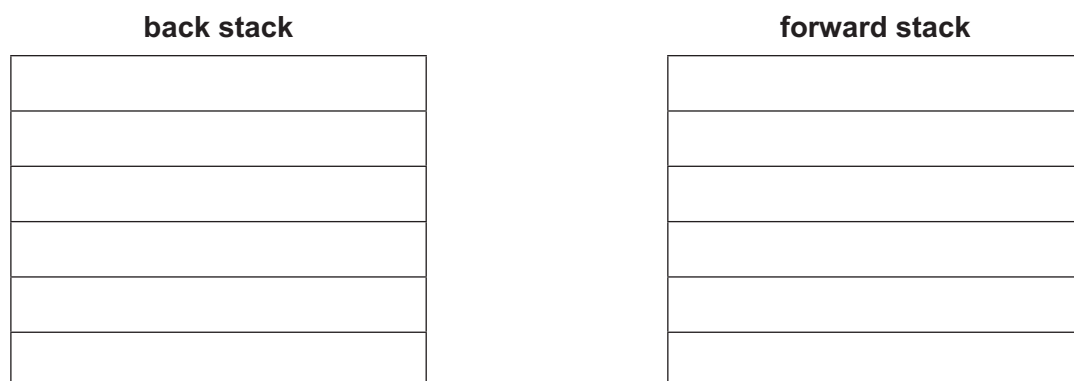
forward button

forward button

- (a) Complete the diagrams to show the state of both stacks after each group of operations has been performed.

Note: you must include the top of stack pointer (TOS) and the current web address being displayed for each group diagram.

after group 1



Web address currently displayed in the web browser:

after group 2

back stack

forward stack

Web address currently displayed in the web browser:

after group 3

back stack

forward stack

Web address currently displayed in the web browser:

[6]

(b) A stack is an Abstract Data Type (ADT)

Identify **one** other ADT and describe **two** features of the ADT.

ADT

feature 1

feature 2

[3]

7. M/J 26/P22/Q5/a,b

- 5 A programmer has defined a global 1D array `Numbers` of type `INTEGER`, which can store 1000 integers. The lower bound of the array `Numbers` is set to 1. Each element of the array has been initialised with a random integer in the range 200 to 99999.

The programmer has written a bubble sort algorithm to sort the array `Numbers` into ascending order.

Study algorithm A.

Assume all the variables have been correctly declared.

algorithm A:

```

FOR LoopCounter ← 1 TO 999                                //Outer Loop
  FOR Index ← 1 TO 999                                    //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    ENDIF
  NEXT Index
NEXT LoopCounter

```

- (a) Describe the change that would have to be made to the bubble sort algorithm (algorithm A) so that the `Numbers` array would be sorted into descending order.

[1]

- (b) The programmer rewrites the bubble sort algorithm to make it more efficient.

Study algorithm B.

Assume all the variables have been correctly declared.

algorithm B:

```

Boundary ← 999
REPEAT                                                    //Outer Loop
  NoSwaps ← TRUE
  FOR Index ← 1 TO Boundary                            //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
      NoSwaps ← FALSE
    ENDIF
  NEXT Index
  Boundary ← Boundary - 1
UNTIL NoSwaps = TRUE

```

- (i) Explain why the second bubble sort algorithm (algorithm B) is more efficient than the first bubble sort algorithm (algorithm A). Your answer must refer to both inner and outer loops.

[4]

- (ii) State the situation that will result in the outer loop iterating for the minimum number of times when the second bubble sort algorithm (algorithm B) is used.

[1]

8. M/J 26/P22/Q6/b

- 6 A program is being developed to encrypt a message.

The following terms are used in the encryption technique to be used:

term	definition
plain text	The original message before it is encrypted.
cipher text	The encrypted version of the plain text.

The encryption method to be used is based on the algorithm for the Caesar Box Cipher. All plain text messages to be encrypted must be 25 characters in length or less. If a message is less than 25 characters in length, then spaces are added to the end to make the padded plain text the required length.

A square grid of five columns and five rows is used. The padded plain text message is put into the grid row by row. The cipher text is obtained by reading the grid column by column.

For example:

plain text: This is a message

The number of characters in the original plain text, including spaces, is 17.

plain text with added spaces at the end: This△is△a△message△△△△△△△△

The character △ is used to indicate a space in the padded plain text.

The grid filled, row by row, with the padded plain text.

T	h	i	s	△
i	s	△	a	△
m	e	s	s	a
g	e	△	△	△
△	△	△	△	△

The cipher text is obtained by reading the grid column by column.

cipher text: Timg△hsee△i△s△△sas△△△△a△△

The programmer has defined the first program module as:

module	description
AddPadding()	• user prompt for and input of plain text • check plain text contains between 1 and 25 characters inclusive • re-input of plain text until it contains between 1 and 25 characters • if required, additional space characters are added to the end of the plain text to ensure it contains exactly 25 characters • return of amended plain text

(b) The programmer has defined a global 2D array `Grid` to be used to create the cipher text.

(i) Complete the pseudocode statement used to declare the global 2D array `Grid`

DECLARE

(ii) The programmer has defined an additional program module as follows:

module	description
PopulateGrid()	- called with one parameter of type string representing the correctly padded plain text - populate the array <code>Grid</code> using the plain text so that it can be used at a later point to create the cipher text - output the contents of the array <code>Grid</code> so it is displayed row-by-row using the original example for the padded plain text: <code>ThisΔisΔaΔmessageΔΔΔΔΔΔΔΔΔ</code> the output would be: <code>This</code> <code>is a</code> <code>messa</code> <code>ge</code>

Write pseudocode for module `PopulateGrid()`

[8]

9. M/J 26/P23/Q2/a

2 A control program is being developed for a robot floor vacuum cleaner. The settings for the robot can be changed and include:

- the strength of the suction (numerical values: 0, 1, 2, 3, 4 or 5)
- the total vacuum cleaning time (a whole number of minutes).

The robot has a number of sensors.

The control program includes the variables:

variable	description
<code>MainsOn</code>	TRUE when the mains power is on
<code>DisplayOn</code>	TRUE when the user display is shown
<code>Suction</code>	TRUE when the program detects that the user has selected a non-zero value for the strength of the suction

(a) Identify the data type used for the **three** variables.

[1]

10. M/J 26/P23/Q5/b

- 5 Data is stored about people who have attended a job interview. Each interview is held at one of three office locations.

Each person's data is stored as a data packet made up of four data items:

<initial><shortened surname><office code><summary of the interview>

- The initial is a single character.
- The shortened surname is the first five characters of the surname.
 - You can assume all surnames are at least five characters.
- The office code is a single character 'A', 'B' or 'C'.
- The interview summary text is of variable length.

Example:

Peter Watkins was interviewed at office 'B'. After the interview it was decided to invite him for a second interview.

His data packet is stored by variable `Applicant`:

```
Applicant ← "PWatkiBSecond interview"
```

- (b) The data for all interviews is saved in this format in a text file `JobApps.txt`

Each line of the file stores one data packet.

A procedure `Search()` is written to search for a particular applicant.

If the applicant is found, the program will output 'Name found', otherwise it will output 'Name not found'.

Complete the pseudocode for procedure `Search()`

```
PROCEDURE Search(Initial : CHAR, ShortSurname : STRING)
  DECLARE ThisApplicant, ThisSurname : STRING
  DECLARE ThisInitial : CHAR
  DECLARE Found : BOOLEAN

  OPENFILE "JobApps.txt" .....
  Found ← FALSE
  REPEAT

    ..... "JobApps.txt", ThisApplicant
    ThisInitial ← LEFT(ThisApplicant, 1)

    ThisSurname ← .....

    IF ThisSurname = ShortSurname ..... THEN
      .....
      OUTPUT "Name found"
    ENDIF

  UNTIL Found = TRUE .....

  IF Found = FALSE THEN
    OUTPUT "Name not found"
  ENDIF

  .....
ENDPROCEDURE
```

11. M/J 26/P23/Q6/a,b,c,d

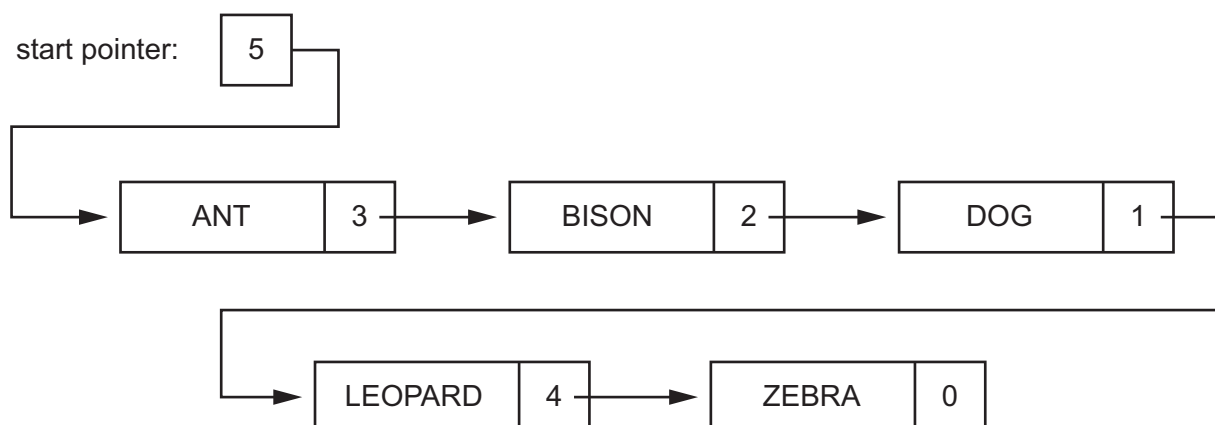
6 A linked list is an example of an Abstract Data Type (ADT).

A linked list is implemented using:

- a 1D array for the data items
- a 1D array for the link pointers
- a start pointer.

The linked list maintains the data items in alphabetical order.

(a) The state of the linked list after five data items have been added is:



Complete the diagram to show the `StartPointer` and the contents of the arrays `Data` and `Pointer`

`StartPointer`

Data		Pointer	
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
19		19	
20		20	

[3]

(b) State what the pointer value 0 (null pointer) indicates.

[1]

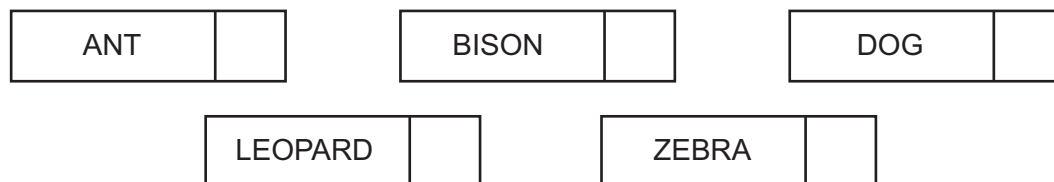
(c) To maintain the linked list, algorithms are required to:

- insert a new data item into the linked list
- delete a data item from the linked list.

- (i) The data item 'DOG' is to be deleted from the linked list given in 6(a).

Complete the diagram to show the new state of the linked list.

start pointer:



[1]

- (ii) To delete a data item, the algorithm will have special cases. One special case is the linked list is empty.

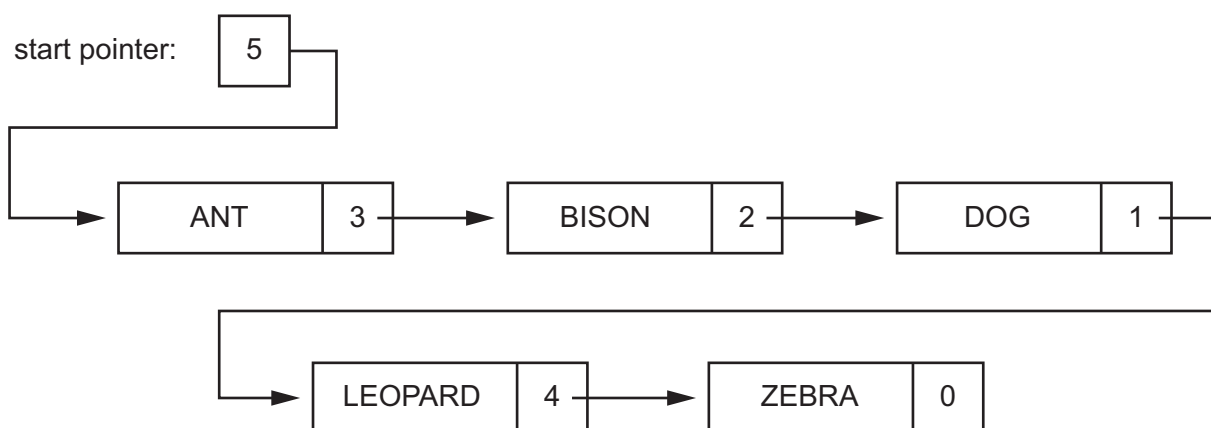
Describe **two** other special cases which must be considered when deleting a data item.

1

2

[2]

- (d) The linked list given in 6(a) is repeated here. Data item 'BAT' is to be added to the linked list.



Describe how the data item 'BAT' is added to the linked list.

Do **not** attempt to write pseudocode.

[6]

12. M/J 26/P23/Q8/a,b

also Unit 12 (3 marks); this unit 16 marks

- 8 A warehouse stores stock in bins. A bin is a container which stores the stock items; each bin can store many identical items. The warehouse is organised on three different levels – level 0 (ground level), then level 1 and level 2. Each level has 120 bins, numbered 1 to 120.

A bin is identified by the level and a bin number. The data item stored for each bin is a Boolean value; this indicates whether the bin is in use (contains stock items) or not in use (contains no stock items).

- (a) (i) Complete the pseudocode to declare a 2D array `Bins`

DECLARE

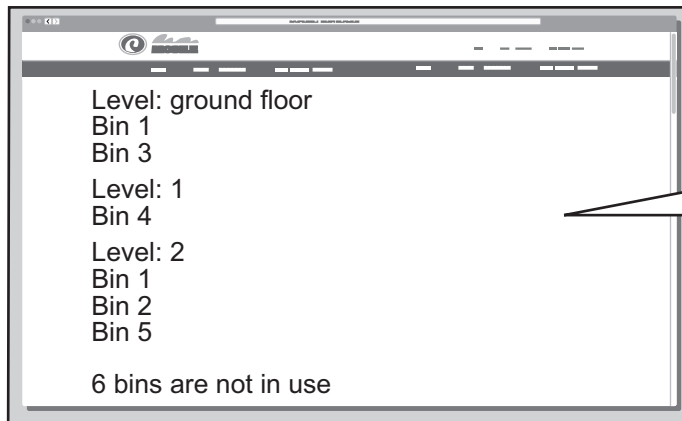
[2]

(ii) A pseudocode module `InactiveReport()` will produce a report that will show:

- a header for each level
- the bin number for each bin that is **not** in use on that level
- the total number of bins that are **not** in use on all levels.

The module is initially developed for only **five** bins on each level.

An example program output:



Shows:

- the header for each level
- a list of bin numbers for bins that are not in use on that level
- the total number of bins not in use.

Complete the pseudocode for the procedure `InactiveReport()`

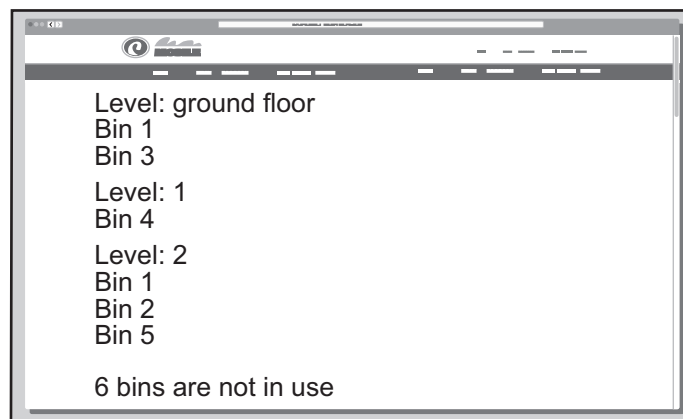
Assume that the `Bins` array is a global variable and has already been populated with data.

```
PROCEDURE InactiveReport()
```

```
ENDPROCEDURE
```

[5]

(iii) Test data were chosen to produce this output:



Complete the pseudocode test data which would produce the output given.
Reminder: The module is initially developed for only **five** bins on each level.

```

// Ground floor
Bins[0, 1] ← FALSE
Bins[0, 2] ← TRUE

Bins[0, 3] ← .....
Bins[0, 4] ← TRUE
Bins[0, 5] ← TRUE

// Level 1
Bins[1, 1] ← TRUE

Bins[1, 2] ← .....
Bins[1, 3] ← TRUE
Bins[1, 4] ← FALSE
Bins[1, 5] ← TRUE

// Level 2

```

[3]

- (b) It is decided to change the 2D array `Bins` so that more information is available about each bin.

Data items for each bin are:

- bin size (coded as 'Small', 'Medium' or 'Large')
- active – is the bin currently in use?
- stock item code for the stock item stored (four-digit string)
- stock level (0 to 500 items).

- (i) Complete the pseudocode to create a record structure with the identifier `BinData`

```

TYPE BinData
ENDTYPE

```

[2]

- (ii) A stock item is to be allocated to an unused bin. A function `FindBin()` is required that finds and returns the location of the first bin not in use which has a suitable size. The function will search on the ground floor first, then search level 1 and finally search level 2.

The function uses the global 2D array `Bins` using the revised record structure `BinData`

The function `FindBin()`:

- takes a parameter for the bin size required (coded as 'Small', 'Medium' or 'Large')
- searches for an available bin that matches the size required
- outputs the level and bin number (as a string) of the first unused bin with the matching size
 - for example, 'Level: 0 Bin: 4'
 - or if there are no available bins 'No bins available'.

Write the pseudocode function `FindBin()`

[7]

13. O/N 25/P21/Q3/a,b

- 3** A program is needed to manage individual rentals in a car-hire business.

The data items for each rental will be held in a record structure of type `RentalRecord`
The programmer has started to define the items that will be needed:

Item	Example value	Comment
<code>RentalID</code>	"AB1234"	a unique alpha-numeric value
<code>CarID</code>	241	a numeric value used as an array index
<code>DisCode</code>	'S'	a letter indicating the type of discount offered
<code>Start</code>	13/06/2025	when the rental starts
<code>Duration</code>	7	the number of days of the rental
<code>Completed</code>	FALSE	TRUE when the car is returned and the rental charge paid

- (a) (i)** Write pseudocode to declare the record structure for type `RentalRecord`

[4]

- (ii)** A 1D array `Rental` containing 500 elements is used to store the data for all rental records.

Write pseudocode to declare the `Rental` array.

[2]

- (b)** State **three** benefits of using an array of records to store the data for all rentals.

1

2

3

[3]

14. O/N 25/P21/Q4/a,b

- 4** A program contains a global 1D array `Number` consisting of 20 elements of type `REAL`

A procedure `Store()` will input a sequence of up to 20 real values, one value at a time. These values will be assigned to elements of the array using four steps:

- Step 1: store the first value in the sequence in the first element of the array
Step 2: check each subsequent value input. If this value is larger than the previous value input, then assign the value to the next array element, otherwise go to **step 4**
Step 3: repeat from **step 2** unless the array is full
Step 4: output the count of the number of values stored in the array together with a suitable message.

- (a)** Complete the pseudocode for `Store()`

All variables used in the algorithm must be declared.

```
PROCEDURE Store()
```

```
ENDPROCEDURE
```

[6]

- (b)** The requirements of the program change:

- the number of values in the sequence is unknown, but may be higher than 20
- the values will need to be accessed by another program as data.

The data will be stored in a text file instead of an array.

- (i)** Give **two** benefits of using a text file instead of an array.

1

2

[2]

- (ii)** State any change that will need to be made before each value is written to the file.

[1]

15. O/N 25/P21/Q5/a

- 5** A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return `-1` if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
  DECLARE Index, FoundAt : INTEGER
  FoundAt ← -1
  FOR Index ← 1 TO 10000
    IF MyData[Index] = SearchString THEN // outer conditional clause
      IF FoundAt = -1 THEN               // inner conditional clause
        FoundAt ← Index
      ENDIF
    ENDIF
  NEXT Index
  RETURN FoundAt
ENDFUNCTION
```

- (a) (i)** Comment on why the programmer chose `-1` as the initial value of `FoundAt`

[1]

- (ii)** Explain the purpose of the **outer** conditional clause.

[1]

- (iii)** The **inner** conditional clause ensures that only the index of the **first** matching element, if any, is returned.

Explain how this clause works.

[1]

16. O/N 25/P21/Q8/a,b

- 8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
<code>OKToBorrow()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - search the array for loan records for the specified student - output a suitable message to say whether the student may, or may not, borrow another book

- (a) Write efficient pseudocode for the module `OKToBorrow()`

[7]

- (b) A second module is defined:

Module	Description
<code>ReturnBook()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - searches the array for the relevant loan record - when found, sets <code>OnLoan</code> to <code>FALSE</code> and returns <code>TRUE</code> - if a loan record is not found, or the book has already been returned, then returns <code>FALSE</code>

Write efficient pseudocode for the module `ReturnBook()`

Assume that each student is only allowed to borrow each book only once. That means that there will be no more than **one** loan record for a given combination of student and book.

[8]

17. O/N 25/P22/Q1/b

(b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
Result	5.42	
MonthLetter	"JFMAMJJASOND"	
Birthday	15/11/2009	

[3]

18. O/N 25/P22/Q3/b,c

- 3 A text file `OldFile.txt` contains IDs, names and email addresses for members of a club. Three information items are stored for each member and each item is stored on a separate line.

The example shows the information for the first two members in the first six lines of the file:

Line in file	Information item	Example data
1	member 1 ID	"AB1234"
2	member 1 name	"Freddie Jones"
3	member 1 email address	"FreddieJ909@Cambridge.org"
4	member 2 ID	"BC2345"
5	member 2 name	"Sue Smith"
6	member 2 email address	"Sue1024@Cambridge.org"

The member ID string is always two letters followed by four digits.

The file design is to be changed so that information for each user is stored as a single line of the file, with the character `'\'` used as a separator between data items.

For example, the single line for member 1 will be:

```
"AB1234\Freddie Jones\FreddieJ909@Cambridge.org"
```

An algorithm will produce a new file `NewFile.txt` from the contents of `OldFile.txt`

Assume:

- `LineX`, `LineY` and `LineZ` are of type `STRING` and are used to store the three items of information for each member
- `NewString` is a temporary variable of type `STRING`
- `OldFile.txt` exists and contains valid data.

(b) The character `'\'` has been chosen as a separator.

Explain why this is a suitable character.

[1]

- (c) Two new information items are to be stored for each member. Both new items are encrypted; they can each contain any character (including ' \ ') and can be of any length up to a maximum of 99 characters.

For example:

Information item	Example data
member 1 ID	"AB1234"
member 1 name	"Freddie Jones"
member 1 email address	"FreddieJ909@Cambridge.org"
member 1 new information 1	"En/98&*(?\ /D7iP"
member 1 new information 2	"23\CoboL"

Explain the additional file design changes needed so that:

- all the information items for each member are stored on a single line of `NewFile.txt`
- it is possible to extract each information item after the line is read.

Assume the ' \ ' character is not used in any email address.

[3]

19. O/N 25/P22/Q4/a,b

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	TRUE	valid total score (no correct answers)
1	FALSE	invalid total score
2	FALSE	invalid total score
3	TRUE	valid total score (one 3-point question correct)
4	FALSE	invalid total score
5	TRUE	valid total score (one 5-point question correct)
6	TRUE	valid total score (two 3-point questions correct)
7	FALSE	invalid total score
8	TRUE	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(a) Write pseudocode to declare `CheckTotal` and to set all elements of the array to `FALSE`

All variables used must be declared.

[4]

(b) The pseudocode represents an algorithm to set the appropriate elements of the array to `TRUE`

Complete the pseudocode:

```

DECLARE EasyQ, HardQ : INTEGER
FOR EasyQ ← ..... TO 15 STEP .....
    FOR HardQ ← 0 TO ..... STEP .....
        CheckTotal[.....] ← TRUE
    NEXT HardQ
NEXT EasyQ

```

[5]

20. O/N 25/P22/Q8/a,b,c

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>CountLoans()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - counts the number of books currently on loan to the specified student - counts the number of books that the student has already returned - output both counts together with a suitable message

(a) Write pseudocode for module `CountLoans()`

[7]

(b) As a reminder, a global array `Loan` stores 7000 loan records with data items for each loan record:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

A new module is defined:

Module	Description
<code>NewLoan()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - searches the array for an unused loan record - if found, updates the loan record and returns <code>TRUE</code>, otherwise returns <code>FALSE</code>

Write efficient pseudocode for `NewLoan()`

[7]

(c) When a book is returned, the loan record will have its `OnLoan` data set to `FALSE`

A new procedure `Archive()` will mark these records as unused after first saving them for future reference.

Explain how these records can be saved for future reference.

[2]

21. O/N 25/P23/Q3

- 3 A program uses a stack to hold up to 30 integer values. The stack is implemented using a global integer variable and a global 1D array.

The array is declared in pseudocode:

```
DECLARE ThisStack : ARRAY[1:30] OF INTEGER
```

Stack design notes:

- The global variable `SP` acts as a stack pointer. `SP` contains the array index of the last value pushed onto the stack.
- If the stack is empty, then `SP` is assigned the value zero.
- The first item added to the stack will be stored in `ThisStack[1]`
- `SP` is incremented each time an item is added to the stack.

A function `Pop()` is written to remove an item from `ThisStack`. The function returns an item of type `PopData` which is defined in pseudocode:

```
TYPE PopData
  DECLARE Data    : INTEGER
  DECLARE Exists  : BOOLEAN
ENDTYPE
```

The value removed from the stack is assigned to `Data` and `Exists` is set to `TRUE`. If it is **not** possible to remove a value from the stack, then `Exists` is set to `FALSE`

Complete the pseudocode for `Pop()`

```
FUNCTION Pop() RETURNS PopData
  DECLARE ThisPop : .....
  IF ..... THEN
    PopData.Exists ← ..... // Stack is empty
  ELSE
    PopData.Data ← .....
    PopData.Exists ← .....
    SP ← .....
  ENDIF
  RETURN ThisPop
ENDFUNCTION
```

[5]

22. O/N 25/P23/Q4

- 4 A program contains a global 1D array `Data` containing 20 elements of type `INTEGER`

A global string `NumString` represents a sequence of three-digit numbers, separated by commas.
For example:

"101,456,219,754,328"

The string contains at least **four** three-digit numbers. The total number of three-digit numbers in the string is unknown.

A procedure `Store()` will:

- extract one three-digit number at a time from `NumString`
- convert each of the three-digit numbers extracted to an integer and assign this to the next array element, starting from index 1
- end when all three-digit numbers have been stored, or when the array is full.

Complete the pseudocode for `Store()`

All local variables used must be declared.

PROCEDURE `Store()`

ENDPROCEDURE

[6]

23. O/N 25/P23/Q8/a,b,c

- 8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
<code>StudentID</code>	<code>STRING</code>	the unique ID of the student who has borrowed the book The first three characters of a <code>StudentID</code> represent a tutor ID. Each student has one tutor.
<code>BookID</code>	<code>STRING</code>	the unique ID of the book being borrowed
<code>OnLoan</code>	<code>BOOLEAN</code>	<code>TRUE</code> if the book has not been returned

The programmer has defined a global array `Loan` to store 8000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>LoanStatus()</code>	- called with two parameters of type <code>STRING</code> representing a <code>StudentID</code> and a <code>BookID</code> - outputs a message saying whether a given loan has been returned or not - outputs a warning message if a record of the given loan is not found

(a) Write efficient pseudocode for the module `LoanStatus()`

Assume that each combination of `StudentID` and `BookID` can occur only once.

[8]

(b) A second module is defined:

Module	Description
<code>LoansPerTutor()</code>	- called with a parameter of type <code>STRING</code> representing a tutor ID (as a reminder, the first three characters of a <code>StudentID</code> represent a tutor ID) - returns an integer value representing the number of books currently on loan to students who have the given tutor

Reminder: unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

Write pseudocode for the module `LoansPerTutor()`

[7]

(c) It is decided to mark as unused all records for book loans that have been returned. The data for these records will first be written to a new text file for archive purposes.

(i) The archive program will automatically generate a meaningful filename each time it is run.

Outline a meaningful format to use for the filename.

[2]

(ii) There is a problem that will need to be overcome before the data items can be written to a text file.

As a reminder, the data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book The first three characters of a <code>StudentID</code> represent a tutor ID. Each student has one tutor.
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

Explain the problem.

[1]

- (iii) One way of storing the data items in a text file is to store each data item on a separate line.

Identify **one** benefit and **one** drawback of this way of storing the data.

Benefit

Drawback

[2]

24. M/J 25/P21/Q5

5 Stacks and queues are both abstract data types.

A stack uses a top-of-stack pointer to indicate the location of the last item added to the stack.

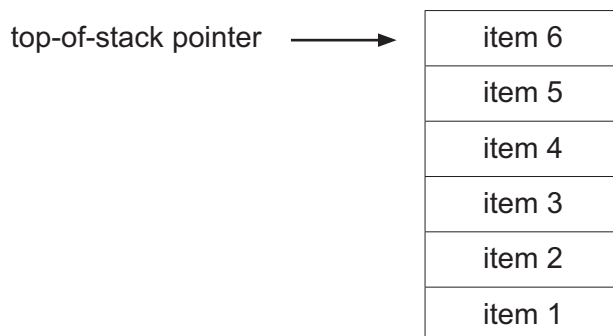
A queue uses two pointers:

- a front pointer to indicate the location of the next item to be removed from the queue
- a rear pointer to indicate the location of the next item to be added to the queue.

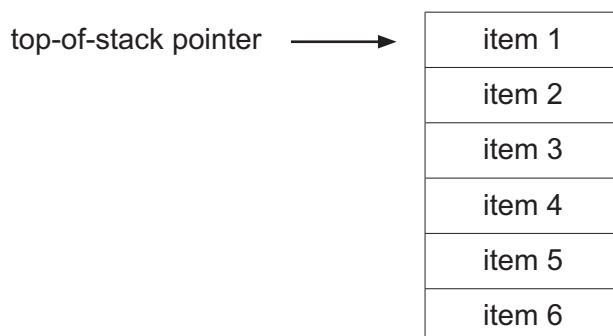
A queue can be used to reverse the items stored on a stack.

For example, if a stack contains six items:

Initial state of the stack:



Final state of the stack when the items have been reversed:



Describe how the queue could be used to reverse the items that are currently stored on the stack.

Your description must include how the pointers are used in both the stack and queue.

Assume:

- The stack initially contains an unknown number of items.
- The queue can store all the items currently stored on the stack.
- The queue is initially empty.

25. M/J 25/P21/Q6/b

- 6** A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

- (b)** Each sensor used in the system has a unique sensor ID number in the range 1 to 50.

The program that is used to monitor the speed of each vehicle reads each sensor value every second and stores this value along with the sensor's unique ID into a global 2D array `Reading`

The global array `Reading` has been declared as follows:

```
DECLARE Reading : ARRAY[1:2000, 1:2] OF INTEGER
```

The array contains 4000 elements organised as 2000 rows and 2 columns.

Column 1 contains the sensor value, and column 2 contains the sensor ID.

When 2000 sensor readings have been taken, the array is full and the system stops taking any more sensor readings until the array has been processed.

A procedure `Sort()` is needed to sort the array into ascending order of sensor value using an efficient bubble sort algorithm.

Write **efficient** pseudocode for the procedure `Sort()`

[8]

26. M/J 25/P21/Q7/b(i)

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	- called with two integer parameters: - the number of coffees ordered - the current number of points - output a suitable message giving the number of free coffees - return a value for the new points total.

- (b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

`<CustomerID>,<Points>`

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at "100001" with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

`"100205,0"`

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
<code>AddNewCustomers()</code>	- called with an integer parameter representing the number of new customers to be added to the loyalty scheme - adds a new line, containing the required information, to the text file <code>Loyalty.txt</code> for each customer added to the loyalty scheme - outputs each new customer ID added to the loyalty scheme

- (i) Write pseudocode for module `AddNewCustomers()`

Assume that there is at least **one** customer already in the loyalty scheme.

27. M/J 25/P22/Q1/c,d

1 A program is being developed to control the production line in a factory.

(c) Pseudocode has been used to design modules for the program to control the production line.

The table shows **four** valid pseudocode expressions.

Complete the table by giving the data type of the evaluated expression.

Expression	Data type
RIGHT (MachineCode, 4)	
Speed * 2.5	
NOT Status	
IS_NUM(Check)	

[4]

(d) A global array `Product` is used as part of the pseudocode design being developed to control the production line. `Product` is used to store the number of rejected items each day.

The following pseudocode statement is used to assign a value to an element of the array:

`Product[x, y] ← 23`

The lower and upper bound values are shown in the table:

Variable	Lower bound	Upper bound
x	0	99
y	0	9

(i) State the number of dimensions of `Product`

[1]

(ii) State the total number of elements in array `Product`

[1]

(iii) Give the pseudocode declaration for the array `Product`

[2]

28. M/J 25/P22/Q5/b

- 5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

- (b) At the end of each sequence of pictures, the time is saved as a string in the format <HH><MM><SS> where:

- HH represents the hours using two digits
- MM represents the minutes using two digits
- SS represents the seconds using two digits.

For example:

- "081230" is stored to represent the time 8:12:30, in the morning
- "152235" is stored to represent the time 15:22:35, in the afternoon.

Each string is stored on a new line in the text file `TimeTaken.txt`

An algorithm is required to process the content of the text file `TimeTaken.txt` once it has been transferred to the computer.

For each hour when pictures are taken, output a suitable message showing the hour and the total number of sequences of pictures taken within that hour.

Example outputs:

```
Hour : 15   Total : 32
Hour : 18   Total : 1
```

Write pseudocode for this algorithm.

Assume the text file `TimeTaken.txt` contains at least **one** line.

29. M/J 25/P22/Q6/b

- 6** Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

Data		Pointer	
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	4
8	0	8	9
9	0	9	10
10	0	10	-1

Study the pseudocode:

```

PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
  DECLARE New, Current, Last : INTEGER
  CONSTANT NullPointer = -1
  New ← Unused
  Last ← NullPointer
  Current ← Start
  WHILE Current <> NullPointer AND Data[Current] < Value
    Last ← Current
    Current ← Pointer[Current]
  ENDWHILE
  Pointer[New] ← Pointer[Last]
  Pointer[Last] ← New
  Data[New] ← Value
ENDPROCEDURE

```

- (b)** The operation carried out by procedure `Place()` together with the arrays form part of the implementation of an Abstract Data Type (ADT).

Identify the ADT and state the operation carried out by procedure `Place()`

[2]

30. M/J 25/P22/Q7/a,b,c

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

Each customer has a unique customer ID starting at 10001 with this value increasing by one each time a new customer joins the loyalty scheme.

For example, the third customer who joins the loyalty scheme is given the customer ID 10003

The loyalty scheme is limited to 1000 customers.

A customer is awarded a loyalty point every time they buy a coffee.

The programmer has decided to use a global 2D array `Loyalty` of type `INTEGER`. The array `Loyalty` is made up of 1000 rows and 2 columns. Each row relates to one customer; column 1 contains the unique customer ID and column 2 contains the number of customer loyalty points.

Rows in the array `Loyalty` that are not currently being used have the value of Column 1 set to 99999

The array is sorted in ascending order by customer ID.

The programmer has defined a program module:

Module	Description
<code>FindCustomer()</code>	- called with parameter of type <code>INTEGER</code> representing a customer ID - searches the <code>Loyalty</code> array for this customer ID - the search will stop as soon as the customer ID is found - the search should efficiently deal with the situation when the customer ID is not stored in the <code>Loyalty</code> array - if the customer ID is found, return an integer value representing the loyalty points, otherwise return <code>-1</code>

(a) Write efficient pseudocode for module `FindCustomer()`

[8]

(b) A customer can claim a free coffee for every 11 loyalty points.

The programmer has defined a second program module:

Module	Description
<code>PointsReport()</code>	- output the customer ID for each customer who has 11 or more loyalty points - output the average loyalty points for all customers in the <code>Loyalty</code> array along with an appropriate message

Write efficient pseudocode for module `PointsReport()`

Assume the array contains the data for at least **one** customer.

[7]

(c) The programmer decides to amend the customer ID; it will be stored as a `STRING` instead of an `INTEGER`. This means that the 2D array `Loyalty` can no longer be used.

(i) Explain why the 2D array `Loyalty` can no longer be used.

[1]

(ii) Explain how a 1D array could be used to store both the loyalty points and the amended customer ID.

[2]

31. M/J 25/P23/Q1/c

1 A program is being developed for the management of a sports centre.

(c) The table lists some of the variables used in the program.

Complete the table by writing the most appropriate data type for each variable.

Variable	Use of variable	Data type
MemberCount	stores how many members have used the sports centre each day	
TotalTakings	stores the total amount of money taken each day	
BookingConfirmed	stores the state of a booking for an exercise class; a booking is either confirmed or not confirmed	
MemberDOB	to calculate when to send an email with a birthday message to a member	

[4]

32. M/J 25/P23/Q2/a,b,c

2 A programmer uses a number of Abstract Data Types (ADT) in the programs that she is developing.

(a) A stack using memory locations 400 to 409 is used in one program.

The diagram represents the current state of the stack.

A variable `TopOfStack` pointer indicates the last value added to the stack.

Stack

Memory location	Value	
409		
408		
407		
406		
405		
404		
403	'P'	← TopOfStack
402	'X'	
401	'D'	
400	'B'	

(i) Complete the answer column in the following table:

Answer

the memory location of the value that has been on the stack for the longest time	
the number of consecutive push operations that will result in the <code>TopOfStack</code> variable containing 409	

[2]

- (ii) The diagram shows the current state of the stack.

Stack	
Memory location	Value
409	
408	
407	'A'
406	'C'
405	'F'
404	'K'
403	'B'
402	'S'
401	'R'
400	'D'

← `TopOfStack`

The following sequence of operations are performed:

PUSH 'T'
 POP
 POP
 PUSH 'Z'
 PUSH 'X'
 POP
 PUSH 'Y'

Complete the following diagram to show the state of the stack after the operations have been performed.

Stack	
Memory location	Value
409	
408	
407	
406	
405	
404	
403	
402	
401	
400	

[3]

- (b) In a different program, the programmer decides to implement a linked list using an array of records.

- (i) The array is represented in the diagram.

A pointer value of -1 indicates the end of the linked list.

Complete the pointer field to produce a linked list that is in alphabetical order.

Index	Data	Pointer
0	"Neptune"	
1	"Jupiter"	
2	"Saturn"	
3	"Earth"	
4	"Mercury"	
5	"Uranus"	

[3]

- (ii) The variable `StartPointer` contains the index of the first item in the linked list.

State the value of the variable `StartPointer`

[1]

- (c) A third program is also being created to manage a queue.

Describe **two** features of a queue.

Feature one

Feature two

[2]

33. M/J 25/P23/Q7/b(i)

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
UpdateVisit()	- called with two parameters: - loyalty points of type <code>INTEGER</code> - last visit date of type <code>DATE</code> - change the loyalty points: - increase by four if the last visit date and the current date are in the same month - otherwise increase by one - change the last visit date to the current date - the changed values must be available to the code that follows the call to <code>UpdateVisit()</code>

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: `DDMMYYYY`.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

`"100123,132,05032025"`

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
FindCustomer() (is already written)	- called with a parameter of type <code>STRING</code> that represents a customer ID - returns the line of data read from the text file <code>Loyalty.txt</code> containing the data items for that customer
MondayCheck()	- called with a parameter of type <code>STRING</code> that represents the customer ID of a customer - calls <code>FindCustomer()</code> - extracts the loyalty points from the string returned by <code>FindCustomer()</code> - increases the loyalty points for the current customer by 10 if the day visited is a Monday - returns an integer value representing the loyalty points

(i) Write pseudocode for module `MondayCheck()`

The module `FindCustomer()` must be used and assume it returns a valid string for the current customer.

[8]

34. O/N 24/P21/Q1/c

1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```

HighRate ← FALSE
CASE OF ItemCost
    ≤ 50 : TaxRate ← 3.75      // tax rate of 3.75%
    ≤ 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25    // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable

```

(c) Give the **appropriate** data type for the variables in the following table, as used in the pseudocode:

Variable name	Data type
HighRate	
TaxPayable	

[2]

35. O/N 24/P21/Q6/a

- 6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

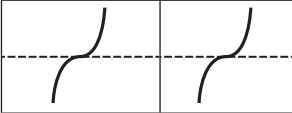
The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
	
11	30
12	31

Note: Leap years are **not** considered

- (a) An algorithm uses the array `DaysInMonth` to calculate a 'use-by' date. An alternative design would involve the use of multiple selection statements.

An array-based technique is often used when there is a large number of different values to check and where no pattern exists.

One advantage of using an array-based technique is the speed of execution compared to the use of multiple selection statements.

Give **two other** advantages of using an array for this type of operation rather than a solution based on multiple selection statements.

1

2

[2]

36. O/N 24/P21/Q8/b,c

- 8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
GradeBoundary[1]	65	The minimum mark for an A grade.
GradeBoundary[2]	57	The minimum mark for a B grade.
GradeBoundary[3]	43	The minimum mark for a C grade.
GradeBoundary[4]	35	The minimum mark for a D grade.
GradeBoundary[5]	27	The minimum mark for an E grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark	Element	ID
Result[4, 1]	56	Result[4, 2]	1074832
Result[5, 1]	54	Result[5, 2]	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
<code>CheckMark()</code>	- called with a parameter of type integer representing a candidate mark - returns <code>TRUE</code> if the mark is within 2 of any of the five grade boundaries, otherwise returns <code>FALSE</code>

(b) A second module is defined:

Module	Description
<code>CheckAll()</code>	- called with a parameter of type integer representing the number of candidate marks in the <code>Result</code> array - uses <code>CheckMark()</code> to check each candidate mark - for each paper that needs to be checked, write the corresponding candidate ID on a separate line in a new file named <code>GRLIST.txt</code> - outputs a message with a count of how many papers need to be checked

Write pseudocode for module `CheckAll()`.

`CheckMark()` must be used to check each individual mark.

[8]

- (c) The requirement changes. Instead of a new file, the module described in part (b) needs to add the corresponding candidate ID for each paper that needs to be checked to an **existing** file.

Explain the change that will need to be made to `CheckAll()`.

[1]

37. O/N 24/P22/Q1/d

- 1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

- (c) To solve the error a programmer decides to create a new module.

The design of the new module has been completed and the module is being coded.

Identify **two** features of an IDE that will help during the coding of this new module.

1

2

[2]

- (d) The new module referred to in part (c) introduces **three** new variables.

Complete the following table by giving the appropriate data type for each.

Variable name	Used to store	Data type
Name	A customer name.	
Index	An array index.	
Result	The result of the division of any two non-zero numbers.	

[3]

38. O/N 24/P22/Q2/b(i)

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character ' * '.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
"1234567890"	"*****7890"
"1234567897652"	"*****7652"
"1234567890123456"	"*****3456"

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(i) Write pseudocode to declare the array.

[2]

39. O/N 24/P22/Q3/a,b

- 3 A program uses a stack to hold up to 60 numeric values.
The stack is implemented using two integer variables and a 1D array.

The array is declared in pseudocode as shown:

```
DECLARE ThisStack : ARRAY[1:60] OF REAL
```

The stack operates as follows:

- Global variable `SP` acts as a stack pointer that points to the next available stack location. The value of `SP` represents an array index.
- Global variable `OnStack` represents the number of values currently on the stack.
- The stack grows upwards from array element index 1.

- (a) (i) Give the initial values that should be assigned to the **two** variables.

`SP`

`OnStack`

[1]

- (ii) Explain why it is **not** necessary to initialise the array elements before the stack is used.

[2]

- (b) A function to add a value to `ThisStack` is expressed in pseudocode as shown.
The function will return a value to indicate whether the operation was successful or not.

Complete the pseudocode by filling in the gaps.

```
FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN
    DECLARE ReturnValue : BOOLEAN
    IF ..... THEN
        RETURN ..... // stack is already full
    ENDIF
    ..... ← ThisValue
    SP ← .....
    OnStack ← OnStack + 1
    RETURN TRUE
ENDFUNCTION
```

[4]

40. O/N 24/P22/Q6/a

- 6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	"Cheese"
2	"Onion"
3	"Salmon"
4	"Anchovies"
5	"Peanut Butter"

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	"White"
2	"Brown"
3	"Pitta"

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

(a) Complete the pseudocode for the procedure `Special()`.

Assume that both arrays are global.

```
PROCEDURE Special()
ENDPROCEDURE
```

[7]

41. O/N 24/P22/Q8/a,b,c,d

- 8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Level	14, 23, 76	The skill level of the character. An integer in the range 0 to 100 inclusive.

The programmer has defined a record type to define each character.

The record type definition is shown in pseudocode as follows:

```

TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE Level : INTEGER
ENDTYPE

```

The `Player` field indicates the player to which the character is assigned (1 to 6). The field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data as follows:

```

DECLARE Character : ARRAY[1:45] OF CharacterType

```

At the start of the game all record fields are initialised, and all `Player` field values are set to 0

The programmer has defined a program module as follows:

Module	Description
<code>Assign()</code>	- called with two parameters: - an integer representing a player - a string representing a character role - search the <code>Character</code> array for an unassigned character with the required role - If found, assign the character to the given player and output a confirmation message, for example: "Bill the Builder has been assigned to player 3" - If no unassigned character with the required role is found, output a suitable message.

(a) Write pseudocode for module `Assign()`.

[7]

(b) A new module will store the contents of the `Character` array in a text file.

The module is defined as follows:

Module	Description
Save ()	- form a string from each record with fields separated by the character '^' - write each string to a separate line of the new file named SaveFile.txt

Complete the pseudocode for module Save () .

PROCEDURE Save ()

ENDPROCEDURE

[7]

(c) The program is changed and the record type definition is modified as follows:

```
TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE Level : INTEGER
    DECLARE Status : BOOLEAN
ENDTYPE
```

Describe how the additional Boolean field may be stored with the rest of the fields on one line of a text file.

[2]

(d) The save operation is to be extended so that multiple files may be saved as the game progresses. This will allow the user to restore the game from any saved position. The filename must reflect the sequence in which the files are saved.

Describe a method that would allow multiple files to be saved **and** give an example of **two** consecutive filenames.

Method

Example

[2]

42. O/N 24/P23/Q1/b

(b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Available	TRUE	
Received	"18/04/2021"	
Index	100	

[3]

43. O/N 24/P23/Q3/a

- 3 The implementation of a linked list uses an integer variable and a 1D array `List` of type `Node`.

Record type `Node` is declared in pseudocode as follows:

```
TYPE Node
  DECLARE Data : STRING
  DECLARE Pointer : INTEGER
ENDTYPE
```

The array `List` is declared in pseudocode as follows:

```
DECLARE List : ARRAY[1:200] OF Node
```

The linked list will operate as follows:

- Integer variable `HeadPointer` will store the array index for the first node in the linked list.
- The `Pointer` field of a node contains the index value of the next array element (the next node) in the linked list.
- The value 0 is used as a null pointer.

- (a) (i) State why the value 0 has been selected as the null pointer.

[1]

- (ii) Give the range of valid values that could be assigned to variable `HeadPointer`.

[1]

44. O/N 24/P23/Q4/a,b

- 4 An examination paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary.

A program is being written to process examination marks.

The five grade boundaries are stored in a global 1D array `GB` of type `INTEGER`, for example:

Index	Value	Comment
1	65	The minimum mark for an A grade.
2	57	The minimum mark for a B grade.
3	43	The minimum mark for a C grade.
4	35	The minimum mark for a D grade.
5	27	The minimum mark for an E grade.

Any paper that achieves a mark within 2 marks of a grade boundary must be checked. Using the given table, a paper with 45 marks would need to be checked.

- (a) The pseudocode algorithm to determine whether a paper should be checked is as shown. The mark for the paper is stored in variable `Mark`. Global variables `Mark`, `Index`, `Upper` and `Lower` are declared as integers.

Complete the pseudocode.

```

FOR Index ← 1 TO .....
    Lower ← GB[Index] - 2
    Upper ← .....
    IF Mark ..... AND Mark ..... THEN
        OUTPUT "Check this paper"
    ENDIF
NEXT Index

```

[4]

- (b) An alternative algorithm to determine if a paper needs to be checked uses a global 1D array `Check`, containing 76 elements of type `BOOLEAN`. The indices of the array are from 0 to 75 (inclusive), corresponding to the range of possible marks.

An element value in `Check` is `TRUE` if the index is within 2 marks of a grade boundary. For example, in the case where the C grade boundary is 43 the corresponding part of the `Check` array would be as follows:

Index	Value
40	FALSE
41	TRUE
42	TRUE
43	TRUE
44	TRUE
45	TRUE
46	FALSE

← The grade boundary for a C grade

- (i) The mark for a given paper is stored in variable `Mark`.

Describe how an algorithm would use the `Check` array to determine whether this paper should be checked.

[2]

- (ii) A procedure `GBInitialise()` will initialise the `Check` array using values from the `GB` array.

Note it can be assumed that the maximum grade boundary value for A is 70 and the minimum value for E is 15.

Write pseudocode for the procedure.

[6]

45. O/N 24/P23/Q8/a,b,c

- 8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Skill level	14, 23, 76	An integer in the range 0 to 100, inclusive.

The programmer has defined a record type to define each character. The record type definition is shown in pseudocode as follows:

```

TYPE CharacterType
    DECLARE Player : INTEGER
    DECLARE Role : STRING
    DECLARE Name : STRING
    DECLARE SkillLevel : INTEGER
ENDTYPE

```

The `Player` field indicates the player to which the character is assigned (1 to 6). This field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data, as follows:

```
DECLARE Character : ARRAY[1:45] OF CharacterType
```

At the start of the game all record fields are initialised, and all `Player` fields are set to 0

The programmer has defined a program module as follows:

Module	Description
<code>Count()</code>	- called with two parameters: - an integer representing a player - a string representing a character role - searches the <code>Character</code> array for characters with the given role that are assigned to the given player - counts the number of assigned characters and sums their total skill level - outputs the result of the search if characters with the given role are found, for example: <pre>"Player 3 has 4 characters with the role of Teacher and the total skill level is 65"</pre> - if no characters with the given role are found, outputs: <pre>"No characters with that role are assigned to this player"</pre>

(a) Complete the pseudocode for module `Count()`.

```

PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)
    ...
ENDPROCEDURE

```

(b) The `Character` array data has been saved in the text file `SaveFile.txt`. Each line of the file contains one element of the array (one record).

New modules are defined:

Module	Description
<code>Extract()</code> (already written)	- called with two parameters: - a string representing a complete line from the text file - an integer representing a field number (see structure below) - returns a string representing the required field
<code>Restore()</code>	- opens the text file <code>SaveFile.txt</code> - reads lines from the file and assigns values to each record in the <code>Character</code> array using data from each line of the file

As a reminder, the record structure is repeated here:

```

TYPE CharacterType
  DECLARE Player : INTEGER           //Field number 1
  DECLARE Role : STRING              //Field number 2
  DECLARE Name : STRING              //Field number 3
  DECLARE SkillLevel : INTEGER       //Field number 4
ENDTYPE

```

Write pseudocode for module `Restore()`.

You must use the module `Extract()`.

[7]

- (c) The game can last for several days and users often find that they have to close and rerun the game program many times in order to complete it.

Describe the benefit of using the file `SaveFile.txt` as described in part (b).

[2]

46. M/J 24/P21/Q1/a

- 1 An algorithm is developed in pseudocode before being coded in a programming language.

- (a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables A, B, C and D.

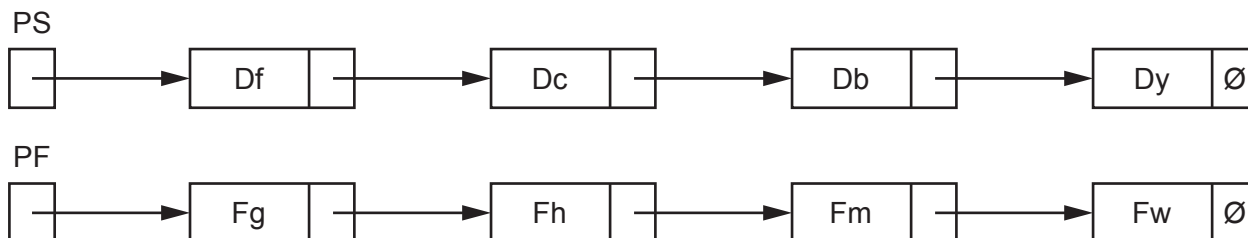
Assignment statement	Data type
<code>A ← LEFT(MyName, 1)</code>	
<code>B ← Total * 2</code>	
<code>C ← INT(ItemCost) / 3</code>	
<code>D ← "Odd OR Even"</code>	

[4]

47. M/J 24/P21/Q3/a,b

- 3 The diagram shows an Abstract Data Type (ADT) representation of a linked list after data items have been added.

- PS is the start pointer.
- PF is the free list pointer.
- Labels Df, Dc, Db and Dy represent the data items of nodes in the list.
- Labels Fg, Fh, Fm and Fw represent the data items of nodes in the free list.
- The symbol $\emptyset$ represents a null pointer.



- (a) Describe the linked list immediately after initialisation, before **any** data items are added.

[3]

- (b) A program will be written to include a linked list to store alphanumeric user IDs.

The design uses two variables and two 1D arrays to implement the linked list.
Each array element contains data of a single data type and **not** a record.

The statements below describe the design.

Complete the statements.

The two variables will be of type

The two variables will be used as to the arrays.

The values stored in the two variables will indicate

The first 1D array will be of type

The first 1D array will be used to

The second 1D array will be of type

The second 1D array will be used to

[5]

48. M/J 24/P21/Q4

4 A global 1D array `Data` contains 100 elements of type integer.

A function `Check()` will:

- total the element values in odd index locations (1, 3, 5 ... 97, 99)
- total the element values in even index locations (2, 4, 6 ... 98, 100)
- return one of three strings 'Odd', 'Even' or 'Same' to indicate which total is the greater, or whether the totals are the same.

Write pseudocode for the function `Check()`.

[6]

49. M/J 24/P21/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
MichaelAday_src.txt	student project file produced by student Michael Aday
MichaelAday_S1.txt	file produced by stage 1
MichaelAday_S2.txt	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
DeleteComment()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: IF X2 > 13 THEN //check if limit exceeded After: IF X2 > 13 THEN

- (b) A second module is defined:

Module	Description
Stage_1()	- called with a parameter of type string representing a student name - creates a new stage 1 file - copies each line from the student's project file to the stage 1 file after removing any comment from each line - does not write blank lines to the stage 1 file - returns the number of lines written to the stage 1 file

Write pseudocode for module Stage_1().

Module DeleteComment() must be used in your solution.

50. M/J 24/P22/Q3/a,b

- 3** A factory needs a program to help manage its production of items.

Data will be stored about each item.

The data for each item will be held in a record structure of type `Component`.

The programmer has started to define the fields that will be needed as shown in the table.

Field	Example value	Comment
Item_Num	123478	a numeric value used as an array index
Reject	FALSE	TRUE if this item has been rejected
Stage	'B'	a letter to indicate the stage of production
Limit_1	13.5	any value in the range 0 to 100 inclusive
Limit_2	26.4	any value in the range 0 to 100 inclusive

- (a) (i)** Write pseudocode to declare the record structure for type `Component`.

[4]

- (ii)** A 1D array `Item` of 2000 elements will store the data for all items.

Write pseudocode to declare the `Item` array.

[2]

- (b)** State **three** benefits of using an array of records to store the data for all items.

1

2

3

[3]

51. M/J 24/P22/Q6/b

- 6** A music player stores music in a digital form and has a display which shows the track being played.

(b) Music is stored as a sequence of digital samples.

Each digital sample is a denary value in the range 0 to 99999999 (8 digits).

The samples are to be stored in a text file. Each sample is converted to a numeric string and 32 samples are concatenated (joined) to form a single line of the text file.

Each numeric string is 8 characters in length; leading '0' characters are added as required.

Example:

Sample	Denary value	String
1	456	"00000456"
2	48	"00000048"
3	37652	"00037652"
		
32	673	"00000673"

The example samples will be stored in the text file as a single line:

"000004560000004800037652...00000673"

- (i)** Identify one drawback of adding leading '0' characters to each numeric string.

[1]

- (ii)** Suggest an alternative method of storing the samples which does **not** involve adding leading '0' characters but which would still allow each individual sample to be extracted.

[1]

- (iii)** State **one** drawback of the alternative method given in part **(b)(ii)**.

[1]

52. M/J 24/P22/Q8/c

- 8** A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

(b) The teacher has defined the first program module as follows:

Module	Description
DeleteSpaces()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: " IF X2 > 13 THEN" After: "IF X2 > 13 THEN"

Complete the pseudocode for module DeleteSpaces().

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
```

```
ENDFUNCTION
```

[6]

(c) Two modules are defined:

Module	Description
DeleteComment() (already written)	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any comment
Stage_2()	- called with two parameters: - a string representing an input file name - a string representing an output file name - copies each line from the input file to the existing output file having first removed all leading spaces and comments from that line - does not write blank lines to the output file - outputs a final message giving the number of blank lines removed

Write pseudocode for module Stage_2().

Modules DeleteComment() and DeleteSpaces() must be used in your solution.

[8]

53. M/J 24/P23/Q3/a

- 3 A record structure is declared to hold data relating to components being produced in a factory:

```
TYPE Component
  DECLARE Item_ID : STRING
  DECLARE Reject : BOOLEAN
  DECLARE Weight : REAL
ENDTYPE
```

The factory normally produces a batch (or set) of 1000 components at a time. A global array is declared to store 1000 records for a batch:

```
DECLARE Batch : ARRAY [1:1000] OF Component
```

Two global variables contain the minimum and maximum acceptable weight for each component. The values represent an inclusive range and are declared as:

```
DECLARE Min, Max : REAL
```

- (a) (i) A program uses a variable `ThisIndex` as the array index to access a record.

Write a pseudocode clause to check whether or **not** the weight of an individual component is within the acceptable range.

[3]

- (ii) When batches of less than 1000 components are processed, it is necessary to indicate that certain elements in the array are unused.

Suggest how an unused array element could be indicated.

[1]

54. M/J 24/P23/Q4/b

- 4 A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:

- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

- (b) The value zero denotes the split between the two parts of the sequence.

The requirement changes and now there may be up to 20 parts.

- (i) Identify a suitable data structure that could be used to store the different total values.

[2]

- (ii) Describe **three** benefits of using the data structure given in part (b)(i).

1

2

3

[3]



55. M/J 24/P23/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

The program analyses a student project and extracts information about each module that is defined (each procedure or function). This information is stored in a global 2D array `ModInfo` of type string.

A module header is the first line of a module definition and starts with either of the keywords `PROCEDURE` or `FUNCTION`.

An example of part of the array is given below. Row 10 of the array shows that a procedure header occurs on line 27 and row 11 shows that a function header occurs on line 35. "P" represents a procedure and "F" represents a function:

	x = 1	x = 2	x = 3
<code>ModInfo[10, x]</code>	"27"	"P"	"MyProc(Z : CHAR)"
<code>ModInfo[11, x]</code>	"35"	"F"	"MyFun(Y : CHAR) RETURNS BOOLEAN"

The string stored in column 3 is called the module description. This is the module header **without** the keyword.

A valid module header will:

- be at least 13 characters long
- start with the keyword `PROCEDURE` or `FUNCTION`. The keyword may appear in either upper or lower case (or a mix of both) and **must** be followed by a space character.

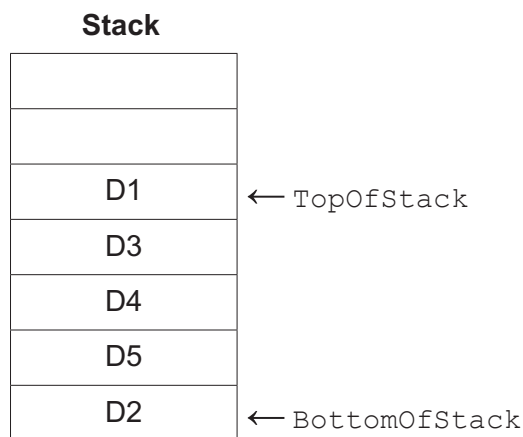
The teacher has defined the first program module as follows:

Module	Description
<code>Header()</code>	• called with a parameter of type string representing a line of pseudocode • if the line is a valid procedure header, returns a string: "P<Module description>" • if the line is a valid function header, returns a string: "F<Module description>" • otherwise, returns an empty string For example, given the string: "FUNCTION Zap(X : INTEGER) RETURNS CHAR" <code>Header()</code> returns the string: "FZap(X : INTEGER) RETURNS CHAR"



The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.

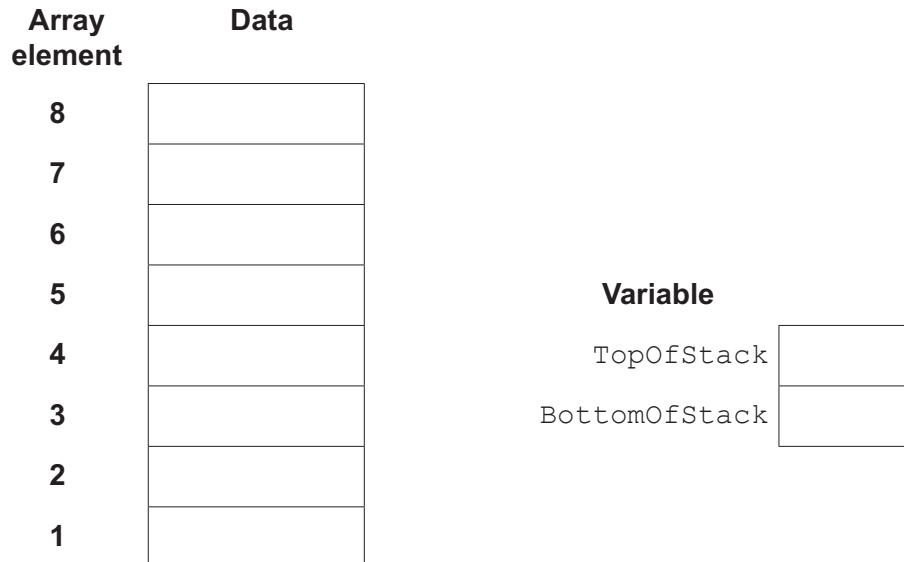


- (a) The stack is implemented using two variables and a 1D array of 8 elements as shown.

The variables are used to reference individual elements of the array, in such a way that:

- the array is filled from the lowest indexed element towards the highest
- all the elements of the array are available for the stack.

Complete the diagram to represent the state of the stack as shown above.



[3]

- (b) A function `Push()` will add a value onto the stack by manipulating the array and variables in part (a).

Before adding a value onto the stack, the algorithm will check that space is available.

If the value is added to the stack, the function will return `TRUE`, otherwise it will return `FALSE`.

The algorithm is expressed in five steps.

Complete the steps.

1. If then return FALSE
2. Otherwise TopOfStack
3. Use TopOfStack as an to the array.
4. Set the element at this to the being added.
5. Return

[5]

58. O/N 23/P21/Q4/a,b

- 4 A global array is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150] OF STRING
```

A function TooMany() will:

1. take two parameters:
 - a string (the search string)
 - an integer (the maximum value)
2. count the number of strings in the array that exactly match the search string
3. return TRUE if the count is greater than the maximum value, otherwise will return FALSE

- (a) Write pseudocode for the function TooMany() .

[6]

- (b) The global array is changed to a 2D array, organised as 150 rows by 2 columns. It is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150, 1:2] OF STRING
```

The algorithm for the function in **part (a)** is changed. Strings will only be counted if **both** of the following conditions are true:

- The current row is an even number.
- The search string exactly matches the value in **either** column.

Write pseudocode to check these conditions.

Assume that the row index is contained in variable Row and the search string in variable Search.

[3]

59. O/N 23/P21/Q8/a,b

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string containing a message - an integer containing a port number - transmits the message using the given port
<code>SendFile()</code>	- takes three parameters: - a string containing a text file name - a string containing a Destination ID - an integer containing a Port number - transmits the file one line at a time - transmits a final message with data string "*****"

(a) Write pseudocode for module `SendFile()`.

Assume:

- module `Transmit()` has already been written and is used to transmit a message
- the value of `MyID` may be used as `SourceID`
- the file specified contains no blank lines
- the file specified does not contain the line "*****"

[7]

(b) Module `SendFile()` is used to copy a file from one computer to another.

A module within the program running on the destination computer will receive the data and write it to a new file.

Explain why module `SendFile()` transmits the message with data string "*****" after the last line of the file.

[2]

60. O/N 23/P22/Q1/c

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer		
3	The number of items in the order		
TRUE	To indicate whether this is a new customer		
15.75	The deposit paid by the customer		

[4]

(c) The data that needs to be stored for each customer order in part (a) is not all of the same type.

Describe an effective way of storing this data for many customer orders while the program is running.

[3]

61. O/N 23/P22/Q3/a,b

- 3 The diagram represents a linked list Abstract Data Type (ADT).

- Ptr1 is the start pointer. Ptr2 is the free list pointer.
- Labels D40, D32, D11 and D100 represent the data items of nodes in the list.
- Labels F1, F2, F3 and F4 represent the data items of nodes in the free list.
- The symbol $\emptyset$ represents a null pointer.

Ptr1



Ptr2



- (a) The linked list is implemented using two variables and two 1D arrays as shown.

The pointer variables and the elements of the Pointer array store the indices (index numbers) of elements in the Data array.

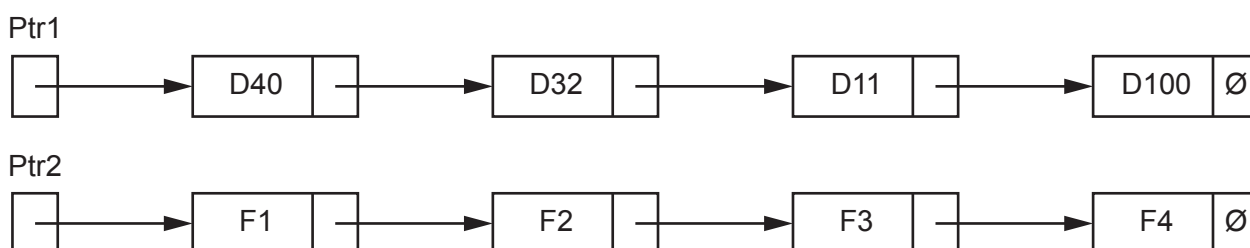
Complete the diagram to show how the linked list as shown above may be represented using the variables and arrays.

Variable		Value
Start_Pointer		
Free_List_Pointer		5

Index	Data array	Pointer array
1	D32	2
2		3
3		
4	D40	
5		
6	F2	7
7		
8		

[5]

- (b) The original linked list is to be modified. A new node D6 is inserted between nodes D32 and D11.



The algorithm required is expressed in four steps as shown.

Complete the steps.

1. Assign the data item to
2. Set the of this node to point to
3. Set Ptr2 to point to
4. Set pointer of to point to

[4]

62. O/N 23/P22/Q6/a,b(iii)

6 (a) A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, `TestData.001`, `TestData.002` and `TestData.003`.

Each file will contain a single line. For example, file `TestData.002` would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

[6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part (a).

`CheckFiles()` will take a string representing a file name and return the number of files found.

(iii) State the file mode that should be used in `CheckFiles()`.

[1]

63. O/N 23/P22/Q8/a,c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.

Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.

When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.

Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	Column 2	Column 3
Row 1	100	199	1
Row 2	200	259	2
Row 3	-1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to -1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
<code>GetPort()</code>	- takes a <code>DestinationID</code> as a parameter of type string - searches for the range corresponding to the <code>DestinationID</code> in the array - returns the port number, or returns -1 if no corresponding range is found

(a) Write pseudocode for module `GetPort()`.

Assume `DestinationID` contains a valid three-digit string.

[7]

(b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
<code>StackMsg()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing the stack number - adds the message to the required stack - returns <code>TRUE</code> if the message is added to the required stack, otherwise returns <code>FALSE</code>
<code>ProcessMsg()</code>	- takes a message as a parameter of type string - ignores any message with a zero-length data field - extract the <code>DestinationID</code> from the message - checks whether the <code>DestinationID</code> is this computer or whether the message is to be forwarded - uses <code>StackMsg()</code> to add the message to the appropriate stack - outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

[7]

- (c) The program contains a module `GetFile()` which receives text files sent from another computer.

Lines from the file are sent one at a time. Each message contains one line and `ProcessMsg()` from part (b) adds each message as it is received onto stack 1.

Module `GetFile()` removes messages from stack 1 and writes the data to a text file.

There is a problem. Under certain circumstances, the received file does not appear as expected.

Assume that while a file is being received `ProcessMsg()` receives only messages containing lines from the file.

- (i) Describe the circumstances and explain the problem.

Circumstances

Explanation

[3]

- (ii) Suggest a more appropriate Abstract Data Type that could be used to store the messages that would not have the same problem.

[1]

64. O/N 23/P23/Q1/a

- 1 A program is being developed in pseudocode before being converted into a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving the data type that should be used to declare the variable underlined in each assignment statement.

Assignment statement	Data type
<u>MyVar1</u> $\leftarrow$ Total1 / Total2	
<u>MyVar2</u> $\leftarrow$ 27/10/2023	
<u>MyVar3</u> $\leftarrow$ "Sum1 / Sum2"	
<u>MyVar4</u> $\leftarrow$ Result1 AND Result2	

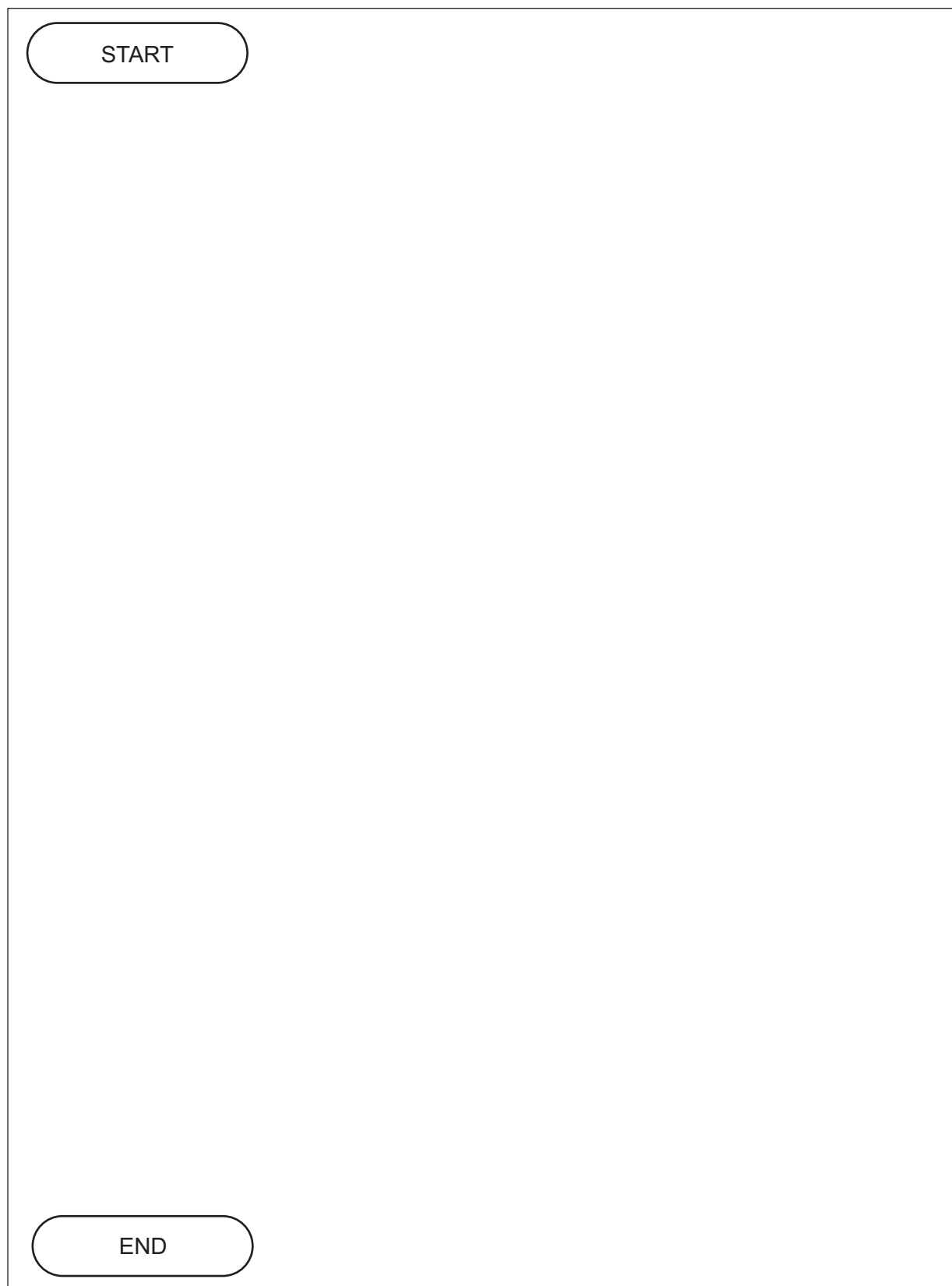
[4]

65. O/N 23/P23/Q2/b,c

- 2 Data is a 1D array of integers, containing 30 elements. All element values are unique.

(a) An algorithm will output the index of the element with the smallest value.

Draw a program flowchart to represent the algorithm.



[5]

(b) The 30 data values could have been stored in separate variables rather than in an array.

Explain the benefits of using an array when designing a solution to part **(a)**.

[2]

- (c) The requirement changes. Array `Data` needs to hold 120 elements and each value may include a decimal place.

Write a pseudocode statement to declare the modified array.

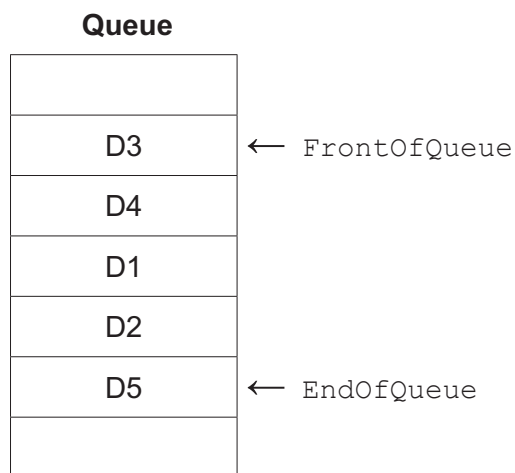
[2]

66. O/N 23/P23/Q3/a,b

- 3 The diagram represents a queue Abstract Data Type (ADT).

The organisation of this queue may be summarised as follows:

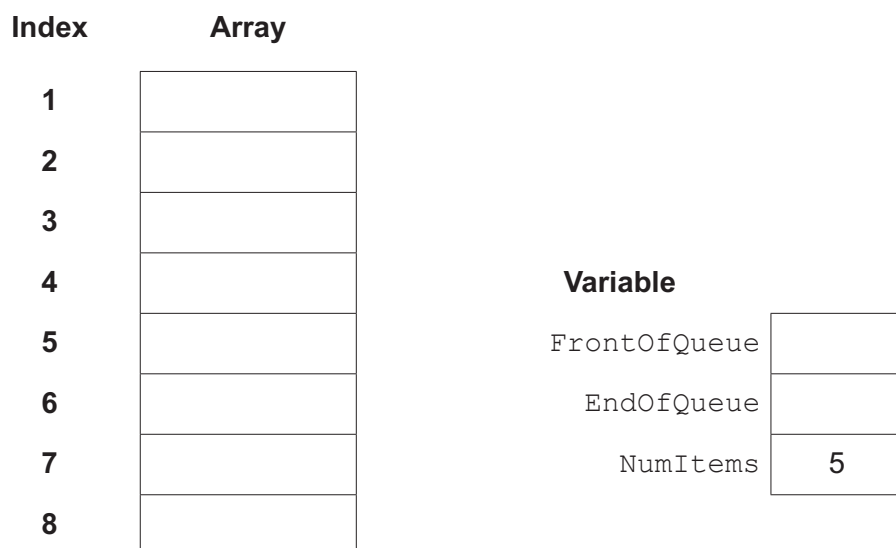
- The `FrontOfQueue` pointer points to the next data item to be removed.
- The `EndOfQueue` pointer points to the last data item added.



The queue is implemented using three variables and a 1D array of eight elements as shown. The variable `NumItems` stores the number of items in the queue.

The pointer variables store indices (index numbers) of the array.

- (a) Complete the diagram to represent the state of the queue as shown above.



[3]

- (b) A module `AddTo()` will add a value to the queue by manipulating the array and variables in part (a).

The queue implementation is circular. When pointers reach the end of the queue, they will 'wrap around' to the beginning.

Before a value can be added to the queue, it is necessary to check the queue is not full.

The algorithm to add a value to the queue is expressed in six steps.

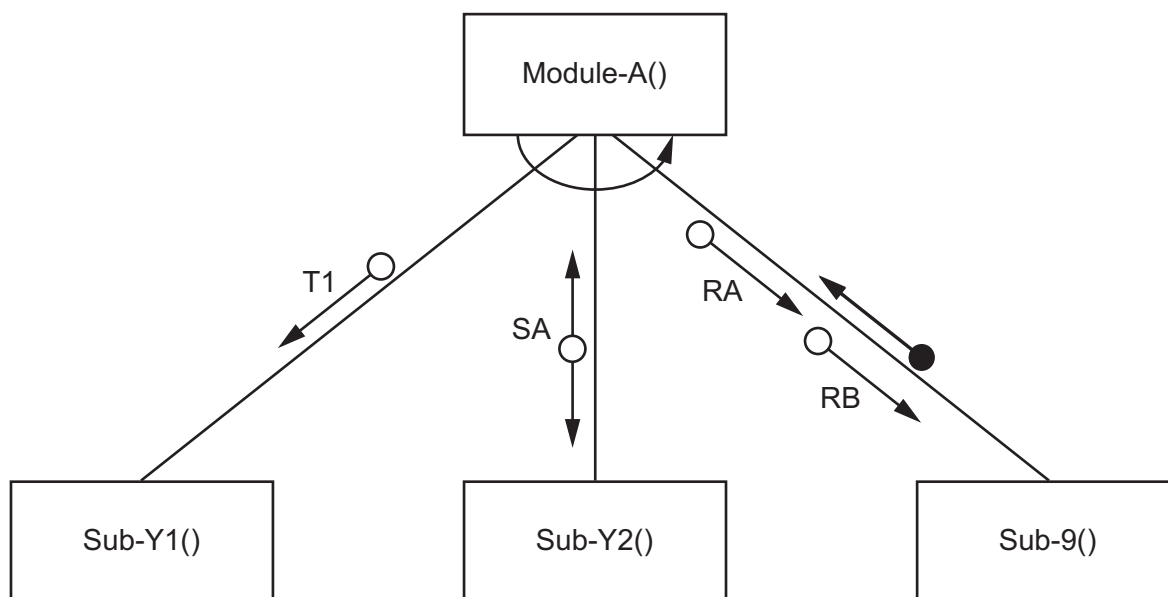
Complete the steps.

1. If NumItems then jump to step 6.
2. Increment
3. If then set EndOfQueue to
4. Increment
5. Set the at the index stored in to the being added.
6. Stop.

[6]

67. O/N 23/P23/Q7/b(i)

- 7 A structure chart shows the modular structure of a program:



- (b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

- (i) Write pseudocode to define `MyType`.

[3]

68. O/N 23/P23/Q8/a

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

`<STX>"454202Hello Jack"<ETX>`

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by <code>GetData()</code> to the file • repeats until the data field is "*****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

- (a) Write pseudocode for module `ReceiveFile()`.

Module `GetData()` has already been written and must be used.

[7]

69. M/J 23/P21/Q3/a

- 3** Customers collect points every time they make a purchase at a store.

A program is used to manage the points system and the table lists some of the information stored for one customer.

Information	Data type required
Name	String
Number of points collected	Integer
Date of birth	Date

- (a) (i)** Identify a suitable structure for storing the information for one customer. Explain the advantage of using this structure.

Structure

Advantage

[4]

- (ii)** Describe a data structure that could be used to store the information for **all** customers.

[2]

70. M/J 23/P21/Q6

- 6 A video-conferencing program supports up to six users. Speech from each user is sampled and digitised (converted from analogue to digital). Digitised values are stored in array `Sample`.

The array `Sample` consists of 6 rows by 128 columns and is of type integer. Each row contains 128 digitised sound samples from one user.

The digitised sound samples from each user are to be processed to produce a single value which will be stored in a 1D array `Result` of type integer. This process will be implemented by procedure `Mix()`.

A procedure `Mix()` will:

- calculate the average of each of the 6 sound samples in a column
- ignore sound sample values of 10 or less
- store the average value in the corresponding position in `Result`
- repeat for each column in array `Sample`

The diagram uses example values to illustrate the process:

		1	2	3	...	126	127	128
Sample:	1	20	20	20		30	30	2
	2	20	20	30		50	30	3
	3	20	20	40		40	40	4
	4	20	20	50		40	50	20
	5	20	3	5		6	60	4
	6	20	4	2		4	70	30
		↓ ↓ ↓			⬇		↓ ↓ ↓	
Result:		20	20	35		40	46	25

Write pseudocode for procedure `Mix()`.

Assume `Sample` and `Result` are global.

[6]

71. M/J 23/P21/Q8/b,d,e

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

(b) A new module has been defined:

Module	Description
<code>CheckNewItem()</code>	- called with a parameter of type string representing a line of item information - checks to see whether an item with the same <code>ItemNum</code> already exists in the file - returns <code>TRUE</code> if the <code>ItemNum</code> is not already in the file, otherwise returns <code>FALSE</code>

Write **efficient** pseudocode for module `CheckNewItem()`.

[7]

(d) A new module `AddItem()` will be used to add information to the `Stock.txt` file.

State the file mode that should be used for the algorithm within this module.

[1]

(e) A new module `FindItem()` searches for a given item in the `Stock.txt` file, which is already organised in ascending order of `ItemNum`.

Describe how this organisation may improve the efficiency of the algorithm.

[3]

72. M/J 23/P22/Q1/c

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75    // set postal cost for item to $3.75
    ItemStatus ← "Valid"    // item can be sent
ENDIF
```

- (c) Give the **appropriate** data types for the following variables:

ValidAddress

ItemPostalCost

ItemStatus

[3]

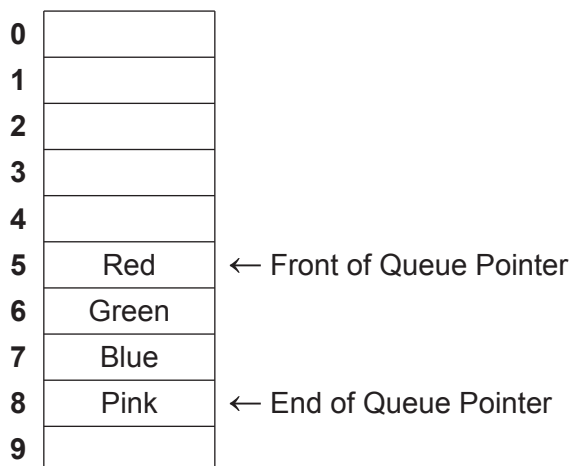
73. M/J 23/P22/Q3/a

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

- (a) The diagram below represents an Abstract Data Type (ADT) implementation of the queue. Each data item is stored in a separate location in the data structure. During initial design, the queue is limited to holding a maximum of 10 data items.

The operation of this queue may be summarised as follows:

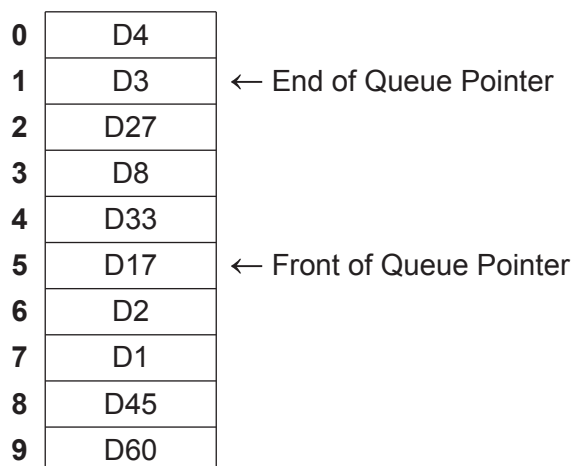
- The Front of Queue Pointer points to the next data item to be removed.
- The End of Queue Pointer points to the last data item added.
- The queue is circular so that locations can be reused.



- (i) Describe how the data items Orange and Yellow are added to the queue shown in the diagram.

[4]

- (ii) The following diagram shows the state of the queue after several operations have been performed. All queue locations have been used at least once.



State the number of data items in the queue.

[1]

74. M/J 23/P22/Q8/b

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

(b) A new module is defined as follows:

Module	Description
<code>AddItem()</code>	- called with a parameter of type string representing valid information for a new item that is not currently in the <code>Stock.txt</code> file - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> and adds the new item information at the appropriate place in the <code>NewStock.txt</code> file

As a reminder, the file `Stock.txt` is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

Write pseudocode for module `AddItem()`.

75. M/J 23/P23/Q1/a,c

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```

10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100  FOR PCount ← 0 TO 49
101      Sheet4[1, PCount] ← 0
102      Sheet4[2, PCount] ← 47
103  NEXT PCount

```

- (a) The pseudocode contains references to an array.

Complete the table by writing the answer for each row.

Answer

The dimension of the array	
The name of the variable used as an array index	
The number of elements in the array	

[3]

- (c) The pseudocode does not include a declaration for PCount.

State the data type that should be used in the declaration.

[1]

76. M/J 23/P23/Q3/a,b(i)

- 3 A program processes data using a stack. The data is copied to a text file before the program ends.

- (a) The following diagram shows the current state of the stack.

The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.
- The stack grows upwards when items are added.

Stack		Pointer
Memory location	Value	
506		
505	WWW	← TopOfStack
504	YYY	
503	XXX	
502	ZZZ	
501	NNN	
500	PPP	← BottomOfStack

- (i) An error will be generated if an attempt is made to POP a value when the stack is empty.

State the maximum number of consecutive POP operations that could be performed on the stack shown above **before** an error is generated.

[1]

- (ii) The following operations are performed:

1. POP and store value in variable `Data1`
2. POP and store value in variable `Data2`
3. PUSH value AAA
4. PUSH value BBB
5. POP and discard value
6. POP and store value in variable `Data2`

Complete the diagram to show the state of the stack and the variables **after** the given operations have been performed.

Stack		Pointer
Memory location	Value	
506		
505		
504		
503		
502		
501		
500		

Variable	Value
<code>Data1</code>	
<code>Data2</code>	

[4]

- (b) The data is copied to a text file before the program ends.

- (i) State an advantage of writing the data from the stack to a text file before the program ends.

[1]

77. M/J 23/P23/Q8/a,b

- 8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
SupplierCode	3 alphabetic characters	code to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
<code>ChangeSupp()</code>	- called with two parameters <code>Code1</code> and <code>Code2</code> of type string that represent valid supplier codes - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> where any reference to <code>Code1</code> is replaced by <code>Code2</code> - returns a count of the number of items that have had their supplier code changed

- (a) Write pseudocode for module `ChangeSupp()`.

[8]

(b) A new module is required:

Module	Description
Report_1()	- takes a parameter of type string that represents a SupplierCode - searches the Stock.txt file for each line of item information that contains the given SupplierCode - produces a formatted report of items for the given SupplierCode, for example, for supplier DRG, the output could be: <pre> Report for Supplier: DRG Item Description 1234 USB Printer Cable 3m 1273 32GB USB Flash Drive 1350 Mouse Mat 320 x 240mm Number of items listed: 3 </pre>

Write pseudocode for module Report_1().

[6]

78. O/N 22/P21/Q1/b

(b) The following pseudocode is part of a program that stores names and test marks for use in other parts of the program.

```

DECLARE Name1, Name2, Name3 : STRING
DECLARE Mark1, Mark2, Mark3 : INTEGER
INPUT Name1
INPUT Mark1
INPUT Name2
INPUT Mark2
INPUT Name3
INPUT Mark3

```

(i) The pseudocode needs to be changed to allow for data to be stored for up to 30 students.

Explain why it would be good practice to use arrays to store the data.

[3]

(ii) The following pseudocode statement includes array references:

```
OUTPUT "Student ", Name[Count], " scored ", Mark[Count]
```

State the purpose of the variable Count and give its data type.

Purpose

Data type

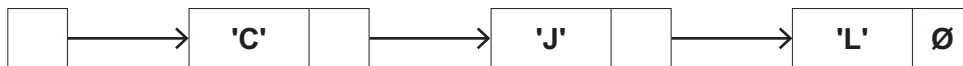
[2]

79. O/N 22/P21/Q4/a,b

- 4 (a) The following diagram shows an Abstract Data Type (ADT) representation of an ordered linked list. The data item stored in each node is a single character. The data will be accessed in alphabetical order.

The symbol $\emptyset$ represents a null pointer.

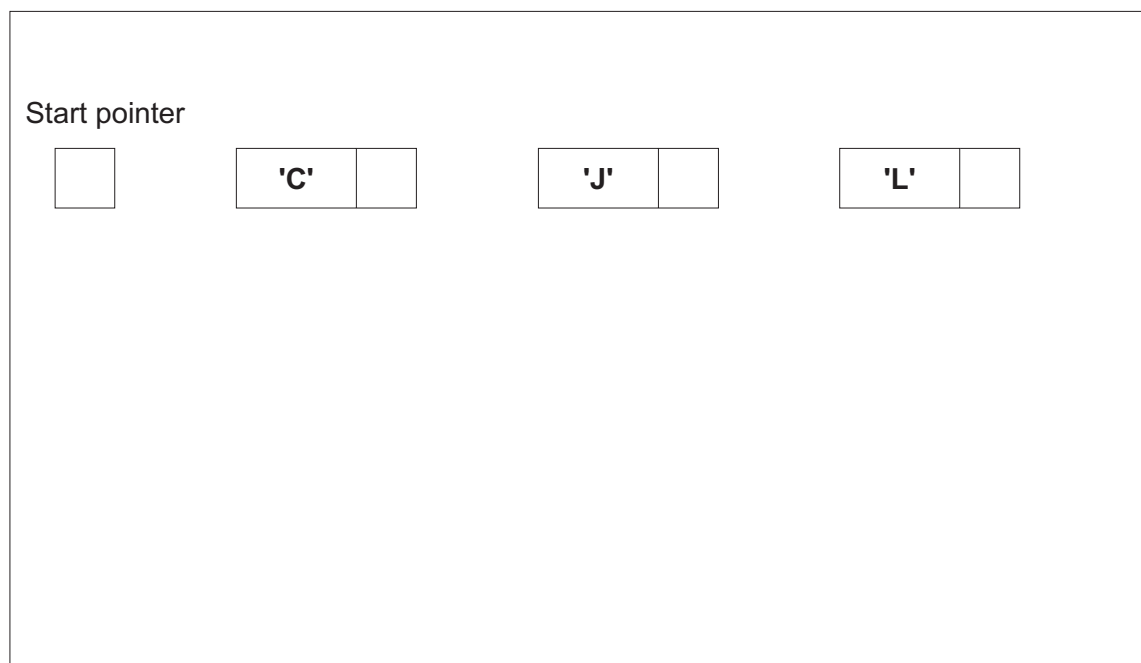
Start pointer



- (i) Nodes with data 'A' and 'K' are added to the linked list. Nodes with data 'J' and 'L' are deleted.

After the changes, the data items still need to be accessed in alphabetical order.

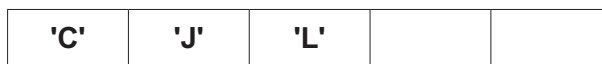
Complete the diagram to show the new state of the linked list.



[4]

- (ii) The original data could have been stored in a 1D array in which each element stores a character.

For example:



Explain the advantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

[2]

- (iii) Explain the disadvantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

[2]

- (b) A program will store data using a linked list like the one shown in **part (a)**.

Explain how the linked list can be implemented.

[4]

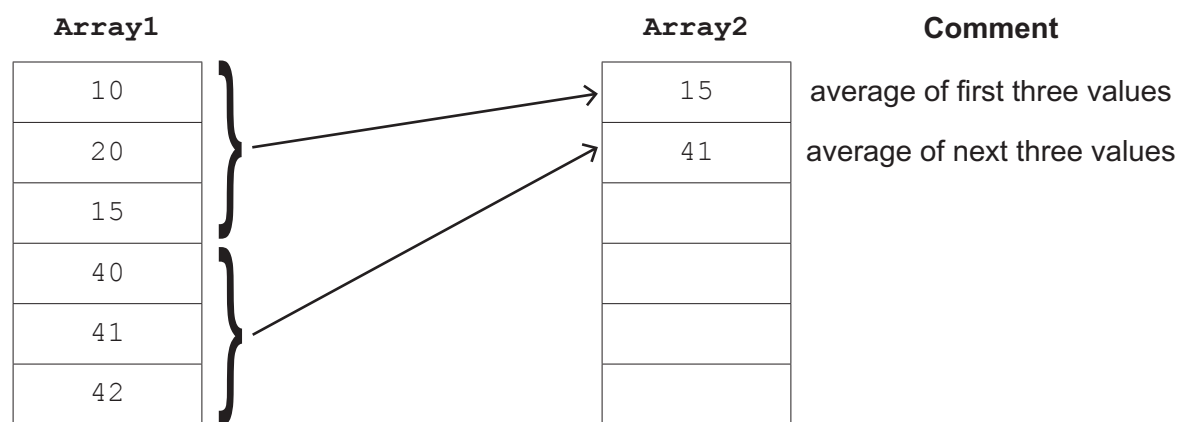
80. O/N 22/P21/Q5

- 5 A program uses two 1D arrays of type integer. `Array1` contains 600 elements and `Array2` contains 200 elements.

`Array1` contains sample values read from a sensor. The sensor always takes three consecutive samples and all of these values are stored in `Array1`.

A procedure `Summarise()` will calculate the average of three consecutive values from `Array1` and write the result to `Array2`. This will be repeated for all values in `Array1`.

The diagram below illustrates the process for the first six entries in `Array1`.



Write pseudocode for the procedure `Summarise()`.

[5]

81. O/N 22/P21/Q8/a,c

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	• takes the name of an existing project file as a parameter of type string • returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	• takes two parameters: ◦ the name of a project file as a string ◦ the maximum number of errors as an integer • uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid • calls <code>CheckLine()</code> for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (c) Write pseudocode for module `CountErrors()`. Assume `CheckFile()` and `CheckLine()` have been written and can be used in your solution.

[8]

82. O/N 22/P22/Q3/a,b,c

- 3 A stack is used in a program to store string data which needs to be accessed in several modules.

- (a) A stack is an example of an Abstract Data Type (ADT).

Identify **one other** example of an ADT **and** describe its main features.

Example

Features

[3]

- (b) Explain how the stack can be implemented using an array.

[5]

- (c) A second stack is used in the program. The diagram below shows the initial state of this stack. Value X is at the top of the stack and was the last item added.

Upper-case letters are used to represent different data values.

Stack operations are performed in three groups as follows:

Group 1:

PUSH D
PUSH E

Group 2:

POP
POP
POP

Group 3:

PUSH A
PUSH B
POP
PUSH C

Complete the diagram to show the state of the stack **after** each group of operations has been performed.

Include the current stack pointer (SP) **after** each group.

Memory location	Initial state		After Group 1	After Group 2	After Group 3
957					
956					
955					
954					
953	X	←SP			
952	Y				
951	Z				
950	P				

[5]

83. O/N 22/P22/Q7/a,b,c

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	"Undeclared variable"
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There may be less than 500 error numbers so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present, for example, there may be no error code 31.

The teacher has defined two program modules as follows:

Module	Description
<code>OutputError()</code>	• takes two parameters as integers: ◦ a line number in the student's program ◦ an error number • searches for the error number in the <code>ErrCode</code> array: ◦ if found, outputs the corresponding error description and the line number, for example: "Bracket mismatch on line 34" ◦ if not found, outputs the line number and a warning, for example: "Unknown error on line 34"
<code>SortArrays()</code>	sorts the arrays into ascending order of <code>ErrCode</code>

(a) Write **efficient** pseudocode for module `OutputError()`.

[6]

(b) Write an **efficient** bubble sort algorithm in pseudocode for module `SortArrays()`.

[8]

(c) Two 1D arrays were described at the beginning of the question. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The two arrays will be replaced by a single array. A user-defined data type (record structure) has been declared as follows:

```
TYPE ErrorRec
    DECLARE ErrCode : STRING
    DECLARE ErrText : STRING
ENDTYPE
```

- (i) State the error in the record declaration.

[1]

- (ii) State **two** benefits of using the single array of the user-defined data type.

1

2

[2]

- (iii) Write the declaration for the single array in pseudocode.

[1]

84. O/N 22/P23/Q1/a

- 1 A program is required for a shopping website.

- (a) Part of the program requires four variables. The following table describes the use of each variable.

Complete the table by adding the most appropriate data type for each variable.

Variable use	Data type
Store the number of days in the current month	
Store the first letter of the customer's first name	
Store an indication of whether a year is a leap year	
Store the average amount spent per customer visit	

[4]

85. O/N 22/P23/Q4/a,b

- 4 (a)** A program contains a 1D array `DataItem` with 100 elements.

State the **one additional** piece of information required before the array can be declared.

[1]

- (b)** A programmer decides to implement a queue Abstract Data Type (ADT) in order to store characters received from the keyboard. The queue will need to store at least 10 characters and will be implemented using an array.

- (i)** Describe **two** operations that are typically required when implementing a queue. State the check that must be carried out before each operation can be completed.

Operation 1

Check 1

Operation 2

Check 2

[4]

- (ii)** Describe the declaration and initialisation of the variables and data structures used to implement the queue.

[5]

86. O/N 22/P23/Q5/b(i)

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
  DECLARE StockID : STRING
  DECLARE Description : STRING
  DECLARE Cost : REAL
ENDTYPE
```

- (i) A variable `LineData` of type `StockItem` is declared.

Write the pseudocode statement to assign the value 12.99 to the `Cost` field of `LineData`.

[1]

87. O/N 22/P23/Q7/a,b(i)

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	" "
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description
<code>OutputRange()</code>	• Prompts for input of two error numbers • Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description • Outputs a warning message when the error description is missing as for error number 50 in the example • Outputs a suitable header and a final count of error numbers found Output based on the example array data above: <pre>List of error numbers from 1 to 60 10 : Invalid identifier name 20 : Bracket mismatch 50 : Error Text Missing 60 : Type mismatch in assignment 4 error numbers output</pre>

- (a) Write pseudocode for module `OutputRange()`. Assume that the two numbers input represent a valid error number range.

(b) (i) Two additional modules are defined:

Module	Description
<code>SortArrays()</code>	Sorts the arrays into ascending order of <code>ErrCode</code>
<code>AddError()</code>	- Takes two parameters: - an error number as an integer - an error description as a string - Writes the error number and error description to the first unused element of the two arrays. Ensures the <code>ErrCode</code> array is still in ascending order - Returns the number of unused elements after the new error number has been added - Returns <code>-1</code> if the new error number could not be added

Write pseudocode for the module `AddError()`. Assume that the error code is **not** already in the `ErrCode` array.

[6]

88. O/N 21/P21/Q1/b

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

89. O/N 21/P21/Q3/a,b,c

- 3 (a) The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

- (i) Describe how “Octopus” is added to the given queue.

[2]

- (ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

[2]

- (iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

[1]

- (b) Some operations are carried out on the original queue given in **part (a)**.

- (i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

(ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

[3]

90. O/N 21/P21/Q5/a,c

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (a) Suggest a suitable separator character **and** give a reason for your choice.

Character

Reason

[2]

- (c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
 - add the value of the array element to the existing text file `LogFile`
 - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

[7]

91. O/N 21/P21/Q6/a,b

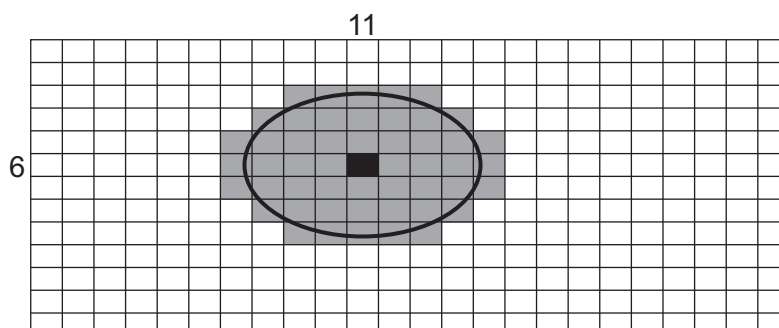
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
SetRow() (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
SearchInRow()	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
SearchInCol()	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

[5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

[8]

92. O/N 21/P22/Q1/b

(b) The following pseudocode assigns a value to an element of an array:

```
ThisArray[n] ← 42
```

Complete the following table by writing the answer for each row.

Answer

The number of dimensions of <code>ThisArray</code>	
The technical terms for minimum and maximum values that the variable <code>n</code> may take	
The technical term for the variable <code>n</code> in the pseudocode statement	

[3]

93. O/N 21/P22/Q3/a,b

3 A programmer is writing a program to help manage clubs in a school.

Data will be stored about each student in the school and each student may join up to three clubs.

The data will be held in a record structure of type `Student`.

The programmer has started to define the fields that will be needed as shown in the following table.

Field	Typical value	Comment
<code>StudentID</code>	"CF1234"	Unique to each student
<code>Email</code>	"Carmen47@xyzmail.com"	Contains letters, numbers and certain symbols
<code>Club_1</code>	1	Any value in the range 1 to 99 inclusive
<code>Club_2</code>	14	Any value in the range 1 to 99 inclusive
<code>Club_3</code>	27	Any value in the range 1 to 99 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Student`.

[3]

(ii) A 1D array `Membership` containing 3000 elements will be used to store the student data.

Write pseudocode to declare the `Membership` array.

[2]

(iii) Some of the elements of the array will be unused.

Give an **appropriate** way of indicating an unused array element.

[1]

(iv) Some students are members of less than three clubs.

State **one** way of indicating an unused club field.

[1]

(b) A procedure `GetIDs()` will:

- prompt and input the number of a club
- output the `StudentID` of all the students who are members of that club
- output a count of all students in the given club.

Write pseudocode for the procedure `GetIDs()`.

[7]

94. O/N 21/P22/Q5/a,b

- 5 A company has several departments. Each department stores the name, email address and the status of each employee in that department in its own text file. All text files have the same format.

Employee details are stored as three separate data strings on three consecutive lines of the file. An example of the first six lines of one of the files is as follows:

File line	Comment
1	First employee name
2	First email address
3	First employee status
4	Second employee name
5	Second email address
6	Second employee status

A procedure `MakeNewFile()` will:

- take three parameters as strings:
 - an existing file name
 - a new file name
 - a search status value
- create a new text file using the new file name
- write all employee details to the new file where the employee status is **not** equal to the search status value
- count the number of sets of employee details that were in the original file
- count the number of sets of employee details that were written to the new file
- produce a summary output.

An example summary output is as follows:

```
File Marketing contained 54 employee details
52 employee sets of details were written to file NewMarketingList
```

- (a) Write pseudocode for the procedure `MakeNewFile()`.

[7]

- (b) An alternative format could be used for storing the data.

A text file will still be used.

- (i) Describe the alternative format.

[1]

- (ii) State **one** advantage **and one** disadvantage of the alternative format.

Advantage

Disadvantage

[2]

95. O/N 21/P22/Q6/a

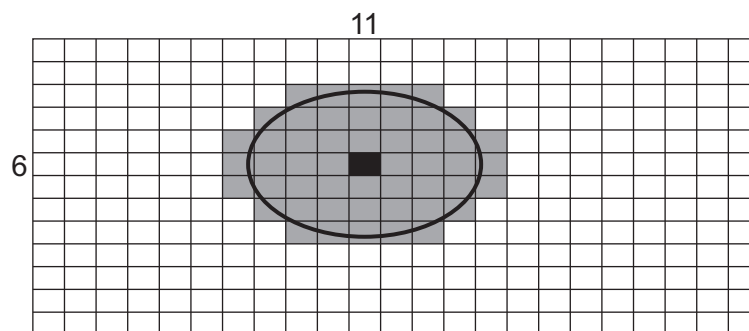
- 6** A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will be set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	- Searches for the first row that has an array element set to 1 - Returns the index of that row (1 is the first row) - Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	- Searches for the last row that has an array element set to 1 - Returns the index of that row - Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	- Searches for the first column that has an array element set to 1 - Returns the index of that column (1 is the first column) - Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	- Searches for the last column that has an array element set to 1 - Returns the index of that column - Returns -1 if there are no elements set to 1

- (a)** Write efficient pseudocode for the module `FirstRowSet()`.

[7]

96. O/N 21/P23/Q1/b

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

97. O/N 21/P23/Q3/a,b,c

- 3 (a) The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

- (i) Describe how “Octopus” is added to the given queue.

[2]

- (ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

[2]

- (iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

[1]

- (b) Some operations are carried out on the original queue given in **part (a)**.

(i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

(ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

[3]

98. O/N 21/P23/Q5/a,c

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

"CJL404Lib01<separator>08:30, June 01, 2021"

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (a) Suggest a suitable separator character **and** give a reason for your choice.

Character

Reason

[2]

- (c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
 - add the value of the array element to the existing text file `LogFile`
 - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

[7]

99. O/N 21/P23/Q6/a,b

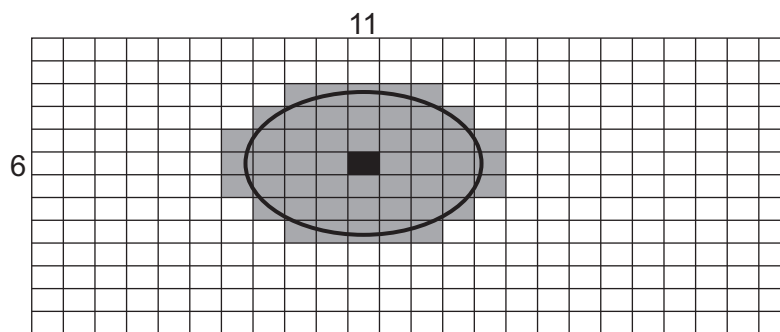
- 6** A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
<code>SetRow()</code> (generates test data)	- Called with three parameters of type <code>INTEGER</code>: - a row number - the number of pixels to be skipped starting from column 1 - the number of pixels that should be set to 1 - Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
<code>SearchInRow()</code>	- Takes two parameters of type <code>INTEGER</code>: - a row number - a start column (1 or 1280) - Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 - Returns the column number of the first element in the given row that is set to 1 - Returns -1 if no element is set to 1
<code>SearchInCol()</code>	- Takes two parameters of type <code>INTEGER</code>: - a column number - a start row (1 or 800) - Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 - Returns the row number of the first element in the given column that is set to 1 - Returns -1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

[5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

[8]

100. M/J 21/P21/Q1/c

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure

Justification

[3]

101. M/J 21/P21/Q3/a,b

- 3 (a)** A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

- (i) Explain how the program could use text files to speed up the process.

[2]

- (ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

[1]

- (iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

[2]

- (b)** A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

[5]

102. M/J 21/P21/Q5

- 5** A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

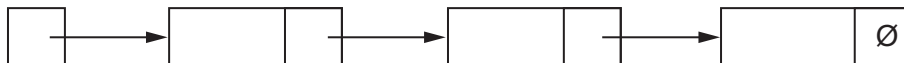
[8]

103. M/J 21/P21/Q6/a,b

6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



(a) Explain how a node containing data value B is added to the list in alphabetic sequence.

[4]

(b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

[2]

104. M/J 21/P22/Q1/a

also Unit 11 (4 marks); this unit 4 marks

1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Name	"Catherine"	
Index	100	
Modified	FALSE	
Holiday	25/12/2020	

[4]

(ii) Evaluate each expression in the following table by using the initial data values shown in **part (a)(i)**.

Expression	Evaluates to
Modified OR Index > 100	
LENGTH("Student: " & Name)	
INT(Index + 2.9)	
MID(Name, 1, 3)	

[4]

105. M/J 21/P22/Q3/a,b,c,d

3 The following diagram represents an Abstract Data Type (ADT).



(a) Identify this type of ADT.

[1]

(b) Give the technical term for the item labelled **A** in the diagram.

[1]

(c) Give the technical term for the item labelled **B** in the diagram.

Explain the meaning of the value given to this item.

Term

Meaning

[2]

(d) Complete the diagram to show the ADT after the data has been sorted in alphabetical order.



[2]

106. M/J 21/P22/Q5/a

also Unit 12 (3 marks); this unit 6 marks

5 (a) A student is learning about arrays.

She wants to write a program to:

- declare a 1D array `RNum` of 100 elements of type `INTEGER`
- assign each element a random value in the range 1 to 200 inclusive
- count and output how many numbers generated were between 66 and 173 inclusive.

(i) Write pseudocode to represent the algorithm.

[6]

(ii) The student decides to modify the algorithm so that each element of the array will contain a unique value.

Describe the changes that the student needs to make to the algorithm.

[3]

107. M/J 21/P22/Q6

6 A procedure `CountVowels()` will:

- be called with a string containing alphanumeric characters as its parameter
- count and output the number of occurrences of each vowel (a, e, i, o, u) in the string
- count and output the number of occurrences of the other alphabetic characters (as a single total).

The string may contain both upper and lower case characters.

Each count value will be stored in a unique element of a global 1D array `CharCount` of type `INTEGER`. The array will contain six elements.

Write pseudocode for the procedure `CountVowels()`.

[8]

108. M/J 21/P22/Q8/a

- 8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as its parameter, representing the number of a word in <code>FNString</code> • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: <code>GetStart(3)</code> applied to "hot and cold" returns 9
<code>GetWord()</code>	• Called with the position of the first character of a word in <code>FNString</code> as its parameter • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"
<code>IgnoreWord()</code>	• Called with a <code>STRING</code> parameter representing a word • Searches for the word in the <code>IgnoreList</code> array • Returns <code>TRUE</code> if the word is found, otherwise returns <code>FALSE</code>
<code>GetInitials()</code>	• Processes the sequence of words in the full name one word at a time • Calls <code>GetStart()</code>, <code>GetWord()</code> and <code>IgnoreWord()</code> to process each word to form the new string • Outputs the new string

- (a) Write pseudocode for the module `IgnoreWord()`.

109. M/J 21/P23/Q1/c

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure

Justification

[3]

110. M/J 21/P23/Q3/a,b

- 3 (a) A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

- (i) Explain how the program could use text files to speed up the process.

[2]

- (ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

[1]

- (iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

[2]

- (b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

[5]

111. M/J 21/P23/Q5

- 5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

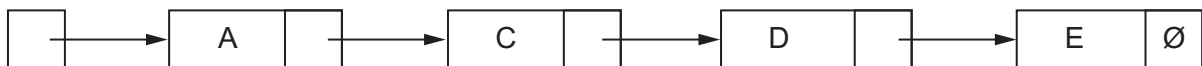
A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

[8]

112. M/J 21/P23/Q6/a,b

- 6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



- (a) Explain how a node containing data value B is added to the list in alphabetic sequence.

[4]

- (b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

[2]

Answers Section

1. M/J 26/P21/Q2/a(i)

2(a)(i)	BOOLEAN	1
---------	---------	----------

2. M/J 26/P21/Q6/c

6(c)	<i>Example solution:</i> <pre> DECLARE ThisPacket : STRING DECLARE ThisLength : STRING DECLARE ThisMachine : CHAR DECLARE ThisDescription : STRING OPENFILE "Rejects.txt" FOR READ WHILE NOT EOF("Rejects.txt") READFILE "Rejects.txt", ThisPacket ThisLength ← LEFT(ThisPacket, 3) ThisMachine ← MID(ThisPacket, 5, 1) ThisDescription ← RejectDescription(ThisPacket) IF STR_TO_NUM(ThisLength) > 560 AND (ThisMachine = 'B' OR ThisMachine = 'C') THEN OUTPUT ThisDescription ENDIF ENDWHILE CLOSEFILE "Rejects.txt" </pre> <i>Mark as follows:</i> MP1 OPENFILE in READ mode and CLOSEFILE MP2 Conditional loop - terminates when EOF("Rejects.txt") MP3 READFILE – with correct syntax MP4 ‘Attempted’ isolation of length and machine data MP5 IF STR_TO_NUM(ThisLength) > 560 AND (ThisMachine = ‘B’ OR ThisMachine = ‘C’) MP6 OUTPUT of return value from RejectDescription() inside the loop	6
6(a)	MP1 LEFT(DataPacket, 3) // MID(DataPacket, 1, 3) MP2 MID(DataPacket, 5, 1)	2

3. M/J 26/P21/Q7/a,b,c

7(a)(i)	Printer queue on a computer network // modelling/simulation of a real-world problem using a queue //accept anything reasonable	1
7(a)(ii)	The queue operates on the principle the first item to join the queue will be the next item to leave MP1 First in – first out ‘feel’ MP2 Must refer to their application stated in part (i)	2

7(b)(i)	MP1 DECLARE MyQueue : ARRAY [1:8] OF STRING MP2 DECLARE Front : INTEGER DECLARE Rear : INTEGER	2																								
7(b)(ii)	Queue <table style="margin-left: auto; margin-right: auto;"> <tr><td>1</td><td>{ "PIG" }</td><td></td></tr> <tr><td>2</td><td>"DOG"</td><td>← Front</td></tr> <tr><td>3</td><td>"RAT"</td><td></td></tr> <tr><td>4</td><td>"SPIDER"</td><td></td></tr> <tr><td>5</td><td>"HORSE"</td><td></td></tr> <tr><td>6</td><td>"CAT"</td><td>← Rear</td></tr> <tr><td>7</td><td></td><td></td></tr> <tr><td>8</td><td></td><td></td></tr> </table> Mark as follows: MP1 Correct sequence of five values in locations <u>2–6</u> MP2 Front points to DOG in location 2 and Rear points to CAT in location 6	1	{ "PIG" }		2	"DOG"	← Front	3	"RAT"		4	"SPIDER"		5	"HORSE"		6	"CAT"	← Rear	7			8			2
1	{ "PIG" }																									
2	"DOG"	← Front																								
3	"RAT"																									
4	"SPIDER"																									
5	"HORSE"																									
6	"CAT"	← Rear																								
7																										
8																										
7(b)(iii)	MyQueue <table style="margin-left: auto; margin-right: auto;"> <tr><td>1</td><td>"MONKEY"</td><td></td></tr> <tr><td>2</td><td>"SNAKE"</td><td>← Rear</td></tr> <tr><td>3</td><td></td><td></td></tr> <tr><td>4</td><td>("LION")</td><td></td></tr> <tr><td>5</td><td>"GIRAFFE"</td><td>← Front</td></tr> <tr><td>6</td><td>"TIGER"</td><td></td></tr> <tr><td>7</td><td>"ANT"</td><td></td></tr> <tr><td>8</td><td>"GERBIL"</td><td></td></tr> </table> Mark as follows: MP1 MONKEY and SNAKE in locations 1, 2 with correct Rear pointer MP2 GIRAFFE in 5 with Front pointer and TIGER, ANT, GERBIL in locations 6, 7 and 8	1	"MONKEY"		2	"SNAKE"	← Rear	3			4	("LION")		5	"GIRAFFE"	← Front	6	"TIGER"		7	"ANT"		8	"GERBIL"		2
1	"MONKEY"																									
2	"SNAKE"	← Rear																								
3																										
4	("LION")																									
5	"GIRAFFE"	← Front																								
6	"TIGER"																									
7	"ANT"																									
8	"GERBIL"																									

7(c)	Example solution: <pre> PROCEDURE InitialiseQueue() DECLARE i : INTEGER Front ← 0 /...7/8 Rear ← 0 / ... 7/8 FOR i ← 1 TO 8 MyQueue[i] ← "" NEXT i ENDPROCEDURE </pre> Mark as follows: MP1 Front and Rear are each assigned a value in range MP2 Correct FOR-NEXT loop // other loop structure MP3 MyQueue[i] assigned an empty string in a loop	3
------	---	----------

4. M/J 26/P21/Q9/a,b,c

9(a)	DECLARE Version1 : ARRAY [0:2, 1:15] OF CHAR MP1 MP2	2
9(b)	Example solution: <pre> DECLARE RoomNumber, FloorNumber, Count : INTEGER Count ← 0 OUTPUT "Floor number: " INPUT FloorNumber OUTPUT "Floor number: " & NUM_TO_STR(FloorNumber) FOR RoomNumber ← 1 TO 15 IF Version1[FloorNumber, RoomNumber] = 'A' THEN OUTPUT RoomNumber Count ← Count + 1 ENDIF NEXT RoomNumber OUTPUT NUM_TO_STR(Count) & " room(s) available" </pre> Mark as follows: MP1 Declaration of all integer variables MP2 Input of the floor number (with prompt) MP3 Loop for 15 iterations MP4 Test array element to check if room is 'available'? MP5 Initialisation and increment of count and Output room number MP6 Output of the final count line including the message outside the loop	6

9(c)(i)	Example solution: <pre>TYPE RoomData (DECLARE Available : <u>CHAR</u>) DECLARE RoomType : CHAR DECLARE View : BOOLEAN DECLARE Grade : STRING ENDTYPE</pre> <i>Mark as follows:</i> MP1 and MP2 Two variables correct 1 mark Three variables correct 2 marks MP3 ENDTYPE	3
---------	---	----------

9(c)(ii)	Example solution: <pre> DECLARE BoolView : BOOLEAN DECLARE MyRoomType : CHAR DECLARE InputView, MyRoomGrade : STRING DECLARE Floor, Room : INTEGER OUTPUT "Room type - S/D/F ?" INPUT MyRoomType OUTPUT "View - True/False ?" INPUT InputView IF TO_UPPER(InputView) = "TRUE" THEN BoolView ← TRUE ELSE BoolView ← FALSE ENDIF OUTPUT "Room grade - ST/DL ?" INPUT MyRoomGrade OUTPUT "Matching floor number and room number:" FOR Floor ← 0 TO 2 FOR Room ← 1 TO 15 IF Version2[Floor, Room].RoomType = MyRoomType AND Version2[Floor, Room].View = BoolView AND Version2[Floor, Room].RoomGrade = MyRoomGrade AND Version2[Floor, Room].Available = 'A' THEN OUTPUT Floor, " ", Room ENDIF NEXT Room NEXT Floor </pre> Marks as follows: MP1 Three correctly formed INPUT statements each with a prompt MP2 OUTPUT of the “Matching floor etc” heading MP3 Outer FOR loop / or other (for the floors) MP4 Inner FOR loop / or other (for the rooms) MP5 Correct use of array with dot notation with at least one data item inside loop MP6 ‘Attempt’ to combine all three tests (room type, view, grade) inside loop MP7 Output of floor and room inside loop	7
----------	---	----------

5. M/J 26/P22/Q1/a(i)

1(a)(i)	STRING	1	
---------	--------	----------	--

6. M/J 26/P22/Q4/a,b

4(a)	After group 1 back stack www.aaa.home www.ccc.my ← TOS forward stack www.bbb.home www.333.corp www.111.home www.999.corp www.222.me ← TOS Web address currently displayed in the browser: www.zzz.corp After group 2 back stack www.zzz.corp www.aaa.home www.ccc.my ← TOS forward stack www.333.corp www.111.home www.999.corp www.222.me ← TOS Website displayed in the browser: www.bbb.home After group 3 back stack www.111.home www.333.corp www.bbb.home www.zzz.corp www.aaa.home www.ccc.my ← TOS forward stack www.222.me ← TOS	6
------	---	---

4(a)	Website displayed in the browser: www.999.corp Mark as follows: Group 1 1 1 mark for back stack as shown and TOS position as shown on both stacks 2 1 mark for forward stack as shown and current web address Group 2 3 1 mark for stacks and correct TOS pointers as shown and current web address Group 3 4 1 mark for back stack as shown and TOS as shown 5 1 mark for forward stack as shown and TOS as shown 6 1 mark current web address		
4(b)	One mark for identifying an ADT. Queue / Linked List One mark per feature Features of a Queue: 1 1 Each queue element contains one data item 2 A front/start pointer and a rear/end pointer 3 Data is added at back / end and removed from front / start // works on a FIFO basis 4 Can be circular or linear MAX 2 marks for features Features of a Linked List: 1 1 Each node / item contains data and a pointer (to the next node/item) 2 A pointer to the start of the list 3 Last node in the list has a null pointer 4 Data may be added / removed by manipulating <u>pointers</u> 5 Pointers are followed to move through nodes in the list (in a given sequence) 6 Unused nodes are stored on a free list // a free-list pointer to the Free List 7 Is a dynamic (data structure) MAX 2 marks for features	3	Need both Reject use of 2 arrays

7. M/J 26/P22/Q5/a,b

5(a)	The greater than logic operator / > in the selection statement would need to be changed to a less than operator / < Or by example <pre>IF Numbers[Index] > Numbers[Index + 1] THEN</pre> would be changed to <pre>IF Numbers[Index] < Numbers[Index + 1] THEN OR IF Numbers[Index] <= Numbers[Index + 1] THEN OR IF Numbers[Index + 1] > Numbers[Index] THEN OR IF Numbers[Index + 1] >= Numbers[Index] THEN</pre>	1	
5(b)(i)	2 marks for inner loop MP 1: The inner loop iterates one less time (in algorithm B) each time the outer / repeat loop iterates. MP 2: this is because for each iteration of the outer / repeat loop (in algorithm B) the last / one more element in the <code>Numbers</code> array will be in the correct place. 2 marks for outer loop MP 3: The outer / repeat loop (in algorithm B) would stop iterating if no swaps are made / all numbers in array are sorted MP 4: so not as many iterations would be needed (if the <code>Numbers</code> array is not initially sorted)	4	Reject answers that just repeat the algorithm Reject program stops rather than iteration ends or algorithm ends
5(b)(ii)	1 mark: When <code>Numbers</code> (array) / array is already in (ascending) order / sorted	1	

8. M/J 26/P22/Q6/b

6(b)(i)	Solutions: • Grid : ARRAY[1 : 5, 1 : 5] OF <u>CHAR</u> • Grid : ARRAY[0 : 4, 0 : 4] OF <u>CHAR</u> 1 mark for boundaries of both array dimensions: • [1 : 5, 1 : 5] • [0 : 4, 0 : 4] 1 mark for all other parts of declaration	2	Reject use of different brackets eg () Reject CHARACTER instead of CHAR Reject STRING instead of CHAR Must be as exactly as above but accept use of lowercase
---------	---	----------	--

6(b)(ii)	Solution One : Using nested loop and output of each row in a loop following nested loop Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER Index ← 1 FOR Row ← 1 TO 5 FOR Column ← 1 TO 5 Grid[Row, Column] ← MID(Plaintext, Index, 1) Index ← Index + 1 NEXT Column Next Row FOR Row ← 1 TO 5 OUTPUT Grid[Row, 1], Grid[Row, 2], Grid[Row, 3], _ Grid[Row, 4], Grid[Row, 5] NEXT Row ENDPROCEDURE </pre> Mark Points 1 Generate Index, 1 to 25, used to extract all characters from Plaintext 2 Generate both Row and Col in a loop 3 Extract an individual character from Plaintext in a loop 4 Attempt to assign the extracted character to Grid in a loop 5 Extraction and assignment of all 25 characters in Plaintext to Grid in a loop 6 Attempt to identify / isolate 5 characters comprising at least one row 7 Identify / isolate all 5 characters for all 5 rows 8 Output all five rows of GRID, each on a separate line	8 Syntax for MID() function must use parameters of correct type – allow other string handling function combinations if they work. Max 7 if Procedure heading/parameter type incorrect Penalise incorrect syntax for indexes i.e. [Row] [Col] instead once.
----------	--	--

6(b)(ii)	Solution Two: Using nested loop and output of each string in a loop following nested loop - Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER DECLARE RowString : STRING FOR Row ← 1 TO 5 FOR Column ← 1 TO 5 Index ← (Row - 1) * 5 + Column Grid[Row, Column] ← MID(Plaintext, Index, 1) NEXT Column NEXT Row FOR Row ← 1 TO 5 RowString ← "" For Col ← 1 TO 5 RowString ← RowString & Grid[Row, Column] Next Col OUTPUT RowString NEXT Row ENDPROCEDURE </pre> Solution Three: Using nested loop where output string is formed in the inner loop and output in string in the outer loop – complex use of Index. No additional loop after nested loop Example Solution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER DECLARE RowString : STRING FOR Row ← 1 TO 5 RowString ← "" FOR Column ← 1 TO 5 Index ← (Row - 1) * 5 + Column Grid[Row, Column] ← MID(Plaintext, Index, 1) RowString ← RowString & MID(Plaintext, Index, 1) NEXT Column OUTPUT RowString NEXT Row ENDPROCEDURE </pre>		
----------	---	--	--

6(b)(ii)	Solution Four: No nested loop Example Soution <pre> PROCEDURE PopulateGrid(Plaintext : STRING) DECLARE Row, Column, Index : INTEGER Row ← 1 Col ← 0 FOR Index ← 1 TO 25 IF Index MOD 5 = 0 THEN Row ← Row + 1 Col ← 0 ENDIF Col ← Col + 1 Grid[Row, Column] ← MID(Plaintext, Index, 1) NEXT Index FOR Row ← 1 TO 5 OUTPUT Grid[Row, 1], Grid[Row, 2], _ Grid[Row, 3], Grid[Row, 4], Grid[Row, 5] NEXT Row ENDPROCEDURE </pre>		
----------	--	--	--

9. M/J 26/P23/Q2/a

2(a)	BOOLEAN	1	
------	---------	----------	--

10. M/J 26/P23/Q5/b

5(b)	<pre> PROCEDURE Search(Initial : CHAR, ShortSurname : STRING) DECLARE ThisApplicant, ThisSurname : STRING DECLARE ThisInitial : CHAR DECLARE Found : BOOLEAN OPENFILE "JobApps.txt" (FOR) READ Found ← FALSE REPEAT READFILE "JobApps.txt", ThisApplicant ThisInitial ← LEFT(ThisApplicant, 1) ThisSurname ← MID(ThisApplicant, 2, 5) IF ThisSurname = ShortSurname AND ThisInitial = Initial THEN Found ← TRUE OUTPUT "Name found" ENDIF UNTIL Found = TRUE OR EOF("JobApps.txt") IF Found = FALSE THEN OUTPUT "Name not Found" ENDIF CLOSE(FILE) "JobApps.txt" ENDPROCEDURE </pre> <i>Mark as follows:</i> 1 mark per gap	7	
------	--	---	--

11. M/J 26/P23/Q6/a,b,c,d

6(a)	MP1 5
------	--

6(c)(i)	Start pointer: 5 Mark as follows: Start is 5 and other pointer values are correct and BISON pointer is changed to 1 and arrows are as shown and DOG is 'isolated'	1	
6(c)(ii)	MP1 The item is not in the list MP2 The data item is the only item in the list MP3 The item is the first in the list	2	
6(d)	MP1 Store 'BAT' at the next available location /with index 6 / in the Data array MP2 Visit the start pointer node MP3 Compare this data value with 'BAT' //compare 'BAT' with 'ANT' MP4 When 'BAT' is less than the current value - 'position is found' MP5 IF not, move to next node // move to 'BISON' MP6 and store the previous position MP7 Repeat until 'BAT' less that current value MP8 Link pointer of the previous node // 'ANT' points/connects to 'BAT' MP9 Link pointer of 'BAT' points to the current node / 3 // 'BAT' points/connects to 'BISON' Max 6	6	

12. M/J 26/P23/Q8/a,b

8(a)(i)	DECLARE Bins : ARRAY [0:2, 1:120] OF BOOLEAN <u>MP1</u> <u>MP2</u>	2	Do not accept variations of syntax.
---------	---	---	-------------------------------------

8(a)(ii)	Example solution: <pre> PROCEDURE InactiveReport() DECLARE Level, BinNo : INTEGER DECLARE BinCount : INTEGER BinCount ← 0 FOR Level ← 0 TO 2 IF Level = 0 THEN OUTPUT "Level: ground floor" ELSE OUTPUT "Level: " & NUM_TO_STR(Level) ENDIF FOR BinNo ← 1 TO 5 IF Bins[Level, BinNo] = FALSE THEN OUTPUT "Bin " & NUM_TO_STR(BinNo) BinCount ← BinCount + 1 ENDIF NEXT BinNo NEXT Level OUTPUT NUM_TO_STR(BinCount) & " bins are not in use" ENDPROCEDURE </pre> Mark as follows: MP1 Bin count declared and initialised before the loop and correctly incremented MP2 (Outer) level loop – iterates 3 times MP3 (Inner) bin loop – iterates 5 times MP4 Correct 2D array syntax and test for not in use with a Boolean comparison MP5 Correct header OUTPUT statement(s) - includes the 'Ground floor' text and 'Bin' text and bin number MP6 Correct final OUTPUT statement with 'total not in use' text and the count Max 5	5	
8(a)(iii)	<pre> Bins[0, 3] ← FALSE Bins[1, 2] ← TRUE Bins[2, 1] ← FALSE Bins[2, 2] ← FALSE Bins[2, 3] ← TRUE Bins[2, 4] ← TRUE Bins[2, 5] ← FALSE </pre> Mark as follows: MP1 Bins(0, 3) ← FALSE MP2 Bins(1, 2) ← TRUE MP3 Level 2 assignment all complete	3	

8(b)(i)	Example solution: <pre> TYPE BinData DECLARE Size : STRING DECLARE Active : BOOLEAN DECLARE StockCode : STRING DECLARE StockLevel : INTEGER ENDTYPE </pre> Mark as follows: Two variables correct – 1 Mark 4 variables correct – 2 marks	2	
8(b)(ii)	Example solution: <pre> FUNCTION FindBin(MySize : STRING) RETURNS STRING DECLARE Level, BinNo : INTEGER DECLARE Text : STRING Text ← "No bins available" Level ← 0 REPEAT BinNo ← 1 REPEAT IF Bins[Level, BinNo].Size = MySize AND Bins[Level, BinNo].Active = FALSE THEN Text ← "Level: " & NUM_TO_STR(Level) & " Bin: " & NUM_TO_STR(BinNo) OUTPUT Text RETURN Text ELSE BinNo ← BinNo + 1 ENDIF UNTIL BinNo = 121 Level ← Level + 1 UNTIL Level = 3 OUTPUT Text RETURN Text ENDFUNCTION </pre> Mark as follows: MP1 Function header and string parameter and return string and ending MP2 Use of any (outer level) loop for 3 iterations MP3 Use of any (inner) bin loop for 5 / 120 iterations MP4 Correct 2D array syntax with either data field – .Size or .Active MP5 Correct IF statement condition including AND MP6 Formation of the text string with correct syntax – MP7 Correct return value (and output) – in both cases and in the correct place	7	Do not accept comma syntax.

13. O/N 25/P21/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE RentalRecord DECLARE RentalID : STRING DECLARE CarID : INTEGER DECLARE DisCode : CHAR DECLARE Start : DATE DECLARE Duration : INTEGER DECLARE Completed : BOOLEAN ENDTYPE </pre> Mark as follows: 1 One mark for TYPE RentalRecord and ENDTYPE statements 2 One mark for RentalID and CarID declared correctly 3 One mark for DisCode and Start declared correctly 4 One mark for Duration and Completed declared correctly	4
3(a)(ii)	<u>DECLARE Rental : ARRAY[1:500] OF RentalRecord</u> One mark per underlined phrase	2
3(b)	One mark per point: 1 Different data types held (for each rental) under a single identifier 2 Allows for iteration through <u>rentals/records</u> // Can access rentals/records in a loop // Can (directly) access a specific <u>rental/record</u> <u>using an</u> (array) <u>index</u> 3 Simplifies the code/program / less code needed / reduces code duplication // Makes code/program easier to understand // Makes testing / debugging easier // Program maintenance easier	3

14. O/N 25/P21/Q4/a,b

4(a)	Example solution - conditional: <pre> PROCEDURE Store() DECLARE ThisNum, LastNum : REAL DECLARE Index : INTEGER // index to global array INPUT ThisNum LastNum ← ThisNum Number[1] ← ThisNum INPUT ThisNum Index ← 2 WHILE Index < 21 AND ThisNum > LastNum Number[Index] ← ThisNum LastNum ← ThisNum Index ← Index + 1 IF Index < 21 THEN INPUT ThisNum ENDIF ENDWHILE OUTPUT Index - 1, " values were stored in the array" ENDPROCEDURE </pre> Alternative loop: <pre> // Loop without using LastNum WHILE Index < 21 AND ThisNum > Number[Index - 1] Number[Index] ← ThisNum Index ← Index + 1 IF Index < 21 THEN //skip input if array full INPUT ThisNum ENDIF ENDWHILE </pre> Mark as follows: 1 Declare local variables <code>Index</code> as integer, <code>ThisNum</code> and <code>LastNum</code> as real 2 Input value and assign to first element 3 Attempt at conditional loop including both tests 4 Completely correct conditional loop including both tests 5 Assign input value to current element if greater than previous element and input next value in a loop 6 Final output giving attempted count of elements stored plus message after a loop	6
------	--	---

4(a)	Alternative FOR loop example solution: <pre> PROCEDURE Store() DECLARE ThisNum, LastNum : REAL DECLARE Index : INTEGER // index to global array DECLARE Count : INTEGER INPUT ThisNum LastNum ← ThisNum Number[1] ← ThisNum Count ← 1 INPUT ThisNum FOR Index ← 2 TO 20 IF ThisNum > LastNum THEN Number[Index] ← ThisNum LastNum ← ThisNum Count ← Count + 1 IF Index < 20 THEN INPUT ThisNum ENDIF ELSE BREAK ENDIF NEXT Index OUTPUT Count, " values were stored in the array" ENDPROCEDURE </pre> Alternative loop: <pre> Count ← 1 INPUT ThisNum FOR Index ← 2 TO 20 IF ThisNum > Number[Index - 1] THEN Number[Index] ← ThisNum Count ← Count + 1 IF Index < 20 THEN INPUT ThisNum ENDIF ELSE BREAK ENDIF NEXT Index </pre> Mark as follows: 1 Declare local variables <code>Index</code> as integer, <code>ThisNum</code> and <code>LastNum</code> as real 2 Input value and assign to first element 3 Count-controlled loop 4 Count-controlled loop with BREAK if current value greater than previous value 5 ... Assign input value to current element following correct comparison in a loop 6 Final output giving count of elements stored plus message following a reasonable attempt after loop	
------	---	--

4(b)(i)	One mark per point: 1 The amount of values stored (in a text file) is not fixed / not limited (other than by secondary storage available) // The amount values stored is not limited to the size of the array 2 The data is stored permanently / non-volatile // Data can be restored / reused without re-entering values	2
4(b)(ii)	Each (real) value would have to be converted to a string	1

15. O/N 25/P21/Q5/a

5(a)(i)	It is a value that is invalid as an (array) index // not in the range of valid (index) values	1
5(a)(ii)	To test whether the current element / value at index matches the given / search string / data / parameter	1
5(a)(iii)	The value of <code>FoundAt</code> is only updated to the (current) index if it currently stores the initial value / -1 // <code>FoundAt</code> is not updated when it currently stores any other value than -1	1

16. O/N 25/P21/Q8/a,b

8(a)	Example solution: <pre> PROCEDURE OKToBorrow(ThisStudentID : STRING) DECLARE Index, Count : INTEGER CONSTANT Max = 5 Count ← 0 Index ← 1 / 0 WHILE Index <= 5000 / Index <= 4999 AND Count < Max IF Loan[Index].StudentID = ThisStudentID AND ____ Loan[Index].OnLoan = TRUE THEN Count ← Count + 1 ENDIF Index ← Index + 1 ENDWHILE IF Count < Max THEN OUTPUT "Student may borrow another book" ELSE OUTPUT "Student may not borrow another book" ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameter and ending 2 Loop through all elements in <code>Loan</code> array 3 ... terminating if <code>Max</code> reached 4 Test for correct <code>StudentID</code> field in a loop 5 ... and that <code>OnLoan = TRUE</code> in a loop ... if so, then increment <code>Count</code> in a loop 6 Output an appropriate message in both cases after the loop	7
------	--	----------

8(b)	Example solution: <pre> FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER Index ← 1 WHILE Index < 5001 IF Loan[Index].StudentID = ThisStudentID AND ___ Loan[Index].BookID = ThisBookID THEN IF Loan[Index].OnLoan = TRUE THEN // Not returned Loan[Index].OnLoan ← FALSE // Mark as returned now RETURN TRUE //Found and not already returned ELSE RETURN FALSE // Found and already returned ENDIF ENDIF Index ← Index + 1 ENDWHILE RETURN False // loan not found ENDFUNCTION </pre> Mark as follows: 1 Loop through all elements in Loan array 2 ...and book loan not found 3 Attempt to reference an individual data item in a loop 4 Correct test for StudentID and correct BookID in a loop 5 ... If book not returned yet, then set OnLoan field to FALSE 6 RETURN TRUE if book loan Found and not already returned 7 RETURN FALSE if book loan Found and loan already returned 8 RETURN FALSE if book loan not found Alterantive example solution – not immediate return <pre> FUNCTION ReturnBook(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE Found, OK : BOOLEAN Index ← 1 Found ← FALSE OK ← TRUE </pre>	8
------	--	---

8(b)	<pre> WHILE Index < 5001 AND NOT Found IF Loan[Index].StudentID = ThisStudentID AND ___ Loan[Index].BookID = ThisBookID THEN Found ← TRUE IF Loan[Index].OnLoan = FALSE THEN // Already returned OK ← FALSE ELSE Loan[Index].OnLoan ← FALSE // Mark as returned now ENDIF ENDIF Index ← Index + 1 ENDWHILE RETURN Found AND OK ENDFUNCTION </pre>	
------	---	--

17. O/N 25/P22/Q1/b

1(b)	<table border="1"> <thead> <tr> <th>Variable</th><th>Example data value</th><th>Data type</th></tr> </thead> <tbody> <tr> <td>Result</td><td>5.42</td><td>REAL</td></tr> <tr> <td>MonthLetter</td><td>"JFMAMJJASOND"</td><td>STRING</td></tr> <tr> <td>Birthday</td><td>15/11/2009</td><td>DATE</td></tr> </tbody> </table> One mark per row	Variable	Example data value	Data type	Result	5.42	REAL	MonthLetter	"JFMAMJJASOND"	STRING	Birthday	15/11/2009	DATE	3
Variable	Example data value	Data type												
Result	5.42	REAL												
MonthLetter	"JFMAMJJASOND"	STRING												
Birthday	15/11/2009	DATE												

18. O/N 25/P22/Q3/b,c

3(b)	It does not appear in (any of) the data items // it does not appear in the ID and Name	1
3(c)	MP1 Add a (single) new separator at the end of the third (original) item MP2 Calculate the length of both new data items MP3 Store / append the length (of the new each items) MP4 as a fixed length / separated digit string // by example Max 3	3

19. O/N 25/P22/Q4/a,b

4(a)	Example solution: <pre> DECLARE CheckTotal : ARRAY [0:35] OF BOOLEAN DECLARE Index : INTEGER FOR Index ← 0 TO 35 CheckTotal[Index] ← FALSE NEXT Index </pre> Mark as follows: MP1 Declaration of CheckTotal array MP2 Declaration of a loop counter MP3 Loop – with correct syntax and number of iterations MP4 Assignment of array element to FALSE	4
4(b)	<pre> DECLARE EasyQ, HardQ : INTEGER FOR EasyQ ← 0 TO 15 STEP 3 MP1 MP2 </pre> <pre> FOR HardQ ← 0 TO 20 STEP 5 MP3 MP4 </pre> <pre> CheckTotal[HardQ + EasyQ] ← TRUE NEXT HardQ MP5 NEXT EasyQ </pre> Mark as follows: One mark per gap (boldened)	5

20. O/N 25/P22/Q8/a,b,c

8(a)	Example solution: <pre> PROCEDURE CountLoans(ThisStudentID : STRING) DECLARE Index, LCount, RCount : INTEGER LCount ← 0 RCount ← 0 FOR Index ← 1 TO 7000 IF Loan[Index].StudentID = ThisStudentID THEN IF Loan[Index].OnLoan = FALSE THEN RCount ← RCount + 1 ELSE LCount ← LCount + 1 ENDIF ENDIF NEXT Index OUTPUT "Student has ", LCount, " books on loan" OUTPUT "and has returned ", RCount, " books" ENDPROCEDURE </pre> Mark as follows: MP1 Declaration of Index and two count variables MP2 Loop for 7000 iterations MP3 Index references an indexed data item with correct .dot notation MP4 Attempt to compare the StudentID field = parameter in a loop MP5 Test of the OnLoan field and correct logic for both outcomes MP6 Both count variables initialised and increment appropriate count in a loop MP7 Output includes both counts with a <u>suitable</u> message	7
------	---	----------

8(b)	Example solution: <pre> FUNCTION NewLoan(ThisStudentID, ThisBookID : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE IsStored : BOOLEAN CONSTANT Blank = "" Index ← 1 IsStored ← FALSE WHILE Index <= 7000 AND NOT IsStored IF Loan[Index].StudentID = Blank THEN Loan[Index].StudentID ← ThisStudentID Loan[Index].BookID ← ThisBookID Loan[Index].OnLoan ← TRUE IsStored ← TRUE ENDIF Index ← Index + 1 ENDWHILE RETURN IsStored ENDFUNCTION </pre> Mark as follows: MP1 Loop through 7000 elements in <code>Loan</code> array – correct loop structure MP2 or until the first unused element is found // Correct <code>RETURN</code> inside a <code>FOR</code> loop MP3 Correct use of the indexed dot notation MP4 Test if record is unused (<code>.StudentID = ""</code>) in a loop MP5 Two item assignment statements correct MP6 <code>Loan[Index].OnLoan</code> assigned <code>TRUE</code> MP7 Correct logic to return <code>BOOLEAN</code> in both cases (position found / no position available) Alternative solution: loop only (no ‘found’ mechanism) <pre> FOR Index ← 1 TO 7000 IF Loan[Index].StudentID = Blank THEN Loan[Index].StudentID ← ThisStudentID Loan[Index].BookID ← ThisBookID Loan[Index].OnLoan ← TRUE RETURN TRUE ENDIF NEXT RETURN FALSE </pre>	7
8(c)	MP1 Save to/Open a (text) file MP2 Single line – form a string with separator characters and write to the file // Multiple lines - write each data item to a separate line in the file	2

21. O/N 25/P23/Q3

3	<pre> FUNCTION Pop() RETURNS PopData DECLARE ThisPop : PopData MP 2 IF SP < 1 // SP = 0 THEN MP 3 PopData.Exists ← FALSE // Stack is empty MP 1 ELSE PopData.Data ← ThisStack[SP] MP 4 PopData.Exists ← TRUE MP 1 SP ← SP - 1 MP 5 ENDIF RETURN ThisPop` ENDFUNCTION </pre> Mark as follows: MP 1 One mark for both assignments to PopData.Exists (FALSE and TRUE) MP 2, 3, 4 ,5 One mark for each of remaining four bold parts	5
---	--	---

22. O/N 25/P23/Q4

4	Example solution: <pre> PROCEDURE Store() DECLARE Index, StartPos, ThisNum : INTEGER DECLARE ThisString : STRING Index ← 1 StartPos ← 1 WHILE StartPos < LENGTH(NumString) AND Index < 21 ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum Index ← Index + 1 StartPos ← StartPos + 4 ENDWHILE ENDPROCEDURE </pre> Mark as follows: 1 Declare all local variables used 2 Conditional loop while end of string not reached 3 ... and array not full 4 Attempted use of MID() // Attempted use of Right() and Left() 5 Correct extraction and use next three letter substring from NumString in a loop 6 Convert to integer and assign value to array element in a loop 7 Correct increment of array index by 1 and start position of substring by 4 Max 6 Alternative FOR loop solution: <pre> PROCEDURE Store() DECLARE Index, StartPos, ThisNum : INTEGER DECLARE ThisString : STRING StartPos ← 1 FOR Index ← 1 TO 20 ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum StartPos ← StartPos + 4 IF StartPos > LENGTH(NumString) THEN Break ENDIF NEXT Index ENDPROCEDURE </pre>	6
---	---	---

4	Alternative where number of 3-digit substrings calculated <pre> StartPos ← 1 NumOfSubStrings ← ((LENGTH(NumString) + 1) DIV 4)) FOR Index ← 1 TO NumOfSubString ThisString ← MID(NumString, StartPos, 3) ThisNum ← STR_TO_NUM(ThisString) Data[Index] ← ThisNum StartPos ← StartPos + 4 IF Index = 20 THEN Break ENDIF NEXT Index </pre> Mark as follows: 1 Declare all variables used 2 Loop for 20 iterations // number of 3-digit numbers in the NumString 3 Attempted use of MID() // Attempted use of Right() and Left() 4 Correct extraction and use next three letter substring from NumString in a loop 5 Convert to integer and assign value to array element in a loop 6 Increment StartPos by 4 in a loop 7 Test for end of NumString and Break // Test for end of array and Break Max 6	
---	--	--

4	Alternative Solution extracting one character at a time and testing for comma <pre> PROCEDURE Store() DECLARE Index, Position : INTEGER DECLARE SubString : STRING Declare ThisCharacter : CHAR Index ← 1 Position ← 0 SubString ← "" WHILE Index <= 20 AND Position <= LENGTH(NumString) Charcter ← MID(NumString, Position, 1) IF Character = ',' Data[Index] ← STR_TO_NUM(SubString) Index ← Index + 1 SubString ← "" ELSE SubString ← SubString & Character ENDIF Position ← Position + 1 ENDWHILE Data[Index] ← STR_TO_NUM(SubString) //last 3 digit string ENDPROCEDURE </pre> Mark as follows: 1 Declare all variables used 2 Conditional loop while end of string not reached 3 ... and array not full 4 Correct extraction and use of a character from NumString in a loop 5 Test for comma in a loop 6 If true convert substring to number ... and store in Data array and set SubString to "" and increment Index 7 Otherwise concatenate next character from NumString to end of Substring Max 6	
---	--	--

23. O/N 25/P23/Q8/a,b,c

8(a)	Example solution: <pre> PROCEDURE LoanStatus(ThisStudentID, ThisBookID : STRING) DECLARE Index : INTEGER DECLARE Found : BOOLEAN Index ← 1 / 0 Found ← FALSE WHILE Index <= 8000 / 7999 AND NOT Found IF Loan[Index].StudentID = ThisStudentID AND Loan[Index].BookID = ThisBookID THEN IF Loan[Index].OnLoan = FALSE THEN OUTPUT "Loan has been returned." ELSE OUTPUT "Loan has not been returned." ENDIF Found ← TRUE ENDIF Index ← Index + 1 ENDWHILE IF NOT Found THEN OUTPUT "Warning - Loan not found." ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameters and ending 2 Loop through all elements in Loan array 3 ... or until record loan is found 4 Test for correct StudentID in a loop 5 ... and correct BookID 6 ... check OnLoan data item 7 ... and output <u>both</u> messages as appropriate regarding book loan status once 8 If no matching loan found, Output 'not found' after the loop	8
------	--	----------

8(b)	Example solution: <pre> FUNCTION LoansPerTutor(TutorID : STRING) RETURNS INTEGER DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 / 0 TO 8000 / 7999 IF LEFT(Loan[Index].StudentID, 3) = TutorID AND Loan[Index].OnLoan = TRUE THEN Count ← Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Initialise local integer for Count 2 Loop through all elements in the Loan array 3 Attempt at referencing a data item in a loop 4 Extract TutorID from Loan[Index].StudentID in a loop 5 Test for Matching Tutor AND test if the book has been returned using correct dot notation in a loop 6 ... if true then increment Count in a loop 7 Return count	7
8(c)(i)	One mark per point: 1 Include (a suffix string based on) the date of the archive / the next number in sequence 2 Concatenate with a root filename such as 'Archive'	2
8(c)(ii)	Boolean data cannot be written to a text file // OnLoan is not a string // OnLoan will have to be converted to text	1
8(c)(iii)	One mark per point: Benefit: Algorithm to store / extract a record is easier // unpacking of data is not required // No need for string concatenation Drawback: More file accesses needed (to read / write a complete record)	2

24. M/J 25/P21/Q5

5	MP1 Pop an item off the stack MP2 Update Stack pointer MP3 Add it to the queue MP4 Update Rear pointer MP5 Repeat until the stack is empty MP6 Remove an item from the queue MP7 Update Front pointer MP8 Push it onto the stack MP9 Update the Stack pointer MP10 Repeat until the queue is empty Max 7	7
---	--	----------

25. M/J 25/P21/Q6/b

6(b)	Example solution: <pre> PROCEDURE Sort() DECLARE Temp, J, Boundary : INTEGER DECLARE NoSwaps : BOOLEAN Boundary ← 1999 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary IF Reading[J, 1] > Reading[J+1, 1] THEN //first swap sensor value Temp ← Reading[J, 1] Reading[J, 1] ← Reading[J+1, 1] Reading[J+1, 1] ← Temp //now swap corresponding ID Temp ← Reading[J, 2] Reading[J, 2] ← Reading[J+1, 2] Reading[J+1, 2] ← Temp NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 Conditional loop correctly formed including Boolean flag declared and initialised MP3 An inner loop MP4 Correct range 1 to 1999 for inner loop MP5 Comparison of element J with J+1 in a loop MP6 Declare <i>Temp</i> variable and swap elements in a loop MP7 Both SensorID and Speed values were swapped in a loop MP8 'No-Swap' mechanism: • Conditional outer loop including flag reset • Flag set in inner loop to indicate swap MP9 Reducing <i>Boundary</i> in the <u>outer</u> loop	8
------	--	----------

26. M/J 25/P21/Q7/b(i)

7(b)(i)	Example solution: <pre> PROCEDURE AddNewCustomers (NumToAdd : INTEGER) DECLARE CustomerID : INTEGER DECLARE Count : INTEGER DECLARE Line : STRING DECLARE NewLine : STRING OPENFILE "Loyalty.txt" FOR READ WHILE NOT EOF("Loyalty.txt") READFILE "Loyalty.txt", Line ENDWHILE CLOSEFILE "Loyalty.txt" OPENFILE "Loyalty.txt" FOR APPEND CustomerID ← STR_TO_NUM((LEFT(Line, 6)) FOR Count ← 1 TO NumToAdd CustomerID ← CustomerID + 1 OUTPUT CustomerID NewLine ← NUM_TO_STR(CustomerID) & ",0" WRITEFILE "Loyalty.txt", NewLine NEXT Count CLOSEFILE "Loyalty.txt" ENDPROCEDURE </pre> Mark as follows: MP1 All variables used are declared using the correct type including a string and an integer MP2 Open file in read mode and close (before opening in append mode) MP3 Conditional loop with <u>EOF("Loyalty.txt")</u> MP4 Read <u>Line</u> from file // Count number of records in the file MP5 Open the file in append mode and subsequently close MP6 Extract <u>CustomerID</u> from <u>Line</u> and convert to integer // use count from MP4 to generate last <u>CustomerID</u> stored MP7 A (count controlled) loop for the number of customers to add MP8 Increment <u>CustomerID</u> and output the new <u>CustomerID</u> in loop MP9 Create string for new <u>CustomerID</u> and write to file in loop Max 8	8
---------	---	----------

27. M/J 25/P22/Q1/c,d

1(c)	1 mark for each correct row		4
	Expression	Data type	
	RIGHT(MachineCode, 4)	STRING	
	Speed * 2.5	REAL	
	NOT Status	BOOLEAN	
	IS_NUM(Check)	BOOLEAN	
1(d)(i)	Two / 2		1
1(d)(ii)	1000		1
1(d)(iii)	DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER 1 mark for correct upper and lower bound • [0:99, 0:9] 1 mark for all other parts of declaration <u>DECLARE Product : ARRAY [0:99, 0:9] OF INTEGER</u>		2

28. M/J 25/P22/Q5/b

5(b)	Conditional Solution Example Solution <pre> DECLARE Count : INTEGER DECLARE Line : STRING DECLARE NextHour, Hour : STRING OPENFILE "TimeTaken.txt" FOR READ Count ← 1 READFILE "TimeTaken.txt", Line Hour ← LEFT(Line, 2) WHILE NOT EOF("TimeTaken.txt") READFILE "TimeTaken.txt", Line NextHour ← LEFT(Line, 2) IF NextHour = Hour THEN Count ← Count + 1 ELSE OUTPUT "Hour : ", Hour, " Total : ", Count Hour ← NextHour Count ← 1 ENDIF ENDWHILE OUTPUT "Hour : ", Hour, " Total : ", Count CLOSEFILE "TimeTaken.txt" </pre> Mark as follows: 1. Any Initialisation of count for number of pictures taken each hour 2. Open "TimeTaken.txt" for read and subsequently close 3. Conditional loop until EOF 4. Read a line from "TimeTaken.txt" in a loop 5. Extract Hour from line read in a loop 6. A mechanism to compare current hour extracted from file with last one read from file in a loop 7. If hours same increment count in a loop 8. If not same output count (with a suitable message) and update Hour ← NextHour and set Count to 1 in a loop 9. Output final Count and Hour (with a suitable message) once only Note: max 8	8
------	--	----------

5(b)	Alternative solution and mark scheme use of an array to hold count for each hour Also, for use of 24 variables and 24 selection conditions <pre> DECLARE HoursArray[0 : 23] OF INTEGER DECLARE Index, Hour : INTEGER DECLARE Line : STRING FOR Index ← 0 TO 23 HoursArray[Index] ← 0 NEXT Index OPENFILE "TimeTaken.txt" FOR READ WHILE NOT EOF("TimeTaken.txt") READFILE "TimeTaken.txt", Line Hour ← STR_TO_NUM(LEFT(Line, 2)) HoursArray[Hour] ← HoursArray[Hour] + 1 ENDWHILE FOR Index ← 0 TO 23 IF HoursArray[Index] <> 0 THEN OUTPUT "Hour : ", Index, " Total : ", HoursArray[Index] ENDIF NEXT Index CLOSEFILE "TimeTaken.txt" </pre> Mark as follows: 1. Initialisation of 24 array elements to 0 // Initialisation of 24 Integer variables to 0 2. Open "TimeTaken.txt" for read and subsequently close 3. Conditional loop until EOF 4. Read a line from "TimeTaken.txt" in a loop 5. Extract Hour from line read in a loop 6. Convert Hour to an integer value in a loop 7. Increment appropriate array element / variable in a loop 8. Output of each hour and corresponding count variable (with a suitable message) for all values where count is not zero	
------	---	--

29. M/J 25/P22/Q6/b

6(b)	Example answer: The ADT is a linked list and the procedure <code>Place()</code> inserts / add a new node / value into it / the linked list Mark as follows: 1. 1 mark for identifying the ADT as a linked list 2. 1 mark for identifying operation as inserting / adding a value / node into a linked list	2
------	---	----------

30. M/J 25/P22/Q7/a,b,c

7(a)	Example solution Conditional Loop <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER DECLARE Found : BOOLEAN CONSTANT Unused = 99999 CONSTANT Upper = 1000 Found ← FALSE Index ← 1 IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF WHILE Found = FALSE AND Loyalty[Index, 1] <> 99999 IF Loyalty[Index,1] = CustomerID THEN Found ← TRUE ELSE Index ← Index + 1 ENDIF IF Index > Upper THEN RETURN -1 ENDIF ENDWHILE IF Found THEN RETURN Loyalty[Index, 2] ELSE RETURN -1 ENDIF ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Check CustomerID is in range and if not return -1 3. (Conditional) loop iterating through each element in array 4. Check for current element in array contains required customer ID in a loop 5. ... If found set value to terminate loop 6. Terminating loop when customer ID found 7. Terminating loop when current customer ID is 99999 8. Return either loyalty points for the customer found or -1 if not found	8
------	---	----------

7(a)	Alternative solution using FOR Loop Example solution <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF FOR Index ← 1 TO 1000 IF Loyalty[Index, 1] = CustomerID THEN RETURN Loyalty[Index, 2] ENDIF IF Loyalty[Index, 1] = 99999 THEN RETURN -1 ENDIF NEXT Index ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Check CustomerID is within range and if not return -1 3. FOR Loop 4. Loop for elements 1 to 1000 / 0 to 999 5. Check if current element in array contains required Customer ID in a loop 6. ... If found correctly return loyalty points // store correct loyalty points and return after loop 7. Check if current element in array is 99999 and return -1 / break loop in a loop 8. return -1 if Customer ID not found Direct access solution Alternative solution subtracting 10000 from CustomerID <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Index : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF </pre>	
------	--	--

7(a)	<pre> Index = CustomerID - 10000 IF Loyalty[Index, 1] = CustomerID THEN RETURN Loyalty[Index, 2] ENDIF RETURN -1 ENDFUNCTION </pre> Mark as follows: 1. Create function header and ending with correct parameter and return type 2. Declare Index as Integer 3. Check CustomerID is less than 10001 and return -1 if it is 4. Check CustomerID is greater than 11000 and return -1 if it is 5. Calculate Index by subtracting a 10000 from CustomerID 6. Check if array contains the CustomerID 7. Return Loyalty Points if CustomerID found 8. Return -1 if not found Binary Search Mark Scheme Example Solution <pre> FUNCTION FindCustomer(CustomerID : INTEGER) RETURNS INTEGER DECLARE Start : INTEGER DECLARE End : INTEGER DECLARE Mid : INTEGER IF CustomerID < 10001 OR CustomerID > 11000 THEN RETURN -1 // Out of range value for customer ID ENDIF Start ← 1 End ← 1000 WHILE Start <= End Mid ← (Start + End) DIV 2 IF Loyalty[Mid, 1] = CustomerID THEN RETURN Loyalty[Mid, 2] ELSE IF Loyalty[Mid, 1] > CustomerID THEN End ← Mid - 1 ELSE Start ← Mid + 1 ENDIF ENDWHILE RETURN -1 ENDFUNCTION </pre>	
------	--	--

7(a)	Mark points 1. Create function header and ending with correct parameter and return type 2. Check <code>CustomerID</code> is within range and if not return <code>-1</code> 3. Conditional loop that halves array search space with each iteration 4. Check if <code>CustomerID</code> is stored in current array element in loop 5. Mechanism to end loop if <code>CustomerID</code> is found in array in a loop 6. Mechanism to end loop if <code>CustomerID</code> is not in array in a loop 7. If <code>CustomerID</code> found in array then return correct loyalty points 8. If <code>CustomerID</code> not in array then return <code>-1</code>	
7(b)	Example solution <pre> PROCEDURE PointsReport () DECLARE Count : INTEGER DECLARE Index : INTEGER DECLARE Sum : INTEGER DECLARE Average : REAL Index ← 1 Count ← 0 Sum ← 0 WHILE Loyalty[Index, 1] <> 99999 AND Index <= 1000 IF Loyalty[Index, 2] >= 11 THEN OUTPUT Loyalty[Index, 1] ENDIF Count ← Count + 1 Sum ← Sum + Loyalty[Index, 2] Index ← Index + 1 ENDWHILE Average ← Sum / Count OUTPUT "The average points of all the customers is ", Average ENDPROCEDURE </pre> Mark as follows: 1. Create procedure header and ending 2. Declare and initialise <code>Index</code>, <code>Sum</code> and <code>Count</code> 3. Loop through all elements in <code>Loyalty</code> array 4. ... or terminates when column1 of <code>Loyalty</code> array equals 99999 in a loop 5. Check if loyalty points greater than or equal to 11 in a loop 6. If true output customer ID 7. Sum points and Increment <code>Count</code> in a loop 8. Calculate and output average with an appropriate message Max 7	7
7(c)(i)	An array can only store data of the same type // An array cannot store data of different types	1

7(c)(ii)	1 mark for each point 1. a new (composite) data type / record is defined (that consists of both <code>INTEGER</code> and <code>STRING</code>) 2. an array based on this new type is declared Alternative 1 mark for each point 1. Converting loyalty points to string and concatenating / joining / append with Customer ID 2. Storing concatenated string in an array of string	2
----------	--	----------

31. M/J 25/P23/Q1/c

1(c)	<table><tr><th>Variable</th><th>Data type</th></tr><tr><td>MemberCount</td><td>INTEGER</td></tr><tr><td>TotalTakings</td><td>REAL</td></tr><tr><td>BookingConfirmed</td><td>BOOLEAN</td></tr><tr><td>MemberDOB</td><td>DATE</td></tr></table> Mark as follows: 1 mark per row	Variable	Data type	MemberCount	INTEGER	TotalTakings	REAL	BookingConfirmed	BOOLEAN	MemberDOB	DATE	4
Variable	Data type											
MemberCount	INTEGER											
TotalTakings	REAL											
BookingConfirmed	BOOLEAN											
MemberDOB	DATE											

32. M/J 25/P23/Q2/a,b,c

2(a)(i)	MP1 400 MP2 6	2
---------	--------------------------------	----------

2(a)(ii)	Stack <table><thead><tr><th>Memory location</th><th>Value</th></tr></thead><tbody><tr><td>409</td><td></td></tr><tr><td>408</td><td>'Y'</td></tr><tr><td>407</td><td>'Z'</td></tr><tr><td>406</td><td>'C'</td></tr><tr><td>405</td><td>'F'</td></tr><tr><td>404</td><td>'K'</td></tr><tr><td>403</td><td>'B'</td></tr><tr><td>402</td><td>'S'</td></tr><tr><td>401</td><td>'R'</td></tr><tr><td>400</td><td>'D'</td></tr></tbody></table> ← TopOfStack MP1 MP2 MP3 TopOfStack pointing to 'Y' and value 'Y' in location 408 Values 'C' and 'Z' in 406 and 407 Values 'D' to 'F' unchanged in 400 to 405 and 409 is empty	Memory location	Value	409		408	'Y'	407	'Z'	406	'C'	405	'F'	404	'K'	403	'B'	402	'S'	401	'R'	400	'D'	3						
Memory location	Value																													
409																														
408	'Y'																													
407	'Z'																													
406	'C'																													
405	'F'																													
404	'K'																													
403	'B'																													
402	'S'																													
401	'R'																													
400	'D'																													
2(b)(i)	<table><thead><tr><th>Index</th><th>Data</th><th>Pointer</th><th></th></tr></thead><tbody><tr><td>0</td><td>"Neptune"</td><td>2</td><td>MP3</td></tr><tr><td>1</td><td>"Jupiter"</td><td>4</td><td></td></tr><tr><td>2</td><td>"Saturn"</td><td>5</td><td></td></tr><tr><td>3</td><td>"Earth"</td><td>1</td><td>MP1</td></tr><tr><td>4</td><td>"Mercury"</td><td>0</td><td></td></tr><tr><td>5</td><td>"Uranus"</td><td>-1</td><td>MP2</td></tr></tbody></table> MP1 MP2 MP3 Earth pointer 1 Uranus pointer -1 Other four correct pointers	Index	Data	Pointer		0	"Neptune"	2	MP3	1	"Jupiter"	4		2	"Saturn"	5		3	"Earth"	1	MP1	4	"Mercury"	0		5	"Uranus"	-1	MP2	3
Index	Data	Pointer																												
0	"Neptune"	2	MP3																											
1	"Jupiter"	4																												
2	"Saturn"	5																												
3	"Earth"	1	MP1																											
4	"Mercury"	0																												
5	"Uranus"	-1	MP2																											
2(b)(ii)	3	1																												

2(c)	MP1 First value added to queue is the first value removed // FIFO // LILO MP2 Two pointers needed to indicate the start and end of the queue MP3 May behave as a circular queue MP4 Manipulated using enqueue () / dequeue () operations // by description Max 2	2
------	--	----------

33. M/J 25/P23/Q7/b(i)

7(b)(i)	Example solution: <pre> FUNCTION (MondayCheck(CustomerID : STRING) RETURNS INTEGER DECLARE LoyaltyPoints : INTEGER DECLARE LoyaltyPointsString : STRING DECLARE Line : STRING Line ← FindCustomer(CustomerID) LoyaltyPointsString ← MID(Line, 8, LENGTH(Line) - 16)) LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString) IF DAYINDEX(TODAY ()) = 2 THEN LoyaltyPoints ← LoyaltyPoints + 10 ENDIF RETURN LoyaltyPoints ENDFUNCTION </pre> MP1 Value is returned from FindCustomer (CustomerID) and stored MP2 Function MID used MP3 Attempt to extract LoyaltyPoints from the string returned by FindCustomer () MP4 Correct extraction of LoyaltyPoints from the string returned by FindCustomer () MP5 Conversion of LoyaltyPoints extracted to an integer MP6 Check - is today's date a Monday using TODAY () MP7 ... increase LoyaltyPoints by 10 MP8 Return LoyaltyPoints	8
---------	---	----------

7(b)(i)	Alternative example solution: Uses a loop to calculate the number of digits in the <code>LoyaltyPoints</code> string <pre> FUNCTION MondayCheck(CustomerID : STRING) RETURNS INTEGER DECLARE LoyaltyPoints : INTEGER DECLARE LoyaltyPointsString : STRING DECLARE Index : INTEGER DECLARE Line : STRING Line ← FindCustomer(CustomerID) Index ← 9 WHILE MID(Line, Index, 1) <> ',' Index ← Index + 1 ENDWHILE // calculate the number of digits in LoyaltyPoints string LoyaltyPointsString ← MID(Line, 8, Index - 8) LoyaltyPoints ← STR_TO_NUM(LoyaltyPointsString) IF DAYINDEX(TODAY()) = 2 THEN LoyaltyPoints ← LoyaltyPoints + 10 ENDIF RETURN LoyaltyPoints ENDFUNCTION </pre>	
---------	--	--

34. O/N 24/P21/Q1/c

1(c)	One mark per row:	2						
	<table><tr><th>Variable name</th><th>Data type</th></tr><tr><td>HighRate</td><td>Boolean</td></tr><tr><td>TaxPayable</td><td>Real</td></tr></table>	Variable name	Data type	HighRate	Boolean	TaxPayable	Real	
Variable name	Data type							
HighRate	Boolean							
TaxPayable	Real							

35. O/N 24/P21/Q6/a

6(a)	One mark per point: - Fewer lines of code are needed - The program is simpler/ less complex // Program is easier to design / code / maintain / modify / test / debug - Direct access to days in a month / data (using month number as index) // Can use index / month to (directly) access days in month / data Max 2 marks	2
------	--	----------

36. O/N 24/P21/Q8/b,c

8(b)	Example Solution: <pre> PROCEDURE CheckAll(CNum : INTEGER) DECLARE Index, Count, ThisMark: INTEGER Count ← 0 OPENFILE "GRList.txt" FOR WRITE FOR Index ← 1 to CNum ThisMark ← Result[Index, 1] // 2D array: mark + ID IF CheckMark(ThisMark) = TRUE THEN WRITE "GRList.txt", NUM_TO_STR(Result[Index, 2]) Count ← Count + 1 ENDIF NEXT Index CLOSEFILE "GRList.txt" OUTPUT "There are ", Count, " papers to check" ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameter and ending. 2 Open file in write mode and subsequently close 3 Loop through all CNum candidates 4 Extract candidate mark from Result array in a loop 5 Use of CheckMark() with candidate mark as parameter in a loop 6 Test return value and if TRUE then increment Count in a loop 7 ... write ID to file ... 8 ... after conversion to a string in a loop 9 Final output of message giving number of papers to check not in a loop Max 8 marks	8
8(c)	The file will need to be opened in <u>APPEND</u> mode	1

37. O/N 24/P22/Q1/d

1(d)	One mark per row: <table border="1" data-bbox="284 1541 1310 1832"> <thead> <tr> <th>Variable name</th><th>Used to store</th><th>Data type</th></tr> </thead> <tbody> <tr> <td>Name</td><td>a customer name</td><td>STRING</td></tr> <tr> <td>Index</td><td>an array index</td><td>INTEGER</td></tr> <tr> <td>Result</td><td>the result of the division of any two non-zero numbers</td><td>REAL</td></tr> </tbody> </table>	Variable name	Used to store	Data type	Name	a customer name	STRING	Index	an array index	INTEGER	Result	the result of the division of any two non-zero numbers	REAL	3
Variable name	Used to store	Data type												
Name	a customer name	STRING												
Index	an array index	INTEGER												
Result	the result of the division of any two non-zero numbers	REAL												

1(c)	Features include: MP1 Editor MP2 <u>Auto</u>- (syntax) complete / <u>auto</u> correction // identify undeclared variables(s) MP3 Prettyprint / <u>auto</u>-indentation / <u>auto</u> (structure) highlighter MP4 <u>Dynamic</u> syntax checking MP5 Expand / collapse code blocks MP6 Context sensitive prompts Max 2 marks	2
------	--	----------

38. O/N 24/P22/Q2/b(i)

2(b)(i)	DECLARE CardNumber : ARRAY [1:100, 1:2] OF STRING MP1 Correct dimensions MP2 All other parts of the statement correct	2
---------	--	----------

39. O/N 24/P22/Q3/a,b

3(a)(i)	SP: 1 OnStack: 0 One mark for both correct values	1
3(a)(ii)	MP1 Unused values cannot be popped / taken off the stack // initialised values would never be used / unused elements cannot be accessed MP2 ... until a value has first been pushed / written // overwrites previous value	2
3(b)	Example solution: <pre> FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN DECLARE ReturnValue : BOOLEAN IF OnStack = 60 / >59 // SP = 61 / SP > 60 // SP outside the range 1 to 60 THEN RETURN FALSE // Stack is already full ENDIF ThisStack[SP] ← ThisValue SP ← SP + 1 OnStack ← OnStack + 1 RETURN TRUE ENDFUNCTION1 </pre> Mark as follows: One mark per gap	4

40. O/N 24/P22/Q6/a

6(a)	Example solution: <pre> PROCEDURE Special() DECLARE Index : INTEGER DECLARE Filling1, Filling2 : STRING REPEAT Index ← INT(RAND(35)) + 1 UNTIL Filling[Index] <> "" Filling1 ← Filling[Index] REPEAT Index ← INT(RAND(35)) + 1 UNTIL Filling[Index] <> "" AND Filling1 <> Filling[Index] Filling2 ← Filling[Index] REPEAT Index ← INT(RAND(10) + 1) UNTIL Bread[Index] <> "" OUTPUT "The daily special is ", Filling1, " and ", — Filling2, " on ", Bread[Index], " bread." ENDPROCEDURE </pre> Mark as follows: MP1 Loop for Filling 1, avoiding unused elements MP2 Loop for Filling 2 avoiding unused elements MP3 Check Filling 2 is different from Filling 1 – could correctly compare either the indices or the array contents MP4 Loop for Bread, avoiding unused elements MP5 Using <code>RAND(10) / RAND(35)</code> MP6 Completely correct use of <code>RAND()</code> – including <code>INT()</code> and <code>+1</code> in all cases MP7 Correct output - once only – following a reasonable attempt at selection of fillings and bread	7
------	--	----------

41. O/N 24/P22/Q8/a,b,c,d

8(a)	Example solution: <pre> PROCEDURE Assign(ThisRole : STRING, ThisPlayer : INTEGER) DECLARE Index : INTEGER DECLARE Done : BOOLEAN Done ← FALSE Index ← 1 WHILE Index < 46 AND Done = FALSE IF Character[Index].Player = 0 AND __ Character[Index].Role = ThisRole THEN Character[Index].Player ← ThisPlayer Done ← TRUE ELSE Index ← Index + 1 ENDIF ENDWHILE IF Done = TRUE THEN OUTPUT Character[Index].Name, " the ", __ Character[Index].Role, __ " has been assigned to player ", ThisPlayer ELSE OUTPUT "No characters with this role are available" ENDIF ENDPROCEDURE </pre> Mark as follows: MP1 Loop until 'found' or all 45 elements considered MP2 Test of <code>Player</code> field – i.e. not value in a loop MP3 ... AND <code>Role</code> – i.e. match for ThisRole parameter in a loop MP4 If available character found, assign <code>ThisPlayer</code> to the character in a loop MP5 When character found set termination condition/flag MP6 Both OUTPUT messages logically <u>correctly placed</u> MP7 Both OUTPUT statements <u>correctly formed</u>	7
------	---	----------

8(b)	Example solution: <pre> PROCEDURE Save() DECLARE Index : INTEGER DECLARE Line : STRING CONSTANT SEP = '^' OPENFILE "SaveFile.txt" FOR WRITE FOR Index ← 1 TO 45 Line ← NUM_TO_STR(Character[Index].Player) & SEP Line ← Line & Character[Index].Role & SEP Line ← Line & Character[Index].Name & SEP Line ← Line & NUM_TO_STR(Character[Index].Level) WRITEFILE "SaveFile.txt", Line NEXT Index CLOSEFILE "SaveFile.txt" ENDPROCEDURE </pre> Mark as follows: MP1 Declaration of local integer for <code>Index</code> (and string type for <code>Line</code>) MP2 Open "SaveFile.txt" in write mode and subsequently close MP3 Loop through 45 elements MP4 Attempt to form <code>Line</code> - four fields in a loop MP5 Correct use of <code>NUM_TO_STR()</code>x2 (<code>Player</code> and <code>Level</code>) in a loop MP6 Correct use of <u>three</u> <code>&</code>separator<code>&</code> strings in a loop MP7 Line from MP4 written to file in a loop	7
8(c)	MP1 Encode <code>Status</code> as a character / string MP2 Append the '^' separator and the character/string	2
8(d)	MP1 Method: Create a filename suffix which is incremented for each file save MP2 Example: SaveFile01.txt, SaveFile02.txt	2

42. O/N 24/P23/Q1/b

1(b)	<table> <tr> <th>Variable</th><th>Example data value</th><th>Data type</th></tr> <tr> <td>Available</td><td>TRUE</td><td>BOOLEAN</td></tr> <tr> <td>Received</td><td>"18/04/2021"</td><td>STRING</td></tr> <tr> <td>Index</td><td>100</td><td>INTEGER</td></tr> </table> One mark per row	Variable	Example data value	Data type	Available	TRUE	BOOLEAN	Received	"18/04/2021"	STRING	Index	100	INTEGER	3
Variable	Example data value	Data type												
Available	TRUE	BOOLEAN												
Received	"18/04/2021"	STRING												
Index	100	INTEGER												

43. O/N 24/P23/Q3/a

3(a)(i)	0 is not a valid <u>array / List index</u> value // No 0 th element in the array	1
3(a)(ii)	0 to 200	1

44. O/N 24/P23/Q4/a,b

4(a)	One mark per highlighted part: <pre> FOR Index ← 1 TO <u>5</u> Lower ← GB[Index] - 2 Upper ← <u>GB[Index] + 2</u> // <u>Lower + 4</u> IF Mark <u>>= Lower</u> AND Mark <u><= Upper</u> THEN //IF Mark <u><= Upper</u> AND Mark <u>>= Lower</u> THEN OUTPUT "Check this paper" ENDIF NEXT Index </pre>	4
4(b)(i)	MP1 Use Mark as the <u>index</u> to the Check array // to specify an array element MP2 If value of indexed element is TRUE, then the paper will need to be checked	2

4(b)(ii)	Example:solution <pre> PROCEDURE GBInitialise() DECLARE GBIndex, GBMark, ThisIndex : INTEGER FOR ThisIndex ← 0 TO 75 Check[ThisIndex] ← FALSE // initialise all values NEXT ThisIndex FOR GBIndex ← 1 TO 5 GBMark ← GB[GBIndex] FOR ThisIndex ← GBMark - 2 TO GBMark + 2 Check[ThisIndex] ← TRUE //Set GBMark ± 2 to TRUE NEXT ThisIndex NEXT GBIndex ENDPROCEDURE </pre> Mark as follows: MP1 Procedure heading and ending MP2 Initialise Check array to FALSE MP3 Loop through GB array MP4 Extract the GB mark MP5 Loop for 5 elements across GBMark ± 2 // Check if within GBMark ± 2 MP6 Assign each element of Check array to TRUE ALTERNATIVE – Example using selection Version 1 <pre> FOR ThisIndex ← 0 TO 75 CASE ThisIndex GB[1] - 2 TO GB[1] + 2 : Check[ThisIndex] ← TRUE GB[2] - 2 TO GB[2] + 2 : Check[ThisIndex] ← TRUE GB[3] - 2 TO GB[3] + 2 : Check[ThisIndex] ← TRUE GB[4] - 2 TO GB[4] + 2 : Check[ThisIndex] ← TRUE GB[5] - 2 TO GB[5] + 2 : Check[ThisIndex] ← TRUE OTHERWISE: Check[ThisIndex] ← FALSE ENDCASE NEXT ThisIndex </pre>	6
----------	---	----------

4(b)(ii)	ALTERNATIVE – Example using selection Version 2 <pre> DECLARE ThisIndex, GBIndex : INTEGER DECLARE Lower, Higher : INTEGER FOR ThisIndex ← 0 TO 75 For GBIndex ← 1 TO 5 Lower ← GB[GBIndex] - 2 Higher ← GB[GBIndex] + 2 IF ThisIndex >= Lower AND ThisIndex <= Higher THEN Check[ThisIndex] ← TRUE ELSE Check[ThisIndex] ← FALSE ENDIF NEXT Index NEXT ThisIndex </pre> Mark as follows: MP1 Procedure heading and ending MP2 Loop through <i>Check</i> array// loop from 0 to 75 MP3 Attempt at using CASE / Selection structure with five clauses/ five checks for range MP4 Extract GB grade MP5 Compare <i>ThisIndex</i> with each element in GB array ± 2 MP6 Assign each element of <i>Check</i> array to TRUE if in range or FALSE if not in range	
----------	---	--

45. O/N 24/P23/Q8/a,b,c

8(a)	Example solution <pre> PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING) DECLARE Index, Num, Total : INTEGER Num ← 0 Total ← 0 FOR Index ← 1 TO 45 IF Character[Index].Player = ThisPlayer AND ____ Character[Index].Role = ThisRole THEN Num ← Num + 1 Total ← Total + Character[Index].SkillLevel NEXT Index IF Num > 0 THEN OUTPUT "Player ", ThisPlayer, ____ " has ", Num, " characters with the role of ", ThisRole, " and the total skill level is ", Total ELSE OUTPUT "No characters with that role are assigned to this player" ENDIF ENDPROCEDURE </pre> MP1 Initialisation of local integers for Num and Total MP2 Loop through 45 elements MP3 Attempt to check Player and Role fields in a loop MP4 Correctly compare Player field with parameter in a loop MP5 Correctly compare Role field with parameter in a loop MP6 ... if player and role found, increment Num and sum Skill Total in a loop MP7 Test for any matches after the loop MP8 Both possible OUTPUT statements correctly formed following an attempt at MP6 but outputting one only Max 7 marks	7
------	--	----------

8(b)	Example solution: <pre> PROCEDURE Restore() DECLARE Index : INTEGER DECLARE Line : STRING OPENFILE "SaveFile.txt" FOR READ FOR Index ← 1 TO 45 READFILE "SaveFile.txt", Line Character[Index].Player ← STR_TO_NUM(Extract(Line, 1)) Character[Index].Role ← Extract(Line, 2) Character[Index].Name ← Extract(Line, 3) Character[Index].Skill ← STR_TO_NUM(Extract(Line, 4)) NEXT Index CLOSEFILE "SaveFile.txt" ENDPROCEDURE </pre> Mark as follows: MP1 Open the file in read mode and subsequently close MP2 Loop through 45 elements MP3 Read a line from the file in a loop MP4 Attempt to use <code>Extract()</code> in a loop MP5 Correct use of <code>Extract()</code> for all fields in a loop MP6 Use of <code>STR_TO_NUM()</code> on Player and Skill in a loop MP7 Completely correct extraction and assignment of all fields in a loop	7
8(c)	MP1 The <u>Character array</u> / <u>character data</u> can be saved before the program is closed MP2 Allowing the game to continue using the same data / from the point it was saved	2

46. M/J 24/P21/Q1/a

1(a)	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td>A ← LEFT(MyName, 1)</td><td>CHAR / STRING</td></tr><tr><td>B ← Total * 2</td><td>INTEGER / REAL</td></tr><tr><td>C ← INT(ItemCost) / 3</td><td>REAL</td></tr><tr><td>D ← "Odd OR Even"</td><td>STRING</td></tr></table>		Assignment statement	Data type	A ← LEFT(MyName, 1)	CHAR / STRING	B ← Total * 2	INTEGER / REAL	C ← INT(ItemCost) / 3	REAL	D ← "Odd OR Even"	STRING	4
	Assignment statement	Data type											
	A ← LEFT(MyName, 1)	CHAR / STRING											
	B ← Total * 2	INTEGER / REAL											
	C ← INT(ItemCost) / 3	REAL											
	D ← "Odd OR Even"	STRING											
One mark per row													

47. M/J 24/P21/Q3/a,b

3(a)	One mark per point: 1 The PS contains a null pointer 2 The PF points to the first element on the free list 3 All the nodes are on the free list	3							
3(b)	Max 2 marks for 'Variables': <table><tr><td>The two variables will be of type Integer</td></tr><tr><td>The two variables will be used as pointers / indexes to the arrays.</td></tr><tr><td>The values stored in the two variables will indicate the first element in each list</td></tr><tr><td>The first 1D array will be of type String</td></tr><tr><td>The first 1D array will be used to store the values // data items // User IDs</td></tr><tr><td>The second 1D array will be of type Integer</td></tr><tr><td>The second 1D array will be used to store the pointers // point to next item</td></tr></table> Mark as follows: One mark for each of the first three rows One mark for both Array 1 rows One mark for both Array 2 rows	The two variables will be of type Integer	The two variables will be used as pointers / indexes to the arrays.	The values stored in the two variables will indicate the first element in each list	The first 1D array will be of type String	The first 1D array will be used to store the values // data items // User IDs	The second 1D array will be of type Integer	The second 1D array will be used to store the pointers // point to next item	5
The two variables will be of type Integer									
The two variables will be used as pointers / indexes to the arrays.									
The values stored in the two variables will indicate the first element in each list									
The first 1D array will be of type String									
The first 1D array will be used to store the values // data items // User IDs									
The second 1D array will be of type Integer									
The second 1D array will be used to store the pointers // point to next item									

48. M/J 24/P21/Q4

4	Example: <pre>FUNCTION Check() RETURNS STRING DECLARE Odd, Even, Index : INTEGER Odd ← 0 Even ← 0 FOR Index ← 1 TO 100 IF Index MOD 2 = 0 THEN Even ← Even + Data[Index] ELSE Odd ← Odd + Data[Index] ENDIF NEXT Index ENDFUNCTION</pre> Mark as follows: 1. Function heading, ending and return type 2. Declare local variables Odd, Even and Index as integers 3. Initialise Odd and Even 4. Loop for 100 iterations // through array 5. Sum Odd and Even element values in a loop 6. Compare Odd and Even after the loop and Return appropriate string	6
---	--	---

49. M/J 24/P21/Q8/b

8(b)	Example: <pre> FUNCTION Stage_1(StudentName : STRING) RETURNS INTEGER DECLARE OldFile, NewFile, Line : STRING DECLARE Count : INTEGER OldFile ← StudentName & "_src.txt" NewFile ← StudentName & "_S1.txt" OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE Count ← 0 WHILE NOT EOF(OldFile) READFILE OldFile, Line Line ← DeleteComment(Line) IF LENGTH(Line) <> 0 THEN WRITEFILE NewFile, Line Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE OldFile CLOSEFILE NewFile RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Generate filenames condone missing “_” 2 Open both files in correct modes and subsequently close 3 Loop to EOF(OldFile) 4 Read a line from OldFile and execute DeleteComment() in a loop 5 Skip blank lines after DeleteComment() in a loop 6 Write Line to stage 1 file and increment Count 7 Return the number of lines after a reasonable attempt at counting	7
------	---	---

50. M/J 24/P22/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE Component DECLARE Item_Num : INTEGER DECLARE Reject : BOOLEAN DECLARE Stage : CHAR DECLARE Limit_1 : REAL DECLARE Limit_2 : REAL ENDTYPE </pre> Mark as follows: 1 One mark for TYPE and ENDTYPE statements 2 One mark for Item_Num and Reject fields 3 One mark for Stage field 4 One mark for Limit fields as REAL	4
---------	--	---

3(a)(ii)	<u>DECLARE Item : ARRAY [1:2000] OF Component //</u> <u>DECLARE Item : ARRAY [2000] OF Component //</u> <u>DECLARE Item : ARRAY [0:1999] OF Component</u> One mark per underlined phrase	2
3(b)	One mark per point: 1 Allows for iteration / can use a loop to access the records / data items 2 Use of index to directly access a record in the array // Example of simplification of code e.g. use of dot notation Item[1].Stage 3 Simplifies the code / algorithm // Reduces duplication of code // Program easier to write / understand / maintain / test / debug // Data items/record easier to search / sort / manipulate	3

51. M/J 24/P22/Q6/b

6(b)(i)	A (very) large file is created // redundant zeroes are stored in the file	1
6(b)(ii)	One mark for: - Values are delimited by a special character / a separator character - First character indicates sample length Max 1 mark	1
6(b)(iii)	The algorithm to store / extract / separate the individual values is more complex / takes longer to execute / run / process NE Algorithm is more complicated	1

52. M/J 24/P22/Q8/c

8(c)	Example: <pre> PROCEDURE Stage_2(F1, F2 : STRING) DECLARE Line : STRING DECLARE Count : INTEGER Count ← 0 OPEN F1 FOR READ OPEN F2 FOR APPEND WHILE NOT EOF(F1) READFILE F1, Line Line ← DeleteSpaces(Line) Line ← DeleteComment(Line) IF Line <> "" THEN WRITEFILE F2, Line // skip blank lines ELSE Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE F1 CLOSEFILE F2 OUTPUT Count, " blank lines were removed" ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, parameters, ending 2 Open both files in correct modes and subsequently close 3 Loop to EOF(F1) 4 Read a line from F1 in a loop 5 Assign return values from DeleteComment() and DeleteSpaces() in a loop 6 Check return value following both MP5 function calls is not an empty string and if so write to F2 in a loop 7 Count the blank lines in a loop 8 Output number of blank lines removed following a reasonable attempt after the loop	8
------	---	---

53. M/J 24/P23/Q3/a

3(a)(i)	Example solution using AND <pre>IF Batch[ThisIndex].Weight >= Min AND Batch[ThisIndex].Weight <= Max THEN</pre> Alternative solution using OR <pre>IF Batch[ThisIndex].Weight < Min OR Batch[ThisIndex].Weight > Max THEN</pre> Mark as follows: 1. Reference to <code>Batch[ThisIndex].Weight</code> 2. A valid check for one boundary 3. A valid check for other boundary with correct logic operator	3
3(a)(ii)	One mark for either: • Set the <code>Item_ID</code> field to an empty string / NULL / invalid value • Set <code>Weight</code> to <code><= 0</code> / zero	1

54. M/J 24/P23/Q4/b

4(b)(i)	(1D) <u>array</u> of <u>20</u> <u>reals</u> Marks as follows: 1 mark for array 1 mark for 20 reals	2
4(b)(ii)	One mark per point: 1 (Multiple instances referenced via a single identifier so) fewer identifiers needed 2 Easier to process / search / organise / access the data // Values may be accessed via a loop-controlled variable / an index // An array / data can be iterated through 3 Makes the program/algorithm easier to write / design / understand / maintain / modify // Simplifies the program // Easier to debug / test	3

55. M/J 24/P23/Q8/b

8(b)	Example: <pre> PROCEDURE FindModules(FileName : STRING) DECLARE Line : STRING DECLARE Index, LineNum : INTEGER OPENFILE FileName FOR READ Index ← 1 LineNum ← 0 WHILE NOT EOF(FileName) READFILE FileName, Line LineNum ← LineNum + 1 Line ← Header(Line) IF Line <> "" THEN ModInfo[Index, 1] ← NUM_TO_STR(LineNum) ModInfo[Index, 2] ← LEFT(Line, 1) ModInfo[Index, 3] ← RIGHT(Line, LENGTH(Line) - 1) Index ← Index + 1 ENDIF ENDWHILE CLOSEFILE FileName ENDPROCEDURE </pre> Mark as follows: 1 Open file in READ mode and subsequently close 2 Loop to EOF(FileName) 3 Read a line from the file and maintain LineNum in a loop 4 Call Header() and use return value in a loop 5 Test return value for "" in a loop 6 Attempt at all three-array assignment for all columns in correct row 7 Correct values assigned to all columns of array 8 Maintain correct array row index	8
------	---	----------

56. O/N 23/P21/Q1/d

1(d)	One mark per point: • ThisValue: INTEGER • Check: REAL • Level: STRING	3
------	---	----------

57. O/N 23/P21/Q3/a,b

3(a)	<table><tr><th>Array element</th><th>Data</th></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td>D1</td></tr><tr><td>4</td><td>D3</td></tr><tr><td>3</td><td>D4</td></tr><tr><td>2</td><td>D5</td></tr><tr><td>1</td><td>D2</td></tr></table> Variables <table><tr><td>TopOfStack</td><td>5</td></tr><tr><td>BottomOfStack</td><td>1</td></tr></table> MP1 all values in the order and location shown MP2 TopOfStack value is index of element containing D1 MP3 BottomOfStack value is index of element containing D2	Array element	Data	8		7		6		5	D1	4	D3	3	D4	2	D5	1	D2	TopOfStack	5	BottomOfStack	1	3
Array element	Data																							
8																								
7																								
6																								
5	D1																							
4	D3																							
3	D4																							
2	D5																							
1	D2																							
TopOfStack	5																							
BottomOfStack	1																							
3(b)	MP1 If TopOfStack = 8 // (stack) full then return FALSE MP2 Otherwise, increment TopOfStack MP3 Use TopOfStack as an index to the Array MP4 Set the element at this index / location / position to the value / data / item being added MP5 Return TRUE	5																						

58. O/N 23/P21/Q4/a,b

4(a)	<pre> FUNCTION TooMany(Search : STRING, Max : INTEGER) RETURNS BOOLEAN DECLARE Count, Index : INTEGER Count ← 0 FOR Index ← 1 TO 150 IF Data[Index] = Search THEN Count ← Count + 1 ENDIF NEXT Index IF Count > Max THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> MP1 Function heading, ending and return type MP2 Declare Count and Index as integers MP3 Initialise Count MP4 Loop (any type) for 150 iterations MP5 Compare Data element with parameter - if equal, increment Count in a loop MP6 Compare Count with Max and return Boolean in both cases outside the loop	6
4(b)	MP1 Test for row being even number MP2 Test for either column value equal to Search <pre> IF <u>Row MOD 2 = 0</u> AND <u>_____</u> (<u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u>) THEN </pre> ALTERNATIVE using nested IFs: <pre> IF <u>Row MOD 2 = 0</u> THEN IF <u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u> THEN </pre> MP3 Selection structure is either: • Single IF statement using AND, or • Two nested IFs using AND, or • Single IF and the use of a two-iteration loop Either of these structures correctly formed scores the mark ALTERNATIVE SOLUTION: A FOR loop using 'STEP 2' <pre> FOR Row ← 2 TO 150 / NEXT Row <u>STEP 2</u> <u>Data[Row, 1] = Search OR Data[Row, 2] = Search</u>) THEN </pre>	3

59. O/N 23/P21/Q8/a,b

8(a)	<pre> PROCEDURE SendFile(FileName, DestID : STRING, Port : INTEGER) DECLARE FileData : STRING CONSTANT STX = CHR(02) CONSTANT ETX = CHR(03) OPENFILE FileName FOR READ WHILE NOT EOF(FileName) READFILE FileName, FileData FileData ← STX & DestID & MyID & FileData & ETX CALL Transmit(FileData, Port) ENDWHILE CLOSEFILE FileName CALL Transmit(STX & DestID & MyID & "*****" & ETX, Port) ENDPROCEDURE </pre> Mark as follows: MP1 OPEN file in READ mode – using parameter - and subsequently CLOSE MP2 Conditional loop to EOF() MP3 Use of READFILE to get a line from the file MP4 'Attempt' to form a message (minimum is DestID, MyID, FileData) MP5 Message formed is completely correct MP6 Call Transmit() with correct MP4 string in a loop MP7 Transmit the "*****" message (all parts present) after the loop	7
8(b)	Max 2 marks MP1 Indicates that all the lines of the file have been sent // it is the end of the transmission / file transfer MP2 So that the receiving program can stop waiting for further data MP3 The file can be closed / saved	2

60. O/N 23/P22/Q1/c

1(c)	One mark per point Max 3 marks Declaration 1 Declare a composite / record (type) 2 Declare an array of the given composite / record (type) Expansion of record: 3 ... containing all data items required // containing items of different data types Expansion of array: 4 ... where each array element represents data for one order / customer (order)	3
------	---	----------

1(a)

4

One mark for each row with appropriate variable name and data type

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer	CustomerName	STRING
3	The number of items in the order	NumItems	INTEGER
TRUE	A value to indicate whether this is a new customer	NewCustomer	BOOLEAN
15.75	The deposit paid by the customer	Deposit	REAL

61. O/N 23/P22/Q3/a,b

3(a)

5

Variable	Value
Start_Pointer	4
Free_List_Pointer	5

Index	Data Array	Pointer Array
1	D32	2
2	D11	3
3	D100	0
4	D40	1
5	F1	6
6	F2	7
7	F3	8
8	F4	0

Mark as follows:

- One mark for Start_Pointer value
- One mark each group of row(s):
- 2
- 3 and 4
- 5
- 7 and 8

For null pointer: accept 0 / ∅ / an out-of-bound index value less than 1, greater than 8

3(b)	One mark per step: 1 Assign the data item D6 to F1 2 Set the pointer of this node to point to D11 3 Set Ptr2 to point to F2 4 Set pointer of D32 to point to D6	4
------	--	----------

62. O/N 23/P22/Q6/a,b(iii)

6(a)	Example Solution <pre> PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER) DECLARE FileName, Suffix : STRING DECLARE Count : INTEGER FOR Count ← 1 TO NumFiles Suffix ← NUM_TO_STR(Count) WHILE LENGTH(Suffix) <> 3 Suffix ← '0' & Suffix ENDWHILE FileName ← NameRoot & '.' & Suffix OPENFILE FileName FOR WRITE WRITEFILE FileName, "This is File " & FileName CLOSEFILE FileName NEXT Count ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, including parameters, and ending 2 Loop for NumFiles iterations 3 Attempt to create filename suffix using NUM_TO_STR() in a loop 4 Completely correct filename 5 OPENFILE in WRITE mode and subsequent CLOSE in a loop 6 WRITE initial first line to the file in a loop	6
6(b)(iii)	Read	1

63. O/N 23/P22/Q8/a,c

8(a)	Solution using a loop Example solution – using a loop <pre> FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER DECLARE Index, DNum, Port : INTEGER DNum ← STR_TO_NUM(ThisDest) Index ← 1 Port ← -1 REPEAT IF RouteTable[Index, 1] <> -1 THEN IF DNum >= RouteTable[Index, 1] AND DNum <= RouteTable[Index, 2] THEN Port ← RouteTable[Index, 3] ENDIF ENDIF Index ← Index + 1 UNTIL Index = 7 OR Port <> -1 // end of array or range found RETURN Port ENDFUNCTION </pre> Mark as follows Max 7: 1. Function heading and ending including parameter and return type 2 Convert parameter to a number 3 (Conditional) loop through array 4 Skip unused element in a loop 5 Attempt to check one range with destination in a loop 6 Test all ranges correctly with destination in a loop 7 Store port value if destination matched in a loop (and exit loop) 8 Return port value including -1 if no match found Solution using selection statement(s) Example solution <pre> FUNCTION GetPort(ThisDest : STRING) RETURNS INTEGER DECLARE Index, DNum, Port : INTEGER DNum ← STR_TO_NUM(ThisDest) Port ← -1 IF RouteTable[1, 1] <> -1 AND RouteTable[1, 1] >= DNum AND RouteTable[1, 2] <= DNum THEN </pre>	7
------	---	----------

8(a)	<pre> Port ← RouteTable[1, 3] END IF IF RouteTable[2, 1] <> -1 AND RouteTable[2, 1] >= DNum AND __ RouteTable[2, 2] <= DNum THEN Port ← RouteTable[2, 3] END IF IF RouteTable[3, 1] <> -1 AND RouteTable[3, 1] >= DNum AND __ RouteTable[3, 2] <= DNum THEN Port ← RouteTable[3, 3] END IF IF RouteTable[4, 1] <> -1 AND RouteTable[4, 1] >= DNum AND __ RouteTable[4, 2] <= DNum THEN Port ← RouteTable[4, 3] END IF IF RouteTable[5, 1] <> -1 AND RouteTable[5, 1] >= DNum AND __ RouteTable[5, 2] <= DNum THEN Port ← RouteTable[5, 3] END IF IF RouteTable[6, 1] <> -1 AND RouteTable[6, 1] >= DNum AND __ RouteTable[6, 2] <= DNum THEN Port ← RouteTable[6, 3] END IF RETURN Port ENDFUNCTION </pre> Mark as follows Max 7: 1 Function heading and ending including parameter and return type 2 Convert parameter to a number 3 Skip all unused elements 4 Attempt to check one range 5 Check two ranges with destination correctly 6 Check all ranges with destination correctly 7 Store port value if destination matched 8 Return port value including -1 if no match found	
------	--	--

8(c)(i)	One mark per point Max 3 marks: Decide on scenario and mark accordingly. Scenario one: • If more than one line is / all lines are stored on the stack (before line(s) are removed) • The stack operates as a FILO device // Last item added to stack will be in first item out • So lines in the file appear out of sequence Scenario two: • Stack is Full • Not all lines can be stored on the stack • so resulting file will not contain all the original lines Scenario three: • (All) the data in a line read can't be stored on the stack • Stack elements have not been allocated enough memory • so only part of each line is stored in the file Scenario four: • Stack is empty • The stack is being read faster than it is being written to • so blank lines may be inserted into the file	3
8(c)(ii)	Queue	1

8(b)	Example solution <pre> PROCEDURE ProcessMsg(ThisMsg : STRING) DECLARE ThisDest : STRING DECLARE Response : BOOLEAN DECLARE StackNum : INTEGER IF LENGTH(ThisMsg) >= 4 THEN ThisDest ← LEFT(ThisMsg, 3) IF ThisDest = MyID THEN // It's for this computer StackNum ← 1 ELSE StackNum ← 2 ENDIF Response ← StackMsg(ThisMsg, StackNum) IF Response = FALSE THEN OUTPUT "Message discarded - no room on stack" ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Ignore message if data field is empty 2. Extract <code>ThisDest</code> from <code>ThisMsg</code> 3. Test if destination is this computer 4. Attempt to use <code>StackMsg()</code> 5. Fully correct use of <code>StackMsg()</code> for both cases / stacks 6. Test <code>StackMsg()</code> return value for both cases 7. Following a reasonable attempt at MP6 output warning if <code>StackMsg()</code> returns <code>FALSE</code>	7
------	---	---

64. O/N 23/P23/Q1/a

1(a)			4									
	<table><tr><th>Assignment statement</th><th>Data type</th></tr><tr><td><u>MyVar1</u> ← Total1 / Total2</td><td>REAL</td></tr><tr><td><u>MyVar2</u> ← 27/10/2023</td><td>DATE</td></tr><tr><td><u>MyVar3</u> ← "Sum1 / Sum2"</td><td>STRING</td></tr><tr><td><u>MyVar4</u> ← Result1 AND Result2</td><td>BOOLEAN</td></tr></table>	Assignment statement	Data type	<u>MyVar1</u> ← Total1 / Total2	REAL	<u>MyVar2</u> ← 27/10/2023	DATE	<u>MyVar3</u> ← "Sum1 / Sum2"	STRING	<u>MyVar4</u> ← Result1 AND Result2	BOOLEAN	
Assignment statement	Data type											
<u>MyVar1</u> ← Total1 / Total2	REAL											
<u>MyVar2</u> ← 27/10/2023	DATE											
<u>MyVar3</u> ← "Sum1 / Sum2"	STRING											
<u>MyVar4</u> ← Result1 AND Result2	BOOLEAN											

65. O/N 23/P23/Q2/b,c

2(b)	MP1 Simplifies the algorithm // easier to write / understand / test / debug MP2 It is possible to iterate through the values // can use a loop // allows the storage of many values using a single identifier	2
------	---	---

2(c)	One mark per underlined section <u>DECLARE</u> <u>Data</u> : <u>ARRAY[1:120]</u> OF <u>REAL</u>	2
------	---	----------

2(a)	<pre> graph TD Start([START]) --> Init[Set Min to Data[1] Set MinIndex to 1] Init --> SetIndex[Set Index to 2] SetIndex --> IsIndex{Is Index > 30 ?} IsIndex -- YES --> Output[/OUTPUT MinIndex/] Output --> End([END]) IsIndex -- NO --> IsData{Is Data[Index] < Min ?} IsData -- YES --> Update[Set MinIndex to Index Set Min to Data[Index]] IsData -- NO --> Increment[Set Index to Index + 1] Update --> IsIndex Increment --> IsIndex </pre> MP1 Initialise Min to first value in Data and MinIndex to 1 MP2 Loop through 29 more values (or 30 values in total) MP3 Compare element from Data[] with Min MP4 Set new Min AND save MinIndex when element value < Min in a loop MP5 Output MinIndex	5
------	---	----------

66. O/N 23/P23/Q3/a,b

3(a)	<table><tr><th>Index</th><th>Array</th></tr><tr><td>1</td><td></td></tr><tr><td>2</td><td>D3</td></tr><tr><td>3</td><td>D4</td></tr><tr><td>4</td><td>D1</td></tr><tr><td>5</td><td>D2</td></tr><tr><td>6</td><td>D5</td></tr><tr><td>7</td><td></td></tr><tr><td>8</td><td></td></tr></table> <table><tr><th colspan="2">Variable</th></tr><tr><td>FrontOfQueue</td><td>2</td></tr><tr><td>EndOfQueue</td><td>6</td></tr><tr><td>NumItems</td><td>5</td></tr></table> : MP1 D3, D4, D1, D2 and D5 in question order or reversed - in any five consecutive locations MP2 FoQ value matches index with D3 MP3 EoQ value matches index with D5	Index	Array	1		2	D3	3	D4	4	D1	5	D2	6	D5	7		8		Variable		FrontOfQueue	2	EndOfQueue	6	NumItems	5	3
Index	Array																											
1																												
2	D3																											
3	D4																											
4	D1																											
5	D2																											
6	D5																											
7																												
8																												
Variable																												
FrontOfQueue	2																											
EndOfQueue	6																											
NumItems	5																											
3(b)	MP1 If NumItems <u>is / = 8 // (queue) full</u> then jump to step 6 MP2 Increment EndOfQueue MP3 If EndOfQueue = 9 then set EndOfQueue to 1 MP4 Increment NumItems MP5 Set the Element at the index ... MP6 stored in EndOfQueue to value/data/item being added Mark as follows: Steps 1 to 4: One mark for gaps filled as shown Step 5: One mark for 'element' and one mark for other two terms	6																										

67. O/N 23/P23/Q7/b(i)

7(b)(i)	<pre> TYPE MyType DECLARE RA : INTEGER DECLARE RB : STRING DECLARE RC : BOOLEAN ENDTYPE MP1 TYPE MyType . . . ENDTYPE MP2 RA as integer and RB as string MP3 RC as Boolean </pre>	3
---------	--	----------

68. O/N 23/P23/Q8/a

8(a)	<pre> PROCEDURE ReceiveFile(FileName : STRING) DECLARE FileData : STRING DECLARE CharCount : INTEGER CONSTANT Terminator = "*****" OPENFILE FileName FOR WRITE FileData ← GetData() CharCount ← 0 WHILE FileData <> Terminator WRITEFILE FileName, FileData CharCount ← CharCount + LENGTH(FileData) FileData ← GetData() ENDWHILE CLOSEFILE FileName OUTPUT CharCount (, " characters were written to ", FileName) ENDPROCEDURE </pre> MP1 OPEN file in WRITE mode and subsequently CLOSE MP2 Conditional loop until terminator received MP3 'Attempted' use of GetData() – Ignore CALL ... MP4 Fully correct use GetData() to return the data in a loop MP5 Maintain CharCount in a loop MP6 Write each line to the file - except the terminator in a loop MP7 Final output of message giving number of characters written outside loop	7
------	---	----------

69. M/J 23/P21/Q3/a

3(a)(i)	One mark for structure Structure: Record One mark for each point Advantage: • A set of data / all data related to one customer • of different types • is held under a single identifier/entity	4
3(a)(ii)	A (1D) <u>array</u> of <u>records</u> // An <u>array</u> of the given <u>type</u> could be used One mark per underlined word	2

70. M/J 23/P21/Q6

6	<pre> PROCEDURE Mix() DECLARE Count, Total ThisNum : INTEGER DECLARE ThisUser, ThisSample : INTEGER FOR ThisSample ← 1 TO 128 Count ← 0 Total ← 0 FOR ThisUser ← 1 TO 6 IF Sample[ThisUser, ThisSample] > 10 THEN Count ← Count + 1 Total ← Total + Sample[ThisUser, ThisSample] ENDIF NEXT ThisUser Result[ThisSample] ← INT(Total / Count) NEXT ThisSample ENDPROCEDURE </pre> Mark as follows: 1 Declaration and initialisation before inner loop of Count and Total 2 Outer Loop for 128 iterations 3 Inner loop for six iterations 4 Test for sample > 10 in a loop 5 and if true sum Total and increment Count 6 Calculate average value and assign to Result array after inner loop and within outer loop 7 Use of INT() / DIV to convert average to integer Max 6 Marks	6
----------	--	----------

71. M/J 23/P21/Q8/b,d,e

8(b)	<pre> FUNCTION CheckNewItem(NewLine : STRING) RETURNS BOOLEAN DECLARE NotFound : BOOLEAN DECLARE NewItemNum, ThisItemNum, ThisLine : STRING NotFound ← TRUE OPENFILE "Stock.txt" FOR READ NewItemNum ← LEFT(NewLine, 4) ThisItemNum ← "0000" //rogue initial value WHILE NOT EOF("Stock.txt") AND NotFound = TRUE AND__ ThisItemNum < NewItemNum READFILE("Stock.txt", ThisLine) //brackets optional ThisItemNum ← LEFT(ThisLine, 4) IF ThisItemNum = NewItemNum THEN NotFound ← FALSE ENDIF ENDWHILE CLOSEFILE "Stock.txt" RETURN NotFound ENDFUNCTION </pre> Mark as follows: 1 Open Stock.txt in READ mode and subsequently close 2 Extract NewItemNum from parameter 3 Conditional loop until EOF("Stock.txt") 4 ... OR NewItemNum found 5 ... OR when ThisItemNum < NewItemNum 6 Read a line from Stock.txt AND extract ThisItemNum in a loop 7 If ThisItemNum = NewItemNum then terminate loop / set flag in a loop 8 Return Boolean after reasonable attempt Max 7 marks Max 6 if function wrapper (heading and ending) missing or incorrect	7
8(d)	Append	1
8(e)	One mark for each part: • The algorithm / search / iteration can stop /only iterates • if the current value read from the file // current line in file • is greater than the value being searched for	3

72. M/J 23/P22/Q1/c

1(c)	One mark per bullet point: • BOOLEAN • REAL • STRING	3
------	--	----------

73. M/J 23/P22/Q3/a

3(a)(i)	One mark per point: 1 Check that the queue is not full 2 EoQ pointer will move to point to location 9 3 Data item Orange will be stored in location referenced by EoQ pointer 4 EoQ pointer will move to point to location 0 5 Data item Yellow will be stored in location referenced by EoQ pointer Note: max 4 marks	4
3(a)(ii)	7	1

74. M/J 23/P22/Q8/b

8(b)	<pre> PROCEDURE AddItem(NewLine : STRING) DECLARE NewItemNum, ThisItemNum : STRING OPENFILE "Stock.txt" FOR READ OPENFILE "NewStock.txt" FOR WRITE NewItemNum ← LEFT(NewLine, 4) WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisItemNum ← LEFT(ThisLine, 4) IF ThisItemNum > NewItemNum THEN WRITEFILE("NewStock.txt", NewLine) // write New Line... NewItemNum ← "9999" // ...once only ENDIF WRITEFILE("NewStock.txt", ThisLine) ENDWHILE IF NewItemNum <> "9999" THEN WRITEFILE("NewStock.txt", NewLine) //New last line in the file ENDIF CLOSEFILE "Stock.txt" CLOSEFILE "NewStock.txt" ENDPROCEDURE </pre> Mark as follows: 1 Open both files, in correct modes, and subsequently close 2 Conditional loop until end of file <code>Stock.txt</code> 3 Read a line from <code>Stock.txt</code> AND extract <code>ThisItemNum</code> in a loop 4 Test <code>ThisItemNum > NewItemNum</code> then write <code>NewLine</code> to <code>NewStock.txt</code> 5 ...including mechanism to only do this once only 6 Write line read from <code>Stock</code> to <code>NewStock</code> 7 Deal with the case where <code>NewLine</code> is the new last line	7
------	--	---

8(b)	Example of array-based solution: <pre> PROCEDURE AddItem(NewLine : STRING) DECLARE ThisItemNum, ThisLine : STRING DECLARE Temp : ARRAY [1:5999] OF STRING DECLARE Index : INTEGER FOR Index ← 1 TO 5999 Temp[Index] ← "" //Initialise array NEXT Index Index ← STR_TO_NUM(LEFT(NewLine, 4)) Temp[Index] ← NewLine //Add new line to array OPENFILE "Stock.txt" FOR READ WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) Index ← STR_TO_NUM(LEFT(ThisLine, 4)) Temp[Index] ← ThisLine //Add line from file to array ENDWHILE CLOSEFILE "Stock.txt" OPENFILE "NewStock.txt" FOR WRITE FOR Index ← 1 TO 5999 IF Temp[Index] <> "" THEN //Write non-blank element... WRITEFILE("NewStock.txt", Temp[Index]) //...to new file ENDIF NEXT Index CLOSEFILE "NewStock.txt" ENDPROCEDURE </pre> Mark as follows: 1 Open both files, in correct modes, and subsequently close 2 Declare AND initialise Temp array 3 Store NewLine in appropriate array element 4 Loop until end of file Stock.txt 5 Read a line from Stock.txt AND extract Index in a loop 6 Assign line read to appropriate array element in a loop 7 Loop through array, writing non-blank elements to file NewStock.txt	
------	---	--

75. M/J 23/P23/Q1/a,c

1(a)	Answer		3
	The dimension of the array	2	
	The name of the variable used as an array index	PCount	
	The number of elements in the array	100	

1(c)	Integer	1
------	---------	---

76. M/J 23/P23/Q3/a,b(i)

3(a)(i)	6	1																						
3(a)(ii)	Stack <table><thead><tr><th>Memory location</th><th>Value</th></tr></thead><tbody><tr><td>506</td><td></td></tr><tr><td>505</td><td>BBB</td></tr><tr><td>504</td><td>AAA</td></tr><tr><td>503</td><td>XXX</td></tr><tr><td>502</td><td>ZZZ</td></tr><tr><td>501</td><td>NNN</td></tr><tr><td>500</td><td>PPP</td></tr></tbody></table> Pointer ← TopOfStack ← BottomOfStack <table><thead><tr><th>Variable</th><th>Value</th></tr></thead><tbody><tr><td>Data1</td><td>WWW</td></tr><tr><td>Data2</td><td>AAA</td></tr></tbody></table> One mark for: MP1 Values 'BBB' and 'AAA' MP2 Values 'XXX' to 'PPP' (unchanged) MP3 Both pointers and labelled22 MP4 Values of both variables	Memory location	Value	506		505	BBB	504	AAA	503	XXX	502	ZZZ	501	NNN	500	PPP	Variable	Value	Data1	WWW	Data2	AAA	4
Memory location	Value																							
506																								
505	BBB																							
504	AAA																							
503	XXX																							
502	ZZZ																							
501	NNN																							
500	PPP																							
Variable	Value																							
Data1	WWW																							
Data2	AAA																							
3(b)(i)	So that the data may be recovered / restored (the next time the program is run) // the data is permanently saved / data is not lost when the <u>program terminates</u>	1																						

77. M/J 23/P23/Q8/a,b

8(a)	<pre> FUNCTION ChangeSupp(Code1, Code2 : STRING) RETURNS INTEGER DECLARE Count : INTEGER DECLARE ThisLine, ThisCode : STRING OPENFILE "Stock.txt" FOR READ OPENFILE "NewStock.txt" FOR WRITE Count ← 0 WHILE NOT EOF("Stock.txt") READFILE("Stock.txt ", ThisLine) // brackets optional ThisCode ← MID(ThisLine, 5, 3) IF ThisCode = Code1 THEN ThisLine ← LEFT(ThisLine, 4) & Code2 & RIGHT(ThisLine, LENGTH(ThisLine) - 7) Count ← Count + 1 ENDIF WRITEFILE("NewStock.txt", ThisLine) // brackets optional ENDWHILE CLOSEFILE "NewStock.txt" CLOSEFILE "Stock.txt" RETURN Count ENDFUNCTION </pre> MP1 Open both files, in correct modes, and subsequently close MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract ThisCode in a loop MP4 Test ThisCode = Code1 AND if true, increment Count (must have been Initialised) in a loop MP5 Update ThisLine using substring functions and '&' in a loop MP6 completely correct update of ThisLine in a loop MP7 Write ThisLine to NewStock.txt in a loop MP8 Return count after loop	8
------	---	---

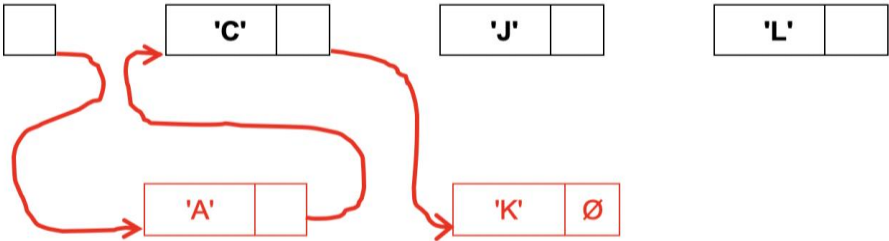
8(b)	<pre> PROCEDURE Report_1(Supp : STRING) DECLARE Count : INTEGER DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode : STRING Count ← 0 OPENFILE "Stock.txt" FOR READ OUTPUT "Report for Supplier:" & Supp OUTPUT "" //Blank line as per example OUTPUT "Item Description" OUTPUT "" //Blank line as per example WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisCode ← Mid(ThisLine, 5, 3) IF ThisCode = Supp THEN ThisItemNum ← LEFT(ThisLine, 4) ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7) OUTPUT ThisItem & " " & ThisDesc Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE "Stock.txt" OUTPUT "" //Blank line as per example OUTPUT "Number of items listed: ", Count ENDPROCEDURE </pre> MP1 Output report header (blank lines optional) – Must contain the parameter code MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract SupplierCode in a loop MP4 Test if SupplierCode = Supp then increment count (must have been Initialised) MP5 Extract AND output item and description in a loop MP6 Output the final line with count	6
------	--	---

78. O/N 22/P21/Q1/b

1(b)(i)	One mark per point: 1 Algorithm to process / search / organise the data is easier to implement // Values may be accessed via a loop-controlled variable / iterated through using index 2 Makes the program easier to design / code / test / understand 3 Multiple instances referenced via a single identifier / so fewer identifiers needed // Easier to amend the program when the number of students increases	3
---------	--	---

1(b)(ii)	One mark per point: Purpose: It identifies / references an individual array element // provides the <u>index</u> to the array Data type: Integer	2
----------	---	---

79. O/N 22/P21/Q4/a,b

4(a)(i)	One mark for each: 1 Data A and K stored in new / existing nodes 2 Start pointer points to Node A 3 Node A points to Node C and Node C points to Node K 4 Node K contains Null Pointer Start pointer 	4
4(a)(ii)	One mark per point: 1 Pointers determine the ordering of data // only the pointers need to be changed when data changed 2 Easier to add / delete data (to maintain correct sequence) in a linked list // description of moving data to maintain correct sequence when array used	2
4(a)(iii)	One mark per point: 1 Need to store pointers as well as data 2 More complex (to setup / implement)	2
4(b)	One mark per point (Max 4): 1 Declare two (1D) arrays 2 One for data, one for pointers 3 Elements from same index represent one node 4 Declare an integer / variable for <code>StartPointer</code> // explain its use 5 Define appropriate value for null pointer // explain its use 6 Declare an integer / variable for <code>FreeList</code> pointer // explain its use 7 Routines are needed to add / delete / search Alternative MP1, 2 and 3 for record-based implementation: 1 Define a record type with fields for data and pointer 2 Declare one (1D) array 3 ...of the defined record type	4

80. O/N 22/P21/Q5

5	Mark as follows (Max 5): 1 Procedure heading and ending and declaration of both indexes 2 Loop to process all elements from <code>Array1</code> 3 Sum (any) three consecutive values from <code>Array1</code> and divide by 3 in a loop 4 Convert result to Integer 5 Assign value to correct element of <code>Array2</code> in a loop 6 Increment <code>Array2</code> index in a loop <pre> PROCEDURE Summarise() DECLARE Value : REAL DECLARE IxA, IxB : INTEGER // Index variables IxB ← 1 FOR IxA ← 1 TO 598 STEP 3 Value ← Array1[IXA] + Array1[IXA + 1] + Array1[IXA + 2] Value ← Value / 3 Array2[IxB] ← INT(Value) IxB ← IxB + 1 NEXT IxA ENDPROCEDURE </pre>	5
---	--	---

81. O/N 22/P21/Q8/a,c

8(a)	One mark for each point (Max 7) as follows: 1 Function heading and ending including parameter and return type 2 Declaration and initialisation of local Integer for Count 3 OPEN in READ mode and CLOSE 4 Conditional loop until EOF() 5 Read a line in a loop 6 If non-blank, increment count in a loop 7 Terminate loop when 10 non-blank lines have been read 8 Return Boolean in both cases <pre> FUNCTION CheckFile(Thisfile : STRING) RETURNS BOOLEAN DECLARE Valid : BOOLEAN DECLARE ThisLine : STRING DECLARE Count : INTEGER Count ← 0 Valid ← FALSE OPEN ThisFile FOR READ WHILE NOT EOF(ThisFile) AND Valid = FALSE READFILE ThisFile, ThisLine IF ThisLine <> "" THEN Count ← Count + 1 IF Count > 9 THEN Valid ← TRUE ENDIF ENDIF ENDWHILE CLOSEFILE ThisFile RETURN Valid ENDFUNCTION </pre>	7
------	--	---

8(c)	Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration and initialisation of local Integer value for <code>ErrCount</code> 3 Use of <code>CheckFile()</code>, output message and terminate if it returns <code>FALSE</code> 4 Conditional loop until <code>EOF()</code> 5 ...or <code>ErrCount > MaxErrors</code> 6 Read line and use as parameter to <code>CheckLine()</code> in a loop 7 Test return value and increment <code>ErrCount</code> if non-zero in a loop 8 Output either message once only as appropriate <pre> PROCEDURE CountErrors(ThisFile : STRING, MaxErrors : INTEGER) DECLARE ErrCount, ThisError : INTEGER DECLARE ThisLine : STRING ErrCount ← 0 IF CheckFile(ThisFile) = FALSE THEN OUTPUT "That program file is not valid" ELSE OPEN ThisFile FOR READ REPEAT READFILE, ThisFile, ThisLine ThisError ← CheckLine(ThisLine) IF ThisError <> 0 THEN ErrCount ← ErrCount + 1 ENDIF UNTIL ErrCount > MaxErrors OR EOF(ThisFile) IF EOF(ThisFile) = FALSE THEN OUTPUT "Check terminated - too many errors" ELSE OUTPUT "There were ", ErrCount, " errors." ENDIF CLOSEFILE ThisFile ENDIF ENDPROCEDURE </pre>	8
------	--	---

82. O/N 22/P22/Q3/a,b,c

3(a)	One mark for name, Max two for features (Max 3 in total) Name: Queue Features: 1 Each queue element contains one data item 2 A Pointer to the front / start of the queue 3 A Pointer to the back / end of the queue 4 Data is added at back / end and removed from front / start // works on a FIFO basis 5 May be circular ALTERNATIVE: Name: Linked List Features: 1 Each node contains data and a pointer to the next node 2 A Pointer to the start of the list 3 Last node in the list has a null pointer 4 Data may be added / removed by manipulating pointers (not moving data) 5 Nodes are traversed in a specific sequence 6 Unused nodes are stored on a free list // a free-list pointer to the Free List	3
3(b)	One mark per point (Max 5): 1 Declare a (1D) array of data type <code>STRING</code> 2 The number of elements in that array corresponds to the size of the required stack 3 Declare an integer / variable for <code>StackPointer</code> 4. Declare an integer / variable for the size of the stack // for the max value of <code>StackPointer</code> 5 Use the <code>StackPointer</code> as an index to the array 6 Pointers and variables initialised to indicate empty stack 7 Store each item on the stack as one array element / Each stack item maps to one array element 8 Attempt to describe Push and Pop operations 9 Push and Pop routines need to check for full or empty conditions	5

3(c)

One mark for each:

5

- 1 Data 'After Group 1' (as shown, including blank cells)
- 2 Data 'After Group 2' (as shown, including blank cells)
- 3 Data 'After Group 3' (as shown, including blank cells)
- 4 SP 'After Group 1' pointing to location 955
- 5 Final two SPs pointing to locations 952 and 954

Memory location	Initial state		After Group 1		After Group 2		After Group 3
957							
956							
955			E	←SP	E		E
954			D		D		C
953	X	←SP	X		X		A
952	Y		Y		Y	←SP	Y
951	Z		Z		Z		Z
950	P		P		P		P

[5]

83. O/N 22/P22/Q7/a,b,c

7(a)	One mark per point (Max 6): 1 Procedure heading and ending including parameters 2 Conditional loop containing incrementing Index... 3 ...terminating when ErrNum found 4 ...terminating when ErrCode[Index] > ErrNum (i.e. ErrNum not found) 5 ... OR after element 500 tested 6 Test if found and OUTPUT 'Found' message 7 ...otherwise OUTPUT 'Not Found' message <pre> PROCEDURE OutputError(LineNum, ErrNum : INTEGER) DECLARE Index : INTEGER Index ← 0 // Search until ErrNum found OR not present OR end of array REPEAT Index ← Index + 1 UNTIL ErrCode[Index] >= ErrNum OR Index = 500 IF ErrCode[Index] = ErrNum THEN OUTPUT ErrText[Index], " on line ", LineNum //Found ELSE OUTPUT "Unknown error on line ", LineNum //Not found ENDIF ENDPROCEDURE </pre>	6
------	---	----------

7(b)	One mark per point (Max 8): 1 Procedure heading and ending as shown 2 Conditional loop correctly terminated 3 An inner loop 4 Correct range for inner loop 5 Comparison (element J with J+1) in a loop 6 Swap elements in both arrays in a loop 7 'No-Swap' mechanism: • Conditional outer loop including flag reset • Flag set in inner loop to indicate swap 8 Efficiency (this scenario): terminate inner loop when ErrCode = 999 9 Reducing Boundary in the <u>outer</u> loop <pre> PROCEDURE SortArrays() DECLARE TempInt, J, Boundary : INTEGER DECLARE TempStr : STRING DECLARE NoSwaps : BOOLEAN Boundary ← 499 REPEAT NoSwaps ← TRUE FOR J ← 1 TO Boundary IF ErrCode[J] > ErrCode[J+1] THEN //first swap ErrCode elements TempInt ← ErrCode[J] ErrCode[J] ← ErrCode[J+1] ErrCode[J+1] ← TempInt //now swap corresponding ErrText elements TempStr ← ErrText[J] ErrText[J] ← ErrText[J+1] ErrText[J+1] ← TempStr NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre>	8
7(c)(i)	ErrCode should be an INTEGER // ErrCode should not be a STRING	1

7(c)(ii)	Benefits include: 1 Array of records can store mixed data types / multiple data types under a single identifier 2 Tighter / closer association between <code>ErrCode</code> and <code>ErrText</code> // simpler code as fields may be referenced together // values cannot get out of step as with two arrays 3 Program easier to design / write / debug / test / maintain / understand One mark per point Note: Max 2 marks	2
7(c)(iii)	<pre>DECLARE Error : ARRAY[1:500] OF ErrorRec</pre>	1

84. O/N 22/P23/Q1/a

1(a)	One mark per row	4										
	<table><tr><th>Variable use</th><th>Data type</th></tr><tr><td>Store the number of days in the current month</td><td>INTEGER</td></tr><tr><td>Store the first letter of the customer's first name</td><td>CHAR</td></tr><tr><td>Store an indication of whether a year is a leap year</td><td>BOOLEAN</td></tr><tr><td>Store the average amount spent per customer visit</td><td>REAL</td></tr></table>	Variable use	Data type	Store the number of days in the current month	INTEGER	Store the first letter of the customer's first name	CHAR	Store an indication of whether a year is a leap year	BOOLEAN	Store the average amount spent per customer visit	REAL	
Variable use	Data type											
Store the number of days in the current month	INTEGER											
Store the first letter of the customer's first name	CHAR											
Store an indication of whether a year is a leap year	BOOLEAN											
Store the average amount spent per customer visit	REAL											

85. O/N 22/P23/Q4/a,b

4(a)	The data type (of the item to be stored)	1
4(b)(i)	Operation: Add an item / Enqueue Check: There are unused elements in the array // The queue is not full Operation: Remove an item / Dequeue Check: There are items in the array // The queue is not empty One mark for reason and one mark for reason it could not be completed.	4

4(b)(ii)	One mark per point (Max 5): 1 Declare a (1D) array of size ≥ 10 2 ...of data type <code>CHAR</code> 3 Declare integer variable for <code>FrontOfQueuePointer</code> 4 Declare integer variable for <code>EndOfQueuePointer</code> 5 Initialise <code>FrontOfQueuePointer</code> and <code>EndOfQueuePointer</code> to represent an empty queue 6 Declare integer variable or <code>NumberInQueue</code> 7 Declare integer variable for <code>SizeOfQueue</code> to count / limit the max number of items allowed // Reference to mechanism for defining 'wrap' of circular queue 8 Initialise <code>SizeOfQueue</code> // Initialise <code>NumberInQueue</code>	5
----------	--	----------

86. O/N 22/P23/Q5/b(i)

5(b)(i)	<code>LineData.Cost ← 12.99</code>	1
---------	------------------------------------	----------

87. O/N 22/P23/Q7/a,b(i)

7(a)	One mark per point (Max 8): 1 Declaration and initialisation of local integer for <code>Count</code> 2 Appropriate prompt and two inputs 3 (Conditional) loop while error number input is in range // error code 999 reached 4 ...and not end of array 5 Check if this <code>ErrCode</code> needs to be output in a loop 6 if so check for blank error text in a loop 7 Output in both cases 8 ...and increment count in a loop 9 OUTPUT of header and summary including count <pre> PROCEDURE OutputRange() DECLARE First, Last, Count, Index, ThisErr : INTEGER DECLARE ThisMess : STRING DECLARE PastLast: BOOLEAN Count ← 0 Index ← 1 PastLast ← FALSE OUTPUT "Please input first error number: " INPUT First OUTPUT "Please input last error number: " INPUT Last OUTPUT "List of error numbers from ", First, " to ", Last WHILE Index < 501 AND NOT PastLast ThisErr ← ErrCode[Index] IF ThisErr > Last THEN PastLast ← TRUE ELSE IF ThisErr >= First THEN ThisMess ← ErrText[Index] IF ThisMess = "" THEN ThisMess ← "Error Text Missing" ENDIF OUTPUT ThisErr, " : ", ThisMess Count ← Count + 1 ENDIF ENDIF Index ← Index + 1 ENDWHILE OUTPUT Count, " error numbers output" ENDPROCEDURE </pre>	8
------	--	---

7(b)(i)	One mark per point: 1 (Conditional) loop terminating when item added OR end of array reached 2 Test for unused element in a loop 3 Assignment of values to arrays // save index of first blank location and assign after loop 4 Set loop termination if empty element found in a loop 5 Call SortArrays() once 6 Calculation of remaining unused elements and return Integer value (for both cases) <pre> FUNCTION AddError(ErrNum : INTEGER, ErrMess : STRING) RETURNS INTEGER DECLARE Index, Remaining : INTEGER CONSTANT Unused = 999 Index ← 1 Remaining ← -1 REPEAT IF ErrCode[Index] = Unused THEN ErrCode[Index] ← ErrNum ErrText[Index] ← ErrMess CALL SortArrays() Remaining ← 500 - Index ENDIF Index ← Index + 1 UNTIL Remaining <> -1 OR Index > 500 RETURN Remaining ENDFUNCTION </pre>	6
---------	---	----------

88. O/N 21/P21/Q1/b

1(b)	<table><tr><th>Use of variable</th><th>Data type</th></tr><tr><td>The average mark in a class of 40 students</td><td>REAL</td></tr><tr><td>An email address</td><td>STRING</td></tr><tr><td>The number of students in the class</td><td>INTEGER</td></tr><tr><td>Indicate whether an email has been read</td><td>BOOLEAN</td></tr></table>	Use of variable	Data type	The average mark in a class of 40 students	REAL	An email address	STRING	The number of students in the class	INTEGER	Indicate whether an email has been read	BOOLEAN	4
	Use of variable	Data type										
	The average mark in a class of 40 students	REAL										
	An email address	STRING										
	The number of students in the class	INTEGER										
	Indicate whether an email has been read	BOOLEAN										
One mark per row												

89. O/N 21/P21/Q3/a,b,c

3(a)(i)	One mark per point: • E_{OQ} pointer will move to point to location 4 // incremented E_{OQ} (by 1) • Data value "Octopus" will be stored in location pointed to be E_{OQ} / location 4	2																		
3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by F_{OQ} / location 0 is assigned to variable <code>AnimalName</code> • F_{OQ} pointer will move to point to location 1 / point to "Cat" // incremented F_{OQ} (by 1) <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="2">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td><td rowspan="2">← End of queue pointer</td></tr> <tr> <td>3</td><td>Elk</td></tr> </table>	0	Frog	← Front of queue pointer	1	Cat	2	Fish	← End of queue pointer	3	Elk	2								
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish	← End of queue pointer																		
3	Elk																			
3(a)(iii)	There is only one data item in the queue	1																		
3(b)(i)	One mark for data values plus one mark for pointers <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="4">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td></tr> <tr> <td>3</td><td>Elk</td></tr> <tr> <td>4</td><td>Wasp</td><td rowspan="4">← End of queue pointer</td></tr> <tr> <td>5</td><td>Bee</td></tr> <tr> <td>6</td><td>Mouse</td></tr> <tr> <td>7</td><td></td></tr> </table> One mark for each pointer One mark for three new data values	0	Frog	← Front of queue pointer	1	Cat	2	Fish	3	Elk	4	Wasp	← End of queue pointer	5	Bee	6	Mouse	7		3
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish																			
3	Elk																			
4	Wasp	← End of queue pointer																		
5	Bee																			
6	Mouse																			
7																				

3(b)(ii)	<table><tr><td>0</td><td>Shark</td><td rowspan="4">← End of queue pointer</td></tr><tr><td>1</td><td>(Cat)</td></tr><tr><td>2</td><td>(Fish)</td></tr><tr><td>3</td><td>(Elk)</td></tr><tr><td>4</td><td>Wasp</td><td rowspan="4">← Front of queue pointer</td></tr><tr><td>5</td><td>Bee</td></tr><tr><td>6</td><td>Mouse</td></tr><tr><td>7</td><td>Dolphin</td></tr></table> One mark for BOTH pointers One mark for all data values as shown	0	Shark	← End of queue pointer	1	(Cat)	2	(Fish)	3	(Elk)	4	Wasp	← Front of queue pointer	5	Bee	6	Mouse	7	Dolphin	2
0	Shark	← End of queue pointer																		
1	(Cat)																			
2	(Fish)																			
3	(Elk)																			
4	Wasp	← Front of queue pointer																		
5	Bee																			
6	Mouse																			
7	Dolphin																			
3(c)	One mark per point: 1 If incremented $E_{OQ} = F_{OQ}$ then error condition: queue is full 2 Increment the E_{OQ} 3 Manage wrap-around	3																		

90. O/N 21/P21/Q5/a,c

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	2
------	---	----------

5(c)	<pre> FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER DECLARE FileData : STRING DECLARE Index, Count : INTEGER CONSTANT LogFile = "LogFile" Count ← 0 OPENFILE LogFile FOR APPEND FOR Index ← 1 TO 2000 FileData ← LogArray[Index] IF LEFT(FileData, 6) = StudentID THEN WRITEFILE (LogFile, FileData) //brackets optional Count ← Count + 1 LogArray[Index] ← "" // clear the element ENDIF NEXT Index CLOSEFILE LogFile RETURN Count ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameter and return type 2 OPEN file LogFile for APPEND and subsequent CLOSE 3 Loop for 2000 iterations 4 Extract first 6 characters from array element in a loop 5 Compare first 6 characters with parameter in a loop 6 If equal: • write whole array element string to file and • increment Count and • clear array element in a loop 7 Return Count (must have been declared and initialised)	7
------	--	---

6(a)	<pre>PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER) DECLARE Col : INTEGER // array is 1280 x 800 FOR Col ← SkipNum + 1 TO SkipNum + SetNum Screen[Row, Col] ← 1 NEXT Index ENDPROCEDURE</pre> ALTERNATIVE 1: <pre>FOR Col ← 1 TO SetNum Screen[Row, SkipNum + Col] ← 1 NEXT Col</pre> ALTERNATIVE 2: <pre>WHILE SetNum > 0 Screen[Row, SkipNum + SetNum] ← 1 SetNum ← SetNum - 1 ENDWHILE</pre> Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration of local Integer for Col 3 Count-controlled loop with meaningful start number 4 correct stop number 5 Reference Screen Array element and set to 1 in a loop	5
------	--	----------

6(b)	<pre> FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS INTEGER DECLARE ThisCol, Step : INTEGER DECLARE Found: BOOLEAN // array is 1280 x 800 Found ← FALSE ThisCol ← StartCol // first decide which way to search IF StartCol = 1 THEN Step ← 1 EndCol ← 1281 ELSE Step ← -1 EndCol ← 0 ENDIF WHILE ThisCol <> EndCol AND Found = FALSE IF Screen[ThisRow, ThisCol] <> 1 THEN ThisCol ← ThisCol + Step ELSE Found ← TRUE ENDIF ENDWHILE IF Found = FALSE THEN ThisCol ← -1 ENDIF RETURN ThisCol ENDFUNCTION </pre> Mark as follows: 1 Interpreting <code>StartCol</code> parameter to determine direction of search 2 An attempt at searching both up and down 3 Conditional Loop / Count-controlled loop with use of <code>ThisCol</code> index 4 Using correct values for <code>StartCol</code>, <code>EndCol</code> and <code>Step</code> 5 Reference a <code>Screen</code> element and compare with 1 in a loop 6 If equal save column or immediately Return column in a loop 7 Return column number or -1 Loop(s) terminate when element with value = 1 found Max 7 marks if function heading, including return type, and ending is incorrect or incomplete	8
------	--	---

92. O/N 21/P22/Q1/b

1(b)	Answer		3
	The number of dimensions of <code>ThisArray</code>	1	
	The technical terms for minimum and maximum values that variable <code>n</code> may take	Lower bound, upper bound	
	The technical term for the variable <code>n</code> in the pseudocode expression.	Index / Subscript	
One mark per row			

93. O/N 21/P22/Q3/a,b

3(a)(i)	Pseudocode: <pre> TYPE Student DECLARE StudentID : STRING DECLARE Email : STRING DECLARE Club_1 : INTEGER DECLARE Club_2 : INTEGER DECLARE Club_3 : INTEGER ENDTYPE </pre> Mark as follows: - One mark for <code>TYPE</code> and <code>ENDTYPE</code> - One mark for <code>StudentID</code> and <code>Email</code> fields as <code>STRING</code> - One mark for all <code>Club</code> fields as <code>INTEGER</code>	3
3(a)(ii)	<u>DECLARE Membership : ARRAY [1:3000] OF Student</u> One mark per underlined phrase	2
3(a)(iii)	One mark for any one of: - Assign a value (of the correct data type) outside the normal range to one of the fields - Assign an empty string to the <code>StudentID</code> field / <code>Email</code> field - or value out of range to any club field	1
3(a)(iv)	A number outside the range 1 to 99	1

3(b)	<pre> PROCEDURE GetIDs() DECLARE Index : INTEGER DECLARE ThisClub, Count : INTEGER OUTPUT "Please Input Club Number: " INPUT ThisClub Count ← 0 FOR Index ← 1 TO 3000 IF Membership[Index].Club_1 = ThisClub OR ____ Membership[Index].Club_2 = ThisClub OR ____ Membership[Index].Club_3 = ThisClub THEN Count ← Count + 1 OUTPUT Membership[Index].StudentID ENDIF NEXT Index OUTPUT "There are ", Count, " Students in the club" ENDPROCEDURE </pre> Mark as follows: 1 Declare and initialise Count 2 Prompt and Input club number before the loop 3 Loop through 3000 elements 4 Compare one club field with number input 5 Compare all Club fields with number input 6 If number found, OUTPUT of StudentID field and increment Count 7 Final OUTPUT of Count outside the loop Note: Max 6 if procedure heading and ending missing or incorrect (but allow array as parameter)	7
------	--	---

94. O/N 21/P22/Q5/a,b

5(a)	<pre> PROCEDURE MakeNewFile(OldFile, NewFile, Status : STRING) DECLARE Line1, Line2, Line3 : STRING DECLARE NumCopied, NumRecs : INTEGER NumRecs ← 0 NumCopied ← 0 OPENFILE OldFile FOR READ OPENFILE NewFile FOR WRITE WHILE NOT EOF(OldFile) READFILE OldFile, Line1 READFILE OldFile, Line2 READFILE OldFile, Line3 NumRecs ← NumRecs + 1 IF Line3 <> Status THEN WRITEFILE NewFile, Line1 WRITEFILE NewFile, Line2 WRITEFILE NewFile, Line3 NumCopied ← NumCopied + 1 ENDIF ENDWHILE OUTPUT "File " , OldFile , " contained " , NumRecs , __ " employee details" OUTPUT Numcopied , " employee sets of details were __ written to file", NewFile CLOSEFILE OldFile CLOSEFILE NewFile ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading and ending, including parameters 2 OPEN OldFile for READ and NewFile for WRITE and subsequently CLOSE both files 3 Conditional loop until EOF(OldFile) 4 Read three lines from OldFile in a loop 5 Compare 3rd line read with Status parameter and if not equal write 3 lines to NewFile in a loop 6 Count number of sets read and those written in a loop 7 Final output including both counts and file names with suitable text after a loop Note: MP6: Both counts must have been declared and initialised	7
5(b)(i)	Store all three items on one line	1
5(b)(ii)	One mark per point: Advantage: Fewer file operations required Disadvantage: Algorithm to combine / extract individual data items is more complex	2

95. O/N 21/P22/Q6/a

6(a)	<pre> FUNCTION FirstRowSet() RETURNS INTEGER DECLARE Row, Col : INTEGER DECLARE Found : BOOLEAN // array is 1280 × 800 Row ← 1 Found ← FALSE WHILE Row <= 800 AND Found = FALSE // top to bottom Col ← 1 WHILE Col <= 1280 AND Found = FALSE // left to right IF Screen[Row,Col] = 1 THEN Found ← TRUE // end function as soon as first // found ENDIF Col ← Col + 1 ENDWHILE Row ← Row + 1 ENDWHILE IF Found = FALSE THEN // nothing found Row ← 0 ENDIF RETURN Row - 1 ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending and return type 2 (Conditional) outer loop 1 to 800 (row) 3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column) 4 Reference <code>Screen</code> element and test for = 1 // <> 0 5 and if true save row number and exit loops 6 Increment index variables in both inner and outer loop 7 Return Row number or -1, following a reasonable attempt	7
------	---	----------

96. O/N 21/P23/Q1/b

1(b)	<table><tr><th>Use of variable</th><th>Data type</th></tr><tr><td>The average mark in a class of 40 students</td><td>REAL</td></tr><tr><td>An email address</td><td>STRING</td></tr><tr><td>The number of students in the class</td><td>INTEGER</td></tr><tr><td>Indicate whether an email has been read</td><td>BOOLEAN</td></tr></table>	Use of variable	Data type	The average mark in a class of 40 students	REAL	An email address	STRING	The number of students in the class	INTEGER	Indicate whether an email has been read	BOOLEAN	4
	Use of variable	Data type										
	The average mark in a class of 40 students	REAL										
	An email address	STRING										
	The number of students in the class	INTEGER										
	Indicate whether an email has been read	BOOLEAN										
One mark per row												

97. O/N 21/P23/Q3/a,b,c

3(a)(i)	One mark per point: • E_{OQ} pointer will move to point to location 4 // incremented E_{OQ} (by 1) • Data value "Octopus" will be stored in location pointed to be E_{OQ} / location 4	2																		
3(a)(ii)	One mark for each bullet • Value "Frog" // value pointed to by F_{OQ} / location 0 is assigned to variable <code>AnimalName</code> • F_{OQ} pointer will move to point to location 1 / point to "Cat" // incremented F_{OQ} (by 1) <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="2">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td><td rowspan="2">← End of queue pointer</td></tr> <tr> <td>3</td><td>Elk</td></tr> </table>	0	Frog	← Front of queue pointer	1	Cat	2	Fish	← End of queue pointer	3	Elk	2								
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish	← End of queue pointer																		
3	Elk																			
3(a)(iii)	There is only one data item in the queue	1																		
3(b)(i)	One mark for data values plus one mark for pointers <table border="1"> <tr> <td>0</td><td>Frog</td><td rowspan="4">← Front of queue pointer</td></tr> <tr> <td>1</td><td>Cat</td></tr> <tr> <td>2</td><td>Fish</td></tr> <tr> <td>3</td><td>Elk</td></tr> <tr> <td>4</td><td>Wasp</td><td rowspan="4">← End of queue pointer</td></tr> <tr> <td>5</td><td>Bee</td></tr> <tr> <td>6</td><td>Mouse</td></tr> <tr> <td>7</td><td></td></tr> </table> One mark for each pointer One mark for three new data values	0	Frog	← Front of queue pointer	1	Cat	2	Fish	3	Elk	4	Wasp	← End of queue pointer	5	Bee	6	Mouse	7		3
0	Frog	← Front of queue pointer																		
1	Cat																			
2	Fish																			
3	Elk																			
4	Wasp	← End of queue pointer																		
5	Bee																			
6	Mouse																			
7																				

3(b)(ii)	<table><tr><td>0</td><td>Shark</td><td rowspan="4">← End of queue pointer</td></tr><tr><td>1</td><td>(Cat)</td></tr><tr><td>2</td><td>(Fish)</td></tr><tr><td>3</td><td>(Elk)</td></tr><tr><td>4</td><td>Wasp</td><td rowspan="4">← Front of queue pointer</td></tr><tr><td>5</td><td>Bee</td></tr><tr><td>6</td><td>Mouse</td></tr><tr><td>7</td><td>Dolphin</td></tr></table> One mark for BOTH pointers One mark for all data values as shown	0	Shark	← End of queue pointer	1	(Cat)	2	(Fish)	3	(Elk)	4	Wasp	← Front of queue pointer	5	Bee	6	Mouse	7	Dolphin	2
0	Shark	← End of queue pointer																		
1	(Cat)																			
2	(Fish)																			
3	(Elk)																			
4	Wasp	← Front of queue pointer																		
5	Bee																			
6	Mouse																			
7	Dolphin																			
3(c)	One mark per point: 1 If incremented $E_{OQ} = F_{OQ}$ then error condition: queue is full 2 Increment the E_{OQ} 3 Manage wrap-around	3																		

98. O/N 21/P23/Q5/a,c

5(a)	One mark for the character and one for the corresponding reason. • Character: Any except alphabetic, numeric, ',' ':' or space • Reason: character doesn't occur in data to be recorded	2
------	---	----------

5(c)	<pre> FUNCTION LogEvents(StudentID : STRING) RETURNS INTEGER DECLARE FileData : STRING DECLARE Index, Count : INTEGER CONSTANT LogFile = "LogFile" Count ← 0 OPENFILE LogFile FOR APPEND FOR Index ← 1 TO 2000 FileData ← LogArray[Index] IF LEFT(FileData, 6) = StudentID THEN WRITEFILE (LogFile, FileData) //brackets optional Count ← Count + 1 LogArray[Index] ← "" // clear the element ENDIF NEXT Index CLOSEFILE LogFile RETURN Count ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading and ending including parameter and return type 2 OPEN file LogFile for APPEND and subsequent CLOSE 3 Loop for 2000 iterations 4 Extract first 6 characters from array element in a loop 5 Compare first 6 characters with parameter in a loop 6 If equal: • write whole array element string to file and • increment Count and • clear array element in a loop 7 Return Count (must have been declared and initialised)	7
------	--	---

6(a)	<pre>PROCEDURE SetRow(Row, SkipNum, SetNum : INTEGER) DECLARE Col : INTEGER // array is 1280 x 800 FOR Col ← SkipNum + 1 TO SkipNum + SetNum Screen[Row, Col] ← 1 NEXT Index ENDPROCEDURE</pre> ALTERNATIVE 1: <pre>FOR Col ← 1 TO SetNum Screen[Row, SkipNum + Col] ← 1 NEXT Col</pre> ALTERNATIVE 2: <pre>WHILE SetNum > 0 Screen[Row, SkipNum + SetNum] ← 1 SetNum ← SetNum - 1 ENDWHILE</pre> Mark as follows: 1 Procedure heading and ending including parameters 2 Declaration of local Integer for Col 3 Count-controlled loop with meaningful start number 4 correct stop number 5 Reference Screen Array element and set to 1 in a loop	5
------	--	----------

6(b)	<pre> FUNCTION SearchInRow(ThisRow, StartCol : INTEGER) RETURNS INTEGER DECLARE ThisCol, Step : INTEGER DECLARE Found: BOOLEAN // array is 1280 x 800 Found ← FALSE ThisCol ← StartCol // first decide which way to search IF StartCol = 1 THEN Step ← 1 EndCol ← 1281 ELSE Step ← -1 EndCol ← 0 ENDIF WHILE ThisCol <> EndCol AND Found = FALSE IF Screen[ThisRow, ThisCol] <> 1 THEN ThisCol ← ThisCol + Step ELSE Found ← TRUE ENDIF ENDWHILE IF Found = FALSE THEN ThisCol ← -1 ENDIF RETURN ThisCol ENDFUNCTION </pre> Mark as follows: 1 Interpreting <code>StartCol</code> parameter to determine direction of search 2 An attempt at searching both up and down 3 Conditional Loop / Count-controlled loop with use of <code>ThisCol</code> index 4 Using correct values for <code>StartCol</code>, <code>EndCol</code> and <code>Step</code> 5 Reference a <code>Screen</code> element and compare with 1 in a loop 6 If equal save column or immediately Return column in a loop 7 Return column number or -1 Loop(s) terminate when element with value = 1 found Max 7 marks if function heading, including return type, and ending is incorrect or incomplete	8
------	--	---

100. M/J 21/P21/Q1/c

1(c)	Structure: - Record Justification: - Allows different data <u>types</u> - to be stored under one <u>identifier</u> Mark as follows: One mark for Structure Two marks for justification. Alternative - Two (1D) arrays - One of string, one of integer - Where same index links the name with the score	3
------	--	----------

101. M/J 21/P21/Q3/a,b

3(a)(i)	One mark per bullet point: - Data from the arrays is written to the <u>files</u> at the end of the day / before the program is terminated / computer is switched off - Data can then be read from the <u>files</u> at the start of the next day and written to / stored in the <u>arrays</u> - No need to (re-)enter the data manually // only need to enter data once Note: Max 2 marks	2
3(a)(ii)	The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	1
3(a)(iii)	One mark per bullet point: - Data items are combined to form a single string / saved as a single line in the file - Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: - Convert all data items / 'number of people' to strings - Consecutive / each line stores a separate data item	2

3(b)	<pre>PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE</pre> Marks as follows: 1 Procedure heading (including parameter) and ending 2 File <code>OPEN</code> and subsequently <code>CLOSE</code> 3 Check if file is empty and output a warning message if it is 4 Conditional Loop 5 Output line (including blank lines) and read next line in a loop	5
------	--	----------

5	<pre> PROCEDURE Sort() DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison in a loop 4 Correct swap of col1 array elements in a loop 5 Correct swap of col2 array elements in a loop (via loop or separate statements) 6 'NoSwap' mechanism: Conditional outer loop including flag reset 7 'NoSwap' mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary in the <u>outer</u> loop	8
---	---	---

103. M/J 21/P21/Q6/a,b

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node Note: max 4 marks	4
------	--	---

6(b)	One mark per point: - An array (1D) to store the data and a second array (1D) to store the pointers - An (integer) variable to hold the start pointer and an (integer) variable to store the next free pointer ALTERNATIVE: - Define a record type comprising a data element and a pointer and declare an array (1D) of this type - An integer variable to hold the start pointer and an integer variable to store the next free pointer	2
------	--	----------

104. M/J 21/P22/Q1/a

1(a)(i)	<table><tr><th>Variable</th><th>Example data value</th><th>Data type</th></tr><tr><td>Name</td><td>"Catherine"</td><td>STRING</td></tr><tr><td>Index</td><td>100</td><td>INTEGER</td></tr><tr><td>Modified</td><td>FALSE</td><td>BOOLEAN</td></tr><tr><td>Holiday</td><td>25/12/2020</td><td>DATE</td></tr></table>	Variable	Example data value	Data type	Name	"Catherine"	STRING	Index	100	INTEGER	Modified	FALSE	BOOLEAN	Holiday	25/12/2020	DATE	4
	Variable	Example data value	Data type														
	Name	"Catherine"	STRING														
	Index	100	INTEGER														
	Modified	FALSE	BOOLEAN														
	Holiday	25/12/2020	DATE														
One mark per data type																	

1(a)(ii)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>Modified OR Index > 100</td><td>FALSE</td></tr><tr><td>LENGTH("Student: " & Name)</td><td>18</td></tr><tr><td>INT(Index + 2.9)</td><td>102</td></tr><tr><td>MID(Name, 1, 3)</td><td>"Cat"</td></tr></table>	Expression	Evaluates to	Modified OR Index > 100	FALSE	LENGTH("Student: " & Name)	18	INT(Index + 2.9)	102	MID(Name, 1, 3)	"Cat"	4
	Expression	Evaluates to										
	Modified OR Index > 100	FALSE										
	LENGTH("Student: " & Name)	18										
	INT(Index + 2.9)	102										
	MID(Name, 1, 3)	"Cat"										
One mark per value												
Quotation marks must be present for final row and must be capital C												

105. M/J 21/P22/Q3/a,b,c,d

3(a)	Linked list	1
3(b)	Start pointer	1
3(c)	One mark for each: Name: Null pointer Meaning: There are no further nodes in the list	2

3(d)	One mark for: - Start Pointer pointing to 'Cat' node - Remaining arrows: Cat → Dolphin → Elk → Fish	2
------	---	---

106. M/J 21/P22/Q5/a

5(a)(i)	<pre> DECLARE RNum : ARRAY[1:100] OF INTEGER DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 TO 100 RNum[Index] ← INT(RAND(200)) + 1 IF RNum[Index] >= 66 AND RNum[Index] <= 173 THEN Count ← Count + 1 ENDIF NEXT Index OUTPUT Count </pre> Mark as follows: 1 Array declaration 2 Loop for 100 iterations 3 Array element index 'syntax' (left-hand side of assignment expression) in a loop 4 Use of <code>RAND()</code> to generate value in range (and assign to array element) in a loop 5 Check if random number within range and if so, increment count in a loop 6 Output of count (following a reasonable attempt) after the loop	6
5(a)(ii)	One mark per bullet / sub-bullet point 1 Initialise the array to a rogue value (to indicate 'unassigned' element) 2 Add a conditional loop to: 3 Generate and store a random number (in the correct range) 4 Check the stored number against values already in the array 5 If the stored number is found then generate another random value 6 Otherwise add it to the array (and exit loop) Note: Max 3 marks	3

107. M/J 21/P22/Q6

6	<pre> PROCEDURE CountVowels(ThisString : STRING) DECLARE Index : INTEGER DECLARE ThisChar : CHAR FOR Index ← 1 to 6 CharCount[Index] ← 0 //initialise elements NEXT Index Index ← 1 FOR Index ← 1 TO LENGTH(ThisString) ThisChar ← LCASE(MID(ThisString, Index, 1)) CASE OF ThisChar 'a' : CharCount[1] ← CharCount[1] + 1 'e' : CharCount[2] ← CharCount[2] + 1 'i' : CharCount[3] ← CharCount[3] + 1 'o' : CharCount[4] ← CharCount[4] + 1 'u' : CharCount[5] ← CharCount[5] + 1 'a' TO 'z': CharCount[6] ← CharCount[6] + 1 ENDCASE NEXT Index FOR Index ← 1 to 6 OUTPUT CharCount[Index] //output results NEXT Index ENDPROCEDURE </pre> 1 mark for each of the following: 1 Procedure heading (with parameter) and ending 2 Declare local variable for Index as loop counter but not CharCount array 3 Initialise elements of CharCount array to zero 4 Loop through all characters in ThisString 5 Use of MID() to extract single character 6 Test for each vowel and increment associated count 7 Test for consonants and increment associated count 8 Output the results (supporting text not necessary) after the loop Note: Max 7 if CharCount not used to store count	8
---	---	---

108. M/J 21/P22/Q8/a

8(a)	<pre> FUNCTION IgnoreWord (ThisWord : STRING) RETURNS BOOLEAN DECLARE Found : BOOLEAN DECLARE Index : INTEGER Found ← False Index ← 1 ThisWord ← TO_LOWER(ThisWord) REPEAT IF TO_LOWER(IgnoreList[Index]) = ThisWord THEN Found ← TRUE ENDIF Index ← Index + 1 UNTIL Found = TRUE OR Index > 10 RETURN Found ENDFUNCTION </pre> 1 mark for each of the following: 1 Loop through array elements 2 Convert both strings to same case 3 Compare array element with parameter in a loop 4 Set a flag (or similar) if match found (after reasonable attempt at MP3) in a loop 5 Return TRUE or FALSE in all cases Note: Max 4 if function declaration incorrect	5
------	--	----------

109. M/J 21/P23/Q1/c

1(c)	Structure: - Record Justification: - Allows different data <u>types</u> - to be stored under one <u>identifier</u> Mark as follows: One mark for Structure Two marks for justification. Alternative - Two (1D) arrays - One of string, one of integer - Where same index links the name with the score	3
------	---	----------

110. M/J 21/P23/Q3/a,b

3(a)(i)	One mark per bullet point: - Data from the arrays is written to the <u>files</u> at the end of the day / before the program is terminated / computer is switched off - Data can then be read from the <u>files</u> at the start of the next day and written to / stored in the <u>arrays</u> - No need to (re-)enter the data manually // only need to enter data once Note: Max 2 marks	2
3(a)(ii)	The data is retained when the program is terminated / after the computer is switched off // data is stored permanently // non-volatile storage	1
3(a)(iii)	One mark per bullet point: - Data items are combined to form a single string / saved as a single line in the file - Data items are separated by a special character // make each data item a fixed length ALTERNATIVE: - Convert all data items / 'number of people' to strings - Consecutive / each line stores a separate data item	2
3(b)	<pre> PROCEDURE Preview (ThisFile : STRING) DECLARE LineNum : INTEGER DECLARE ThisLine : STRING OPENFILE ThisFile FOR READ IF EOF(ThisFile) THEN OUTPUT "Warning Message" ELSE LineNum ← 1 WHILE LineNum < 6 AND NOT EOF(ThisFile) READFILE Thisfile, ThisLine OUTPUT ThisLine LineNum ← LineNum + 1 ENDWHILE ENDIF CLOSEFILE ThisFile ENDPROCEDURE </pre> Marks as follows: - Procedure heading (including parameter) and ending - File OPEN and subsequently CLOSE - Check if file is empty and output a warning message if it is - Conditional Loop - Output line (including blank lines) and read next line in a loop	5

111. M/J 21/P23/Q5

5	<pre> PROCEDURE Sort () DECLARE Temp : INTEGER DECLARE NoSwaps : BOOLEAN DECLARE Boundary, Row, Col : INTEGER Boundary ← 999 REPEAT NoSwaps ← TRUE FOR Row ← 1 TO Boundary IF Result[Row, 2] > Result[Row + 1, 2] THEN FOR Col ← 1 TO 2 Temp ← Result [Row, Col] Result [Row, Col] ← Result [Row + 1, Col] Result [Row + 1, Col] ← Temp NEXT Col NoSwaps ← FALSE ENDIF NEXT J Boundary ← Boundary - 1 UNTIL NoSwaps = TRUE ENDPROCEDURE </pre> Mark as follows: 1 Outer loop 2 Inner loop 3 Correct comparison in a loop 4 Correct swap of col1 array elements in a loop 5 Correct swap of col2 array elements in a loop (via loop or separate statements) 6 'NoSwap' mechanism: Conditional outer loop including flag reset 7 'NoSwap' mechanism: Set flag in inner loop to indicate swap 8 Reducing Boundary in the <u>outer</u> loop	8
---	--	---

112. M/J 21/P23/Q6/a,b

6(a)	One mark per point: 1 Check for a free node 2 Search for correct insertion point 3 Assign data value B to first node in free list / node pointed to by start pointer of free list 4 Pointer from A will be changed to point to node containing B (instead of C) 5 Pointer from B will be changed to point to node containing C 6 Start pointer in free list moved to point to next free node Note: max 4 marks	4
------	--	---

6(b)	One mark per point: • An array (1D) to store the data and a second array (1D) to store the pointers • An (integer) variable to hold the start pointer and an (integer) variable to store the next free pointer ALTERNATIVE: • Define a record type comprising a data element and a pointer and declare an array (1D) of this type • An integer variable to hold the start pointer and an integer variable to store the next free pointer	2
------	--	----------

Unit 11

PROGRAMMING

96 items · 540 marks

Syllabus sections in this unit

- 11.1 Programming Basics
- 11.2 Constructs
- 11.3 Structured Programming

Unit 11: Programming

1. M/J 26/P21/Q1/a,b

1 Study this pseudocode:

```
FOR i ← 3 TO 12 STEP 3
    OUTPUT i, i * i
NEXT i
```

(a) Identify the type of construct used in this pseudocode.

[1]

(b) State how many iterations are performed by this pseudocode.

[1]

2. M/J 26/P21/Q5/a,b

5 Electric light bulbs are advertised online with the price showing:

\$1.75 each or 20 bulbs for \$30.00

A pseudocode function `BulbCost()`

- takes the quantity of bulbs purchased as its single parameter
- returns the total cost in dollars.

Pseudocode for the main program:

- inputs the number of bulbs purchased
- returns the total cost from the function `BulbCost()`
- outputs the total cost.

(a) Write pseudocode for the function `BulbCost()`

[4]

(b) The main program accepts a user input for the quantity of bulbs purchased and calls function `BulbCost()`

Complete the pseudocode for the main program.

```
// Main program
DECLARE NumberPurchased : INTEGER
DECLARE TotalCost : REAL

OUTPUT "How many bulbs? "
INPUT NumberPurchased

TotalCost ←

OUTPUT '$' , TotalCost
// end main program
```

[1]

3. M/J 26/P21/Q6/a,b

- 6 Fence parts are manufactured by a number of similar machines. The length of the part must fall within a certain range. Parts **not** within the length range are rejected.

The data for each rejected part forms a data packet made up of three data items, each separated by the '/' character delimiter. The format of a data packet is:

<length>/<machine ID>/<description of the fault>

- The length is 3 digits (in millimetres).
- The machine ID is a single character ('A', 'B', or 'C').
- The description of the fault is a variable length string.

A pseudocode program will process the data for several data packets.

The program contains the statement:

DataPacket ← "563/B/the fence length is too long"

- (a) Complete the **two** pseudocode expressions which extract the appropriate data item.

PartLength ←

MachineID ←

[2]

- (b) The program will create a module `RejectDescription()` to return the fault description.

Write the pseudocode for the module `RejectDescription()`

[4]

4. M/J 26/P22/Q1/a(ii,iii)

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (ii) Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

IF (Pass) < 6 OR (Pass) > THEN

OUTPUT "Error"

OUTPUT "The password has to contain between 6 and 8 characters."

ENDIF

[2]

- (iii) Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```
DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ..... , ..... )
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF
```

[7]

5. M/J 26/P22/Q5/c

- 5 A programmer has defined a global 1D array `Numbers` of type `INTEGER`, which can store 1000 integers. The lower bound of the array `Numbers` is set to 1. Each element of the array has been initialised with a random integer in the range 200 to 99999.

The programmer has written a bubble sort algorithm to sort the array `Numbers` into ascending order.

Study algorithm A.

Assume all the variables have been correctly declared.

algorithm A:

```
FOR LoopCounter ← 1 TO 999                                //Outer Loop
  FOR Index ← 1 TO 999                                    //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    ENDIF
  NEXT Index
NEXT LoopCounter
```

(c) The programmer has defined a new procedure:

```
PROCEDURE Swap (Num1 : INTEGER, Num2 : INTEGER)
  DECLARE Temp : INTEGER
  Temp ← Num1
  Num1 ← Num2
  Num2 ← Temp
ENDPROCEDURE
```

The programmer replaces the block of pseudocode in the algorithm that swaps the values in two neighbouring elements in the array with a call to this procedure.

The block of code:

```
Temp ← Numbers[Index]
Numbers[Index] ← Numbers[Index + 1]
Numbers[Index + 1] ← Temp
```

Is replaced by:

```
CALL Swap (Numbers[Index], Numbers[Index + 1])
```

The `Swap ()` procedure will **not** work as expected when used in the bubble sort algorithm.

- (i) Explain why the procedure `Swap ()` will **not** work as expected.

[2]

- (ii) Write pseudocode to amend the version of the `Swap ()` procedure given so that it will work as expected.

The amended version of the `Swap ()` procedure must **not** use global variables.

[2]

6. M/J 26/P22/Q6/a

6 A program is being developed to encrypt a message.

The following terms are used in the encryption technique to be used:

term	definition
plain text	The original message before it is encrypted.
cipher text	The encrypted version of the plain text.

The encryption method to be used is based on the algorithm for the Caesar Box Cipher. All plain text messages to be encrypted must be 25 characters in length or less. If a message is less than 25 characters in length, then spaces are added to the end to make the padded plain text the required length.

A square grid of five columns and five rows is used. The padded plain text message is put into the grid row by row. The cipher text is obtained by reading the grid column by column.

For example:

plain text: This is a message

The number of characters in the original plain text, including spaces, is 17.

plain text with added spaces at the end: This△is△a△message△△△△△△△△

The character △ is used to indicate a space in the padded plain text.

The grid filled, row by row, with the padded plain text.

T	h	i	s	△
i	s	△	a	△
m	e	s	s	a
g	e	△	△	△
△	△	△	△	△

The cipher text is obtained by reading the grid column by column.

cipher text: Timg△hsee△i△s△△sas△△△△a△△

The programmer has defined the first program module as:

module	description
AddPadding()	• user prompt for and input of plain text • check plain text contains between 1 and 25 characters inclusive • re-input of plain text until it contains between 1 and 25 characters • if required, additional space characters are added to the end of the plain text to ensure it contains exactly 25 characters • return of amended plain text

(a) Write pseudocode for module AddPadding()

7. M/J 26/P23/Q1/a,b

1 Study the pseudocode.

```
i ← 1
REPEAT
    OUTPUT i*i
    i ← i + 1
UNTIL i = 6
```

(a) Identify the type of programming construct used in the pseudocode.

[1]

(b) State how many iterations are carried out by the pseudocode.

[1]

8. M/J 26/P23/Q5/a

5 Data is stored about people who have attended a job interview. Each interview is held at one of three office locations.

Each person's data is stored as a data packet made up of four data items:

<initial><shortened surname><office code><summary of the interview>

- The initial is a single character.
- The shortened surname is the first five characters of the surname.
 - You can assume all surnames are at least five characters.
- The office code is a single character 'A', 'B' or 'C'.
- The interview summary text is of variable length.

Example:

Peter Watkins was interviewed at office 'B'. After the interview it was decided to invite him for a second interview.

His data packet is stored by variable `Applicant`:

```
Applicant ← "PWatkiBSecond interview"
```

(a) A pseudocode function `InterviewSummary()` will:

- take the data packet as a single parameter
- return the interview summary text.

Write the pseudocode for the function `InterviewSummary()`

[4]

9. O/N 25/P21/Q1/a

Uses the insert (Appendix)

1 A program is being developed to meet a particular customer requirement.**(a)** The program contains these variables:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER
MyDOB	DATE

Complete the table by filling in the gaps using functions and/or operators from the **insert**.

Each expression must be valid.

Expression
MyString ← 'X' MyString
MyChar ← ("ABCD", ,)
MyString ← (..... (MyDOB))
MyInt ← (..... (MyString) / 2)

[4]

10. O/N 25/P21/Q5/b

- 5** A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return `-1` if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
  DECLARE Index, FoundAt : INTEGER
  FoundAt ← -1
  FOR Index ← 1 TO 10000
    IF MyData[Index] = SearchString THEN // outer conditional clause
      IF FoundAt = -1 THEN               // inner conditional clause
        FoundAt ← Index
      ENDIF
    ENDIF
  NEXT Index
  RETURN FoundAt
ENDFUNCTION
```

- (b)** The pseudocode does **not** use the most appropriate loop construct.

- (i)** Explain why this is **not** the most appropriate loop construct.

[1]

- (ii)** Suggest and justify a more appropriate loop construct that could be used.

Construct

Justification

[2]

11. O/N 25/P21/Q6/b

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The method described can be used to detect single-digit errors. For example, if the new number is incorrectly input as 41633, then the check digit does not match and the input will be rejected.

- (b) A module `Generate()` will take an integer value representing an original number and return an integer value representing a new number which includes the check digit.

The original number is always at least three digits in length.

Write pseudocode for the module `Generate()`

Assume that the parameter is valid.

[7]

12. O/N 25/P22/Q1/c

- (b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
Result	5.42	
MonthLetter	"JFMAMJJASOND"	
Birthday	15/11/2009	

[3]

- (c) Evaluate each expression in the table by using the data values shown in (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
<code>INT(Result) + 1 > 6</code>	
<code>NUM_TO_STR(LENGTH(MonthLetter))</code>	
<code>NUM_TO_STR(Result + "3.2")</code>	
<code>MID(MonthLetter, MONTH(Birthday) - 2, 1)</code>	

[4]

13. O/N 25/P22/Q6

6 A string function `Compare()` will compare two strings.

The function will:

- take **four** parameters:
 - two strings, `String1` and `String2`
 - a character, `Position`, to indicate whether `String1` will be compared with the start ('s'), or the end ('e') of `String2`
 - a Boolean, `CaseMatters`, to indicate whether an upper case character and lower case character (for example, 'A' and 'a') are regarded as different (TRUE), or as the same (FALSE)
- return FALSE if `String2` has fewer characters than `String1`
- return TRUE if the comparison is true, otherwise return FALSE

For example:

Parameter				Return value
String1	String2	CaseMatters	Position	
"Cat"	"Catalogue"	TRUE	's'	TRUE
"CAT"	"Catalogue"	TRUE	's'	FALSE
"CAT"	"Catalogue"	FALSE	's'	TRUE
"Cat"	"Catalogue"	TRUE	'e'	FALSE
"GUE"	"Catalogue"	FALSE	'e'	TRUE
"Catalogue"	"Cat"	TRUE	's'	FALSE

Write pseudocode for the function `Compare()`

[7]

14. O/N 25/P23/Q1/a,b,c

1 A user requires a program that implements complex data encryption to a recognised international standard. A programmer has started the design of the program.

- (a) The programmer decides to use a library routine to provide part of the solution. One reason for this is that library routines may perform functions that the programmer is unable to program themselves.

State **three** other benefits of using library routines in the development of the program.

1

2

3

[3]

- (b) The programmer has identified a different part of the algorithm that would be appropriate to implement as a subroutine (a procedure or a function); no suitable library routine exists for this part.

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A function header in pseudocode is defined as:

```
FUNCTION Pass2 (Count : INTEGER) RETURNS BOOLEAN
```

Complete the table by describing the terms used in the function header.

The first row has been completed.

Term	Description
Pass2	the name of the function
Count	
BOOLEAN	

[2]

(c) Variables in the program have example values:

Variable	Example value
DoB	23/6/2011
Multiplier	2.5
AddressLine[1]	"35 Lincoln Avenue"

Complete the table by evaluating each expression using the example values:

Expression	Evaluates to
LENGTH (NUM_TO_STR (Multiplier))	
MONTH (DoB) > 4	
15 + STR_TO_NUM (MID (AddressLine[1], 2, 1))	

[3]

15. O/N 25/P23/Q5/b

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
```

```
Num[2] ← 2
```

```
Num[3] ← 5
```

```
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count ≤ 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE
```

(b) As a reminder, the CASE structure in the pseudocode is:

```
CASE OF CaseVar
  1 : Num[Index] ← Num[Index] + Index //clause one
    Index ← Index + 1
    Count ← Count + 1

  2 : Num[Index] ← Num[Index] + Index //clause two
    Index ← Index + 2
    Count ← Count + 2

  3 : Num[Index] ← Num[Index] * 2      //clause three
    Index ← Index + 3
    Count ← Count - 1

  4 : Count ← Count + 4                //clause four
  OTHERWISE : Count ← 20
ENDCASE
```

The CASE structure could be optimised by combining two existing clauses.

(i) Identify the CaseVar values for these **two** existing clauses.

[1]

(ii) Write pseudocode for a single clause to replace the **two** clauses identified in **(b) (i)**.

[2]

16. O/N 25/P23/Q6/b

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (b) A function `CheckNumber()` will take an integer value and return the Boolean value `TRUE` if the check digit is correct, otherwise return `FALSE`

Write pseudocode for the function `CheckNumber()`

Assume that the parameter is valid.

[7]

17. M/J 25/P21/Q4/a**4** Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I ≤ 4                                //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J ≥ 0 AND Chars[J] > Key          //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(a) The outer loop structure used in the algorithm is **not** the most appropriate one to use.

State the type of loop structure that would be the most appropriate to use and justify why it is the most appropriate.

Loop structure

Justification

[2]

18. M/J 25/P21/Q7/a

- 7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	• called with two integer parameters: ◦ the number of coffees ordered ◦ the current number of points • output a suitable message giving the number of free coffees • return a value for the new points total.

- (a) Write pseudocode for module `CustomerOrder()`

[6]

19. M/J 25/P23/Q1/a,b

Uses the insert (Appendix)

1 A program is being developed for the management of a sports centre.**(a)** The programmer developing the software decides to use modules (procedures or functions) and local variables. They also decide not to use global variables.**(i)** State **two** reasons why the programmer decides to use modules.

1

2

[2]

(ii) State **one** difference between local and global variables.

[1]

(iii) State **two** benefits of using local variables.

1

2

[2]

(b) The pseudocode design contains a number of expressions.

Complete each pseudocode expression so that it evaluates to the value shown.

Refer to the **insert** for the list of pseudocode functions and operators.

Expression	Evaluates to
(68)	' D '
..... (04 / 02 / 2025)	2
TRUE	FALSE
..... (..... ("Court1.4Upper" , ,))	1 . 4

[5]

20. M/J 25/P23/Q5

- 5 A module `Parity()` takes a string as a parameter. The parameter has the identifier `BitString` and it represents a binary value.

The module will concatenate a single character to the end of `BitString` by applying one of the two rules:

- '0' if `BitString` contains an even number of 1s
- '1' if `BitString` contains an odd number of 1s.

The modified value of `BitString` is then returned.

For example:

Parameter	String returned	Explanation
"0010010110"	"00100101100"	there are an even number of 1s in the string passed to <code>Parity()</code> so a '0' is concatenated to the end of <code>BitString</code>
"101010"	"1010101"	there are an odd number of 1s in the string passed to <code>Parity()</code> so a '1' is concatenated to the end of <code>BitString</code>

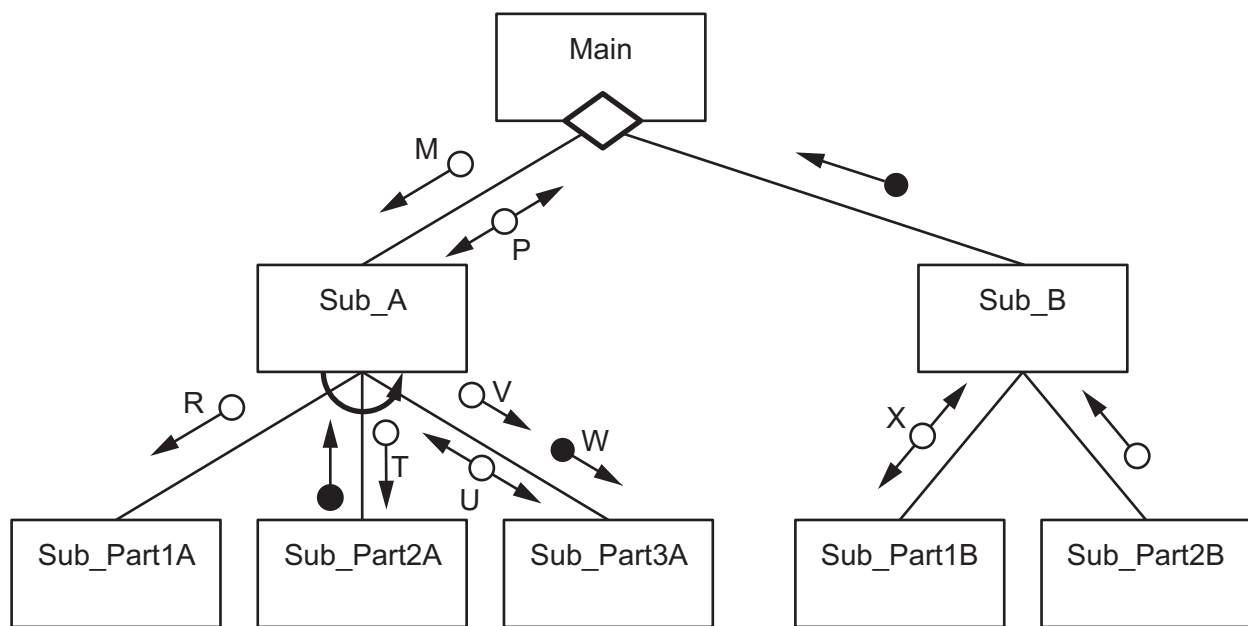
Write pseudocode for the module `Parity()`

Assume the parameter `BitString` can only contain the characters '0' and '1'

[8]

21. M/J 25/P23/Q6/a

6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(a) Explain **one** difference between functions and procedures.

[1]

22. M/J 25/P23/Q7/a,b(ii)

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
UpdateVisit()	- called with two parameters: - loyalty points of type <code>INTEGER</code> - last visit date of type <code>DATE</code> - change the loyalty points: - increase by four if the last visit date and the current date are in the same month - otherwise increase by one - change the last visit date to the current date - the changed values must be available to the code that follows the call to <code>UpdateVisit()</code>

(a) (i) Write pseudocode for module `UpdateVisit()`

Assume the customer has made at least **one** previous visit.

[5]

(ii) The programmer decides to amend the `UpdateVisit()` module so that loyalty points are further increased if the customer visits the coffee shop the same day of the week as their last visit.

Write the pseudocode for this condition.

[1]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: DDMMYYYY.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

`"100123,132,05032025"`

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
<code>FindCustomer()</code> (is already written)	- called with a parameter of type <code>STRING</code> that represents a customer ID - returns the line of data read from the text file <code>Loyalty.txt</code> containing the data items for that customer
<code>MondayCheck()</code>	- called with a parameter of type <code>STRING</code> that represents the customer ID of a customer - calls <code>FindCustomer()</code> - extracts the loyalty points from the string returned by <code>FindCustomer()</code> - increases the loyalty points for the current customer by 10 if the day visited is a Monday - returns an integer value representing the loyalty points

(ii) A date is stored in `LastVisitDateString` in the format:

DDMMYYYY

DD is a 2-digit string

MM is a 2-digit string

YYYY is a 4-digit string.

For example, the date 03/09/2024 is stored as "03092024"

An algorithm is needed to convert the string stored in `LastVisitDateString` to data type `DATE` which is then stored in the variable `LastVisitDate`.

Complete the pseudocode for this algorithm:

Assume that `LastVisitDateString` has been declared and contains valid data.

```
DECLARE DayInt, MonthInt, YearInt : INTEGER
```

```
DECLARE LastVisitDate : DATE
```

[4]

23. O/N 24/P21/Q1/b,d

- 1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23     // tax rate of 5.23%
    > 200 : TaxRate ← 6.25     // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

- (b) During the design of the program, tax rate values have been used wherever they are needed as shown in the pseudocode example above. Tax rates do not change while the program runs.

- (i) Identify a more appropriate way of representing the tax rate values in the final program.

[1]

- (ii) Describe the benefits of your answer to part (b)(i) with reference to this program.

[3]

- (d) The final CASE condition (> 200) in the pseudocode example could be replaced with a keyword.

Give the keyword.

[1]

24. O/N 24/P21/Q2

- 2 A program uses three global integer variables HH, MM and SS to represent the current time in hours, minutes and seconds using the 24-hour clock notation.

Midnight would be represented as 00:00:00 (HH:MM:SS). If the variables HH, MM and SS contained the values 16, 30 and 10 respectively, then the time would be 16:30:10 or just after 4.30 in the afternoon.

A procedure Tick() will be called every second.

The procedure Tick() will:

- update the value in SS each time it is called
- update the values in HH and MM as appropriate
- call a procedure CheckAlarm() at the start of each minute
- call a procedure NewDay() whenever the time reaches midnight.

Complete the pseudocode for procedure Tick().

```
PROCEDURE Tick()
ENDPROCEDURE
```

[6]

25. O/N 24/P21/Q4/b,c

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (b) The designer considers implementing the **checks and calculation** as a module (a procedure or a function). One reason for this is that the same checks and calculations are performed at several places in the program.

Give **another** reason why this is a suitable approach **and** state what is avoided by this approach.

Reason

Avoided

[2]

- (c) The module to perform the checks and calculation will be implemented as a function. The function will need to return both a real **and** a Boolean value. To achieve this a record type is defined in pseudocode as follows:

```
TYPE Result
    DECLARE Done : BOOLEAN
    DECLARE Value : REAL
ENDTYPE
```

The function `Evaluate()` will:

- take two parameters of type string representing the two numeric values
- return a variable of type `Result` with the `Done` field set to `FALSE` if either of the following applies:
 - at least one of the strings does **not** represent a valid numeric value
 - the numeric value of the string representing value `y` is zero
- otherwise return a variable of type `Result` with the `Done` field set to `TRUE` and the `Value` field assigned the result of the formula (based on the numeric value of the two parameters).

Write pseudocode for the function `Evaluate()`.

[6]

26. O/N 24/P21/Q6/b

- 6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

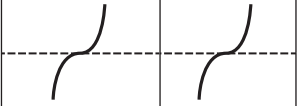
The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
	
11	30
12	31

Note: Leap years are **not** considered

(b) Complete the pseudocode for the function `GetDate()`.

Date functions from the **insert** should be used in your solution.

```
FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE
ENDFUNCTION
```

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

```
RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

```
MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
```

returns a string of length y starting at position x from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

`DAY(ThisDate : DATE) RETURNS INTEGER`

returns the day number from `ThisDate`

Example: `DAY(04/10/2003)` returns 4

`MONTH(ThisDate : DATE) RETURNS INTEGER`

returns the month number from `ThisDate`

Example: `MONTH(04/10/2003)` returns 10

`YEAR(ThisDate : DATE) RETURNS INTEGER`

returns the year number from `ThisDate`

Example: `YEAR(04/10/2003)` returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes: - may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR and/or STRING - cannot be used to compare two records
<	
>=	
<=	
<>	
	Examples:
	"Program" = "program" evaluates to FALSE
	Count = 4 evaluates to TRUE when Count contains the value 4

27. O/N 24/P21/Q8/a

- 8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
<code>GradeBoundary[1]</code>	65	The minimum mark for an A grade.
<code>GradeBoundary[2]</code>	57	The minimum mark for a B grade.
<code>GradeBoundary[3]</code>	43	The minimum mark for a C grade.
<code>GradeBoundary[4]</code>	35	The minimum mark for a D grade.
<code>GradeBoundary[5]</code>	27	The minimum mark for an E grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark	Element	ID
<code>Result[4, 1]</code>	56	<code>Result[4, 2]</code>	1074832
<code>Result[5, 1]</code>	54	<code>Result[5, 2]</code>	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
<code>CheckMark()</code>	- called with a parameter of type integer representing a candidate mark - returns <code>TRUE</code> if the mark is within 2 of any of the five grade boundaries, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckMark()`.

28. O/N 24/P22/Q2/a,b(ii)

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character ' * '.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
"1234567890"	"*****7890"
"1234567897652"	"*****7652"
"1234567890123456"	"*****3456"

(a) The function `Conceal()` will:

- take a numeric string as a parameter representing the card number
- return a string in which the asterisk character replaces all except the last four characters of the card number parameter.

Write pseudocode for the function `Conceal()`.

[6]

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(ii) The new procedure `Conceal()` will write the modified string to the corresponding element in column two.

The array `CardNumber` is passed as a parameter to the new procedure `Conceal()`.

Identify how this parameter should be specified in the new procedure header.

[1]

29. O/N 24/P22/Q4

- 4 A global integer variable `Tick` is always incremented every millisecond (1000 times per second) regardless of the other programs running.

The value of `Tick` can be read by any program but the value should not be changed.

Assume that the value of `Tick` does not overflow.

As an example, the following pseudocode algorithm would output "Goodbye" 40 seconds after outputting "Hello".

```
DECLARE Start : INTEGER
OUTPUT "Hello"
Start ← Tick
REPEAT
    //do nothing
UNTIL Tick = Start + 40000
OUTPUT "Goodbye"
```

A program is needed to help a user to time an event such as boiling an egg.

The time taken for the event is known as the elapsed time.

The program contains a procedure `Timer()` which will:

- take two integer values representing an elapsed time in minutes and seconds
- use the value of variable `Tick` to calculate the elapsed time
- output a warning message 30 seconds before the elapsed time is up
- output a final message when the total time has elapsed.

For example, to set an alarm for 5 minutes and 45 seconds the program makes the following call:

```
CALL Timer(5, 45)
```

When 5 minutes and 15 seconds have elapsed, the program will output:

```
"30 seconds to go"
```

When 5 minutes and 45 seconds have elapsed, the program will output:

```
"The time is up!"
```

Write pseudocode for the procedure `Timer()`.

30. O/N 24/P22/Q5/c

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```

FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN ReturnValue
ENDFUNCTION

```

- (c) The function `Process()` contains a selection construct using a `CASE` structure.

Write pseudocode using a single selection construct with the same functionality **without** using a `CASE` structure.

[2]

31. O/N 24/P23/Q1/c

- (b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Available	TRUE	
Received	"18/04/2021"	
Index	100	

[3]

- (c) Evaluate each expression in the table by using the data values shown in part (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
Available AND NOT(Index > 100)	
Index MOD 30	
NUM_TO_STR(Index + "33")	

[3]

32. O/N 24/P23/Q6

- 6 In some countries, on the third Sunday in March, daylight saving time begins when clocks move forward by one hour.

A module `AdjustClock()` will take an integer parameter representing a year. The module will return an integer value representing the number of the day in March on which the clocks move forward.

For example, the following line of pseudocode would assign `DayNumber` the value 20:

DayNumber $\leftarrow$ AdjustClock(2022)

Write pseudocode for the function AdjustClock().

Date functions from the **insert** should be used in your solution.

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type CHAR or STRING
- A CHAR may be assigned to, or concatenated with, a STRING
- A STRING of length greater than 1 cannot be assigned to a CHAR

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER

returns the day number from ThisDate

Example: DAY(04/10/2003) returns 4

MONTH(ThisDate : DATE) RETURNS INTEGER

returns the month number from ThisDate

Example: MONTH(04/10/2003) returns 10

YEAR(ThisDate : DATE) RETURNS INTEGER

returns the year number from ThisDate

Example: YEAR(04/10/2003) returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

<code>&</code>	concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> evaluates to <code>"Summer Pudding"</code> may also be used to concatenate a <code>CHAR</code> with a <code>STRING</code>
<code>AND</code>	performs a logical <code>AND</code> on two Boolean values. Example: <code>TRUE AND FALSE</code> evaluates to <code>FALSE</code>
<code>OR</code>	performs a logical <code>OR</code> on two Boolean values. Example: <code>TRUE OR FALSE</code> evaluates to <code>TRUE</code>
<code>NOT</code>	performs a logical <code>NOT</code> on a Boolean value. Example: <code>NOT TRUE</code> evaluates to <code>FALSE</code>
<code>MOD</code>	finds the remainder when one number is divided by another. Example: <code>10 MOD 3</code> evaluates to 1
<code>DIV</code>	finds the quotient when one number is divided by another. Example <code>10 DIV 3</code> evaluates to 3

Comparison operators

<code>=</code>	used to compare two items of the same type. evaluates to <code>TRUE</code> if the condition is true, otherwise evaluates to <code>FALSE</code>
<code>></code>	Notes: - may be used to compare types <code>REAL</code> and <code>INTEGER</code> - may be used to compare types <code>CHAR</code> and <code>STRING</code> - case sensitive when used to compare types <code>CHAR</code> and/or <code>STRING</code> - cannot be used to compare two records
<code><</code>	
<code>>=</code>	
<code><=</code>	
<code><></code>	
	Examples: <code>"Program" = "program"</code> evaluates to <code>FALSE</code> <code>Count = 4</code> evaluates to <code>TRUE</code> when <code>Count</code> contains the value 4

33. M/J 24/P21/Q1/b

1 An algorithm is developed in pseudocode before being coded in a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Sorted	False
Tries	9
ID	"ZGAC001"

Complete the table by evaluating each expression, using the example values.

Expression	Evaluates to
<code>Tries < 10 AND NOT Sorted</code>	
<code>Tries MOD 4</code>	
<code>TO_LOWER(MID(ID, 3, 1))</code>	
<code>LENGTH(ID & "xx") >= Tries</code>	

[4]

34. M/J 24/P21/Q2/b

2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values A and B
2. count the number of pairs that are input until A has been greater than B 10 times
3. output the number of pairs that were input.

(b) Step 1 of the algorithm is changed.

A variable `ThisSequence` is used to enter a sequence of 10 pairs of numeric values, using a single input statement.

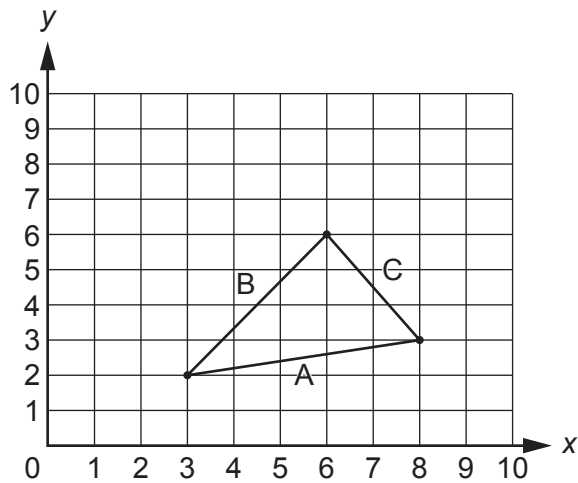
Following the input of `ThisSequence` the revised algorithm will extract the pairs of numbers.

Describe the variable `ThisSequence` and how the numbers are extracted.

[2]

35. M/J 24/P21/Q6/a

6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



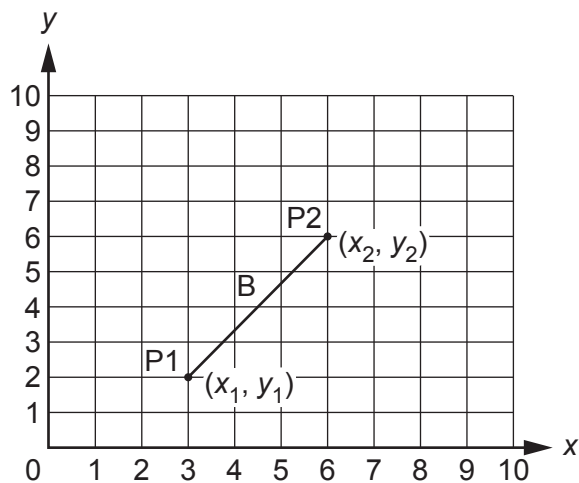
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator ‘^’ represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
    ENDFUNCTION
```

[6]

36. M/J 24/P21/Q8/a

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
MichaelAday_src.txt	student project file produced by student Michael Aday
MichaelAday_S1.txt	file produced by stage 1
MichaelAday_S2.txt	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
<code>DeleteComment()</code>	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: <code>IF X2 > 13 THEN //check if limit exceeded</code> After: <code>IF X2 > 13 THEN</code>

- (a) Complete the pseudocode for module `DeleteComment()`.

```
FUNCTION DeleteComment(Line : STRING) RETURNS STRING
    ENDFUNCTION
```

[8]

37. M/J 24/P22/Q1/b

Uses the insert (Appendix)

(b) Program variables have data types as follows:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER

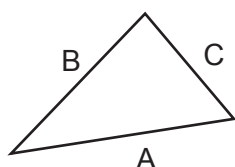
Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
MyInt ← (3.1415926)
MyChar ← ("Elwood", 3, 1)
MyString ← (..... (27.509))
MyInt ← (..... ("ABC123", 3))

[4]

38. M/J 24/P22/Q4

4 A triangle has sides of length A, B and C.



In this example, A is the length of the longest side.

This triangle is said to be right-angled if the following equation is true:

$$A \times A = (B \times B) + (C \times C)$$

A procedure will be written to check whether three lengths represent a right-angled triangle. The lengths will be input in any sequence.

The procedure `IsRA()` will:

- prompt and input three integer values representing the three lengths
- test whether the three lengths correspond to the sides of a right-angled triangle
- output a suitable message.

The length of the longest side may **not** be the first value input.

Write pseudocode for the procedure `IsRA()`.

[5]

39. M/J 24/P22/Q5/a(ii)

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(ii) The procedure contains a statement that is **not** needed.

Identify the pseudocode statement **and** explain why it is **not** needed.

Statement

Explanation

[2]

40. M/J 24/P22/Q6/a

- 6** A music player stores music in a digital form and has a display which shows the track being played.

(a) Up to 16 characters can be displayed. Track titles longer than 16 characters will need to be trimmed as follows:

- Words must be removed from the end of the track title until the resulting title is less than 14 characters.
- When a word is removed, the space in front of that word is also removed.
- Three dots are added to the end of the last word displayed when one or more words have been removed.

The table below shows some examples:

Original title	Display string															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bat out of Hull	B	a	t		o	u	t		o	f		H	u	l	l	
Bohemian Symphony	B	o	h	e	m	i	a	n	.	.	.					
Paperbook Writer	P	a	p	e	r	b	o	o	k		W	r	i	t	e	r
Chris Sings the Blues	C	h	r	i	s		S	i	n	g	s	.	.	.		
Green Home Alabama	G	r	e	e	n		H	o	m	e	.	.	.			

A function `Trim()` will:

- take a string representing the original title
- return the string to be displayed.

Assume:

- Words in the original title are separated by a single space character.
- There are no spaces before the first word or after the last word of the original title.
- The first word of the original title is less than 14 characters.

Write pseudocode for the function `Trim()`.

41. M/J 24/P22/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

- (b) The teacher has defined the first program module as follows:

Module	Description
DeleteSpaces()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: " IF X2 > 13 THEN" After: "IF X2 > 13 THEN"

Complete the pseudocode for module DeleteSpaces().

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
ENDFUNCTION
```

[6]

42. M/J 24/P23/Q1/a,b

Uses the insert (Appendix)

- 1** A program uses many complex algorithms.

One algorithm is repeated in several places. The code for the algorithm is the same wherever it is used, but the calculations within the algorithm may operate on different data.

The result of each calculation is used by the code that follows it.

It is decided to modify the program and implement the algorithm as a separate module.

- (a) (i)** State **two** benefits of this modification to the existing program.

1

2

[2]

- (ii)** Describe how the modification would be implemented.

[3]



- (b)** Four of the expressions used in the program are represented by pseudocode in the table.

Complete each pseudocode expression with a function or operator so that it evaluates to the value shown.

Any functions and operators used must be defined in the **insert**.

Pseudocode expression	Evaluates to
..... ("Random", 2, 3)	"and"
5 + (10/11/2023)	15
..... ("45000")	TRUE
(20 3) + 1	3

[4]



43. M/J 24/P23/Q4/a

- 4 A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:

- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

- (a) Write pseudocode for the procedure `TwoParts()`.

[6]

**44. M/J 24/P23/Q5/a,b**

- 5 A program is being designed in pseudocode.

The program contains the following declaration:

```
DECLARE Data : ARRAY[1:1000] OF STRING
```

A procedure `ArrayInitialise()` is written to initialise the values in the array:

```
PROCEDURE ArrayInitialise(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 1
  WHILE Index ≤ 1000
    CASE OF (Index MOD 2)
      0 : Data[Index] ← FormatA(Label)
          Index ← Index + 1
      1 : Data[Index] ← FormatB(Label)
          Index ← Index + 1
    ENDCASE
  ENDWHILE
ENDPROCEDURE
```

Functions `FormatA()` and `FormatB()` apply fixed format case changes to the parameter string.

- (a) The design of the procedure does **not** use the most appropriate loop construct.

Suggest a **more appropriate** construct that could be used **and** explain your choice.

Construct

Explanation

[2]

- (b) The algorithm calls one of the functions `FormatA()` and `FormatB()` each time within the loop.

Explain why this is **not** efficient **and** suggest a more efficient solution.

[4]



45. M/J 24/P23/Q6/a,b(i)

- 6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.



- (a) Write pseudocode for procedure `Progress()`.

[6]



- (b)** The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

- (i) Write pseudocode for the new module header.

[2]

46. M/J 24/P23/Q7/a(ii)

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (ii) Modules Mod-F and Mod-G are both called with Par3 as a parameter.
 In the case of Mod-F, the parameter is passed by value.
 In the case of Mod-G, the parameter is passed by reference.

Explain the effect of the **two** different ways of passing the parameter Par3.

[2]



48. O/N 23/P21/Q1/a,b,c

- 1 The following pseudocode represents part of the algorithm for a program:

```

CASE OF ThisValue
    < 30      : Level ← "Low"           // less than 30
               Check ← 1
    < 20      : Level ← "Very Low"      // less than 20
               Check ← ThisValue / 2
    30 TO 40  : Level ← "Medium"        // between 30 and 40
               Check ← ThisValue / 3
               Data[ThisValue] ← Data[ThisValue] + 1
    > 40      : Level ← "High"
ENDCASE

```

- (a) Complete the table by writing the answer for each row:

	Answer
The value assigned to <code>Level</code> when <code>ThisValue</code> is 40	
The value assigned to <code>Check</code> when <code>ThisValue</code> is 36	
The value assigned to <code>Level</code> when <code>ThisValue</code> is 18	
The number of elements in array <code>Data</code> that may be incremented	

[4]

- (b) The pseudocode contains four assignments to variable `Level`. One of these assignments will never be performed.

Identify this assignment **and** explain why this is the case.

[3]

- (c) The following line is added immediately before the `ENDCASE` statement:

```
OTHERWISE : Level ← "Undefined"
```

State why this assignment is never performed.

[1]

49. O/N 23/P21/Q6/a,b

6 The pseudocode `OUTPUT` command starts each output on a new line.

- (a) A new procedure `MyOutput()` will take a string and a Boolean parameter. `MyOutput()` may be called repeatedly and will use concatenation to build a string using a global variable `MyString`, up to a maximum length of 255 characters.

`MyString` will be output in either of these two cases:

1. The Boolean parameter value is `TRUE`
2. The resulting string (after concatenation) would be longer than 255 characters.

If `MyString` is not output, the string is concatenated with `MyString`.

For example, the calls to `MyOutput()` given below would result in the output as shown:

```
MyOutput("Hello ", FALSE)
MyOutput("ginger ", FALSE)
MyOutput("cat", TRUE)
MyOutput("How are you?", TRUE)
```

Resulting output:

```
Hello ginger cat
How are you?
```

Notes:

- `MyString` is initialised to an empty string before `MyOutput()` is called for the first time.
- No string passed to `MyOutput()` will be longer than 255 characters.

Write pseudocode for `MyOutput()`.

[7]

- (b) The design of the procedure given in **part (a)** is modified and `MyString` is changed from a global to a local variable declared in `MyOutput()`.

When the modified procedure is converted into program code, it does not work as expected.

Explain why it does not work as expected.

[2]

50. O/N 23/P21/Q8/d

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

<STX>"232101Hello Kevin"<ETX>

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "*****"

(d) A new module has been defined:

Module	Description
<code>GetField()</code>	• takes two parameters: ◦ a string containing a message ◦ an integer containing a field number • If the field number is valid (in the range 1 to 3, inclusive), it returns a string containing the required field, otherwise it returns an empty string.

As a reminder, a message is defined as follows:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
Not applicable	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
Not applicable	ETX	a single character marking the end of the message (ASCII value 03)

Write pseudocode for module `GetField()`.

[6]

51. O/N 23/P22/Q1/b

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(b) Other variables in the program have example values as shown:

Variable	Example value
Total	124.00
DepRate	2.00
Description	"AB12345:Cleaning Brush (small)"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
<code>(Total * DepRate) + 1.5</code>	
<code>RIGHT(Description, 7)</code>	
<code>(LENGTH(Description) - 8) > 16</code>	
<code>NUM_TO_STR(INT(DepRate * 10)) & '%'</code>	

[4]

52. O/N 23/P22/Q2/b

2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(b) The solution to the algorithm includes iteration.

Give the name of a suitable loop structure that could be used.

Justify your answer.

Name

Justification

[2]

53. O/N 23/P22/Q4/a

4 A procedure `Count()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(a) Write pseudocode for the procedure `Count()`.

[6]

54. O/N 23/P22/Q6/b(i,ii)

6 (a) A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, `TestData.001`, `TestData.002` and `TestData.003`.

Each file will contain a single line. For example, file `TestData.002` would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

[6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(i) Identify the type of module that should be used for `CheckFiles()`.

[1]

(ii) Write the module header for `CheckFiles()`.

[1]

55. O/N 23/P22/Q8/b

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.

Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.

When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.

Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	Column 2	Column 3
Row 1	100	199	1
Row 2	200	259	2
Row 3	-1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to -1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
<code>GetPort()</code>	- takes a <code>DestinationID</code> as a parameter of type string - searches for the range corresponding to the <code>DestinationID</code> in the array - returns the port number, or returns -1 if no corresponding range is found

- (b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
<code>StackMsg()</code> (already written)	• takes two parameters: ◦ a string representing a message ◦ an integer representing the stack number • adds the message to the required stack • returns <code>TRUE</code> if the message is added to the required stack, otherwise returns <code>FALSE</code>
<code>ProcessMsg()</code>	• takes a message as a parameter of type string • ignores any message with a zero-length data field • extract the <code>DestinationID</code> from the message • checks whether the <code>DestinationID</code> is this computer or whether the message is to be forwarded • uses <code>StackMsg()</code> to add the message to the appropriate stack • outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

56. O/N 23/P23/Q1/b,c

- 1 A program is being developed in pseudocode before being converted into a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Active	TRUE
Fraction	0.2
Code	"Ab12345"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
Fraction >= 0.2 AND NOT Active	
INT((Fraction * 100) + 13.3)	
STR_TO_NUM(MID(Code, 4, 2)) + 5	
LENGTH("TRUE" & Code)	

[4]

- (c) The program makes use of complex statistical functions. The required functions are not built-in to the programming language and are too complicated for the programmer to write.

One solution would be to employ another programmer who has experience of writing these functions, as there is no time to train the existing programmer.

State **one other** way that these functions may be provided for inclusion in the program.

[1]

57. O/N 23/P23/Q4/a,b

- 4 A procedure `RandList()` will output a sequence of 25 random integers, where each integer is larger than the previous one.

(a) Write pseudocode for procedure `RandList()`.

[6]

- (b) Procedure `RandList()` is modified so that the random numbers are also written into a 1D array `Result`.

A new module is written to confirm that the numbers in the array are in ascending order.

This module contains an `IF` statement that performs a comparison between elements:

```
IF (Result[x + 1] = Result[x]) OR (Result[x] > Result[x + 1]) THEN
    Sequence ← FALSE
ENDIF
```

Write a simplified version of the conditional clause.

[1]

58. O/N 23/P23/Q6

- 6 A function `TestNum()` will take a six-digit string as a parameter.

The function will test whether the string meets certain conditions and will return an integer value as follows:

Return value	Condition	Example
1	The last three digits are the same but non-zero.	"253444"
2	The last three digits are zero.	"253000"
3	The first three and last three digits are the same.	"410410"

The function will return the highest possible value for the given string.

If the string does not meet any of the conditions, zero is returned.

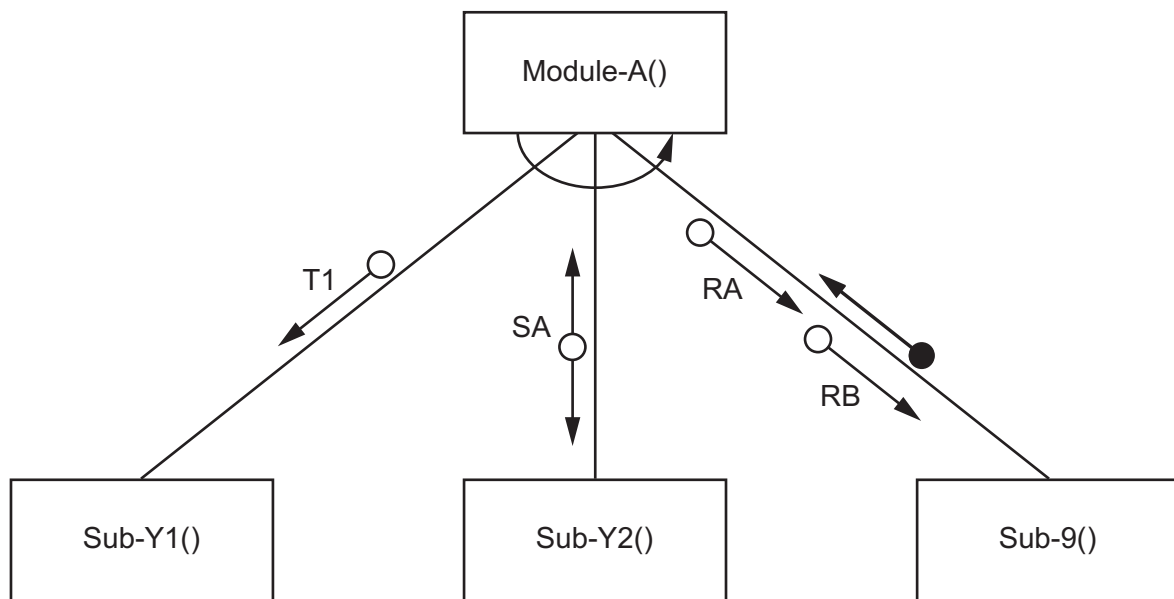
Write pseudocode for function `TestNum()`.

Assume that the parameter is valid.

[6]

59. O/N 23/P23/Q7/b(ii)

7 A structure chart shows the modular structure of a program:



(b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

(ii) The design is modified and Sub-9() is changed to a procedure.

The procedure will be called with a single parameter of type `MyType`.

Write the pseudocode header for procedure `Sub-9()`.

[2]

60. O/N 23/P23/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	- returns the data field from a message that has been received - If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	- takes a file name as a parameter of type string - creates a text file with the given file name (no checking required) - writes the data field returned by <code>GetData()</code> to the file - repeats until the data field is "*****", which is not written to the file - outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string representing a message ◦ an integer representing a port number • transmits the message using the given port
<code>Chat()</code>	• takes two parameters: ◦ a string representing a Destination ID ◦ an integer representing a port number • extracts data from a received message using <code>GetData()</code> and outputs it • forms a message using data input by the user and sends it using <code>Transmit()</code> • repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

61. M/J 23/P21/Q1/b,c

- 1 A programmer has written a program which includes the function `Calculate()`. When the program is run, the function returns an unexpected value.

- (b) The algorithm for function `Calculate()` contains the three pseudocode statements shown.

Describe the error in each statement or write 'no error' if the statement contains no error.

Assume any variables used are of the correct type for the given function.

Statement 1: `Index ← STR_TO_NUM("27") + 2)`

Error

Statement 2: `Index ← STR_TO_NUM(MID("CPE1704TKS", 4, 2))`

Error

Statement 3: `IF MONTH(ThisDate) > '6' THEN`

Error

[3]

- (c) The program contains variables with values as follows:

Variable	Value
Active	TRUE
Points	75
Exempt	FALSE

- (i) Complete the table by evaluating each expression.

	Expression	Evaluation
1	<code>(Points > 99) OR Active</code>	
2	<code>(Points MOD 2 = 0) OR Exempt</code>	
3	<code>(Points <= 75) AND (Active OR Exempt)</code>	
4	<code>(Active OR NOT Active) AND NOT Exempt</code>	

[2]

- (ii) Write expression 4 from the table in part (c)(i) in its simplest form.

[1]

62. M/J 23/P21/Q2/b

- 2 A program contains an algorithm to output a string of a specified length containing identical characters.
- (b) A different part of the program uses the variable `StartDate`.

Write pseudocode statements to declare `StartDate` **and** assign to it the date corresponding to 15/11/2005.

Declaration

Assignment

[3]

63. M/J 23/P21/Q4

- 4 Function `Replace()` will:
1. take three parameters:
 - a string (the original string)
 - a char (the original character)
 - a char (the new character)
 2. form a new string from the original string where all instances of the original character are replaced by the new character
 3. return the new string.

Write pseudocode for function `Replace()`.

[6]

64. M/J 23/P21/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `IsNewSupp()`.

Module `SuppExists()` has already been written and should be used as part of your solution. Module `SuppExists()` will generate a run-time error if the given parameter is not 5 characters in length.

65. M/J 23/P22/Q1/a

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (a) (i) Identify a more appropriate way of representing the postal costs.

[1]

- (ii) Describe the advantages of your answer to **part (a)(i)** with reference to this program.

[3]

66. M/J 23/P22/Q2/a,b

- 2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

- (a) Write a pseudocode statement, using a function from the **insert**, to assign the value corresponding to 17/11/2007 to `MyDOB`.

[1]

- (b) `MyDOB` has been assigned a valid value representing the user's date of birth.

Write a pseudocode statement to calculate the number of months from the month of the user's birth until the end of the year and to assign this to the variable `NumMonths`.

For example, if `MyDOB` contains a value representing 02/07/2008, the value 5 would be assigned to `NumMonths`.

[2]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and Character Functions

- A string of length 1 may be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns rightmost x characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`

returns a string of length y starting at position x from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns string "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`TO_UPPER(x : <datatype>) RETURNS <datatype>`

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- `TO_UPPER("Error 803")` returns "ERROR 803"
- `TO_UPPER('a')` returns 'A'

`TO_LOWER(x : <datatype>) RETURNS <datatype>`

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- `TO_LOWER("JIM 803")` returns "jim 803"
- `TO_LOWER('W')` returns 'w'

`NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>`

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: `NUM_TO_STR(87.5)` returns "87.5"

`STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>`

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: `STR_TO_NUM("23.45")` returns 23.45

`IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN`

returns TRUE if `ThisString` represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: `IS_NUM("-12.36")` returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of character ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66, etc.

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B', etc.

Numeric Functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (**not** inclusive of x).

Example: RAND(87) may return 35.43

Date Functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER

returns the current day number from ThisDate

Example: DAY(04/10/2003) returns 4

MONTH(ThisDate : DATE) RETURNS INTEGER

returns the current month number from ThisDate

Example: MONTH(04/10/2003) returns 10

YEAR(ThisDate : DATE) RETURNS INTEGER

returns the current year number from ThisDate

Example: YEAR(04/10/2003) returns 2003

DAYINDEX(ThisDate : DATE) RETURNS INTEGER

returns the day index number from ThisDate where Sunday = 1, Monday = 2 etc.

Example: DAYINDEX(09/05/2023) returns 3

SETDATE(Day, Month, Year : INTEGER) RETURNS DATE

returns a value of type DATE with the value of <Day>/<Month>/<Year>

Example: SETDATE(26, 10, 2003) returns a date corresponding to 26/10/2003

TODAY() RETURNS DATE

returns a value of type DATE corresponding to the current date.

Text File Functions

EOF(FileName : STRING) RETURNS BOOLEAN

returns TRUE if there are no more lines to be read from file FileName

The function will generate an error if the file is not already open in READ mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" The operator may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison Operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes:
<	- may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR or STRING - cannot be used to compare two records
>=	
<=	Examples:
<>	"Program" = "program" evaluates to FALSE - Count = 4 evaluates to TRUE when variable Count contains the value 4

67. M/J 23/P22/Q4

4 A function `GetNum()` will:

- take two parameters: a string and a character
- count the number of times that the character occurs in the string
- return the count.

Any comparison between characters needs to be case sensitive. For example, character 'a' and character 'A' are not identical.

Write pseudocode for function `GetNum()`.

68. M/J 23/P22/Q6

- 6** A procedure `Square()` will take an integer value in the range 1 to 9 as a parameter and output a number square.

The boundary of a number square is made up of the character representing the parameter value. The inside of the number square is made up of the asterisk character (*).

Parameter value	1	2	3	4	...	9
Output	1	22 22	333 3*3 333	4444 4**4 4**4 4444	...	999999999 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 999999999

The pseudocode `OUTPUT` command starts each output on a new line. For example, the following three `OUTPUT` statements would result in the outputs as shown:

```
OUTPUT "Hello"
OUTPUT "ginger"
OUTPUT "cat"
```

Resulting output:

```
Hello
ginger
cat
```

Write pseudocode for procedure `Square()`.

Parameter validation is not required.

69. M/J 23/P22/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckInfo()`.

Module `OnlyAlpha()` should be used as part of your solution.

[7]

70. M/J 23/P23/Q2/a

- 2 A program stores a date of birth for a student using a variable, `MyDOB`, of type `DATE`.

- (a) `MyDOB` has been assigned a valid value corresponding to Kevin's date of birth.

Complete the pseudocode statement to test whether Kevin was born on a Thursday.

IF

THEN [2]

71. M/J 23/P23/Q4

4 A function `MakeString()` will:

1. take two parameters:
 - a count as an integer
 - a character
2. generate a string of length equal to the count, made up of the character
3. return the string generated, or return "ERROR" if the count is less than 1.

For example, the function call:

`MakeString(3, 'Z')` will return the string "ZZZ"

Write pseudocode for function `MakeString()`.

[6]

72. M/J 23/P23/Q6/a,b

6 A procedure `Select()` will:

1. take two integer values as parameters representing start and end values where both values are greater than 9 and the end value is greater than the start value
2. output **each** integer value between the start and the end value (**not** including the start and end values), where the sum of the last two digits is 6, for example, 142.

(a) Write pseudocode for procedure `Select()`.

Parameter validation is **not** required.

[7]

(b) The check performed by procedure `Select()` on the last two digits is needed at several places in the program and will be implemented using a new function.

The new function `CheckNum()` will:

- allow the required sum to be specified (not just 6)
- check one number
- return an appropriate value.

Describe the function interface **and two** advantages of this modular approach.

Interface

Advantage 1

Advantage 2

[4]

73. O/N 22/P21/Q1/a,c

- 1 (a) An algorithm includes a number of complex calculations. A programmer is writing a program to implement the algorithm and decides to use library routines to provide part of the solution.

State **three** possible benefits of using library routines in the development of the program.

1

2

3

[3]

- (c) The pseudocode statements in the following table may contain errors.

State the error in each case or write 'NO ERROR' if the statement contains no error.

Assume that any variables used are of the correct type for the given function.

Statement	Error
IF EMPTY ← "" THEN	
Status ← IS_NUM(-23.4)	
X ← STR_TO_NUM("37") + 5	
Y ← STR_TO_NUM("37" + "5")	

[4]

74. O/N 22/P21/Q6/a

- 6** The following pseudocode algorithm attempts to check whether a string is a valid email address.

```

FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
            IF Ats > 1 THEN
              Valid ← FALSE
            ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF

  UNTIL Index > LENGTH(InString) OR Valid = FALSE

  IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
  ENDIF

  RETURN Valid

ENDFUNCTION

```

- (a)** Part of the validation is implemented by the line:

```
IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
```

State the values that would result in the condition evaluating to TRUE.

75. O/N 22/P21/Q7/a

7 A simple arithmetic expression is stored as a string in the format:

`<Value1><Operator><Value2>`

An operator character is one of the following: `'+'` `'-'` `'*'` `'/'`

Example arithmetic expression strings:

`"803+1904"`

`"34/7"`

(a) A procedure `Calculate()` will:

- take an arithmetic expression string as a parameter
- evaluate the expression
- output the result.

Assume:

- the string contains only numeric digits and a single operator character
- `Value1` and `Value2` represent integer values
- `Value1` and `Value2` are unsigned (they will not be preceded by `'+'` or `'-'`).

(i) Write pseudocode for the procedure `Calculate()`.

[7]

(ii) `Calculate()` is changed to a function that returns the value of the evaluated expression.

Write the header for the function in pseudocode.

[1]

76. O/N 22/P21/Q8/b

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	• takes the name of an existing project file as a parameter of type string • returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	• takes two parameters: ◦ the name of a project file as a string ◦ the maximum number of errors as an integer • uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid • calls <code>CheckLine()</code> for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (b) `CountErrors()` is called to check the project file `Jim01Prog.txt` and to stop if more than 20 errors are found.

Write the pseudocode statement for this call.

[2]

77. O/N 22/P22/Q1/a,c

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc(Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers **Count** and **Message** **and** explain their use.

Term

Use

[2]

(c) Part of an identifier table is shown:

Variable	Type	Example value
FlagDay	DATE	23/04/2004
CharList	STRING	"ABCDEF"
Count	INTEGER	29

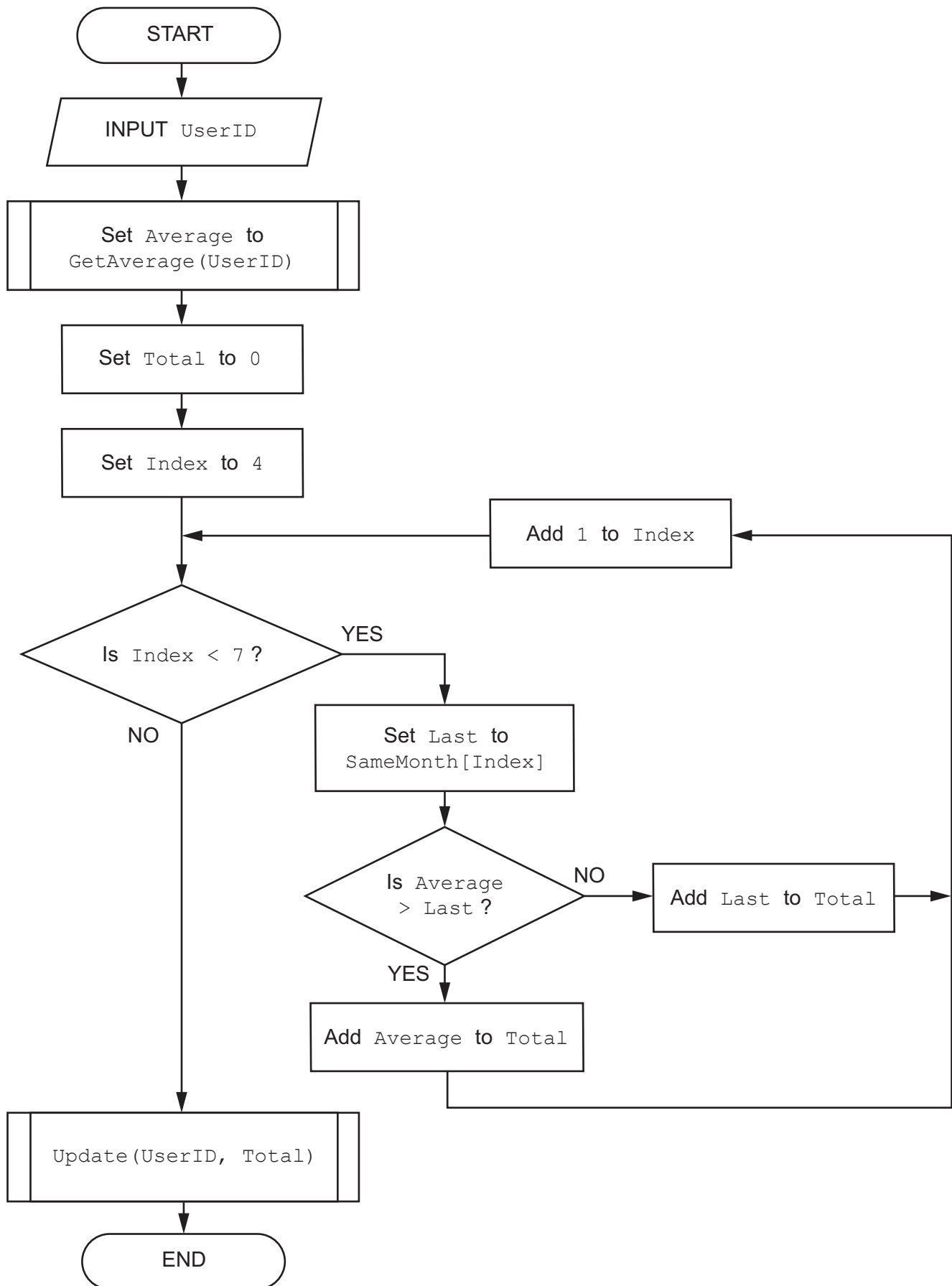
Complete the table by evaluating each expression using the example values.

Expression	Evaluation
MID(CharList, MONTH(FlagDay), 1)	
INT(Count / LENGTH(CharList))	
(Count >= 29) AND (DAY(FlagDay) > 23)	

[3]

78. O/N 22/P22/Q4/b

- 4 The program flowchart represents a simple algorithm.



(b) Give the name of the iterative construct in the flowchart.

79. O/N 22/P22/Q6/a

- 6 (a)** The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter
- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

80. O/N 22/P23/Q5/a,b(ii)

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (i) Write pseudocode for the procedure `Unpack()`.

[6]

- (ii) Explain the term **procedure interface** with reference to procedure `Unpack()`.

[2]

- (b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
  DECLARE StockID : STRING
  DECLARE Description : STRING
  DECLARE Cost : REAL
ENDTYPE
```

- (ii) Procedure `Unpack()` is modified and converted to a function which takes the original text string as the only parameter.

Explain the other changes that need to be made to convert the procedure into a function.

[2]

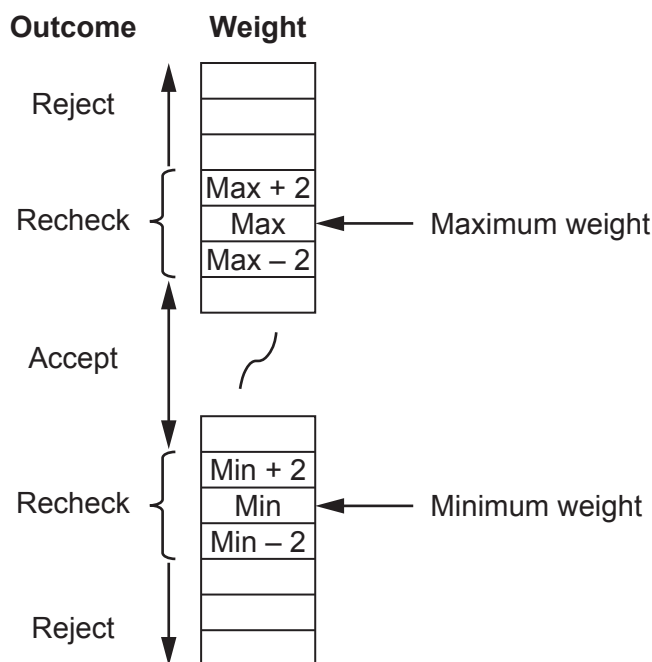
81. O/N 22/P23/Q6/b

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (b) Write pseudocode for `Status()`.

[6]

82. O/N 21/P21/Q6/c

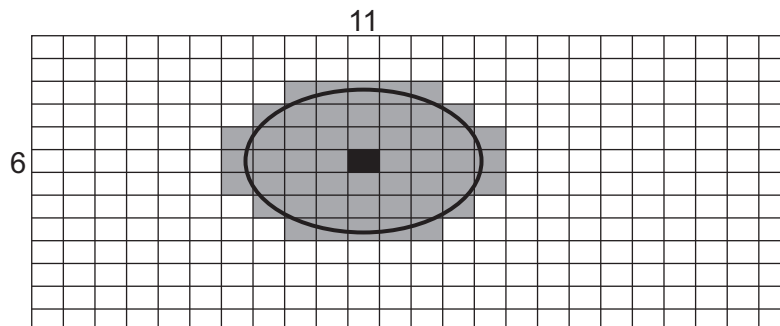
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
SetRow() (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
SearchInRow()	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
SearchInCol()	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
GetCentreCol()	- Called with a row number as an <code>INTEGER</code> - Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 - Returns the index of the column midway between the first and last columns - Returns -1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

[6]

83. O/N 21/P22/Q1/c,d

(c) Complete the pseudocode expressions so that they evaluate to the values shown.

Any functions and operators used must be defined in the **insert**.

Expression	Evaluates to
('C')	67
2 * ("27")	54
(27 /)	13
"Sub" & ("Abstraction" , ,)	"Subtract"

[4]

(d) Evaluate the expressions given in the following table. The variables have been assigned values as follows:

```
PumpOn ← TRUE
PressureOK ← TRUE
HiFlow ← FALSE
```

Expression	Evaluates to
PressureOK AND HiFlow	
PumpOn OR PressureOK	
NOT PumpOn OR (PressureOK AND NOT HiFlow)	
NOT (PumpOn OR PressureOK) AND NOT HiFlow	

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost x characters from ThisString

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Alphabetic characters that are not upper case are unchanged.

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Alphabetic characters that are not lower case are unchanged.

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case.

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case.

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type1>) RETURNS <data type2>`
returns a string representation of a numeric value.

Note: <data type1> may be `REAL` or `INTEGER`
<data type2> may be `CHAR` or `STRING`

Example: `NUM_TO_STR(87.5)` returns `"87.5"`

`STR_TO_NUM(x : <data type1>) RETURNS <data type2>`
returns a numeric representation of a string.

Note: <data type1> may be `CHAR` or `STRING`
<data type2> may be `REAL` or `INTEGER`

Example: `STR_TO_NUM("23.45")` returns `23.45`

`IS_NUM(ThisString : STRING) RETURNS BOOLEAN`
returns the value `TRUE` if `ThisString` represents a valid numeric value.

Example 1: `IS_NUM("12.36")` returns `TRUE`

Example 2: `IS_NUM("-12.36")` returns `TRUE`

Example 3: `IS_NUM("12.3a")` returns `FALSE`

`ASC(ThisChar : CHAR) RETURNS INTEGER`
returns an integer value (the ASCII value) of `ThisChar`

Example: `ASC('A')` returns `65`

`CHR(x : INTEGER) RETURNS CHAR`
returns the character whose integer value (the ASCII value) is `x`

Example: `CHR(87)` returns `'W'`

NUMERIC Functions

`INT(x : REAL) RETURNS INTEGER`
returns the integer part of `x`

Example: `INT(27.5415)` returns `27`

`RAND(x : INTEGER) RETURNS REAL`
returns a real number in the range 0 to `x` (**not** inclusive of `x`).

Example: `RAND(87)` could return `35.43`

OTHER Functions

EOF(FileName : STRING) RETURNS BOOLEAN
returns TRUE if there are no more lines to be read from file FileName

Note: The function will generate an error if the file is not already open in READ mode.

Note: An error occurs if an operator with a value of an incorrect type is used.

OPERATORS

&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE evaluates to FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE evaluates to TRUE
NOT	Performs a logical NOT on a Boolean value Example: NOT TRUE evaluates to FALSE
MOD	Finds the remainder when one number is divided by another Example: 10 MOD 3 evaluates to 1
DIV	Finds the quotient when one number is divided by another Example: 10 DIV 3 evaluates to 3

84. O/N 21/P22/Q4/a,c

4 The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```

10  PROCEDURE Check(InString : STRING)
11      DECLARE Odds, Evens, Index : INTEGER
12
13      Odds ← 0
14      Evens ← 0
15      Index ← 1
16
17      WHILE Index <= LENGTH(InString)
18          IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19              Odds ← Odds + 1
20          ELSE
21              Evens ← Evens + 1
22          ENDIF
23          Index ← Index + 1
24      ENDWHILE
25
26      CALL Result(Odds, Evens)
27  ENDPROCEDURE

```

(a) Complete the following table by giving the answers, using the given pseudocode.

Answer

A line number containing a variable being incremented	
The type of loop structure	
The number of functions used	
The number of parameters passed to STR_TO_NUM()	
The name of a procedure other than Check()	

[5]

(c) (i) The loop structure used in the pseudocode is not the most appropriate.

State a more appropriate loop structure **and** justify your choice.

Loop structure

Justification

[2]

(ii) The appropriate loop structure is now used. Two lines of pseudocode are changed and two lines are removed.

Write the line numbers of the two lines that are removed.

[1]

85. O/N 21/P22/Q6/b,c(ii),d

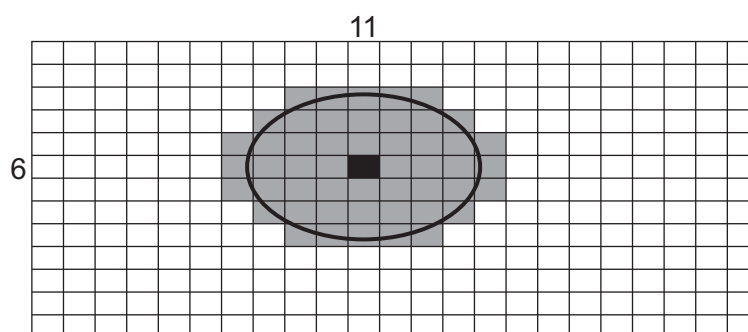
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	- Searches for the first row that has an array element set to 1 - Returns the index of that row (1 is the first row) - Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	- Searches for the last row that has an array element set to 1 - Returns the index of that row - Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	- Searches for the first column that has an array element set to 1 - Returns the index of that column (1 is the first column) - Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	- Searches for the last column that has an array element set to 1 - Returns the index of that column - Returns -1 if there are no elements set to 1

- (a) Write efficient pseudocode for the module `FirstRowSet()`.

- (b) Describe a feature of your solution to **part (a)** that indicates the pseudocode represents an efficient algorithm.

[2]

- (c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (ii) Give one possible advantage **and** one possible disadvantage of combining the four searches into a single module.

Advantage

Disadvantage

[2]

- (d) An additional module `GetCentre()` is defined as follows:

Module	Description
<code>GetCentre()</code>	- Calculates the coordinates of the centre point - Uses the four modules: <code>FirstRowSet()</code>, <code>LastRowSet()</code>, <code>FirstColSet()</code>, <code>LastColSet()</code> - Assigns values to <code>CentreRow</code> and <code>CentreCol</code> - Sets <code>CentreRow</code> to <code>-1</code> if no screen touch is detected

Write pseudocode for the module `GetCentre()`.

[6]

86. O/N 21/P23/Q6/c

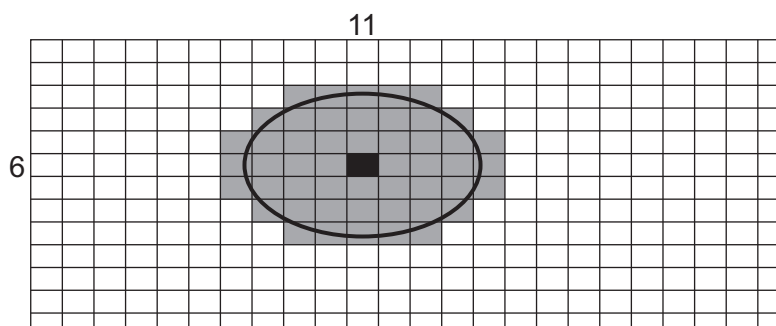
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
<code>SetRow()</code> (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
<code>SearchInCol()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
<code>GetCentreCol()</code>	• Called with a row number as an <code>INTEGER</code> • Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 • Returns the index of the column midway between the first and last columns • Returns -1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

87. M/J 21/P21/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

88. M/J 21/P21/Q2/b

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either LoanExtend() or LoanReturn()
LoanExtend()	• Called with parameters LoanID and BookID • Calls CheckReserve() to see whether the book has been reserved for another library user • Returns TRUE if the loan has been extended, otherwise returns FALSE
CheckReserve()	• Called with BookID • Returns TRUE if the book has been reserved, otherwise returns FALSE
LoanReturn()	• Called with parameters LoanID and BookID • Returns a REAL (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
<code>LoanReturn()</code>	Called with parameters <code>LoanID</code> , <code>BookID</code> and <code>Fine</code> The module code checks whether the book has been returned on time and then assigns a new value to <code>Fine</code>

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

[2]

89. M/J 21/P21/Q4/b

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

90. M/J 21/P21/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

91. M/J 21/P22/Q5/b

(b) The following is a pseudocode function.

Line numbers are given for reference only.

```

01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03      DECLARE NextChar : CHAR
04      DECLARE OutString : STRING
05      DECLARE Counter : INTEGER
06
07      OutString ← ""
08
09      FOR Counter ← 1 TO LENGTH(InString)
10          NextChar ← MID(InString, Counter, 1)
11          NextChar ← LCASE(NextChar)
12          IF NOT((NextChar < 'a') OR (NextChar > 'z')) THEN
13              OutString ← OutString & NextChar
14          ENDIF
15      NEXT Counter
16
17      RETURN OutString
18
19  ENDFUNCTION

```

(i) Examine the pseudocode and complete the following table.

Answer

Give a line number containing an example of an initialisation statement.	
Give a line number containing the start of a repeating block of code.	
Give a line number containing a logic operation.	
Give the number of parameters to the function MID().	

[4]

(ii) Write a simplified version of the statement in line 12.

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

92. M/J 21/P22/Q8/b

- 8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as its parameter, representing the number of a word in <code>FNString</code> • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: <code>GetStart(3)</code> applied to "hot and cold" returns 9
<code>GetWord()</code>	• Called with the position of the first character of a word in <code>FNString</code> as its parameter • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"
<code>IgnoreWord()</code>	• Called with a <code>STRING</code> parameter representing a word • Searches for the word in the <code>IgnoreList</code> array • Returns <code>TRUE</code> if the word is found, otherwise returns <code>FALSE</code>
<code>GetInitials()</code>	• Processes the sequence of words in the full name one word at a time • Calls <code>GetStart()</code>, <code>GetWord()</code> and <code>IgnoreWord()</code> to process each word to form the new string • Outputs the new string

- (b) Write pseudocode for the module `GetInitials()`.

93. M/J 21/P23/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

94. M/J 21/P23/Q2/b

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either LoanExtend() or LoanReturn()
LoanExtend()	• Called with parameters LoanID and BookID • Calls CheckReserve() to see whether the book has been reserved for another library user • Returns TRUE if the loan has been extended, otherwise returns FALSE
CheckReserve()	• Called with BookID • Returns TRUE if the book has been reserved, otherwise returns FALSE
LoanReturn()	• Called with parameters LoanID and BookID • Returns a REAL (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
<code>LoanReturn()</code>	Called with parameters <code>LoanID</code> , <code>BookID</code> and <code>Fine</code> The module code checks whether the book has been returned on time and then assigns a new value to <code>Fine</code>

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

[2]

95. M/J 21/P23/Q4/b

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

96. M/J 21/P23/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

Answers Section

1. M/J 26/P21/Q1/a,b

1(a)	<u>Count-control(led) loop // Count-control(led) iteration</u>	1
1(b)	4	1

2. M/J 26/P21/Q5/a,b

5(a)	<i>Example solution:</i> <pre> FUNCTION BulbCost(Quantity : INTEGER) RETURNS REAL DECLARE Cost : REAL DECLARE Twenty : INTEGER DECLARE Other : INTEGER Twenty ← Quantity DIV 20 Other ← Quantity MOD 20 Cost ← (Twenty * 30) + (Other * 1.75) RETURN Cost ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and single parameter and returns REAL and end MP2 Declaration of required local variable(s) MP3 Correct calculation of 'multiples of 20' and 'other' amounts using MOD and DIV // alternative method MP4 Cost is returned	4
5(b)	TotalCost ← BulbCost (NumberPurchased)	1

3. M/J 26/P21/Q6/a,b

6(a)	MP1 LEFT(DataPacket, 3) // MID(DataPacket, 1, 3) MP2 MID(DataPacket, 5, 1)	2
------	--	----------

6(b)	<i>Example solution:</i> <pre> FUNCTION RejectDescription(ThisReject : STRING) RETURNS STRING DECLARE Description : STRING Description ← <u>MID(ThisReject, 7,</u> <i>Part 1</i> <u>LENGTH(ThisReject) - 6)</u> <i>Part 2</i> OR: Description ← RIGHT(ThisReject, LENGTH(ThisReject) - 6) <i>Part 1</i> <i>Part 2</i> RETURN Description ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and end function (and string parameter) MP2 Calculation for description expression (<i>part 1</i>) using MID/RIGHT MP3 Calculation for description expression (<i>part 2</i>) using the LENGTH function MP4 Return of a calculated value and RETURN STRING in the header	4
------	---	---

4. M/J 26/P22/Q1/a(ii,iii)

1(a)(ii)	<pre> IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN </pre> Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
----------	---	---	--

1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for BOOLEAN 2 LetterFlag ← <u>FALSE</u> 1 mark for FALSE 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for LENGTH 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for MID, Index and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for IS_NUM 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for NOT	7	
-----------	---	---	--

5. M/J 26/P22/Q5/c

5(c)(i)	1 mark: The parameters / Num1 and Num2 are passed by value / BYVAL // The parameters / Num1 and Num2 are not passed / should be passed by reference / BYREF 1 mark: (The swap made inside) the procedure does not change the arguments used /parameters (passed) / the values stored in the Numbers elements passed // The procedure does not swap / change the values stored in the array	2	
5(c)(ii)	Solution <pre>PROCEDURE Swap (BYREF Num1, Num2 : INTEGER) DECLARE Temp : INTEGER Temp ← Num1 Num1 ← Num2 Num2 ← Temp ENDPROCEDURE</pre> 2 marks: code as above 1 mark: just the procedure header as above	2	NE for header mark If ByRef is mentioned in the header but syntax is not correct but allow FT for second mark

6. M/J 26/P22/Q6/a

6(a)	Mark Scheme One – loop used to form return string (See MS Two for solutions using LEFT () /MID () to form return string) Example solution <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Space = ' ' DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 FOR Index ← LenOfPlaintext + 1 TO 25 Plaintext ← Plaintext & Space NEXT Index // Alternative way to append spaces using while loop WHILE LENGTH(Plaintext) <> 25 Plaintext ← Plaintext & Space ENDWHILE //Alternative way to append spaces DECLARE SpaceString : STRING SpaceString ← "" FOR Index ← LenOfPlaintext + 1 TO 25 SpaceString ← SpaceString & Space NEXT Index Plaintext ← Plaintext & SpaceString RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values 4 Conditional loop that correctly checks both boundaries of plaintext 5 ...Re-input of plaintext in a loop 6 Loop that iterates for number of spaces to add to end of plaintext 7 ...Spaces appended to plaintext // Spaces appended to an empty string 8 Correctly padded plaintext returned following a reasonable attempt at MP7	8	MP 4 Must be a conditional loop If no loop used then MP3, MP4 and MP5 can not be given
------	---	---	---

6(a)	Mark Scheme Two - No Loop used to form return string – use of LEFT () /MID () Function Example Solution: <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Fill = " " //24 Spaces DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 Plaintext ← LEFT(Plaintext & Fill, 25) RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values Conditional loop that correctly checks for length of plaintext 4 ...Re-input of plaintext in a loop 5 Use of string of at least 24 spaces Correct use of LEFT () / MID () function and & to append padding characters 6 Correctly padded plaintext returned following a reasonable attempt at MP 6 and MP 7		Do not accept use of language specific string manipulation which is not the compatible with the pseudocode used for the syllabus
------	---	--	---

7. M/J 26/P23/Q1/a,b

1(a)	<u>Post-condition(al) loop // post-condition(al) iteration</u>	1	
1(b)	5	1	

8. M/J 26/P23/Q5/a

5(a)	<pre> FUNCTION InterviewSummary(ThisApplicant : STRING) RETURNS STRING DECLARE Summary : STRING Summary ← MID(ThisApplicant, 8, <u>MP2</u> LENGTH(ThisApplicant) - 7) <u>MP3</u> Or: Summary ← RIGHT(ThisApplicant, LENGTH(ThisApplicant) - 7) <u>MP2</u> <u>MP3</u> RETURN Summary ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function identifier and function ending and parameter MP2 Calculation for summary expression (part 1) – using MID or RIGHT MP3 Calculation for summary expression (part 2) – using LENGTH MP4 Return of the calculated STRING value (earlier declaration optional) and RETURNS STRING in the header –	4	Do not accept a final missing close bracket (for the MID or RIGHT function).
------	--	----------	---

9. O/N 25/P21/Q1/a

1(a)		4
	Expression	
	MyString ← 'X' & MyString	
	MyChar ← MID ("ABCD", 1 / 2 / 3 / 4 , 1)	
	MyString ← NUM_TO_STR (DAY / MONTH / YEAR / DAYINDEX (MyDOB))	
	MyInt ← INT (LENGTH (MyString) / 2)	
One mark per row		

10. O/N 25/P21/Q5/b

5(b)(i)	The loop continues after the (first instance) of the search value / string / matching element has been found Comparisons/tests continue to be made even if the search value / string / matching element has been found	1
---------	--	----------

5(b)(ii)	One mark per point Construct: A conditional loop Justification: The loop / search can be terminated as soon as the (first occurrence of) SearchString / required value / required string / matching element is found	2
----------	---	----------

11. O/N 25/P21/Q6/b

6(b)	Example solution – conversion of Number to a string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR (Number) FOR Index ← 1 TO LENGTH (NumString) Total ← Total + STR_TO_NUM (MID (NumString, Index, 1)) NEXT Index CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit // NumString ← NumString & NUM_TO_STR (CheckDigit) RETURN Number // STR_TO_NUM (NumString) ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Convert Number to a string and loop for length of converted Number 4 attempt to form sum of digits in a loop 5 completely correct sum of digits in a loop 6 Calculate CheckDigit 7 Add CheckDigit to original number multiplied by 10 and return number // Convert CheckDigit to a character and appended to NumString and then convert to a number and return	7
------	---	----------

6(b)	Example Solution – no conversion to string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, CheckDigit, Remainder, Num: INTEGER Num ← Number Total ← 0 WHILE Num > 0 Remainder ← Num MOD 10 Num ← Num DIV 10 Total ← Total + Remainder ENDWHILE CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit RETURN Number ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Loop while Num > 0 4 attempt to use MOD and DIV to sum successive digits in Number 5 completely correct sum of digits 6 Calculate CheckDigit 7 Add CheckDigit to original number and return number	
------	--	--

12. O/N 25/P22/Q1/c

1(c)	Expression		Evaluates to	4
	INT(Result) + 1 > 6		FALSE	
	NUM_TO_STR(LENGTH(MonthLetter))		"12"	
	NUM_TO_STR(Result + "3.2")		ERROR	
	MID(MonthLetter, MONTH(Birthday) - 2, 1)		"S"	
	One mark per row			

1(b)	Variable	Example data value	Data type	3
	Result	5.42	REAL	
	MonthLetter	"JFMAMJJASOND"	STRING	
	Birthday	15/11/2009	DATE	
	One mark per row			

13. O/N 25/P22/Q6

6	Example solution: <pre> FUNCTION Compare(String1, String2 : STRING, __ CaseMatters : BOOLEAN, Position : CHAR) RETURNS BOOLEAN DECLARE S1Length : INTEGER S1Length ← LENGTH(String1) IF LENGTH(String2) < S1Length THEN RETURN FALSE // String2 is shorter than String1 ENDIF IF CaseMatters = FALSE THEN String1 ← TO_UPPER(String1) String2 ← TO_UPPER(String2) ENDIF CASE Position 'e' : String2 ← RIGHT(String2, S1Length) 's' : String2 ← LEFT(String2, S1Length) ENDCASE IF String1 = String2 THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameters and ending and return type MP2 Calculate length of String1 MP3 if String2 is shorter than String1, return FALSE MP4 Compare (modified) strings MP5 Test CaseMatters - if FALSE and convert two string(s) to same case MP6 Test for the value of 's'/'e' and form modified string(s) in one or both cases MP7 Return appropriate BOOLEAN value in all cases	7
---	---	---

14. O/N 25/P23/Q1/a,b,c

1(a)	Answers include: 1 They are tried and tested // Free from errors 2 They are readily available // Speed up development time 3 They will be updated automatically when improvements are made // No maintenance of routine is needed (as automatically updated) 4 The algorithm will be compliant with the international standard One mark per point Max 3	3								
1(b)(i)	1 When a part of the algorithm performs a specific task 2 Part is repeated // performed in several places 3 Testing / debugging / maintenance is easier One mark for each point Max 2	2								
1(b)(ii)	<table><thead><tr><th>Term</th><th>Description</th></tr></thead><tbody><tr><td>Pass2</td><td>the name of the function</td></tr><tr><td>Count</td><td>The parameter / value passed to the function</td></tr><tr><td>BOOLEAN</td><td>The (data) type <u>returned</u> (by the function)</td></tr></tbody></table> One mark per description (excluding first row)	Term	Description	Pass2	the name of the function	Count	The parameter / value passed to the function	BOOLEAN	The (data) type <u>returned</u> (by the function)	2
Term	Description									
Pass2	the name of the function									
Count	The parameter / value passed to the function									
BOOLEAN	The (data) type <u>returned</u> (by the function)									
1(c)	<table><thead><tr><th>Expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>LENGTH(NUM_TO_STR(Multiplier))</td><td>3</td></tr><tr><td>MONTH (DoB) > 4</td><td>TRUE</td></tr><tr><td>15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))</td><td>20</td></tr></tbody></table> One mark per row	Expression	Evaluates to	LENGTH(NUM_TO_STR(Multiplier))	3	MONTH (DoB) > 4	TRUE	15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20	3
Expression	Evaluates to									
LENGTH(NUM_TO_STR(Multiplier))	3									
MONTH (DoB) > 4	TRUE									
15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20									

15. O/N 25/P23/Q5/b

5(b)(i)	1 and 2	1
---------	---------	----------

5(b)(ii)	<pre> 1 TO 2 : Num[Index] ← Num[Index] + Index Index ← Index + CaseVar Count ← Count + CaseVar </pre> One mark per point: • use of variable <code>CaseVar</code> rather than literal values • completely correct clause	2
----------	---	----------

16. O/N 25/P23/Q6/b

6(b)	Note 2 different solutions with different mark scheme Example solution – conversion to string and using a loop <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR(Number) CheckDigit ← STR_TO_NUM(RIGHT(NumString, 1)) // CheckDigit ← Number MOD 10 FOR Index ← 1 TO LENGTH(NumString) - 1 // 1 TO LENGTH(NUM_TO_STR(Number DIV 10)) Total ← Total + STR_TO_NUM(MID(NumString, Index, 1)) ENDFOR IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Function Header including Identifier, parameters and return type 2 Use of <code>Num_TO_STR</code> to convert parameter // Use of <code>Num_TO_STR</code> to convert parameter excluding least significant digit to a string 3 Extract <code>CheckDigit</code> from <code>NumString</code> // Extract least significant digit from parameter 4 Loop for length of <code>NumString</code> - 1 following correct conversion to string 5 attempt to form sum of digits in a loop 6 completely correct sum of digits in a loop and of sum initialised before loop 7 Calculate check digit from sum 8 Compare sum with final digit of parameter and return appropriate Boolean value Max 7	7
------	---	----------

6(b)	Example solution – numeric version <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Sum, CheckDigit, Remainder: INTEGER Remainder ← Number MOD 10 Number ← Number DIV 10 CheckDigit ← Remainder Sum ← 0 WHILE Number > 0 Remainder ← Number MOD 10 Number ← Number DIV 10 Total ← Total + Remainder ENDWHILE IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> 1 Function Header including Identifier, parameters and return type 2 Initialise Sum 3 Set CheckDigit to least significant digit in Number 4 Loop while Number > 0 5 attempt to use MOD and DIV to sum successive digits in Number in loop 6 completely correct sum of all digits in Number apart from least significant digit in loop 7 Calculate check digit from sum 8 Compare with CheckDigit and return appropriate Boolean value Max 7	
------	--	--

17. M/J 25/P21/Q4/a

4(a)	MP1 Count-controlled MP2 The number of iterations required is known	2
------	---	----------

18. M/J 25/P21/Q7/a

7(a)	Example solution: <pre> FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER DECLARE NewPoints, NumberFree: INTEGER NewPoints ← Points + Number NumberFree ← 0 WHILE NewPoints >= 11 AND Number > 0 NumberFree ← NumberFree + 1 NewPoints ← NewPoints -11 Number ← Number - 1 END WHILE OUTPUT "Number of free coffees is: ", NumberFree RETURN NewPoints ENDFUNCTION </pre> MP1 Function header and ending and parameters and return type MP2 Number of coffees ordered added to points MP3 Correct calculation of one free coffee MP4 Attempted calculation of multiple free coffees and reduced <code>NewPoints</code> MP5 Output number of free drinks with suitable message MP6 Return of <code>NewPoints</code>	6
------	---	----------

19. M/J 25/P23/Q1/a,b

1(a)(i)	MP1 When a task/module is repeated / reused / called // performed in several places MP2 A specific task can be identified /coded as a module / subroutine / procedure / function. MP3 Reduces complexity of <u>program / code</u> // Program / code is simplified MP4 Module / subroutine / procedure / function already available. MP5 <u>Testing</u> / <u>debugging</u> / <u>maintenance</u> is easier Max 2	2
1(a)(ii)	MP1 Local variables are used within a module // are accessible only within the module (in which they are declared) MP2 Global variables are accessible to all parts of the program / MP3 Local variables use memory only while the module is executing // Global variables use memory for the whole time the program is running Max 1	1

1(a)(iii)	MP1 The same variable name can be reused in other parts of the program MP2 Using local variables makes modules self-contained // cannot accidentally change the same identifier outside of the module MP3 Using local variables aids modularisation. MP4 Memory allocated to local variables can be reused when module not in use Max 2	2																		
1(b)	Mark as follows: <table border="1"> <thead> <tr> <th>Expression</th><th>Evaluates to</th><th>Mark</th></tr> </thead> <tbody> <tr> <td>CHR(68)</td><td>'D'</td><td>1</td></tr> <tr> <td>MONTH(04/02/2025)</td><td>2</td><td>1</td></tr> <tr> <td>// -2 + DAY(04/02/2025) // -2023 + YEAR(04/02/2025)</td><td></td><td></td></tr> <tr> <td>NOT TRUE // FALSE = TRUE</td><td>FALSE</td><td>1</td></tr> <tr> <td>STR_TO_NUM(MID("Court1.4Upper", 6, 3))</td><td>1.4</td><td>2</td></tr> </tbody> </table> Mark Row 4 as follows: STR_TO_NUM 1 mark MID("Court1.4Upper", 6, 3) 1 mark	Expression	Evaluates to	Mark	CHR (68)	'D'	1	MONTH (04/02/2025)	2	1	// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)			NOT TRUE // FALSE = TRUE	FALSE	1	STR_TO_NUM (MID ("Court1.4Upper", 6, 3))	1.4	2	5
Expression	Evaluates to	Mark																		
CHR (68)	'D'	1																		
MONTH (04/02/2025)	2	1																		
// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)																				
NOT TRUE // FALSE = TRUE	FALSE	1																		
STR_TO_NUM (MID ("Court1.4Upper", 6, 3))	1.4	2																		

5	Example solution: <pre> FUNCTION Parity(BitString : STRING) RETURNS STRING DECLARE Index, Count: INTEGER Count ← 0 FOR Index ← 1 TO LENGTH(BitString) IF MID(BitString, Index, 1) = '1' THEN Count ← Count + 1 ENDIF NEXT Index IF Count MOD 2 = 1 THEN BitString ← BitString & '1' ELSE BitString ← BitString & '0' ENDIF RETURN BitString ENDFUNCTION </pre> MP1 Function heading and ending and parameter (STRING) and return type STRING MP2 Loop for length of BitString MP3 Extract current character in a loop MP4 Compare current character to '1' or '0' in a loop MP5 Increment count and was Initialised earlier in a loop MP6 Check if even/odd number of 1s MP7 Append '1' or '0' as appropriate MP8 Return amended BitString	8
---	--	---

21. M/J 25/P23/Q6/a

6(a)	MP1 A function returns a (single) value and a procedure does not MP2 A procedure can pass parameters by ByRef Max 1	1
------	---	---

22. M/J 25/P23/Q7/a,b(ii)

7(a)(i)	Example solution: <pre> PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE, __ BYREF LoyaltyPoints : INTEGER) IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN LoyaltyPoints ← LoyaltyPoints + 4 ELSE LoyaltyPoints ← LoyaltyPoints + 1 ENDIF LastVisitDate ← TODAY() ENDPROCEDURE </pre> MP1 Procedure heading and ending and two parameters of correct type MP2 ... both passed BYREF MP3 Use of TODAY () function to obtain current date MP4 Use MONTH function (×2) and one parameter is the header parameter and 2nd parameter is TODAY () / assigned variable MP5 If visit is same month, increase LoyaltyPoints by 4 otherwise increase by 1 MP6 LastVisitDate set to TODAY () / 'current date' variable (assigned or unassigned) Max 5	5
7(a)(ii)	DAYINDEX (LastVisitDate) = DAYINDEX (TODAY ())	1
7(b)(ii)	Example solution: <pre> DayInt ← STR_TO_NUM(LEFT(LastVisitDateString, 2)) MonthInt ← STR_TO_NUM(MID(LastVisitDateString, 3, 2)) YearInt ← STR_TO_NUM(RIGHT(LastVisitDateString, 4)) LastVisitDate ← SETDATE(DayInt, MonthInt, YearInt) </pre> Mark as follows: MP1 Use of one substring function MP2 DayInt, MonthInt and YearInt all correctly extracted MP3 Use of STR_TO_NUM x 3 MP4 SETDATE () function used to convert to a date type and assigned to LastVisitDate	4

23. O/N 24/P21/Q1/b,d

1(b)(i)	Use constants (to represent the tax rate values)	1
1(b)(ii)	One mark per bullet point (or equivalent to max 3): Tax rates are entered once only - Avoids / Minimise (input) error(s) / changing the Tax rates accidentally // avoids different values for tax rates at different points in the program - When required, the constant value (representing a tax rate) is changed (once) // Easier to maintain / update the program (when the tax rates change) - Makes the program / code easier to understand	3
1(d)	OTHERWISE	1

24. O/N 24/P21/Q2

2	Example solution: <pre> PROCEDURE Tick() SS ← SS + 1 IF SS = 60 THEN SS ← 0 MM ← MM + 1 IF MM = 60 THEN MM ← 0 HH ← HH + 1 IF HH = 24 THEN HH ← 0 CALL NewDay() ENDIF ENDIF CALL CheckAlarm() ENDIF ENDPROCEDURE </pre> Mark as follows: - Increment SS anywhere in the code - Call <code>CheckAlarm()</code> when <u>SS is set to zero</u> - Test if <code>SS = 60</code> anywhere in the code ... if so: - set SS to 0 - increment MM - When MM is set to 60 set MM to 0 and increment HH - When HH is set to 24 set HH to 0 and call <code>NewDay()</code> Max 6 marks	6
---	---	----------

25. O/N 24/P21/Q4/b,c

4(b)	Answers include: Reason: • This part of the algorithm performs a specific task // if the check or calculation is changed it is changed only once • A (similar) subroutine is already available // Library routine is already available • The program is simplified / easier to understand / easier to design / code / test / debug / maintain Max 1 marks for 'Reason' Avoided: • Unnecessary code duplication • Errors caused by differences where several copies of the check and calculation exist Max 1 marks for 'Avoided'	2
------	--	----------

4(c)	Example solution: <pre> FUNCTION Evaluate (NumStr1, NumStr2 : STRING) RETURNS Result DECLARE RetVal : Result DECLARE Num1, Num2 : REAL RetVal.Done ← FALSE IF IS_NUM(NumStr1) = TRUE AND IS_NUM(NumStr2) = TRUE THEN Num1 ← STR_TO_NUM(NumStr1) Num2 ← STR_TO_NUM(NumStr2) IF Num2 <> 0 THEN RetVal.Value ← (Num1 / Num2) RetVal.Done ← TRUE ENDIF ENDIF RETURN RetVal ENDFUNCTION </pre> Note that order of parameters is not specified so divisor could be either parameter. Mark as follows: 1 Function heading, parameters, ending 2 ... Correct return type of <code>Result</code> 3 Local declaration of type <code>Result</code> 4 Attempt at using <code>IS_NUM()</code> to check both parameters/strings 5 Attempt at using of <code>STR_TO_NUM()</code> to convert both parameters/strings 6 Check if the divisor, is non-zero ... 7 ... and if so correct calculation and assignment to <code>Value</code> field 8 Return variable of type <code>Result</code> having assigned <u>both</u> fields Max 6 marks	6
------	---	----------

26. O/N 24/P21/Q6/b

6(b)	Example Solution <pre> FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE DECLARE NewDate : DATE DECLARE DD, MM, YY, LastDay : INTEGER DD ← DAY(ProductionDate) MM ← MONTH(ProductionDate) YY ← YEAR(ProductionDate) DD ← DD + ShelfLife LastDay ← DaysInMonth[MM] IF DD > LastDay THEN MM ← MM + 1 DD ← DD - LastDay IF MM = 13 THEN MM ← 1 YY ← YY + 1 ENDIF ENDIF NewDate ← SETDATE(DD, MM, YY) RETURN NewDate ENDFUNCTION </pre> Mark as follows: 1 Declare three variables of TYPE INTEGER to store DD, MM and YY 2 Correct use of date functions from Insert to extract three integer values representing DD, MM and YY and use/assign to a variable 3 Add parameter ShelfLife to DD and use / assign result to a variable 4 Extract LastDay from DaysInMonth using MM as Index 5 Test for new DD after end of month / DaysInMonth[MM] ... and if DD > DaysInMonth[MM]: 6 MM is incremented by 1 and DD is set to DD - DaysInMonth[MM] 7 If MM = 13 / MM > 12 then increment YY and set MM to 1 8 Use of SETDATE to assign value to NewDate, NewDate must have been declared as a type DATE 9 Return date Max 7 marks	7
------	--	----------

27. O/N 24/P21/Q8/a

8(a)	Loop example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN DECLARE Index, Lower, Upper : INTEGER FOR Index ← 1 TO 5 Lower ← GradeBoundary[Index] - 2 Upper ← GradeBoundary[Index] + 2 IF Mark >= Lower AND Mark <= Upper THEN RETURN TRUE ENDIF NEXT Index RETURN FALSE ENDFUNCTION </pre> Mark as follows: 1 Loop through all elements in <i>GradeBoundary</i> array 2 Attempt to calculate range, both <i>Lower</i> and <i>Upper</i>, in a loop 3 Completely correct range calculation in a loop 4 Test if given mark / parameter, is within range in a loop 5 Set variable / immediate RETURN if recheck required in a loop 6 Return Boolean in both cases following a reasonable attempt Selection example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN CASE OF Mark GradeBoundary[1] - 2 TO GradeBoundary[1] + 2 : RETURN TRUE GradeBoundary[2] - 2 TO GradeBoundary[2] + 2 : RETURN TRUE GradeBoundary[3] - 2 TO GradeBoundary[3] + 2 : RETURN TRUE GradeBoundary[4] - 2 TO GradeBoundary[4] + 2 : RETURN TRUE GradeBoundary[5] - 2 TO GradeBoundary[5] + 2 : RETURN TRUE OTHERWISE : RETURN FALSE ENDCASE ENDFUNCTION </pre> Mark as follows: 1 Correct syntax used for selection structure(s) 2 Correct checks for two ranges 3 Correct checks for three ranges 4 Correct checks for four ranges 5 Correct checks for all five ranges 6 Return Boolean in both cases following a reasonable attempt	6
------	--	----------

28. O/N 24/P22/Q2/a,b(ii)

2(a)	Example 'loop solution': <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisk = '*' MaskedString ← RIGHT(CardNumber, 4) FOR Count ← 1 TO LENGTH(CardNumber) - 4 MaskedString ← Asterisk & MaskedString NEXT Count RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used - including the loop counter MP3 Calculate number of digits to mask / number of asterisks required MP4 use of a 'relevant' Loop MP5 Correct number of iterations MP6 Concatenate asterisk to start/end of MaskedString in a loop MP7 Assign last four characters to MaskedString // Concatenate the retained original last four digits MP8 Return masked string Max 6 marks ALTERNATIVE 'non loop' solution: <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisks = "*****" //20 asterisks Count ← LENGTH(CardNumber) - 4 MaskedString ← LEFT(Asterisks, Count) & RIGHT(CardNumber, 4) RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used MP3 Calculate number of digits to mask / number of asterisks required	6
------	--	----------

2(a)	MP4 Trim asterisk string to number calculated in MP3 MP5 Extract the last four characters MP6 Concatenate trimmed asterisk string with last four characters of CardNumber MP7 Return masked string Max 6 marks	
2(b)(ii)	Any reference to BYREF // 'by reference'	1

29. O/N 24/P22/Q4

4	Example solution: <pre> PROCEDURE Timer(Mins, Secs : INTEGER) DECLARE WarningTick, EndTick : INTEGER EndTick ← Tick + 1000 * ((Mins * 60) + Secs) WarningTick ← EndTick - (30 * 1000) REPEAT //do nothing UNTIL Tick = WarningTick OUTPUT "30 seconds to go" REPEAT //do nothing UNTIL Tick = EndTick OUTPUT "The time is up!" ENDPROCEDURE </pre> Mark as follows: MP1 Procedure heading and parameters and ending MP2 'Attempt' to calculate 'total time'/EndTick // 'elapsed time' // WarningTick MP3 Correct calculation of EndTick and WarningTick MP4 (Design mark) • Two separate loops – checking warning time then the final time, OR ... • Single loop checking the final time with an IF statement to check for warning time, OR ... • Single loop with two IF statements checking the warning time and final time MP5 Completely correct MP4 MP6 Output both messages (must be meaningful and follow successful MP4)	6
---	--	----------

30. O/N 24/P22/Q5/c

5(c)	Example solution: <pre>IF Index Mod 2 = 0 THEN ReturnValue ← TO_UPPER(RIGHT(Label, Count)) ELSE ReturnValue ← "*****" ENDIF</pre> Mark as follows: MP1 IF...THEN...ELSE...ENDIF MP2 Both correct assignments and the correct test/logic	2
------	--	----------

31. O/N 24/P23/Q1/c

1(c)	Expression		Evaluates to	3
	Available AND NOT(Index > 100)		TRUE	
	Index MOD 30		10	
	NUM_TO_STR(Index + "33")		ERROR	
	One mark per row			

1(b)	Variable	Example data value	Data type	3
	Available	TRUE	BOOLEAN	
	Received	"18/04/2021"	STRING	
	Index	100	INTEGER	
	One mark per row			

32. O/N 24/P23/Q6

6	Loop Solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE ThisDayNumber, SundayCount : INTEGER DECLARE ThisDate : DATE ThisDayNumber ← 0 SundayCount ← 0 REPEAT ThisDayNumber ← ThisDayNumber + 1 ThisDate ← SETDATE(ThisDayNumber, 3, ThisYear) IF DAYINDEX(ThisDate) = 1 THEN SundayCount ← SundayCount + 1 ENDIF UNTIL SundayCount = 3 RETURN ThisDayNumber ENDFUNCTION </pre> Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to create a date MP3 Loop until 3rd Sunday found MP4 Attempt to use both SETDATE() and DAYINDEX() in a loop MP5 Correctly generate value of type DATE using SETDATE() in a loop MP6 Test if value represents a Sunday using DAYINDEX() in a loop MP7 Increment Sunday count in a loop and initialised correctly before loop MP8 Return day number of third Sunday Max 7 marks Non-loop solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE FirstDayIndex: INTEGER DECLARE ThisDate : DATE ThisDate ← SETDATE(1, 3, ThisYear) FirstDayIndex ← DAYINDEX(ThisDate) IF FirstDayIndex = 1 THEN ThirdSunday ← FirstDayIndex + 14 //First day is Sunday ELSE ThirdSunday ← 23 - FirstDayIndex // Other days ENDIF RETURN ThirdSunday ENDFUNCTION </pre>	7
---	---	---

6	Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to hold a day number MP4 Attempt to use both <code>SETDATE()</code> and <code>DAYINDEX()</code> MP5 Correctly generate value of type <code>DATE</code> using <code>SETDATE()</code> for first of March / specific day in March MP6 Test if date represents a Sunday / specific day using <code>DAYINDEX()</code> and calculate third Sunday // Correctly calculate third Sunday for a value returned by <code>DAYINDEX()</code> for one day MP7 Calculate third Sunday for other six days MP8 Return day number of third Sunday Max 7 marks	
---	---	--

33. M/J 24/P21/Q1/b

1(b)			4

34. M/J 24/P21/Q2/b

2(b)	One mark per point: 1 A (variable of type) string will be input // by example e.g. "67,72" 2 A special / identified character would need to be used to separate each numeric value // all numbers are fixed length	2
------	---	---

35. M/J 24/P21/Q6/a

6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR___ (Len2 = Len1 + Len3) OR___ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 One correct comparison of square of three lengths 4 All three comparisons... 5 combined using logical operators / nested IF // completely correct selection 6 Return result correctly in both cases	6
------	--	---

8(a)	<pre> FUNCTION DeleteComment(Line : STRING) RETURNS STRING DECLARE NewLine, TwoChars : STRING DECLARE Count, TrimTo : INTEGER CONSTANT Comment = "/" NewLine ← Line TrimTo ← 0 Count ← 1 WHILE Count < LENGTH(Line) AND TrimTo = 0 TwoChars ← MID(Line, Count, 2) // extract 2 chars IF TwoChars = Comment THEN TrimTo ← Count ENDIF Count ← Count + 1 ENDWHILE IF TrimTo <> 0 THEN NewLine ← LEFT(Line, TrimTo - 1) ENDIF RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of <code>Line</code> (parameter) // length of <code>Line</code> -1 2 Terminate loop on first double slash 3 Attempt to extract one/two characters in a loop 4 Check for <code>"/"</code> after attempt at extraction in a loop 5 Record start position of comment // Calculate amount of trim required 6 Attempt to trim <code>Line</code> from start of comment 7 Completely correct trimmed <code>Line</code> from start of comment 8 Return <code>Line</code> after a reasonable overall attempt	8
------	---	---

1(b)	Expression	4
	MyInt ← INT (3.1415926)	
	MyChar ← MID ("Elwood", 3, 1)	
	Any of: • MyString ← NUM_TO_STR (INT (27.509)) • MyString ← CHR (INT (27.509)) • MyString ← TO_UPPER(NUM_TO_STR(27.509)) • MyString ← TO_LOWER(NUM_TO_STR(27.509))	
	Any of: • MyInt ← STR_TO_NUM (RIGHT ("ABC123", 3)) • MyInt ← LENGTH (RIGHT ("ABC123", 3)) • MyInt ← LENGTH (LEFT ("ABC123", 3))	

One mark per row

38. M/J 24/P22/Q4

4	Example solution: <pre> PROCEDURE IsRA() DECLARE a, b, c : INTEGER OUTPUT "Input length of the first side" INPUT a OUTPUT "Input length of the second side" INPUT b OUTPUT "Input length of the third side" INPUT c IF (a * a = (b * b) + (c * c)) OR__ (b * b = (a * a) + (c * c)) OR__ (c * c = (a * a) + (b * b)) THEN OUTPUT "It is right-angled" ELSE OUTPUT "Not right-angled" ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending and declaration of all variables used 2. Appropriate prompt and input for each length 3. One correct length test 4. All three length tests // selection of which test is required 5. Output one of two messages following a reasonable attempt at MP3	5
---	---	---

39. M/J 24/P22/Q5/a(ii)

5(a)(ii)	One mark per point: Statement: • The OTHERWISE statement Explanation: • The result of MOD 2 can only be 0 or 1	2
----------	---	---

40. M/J 24/P22/Q6/a

6(a)	Example solution: <pre> Function Trim(Name : STRING) RETURNS STRING CONSTANT Dots = "... " CONSTANT Space = " " IF LENGTH(Name) <= 16 THEN RETURN Name ENDIF // Otherwise it has to be trimmed WHILE LENGTH(Name) > 13 REPEAT Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char UNTIL RIGHT(Name, 1) = Space // back to SPACE ENDWHILE Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space Name ← Name & Dots RETURN Name ENDFUNCTION </pre> Mark as follows: 1. Function heading, ending, parameter and return type 2. If length of original string <= 16 then return original string 3. Any Conditional loop... 4. until string is short enough 5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string 6. Attempt to strip characters back to space 7. Correct removal of word/words from end of string and remove final space 8. Concatenate <code>Dots</code> and return result Max 7 marks	7
------	---	----------

41. M/J 24/P22/Q8/b

8(b)	Example algorithm based on finding position of first non-space character and then using substring function: <pre> FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING DECLARE NewLine : STRING DECLARE EndOfLeading : BOOLEAN DECLARE Count, NumSpaces : INTEGER DECLARE NextChar : CHAR CONSTANT Space = " " NumSpaces ← 0 EndOfLeading ← FALSE FOR Count ← 1 TO LENGTH(Line) NextChar ← MID(Line, Count, 1) IF NextChar <> Space AND EndOfLeading = FALSE THEN NumSpaces ← Count - 1 // the number to trim EndOfLeading = TRUE ENDIF NEXT Count NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces) RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of parameter // Loop until first non-space character in Line 2 Extract a character in a loop 3 Identify <u>first</u> non-space character in a loop 4 Attempt at removing leading spaces in Line 5 Leading spaces removed from Line // Create new string without leading space 6 Return a string following a reasonable attempt at removing leading spaces in Line	6
------	--	----------

42. M/J 24/P23/Q1/a,b

1(a)(i)	Two marks for the benefits: 1. The code can be called when needed 2. Any subsequent change to the algorithm / calculation needs to be made once only // Easier to manage / maintain (the program) 3. Less code / no code duplication 4. The algorithm / code / calculation can be designed / coded / tested once Note: Max 2 marks	2
---------	---	----------

1(a)(ii)	One mark per point: 1. Create a module / function / procedure / subroutine using the (old) code / algorithm 2. Replace the old code / algorithm wherever it appears with a call to the module / function / procedure / subroutine 3. Pass the data as parameter(s) // Use of <u>global variable(s)</u> 4. <u>Return</u> the result (of the calculation) // Assign result to <u>global variable(s)</u> / <u>BYREF</u> parameter(s) 5. Create <u>local</u> variables as needed (to perform the calculation) Note: Max 3 marks	3										
1(b)	<table><thead><tr><th>Pseudocode expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>MID ("Random", 2, 3)</td><td>"and"</td></tr><tr><td>5 + DAY (10/11/2023)</td><td>15</td></tr><tr><td>IS_NUM ("45000")</td><td>TRUE</td></tr><tr><td>(20 MOD 3) + 1</td><td>3</td></tr></tbody></table> One mark per row	Pseudocode expression	Evaluates to	MID ("Random", 2, 3)	"and"	5 + DAY (10/11/2023)	15	IS_NUM ("45000")	TRUE	(20 MOD 3) + 1	3	4
Pseudocode expression	Evaluates to											
MID ("Random", 2, 3)	"and"											
5 + DAY (10/11/2023)	15											
IS_NUM ("45000")	TRUE											
(20 MOD 3) + 1	3											

43. M/J 24/P23/Q4/a

4(a)	Example solution: <pre> PROCEDURE TwoParts() DECLARE NextNum, Average : REAL TotalA ← 0.0 // 0 TotalB ← 0.0 // 0 REPEAT INPUT NextNum TotalA ← TotalA + NextNum UNTIL NextNum = 0 REPEAT INPUT NextNum TotalB ← TotalB + NextNum UNTIL NextNum = 0 Average ← (TotalA + TotalB) / 2 OUTPUT "The average is ", Average ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending 2. Declare all local variables 3. Initialise <code>TotalA</code> and <code>TotalB</code> 4. First conditional loop until zero entered, summing <code>TotalA</code> // Loop until both parts (sequences) have been entered 5. Second conditional loop until zero entered, summing <code>TotalB</code> // Loop summing appropriate Totals 6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message	6
------	--	----------

44. M/J 24/P23/Q5/a,b

5(a)	One mark per point • Count-controlled loop • the number of iterations is known	2
5(b)	Two mark for Statement of Problem: 1 The functions will return the same value every time they are called 2 ... because <code>Label</code> / the parameter value does not change within the loop Two marks for Solution: 3 Assign <code>FormatA(Label)</code> and <code>FormatB(Label)</code> to two (local) variables <u>before</u> the loop 4 Use the (new) variables in place of the function calls / in the loop // Replace the references to <code>FormatA()</code> and <code>FormatB()</code> in the CASE clauses with the new variable names	4

45. M/J 24/P23/Q6/a,b(i)

6(a)	Example Solutions: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING StepValue ← (Percent DIV 10) + 1 // INT(Percent / 10) + 1 FileName ← Root & "-" & NUM_TO_STR(StepValue) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Alternative: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING DECLARE Found : BOOLEAN StepValue ← 1 Found ← FALSE REPEAT IF Percent < StepValue * 10 THEN Found ← TRUE ENDIF StepValue ← StepValue + 1 UNTIL Found Filename ← Root & "-" & NUM_TO_STR(StepValue - 1) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for use of DIV/INT or loop solution: 1 Procedure heading, parameters and ending 2 Calculate StepValue as integer value // Attempt at calculating file number 3 Add 1 to obtain file number // Completely correct file number calculation 4 Use of NUM_TO_STR() to convert file number to string and use 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() with filename as parameter following a reasonable attempt	6
------	--	----------

6(a)	Example Selection Solution: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE Filename, StepValue : STRING CASE OF Percent < 10 : StepValue ← "1" < 20 : StepValue ← "2" < 30 : StepValue ← "3" < 40 : StepValue ← "4" < 50 : StepValue ← "5" < 60 : StepValue ← "6" < 70 : StepValue ← "7" < 80 : StepValue ← "8" < 90 : StepValue ← "9" < 100 : StepValue ← "10" OTHERWISE : StepValue ← "11" ENDCASE Filename ← Root & "-" & StepValue & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for loop solution: 1 Procedure heading, parameters and ending 2 Correct selection construct(s) structure 3 Use of selection statement to obtain two file numbers 4 Use of selection statement to obtain all the file numbers 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() once with filename as parameter following a reasonable attempt	
6(b)(i)	Example answer: <pre> FUNCTION Progress2(Percent, Steps : INTEGER, Root : STRING) RETURNS STRING </pre> One mark for each: • Keyword <code>FUNCTION Progress2</code> and three parameters of correct type • Keyword <code>RETURNS</code> and type string	2

46. M/J 24/P23/Q7/a(ii)

7(a)(ii)	Points required: 1 any change made to the parameter value / Par3 within Mod-G() is reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-G() terminates) 2 any change made to the parameter value / Par3 within Mod-F() is NOT reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-F() terminates) Mark as follows: 1 mark for a reasonable attempt to explain 2 marks for full explanation including context	2
----------	--	----------

47. M/J 24/P23/Q8/a

8(a)	Example: <pre> FUNCTION Header(Line : STRING) RETURNS STRING IF LENGTH(Line) >= 13 THEN IF TO_UPPER(LEFT(Line, 9)) = "FUNCTION " THEN RETURN "F" & RIGHT(Line, LENGTH(Line) - 9) ENDIF IF TO_UPPER(LEFT(Line, 10)) = "PROCEDURE " THEN RETURN "P" & RIGHT(Line, LENGTH(Line) - 10) ENDIF ENDIF RETURN "" ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameter, return type and ending 2 Check that the line is at least 13 characters long before attempting to extract and return empty string 3 Attempt at: Extract characters, 9 or 10 characters, corresponding to keyword plus space and compare with appropriate keyword plus space 4 Completely correct MP3 5 Use of type case conversion to allow for 'any case' 6 Calculation of 'rest of line' and concatenation with 'P' or 'F' 7 Return string	7
------	--	----------

48. O/N 23/P21/Q1/a,b,c

1(a)	One mark per row:		4
		Answer	
	The value assigned to Level when ThisValue is 40	"Medium"	
	The value assigned to Check when ThisValue is 36	12	
	The value assigned to Level when ThisValue is 18	"Low"	
	The number of elements in array Data that may be incremented	11	

1(b)	One mark for identifying assignment: MP1 <code>Level ← "Very Low"</code> // the level is assigned value "very low" Explanation points: MP2 because CASE clauses are checked in sequence // because of the order of the clauses MP3 a value < 30 satisfies the first clause // Clause '<u>< 20</u>' will never be tested	3
1(c)	MP1 all of the possible values are addressed via all / four / three / the other clauses // there are no other possible values to map to OTHERWISE	1

49. O/N 23/P21/Q6/a,b

6(a)	<pre> PROCEDURE MyOutput(NewString : STRING, EOL : BOOLEAN) IF LENGTH(MyString) + LENGTH(NewString) > 255 THEN OUTPUT MyString // Resulting string would be too long MyString ← NewString ELSE MyString ← MyString & NewString // Concat with MyString IF EOL = TRUE THEN OUTPUT MyString MyString ← "" ENDIF ENDIF ENDPROCEDURE </pre> MP1 Procedure heading, including parameters, and ending MP2 Produce concatenated string MP3 ... Check whether resulting string would be too long MP4 If so, then output old MyString MP5 ... and assign NewString to MyString MP6 Else concatenate NewString to MyString MP7 (test for length < 255) Test EOL – If TRUE then Output MP8 ... and reset MyString to empty string	7
6(b)	MP1 A new (instance of) variable MyString is created each time the procedure is called / executed MP2 So the previous contents are lost	2

50. O/N 23/P21/Q8/d

8(d)	<pre> FUNCTION GetField(Msg : STRING, FieldNo : INTEGER) RETURNS STRING DECLARE RetString : STRING CASE OF FieldNo 1 : RetString ← MID(Msg, 2, 3) 2 : RetString ← MID(Msg, 5, 3) 3 : RetString ← MID(Msg, 8, LENGTH(Msg) - 8) OTHERWISE : RetString ← "" ENDCASE RETURN RetString ENDFUNCTION </pre> MP1 Use of CASE ... ENDCASE or IF ... THEN ... ENDIF MP2 Field 1 and Field 2 extracted correctly MP3 Calculate a length of field 3 MP4 Field 3 extracted <u>correctly</u> MP5 Return empty string in case of invalid parameter (via OTHERWISE or initialisation) MP6 Final RETURN, after a reasonable attempt	6
------	--	----------

51. O/N 23/P22/Q1/b

1(b)	One mark per row	4										
<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>(Total * DepRate) + 1.5</td><td>249.50</td></tr><tr><td>RIGHT(Description, 7)</td><td>"(small)"</td></tr><tr><td>(LENGTH(Description) - 8) > 16</td><td>TRUE</td></tr><tr><td>NUM_TO_STR(INT(DepRate * 10)) & '%'</td><td>"20%"</td></tr></table>			Expression	Evaluates to	(Total * DepRate) + 1.5	249.50	RIGHT(Description, 7)	"(small)"	(LENGTH(Description) - 8) > 16	TRUE	NUM_TO_STR(INT(DepRate * 10)) & '%'	"20%"
Expression	Evaluates to											
(Total * DepRate) + 1.5	249.50											
RIGHT(Description, 7)	"(small)"											
(LENGTH(Description) - 8) > 16	TRUE											
NUM_TO_STR(INT(DepRate * 10)) & '%'	"20%"											

52. O/N 23/P22/Q2/b

2(b)	One mark per point: - Name: (pre / post) conditional loop - Justification: the number of iterations is not known // loop ends following a specific input (in the loop)	2
------	---	----------

53. O/N 23/P22/Q4/a

4(a)	Example Solution <pre> PROCEDURE Count() DECLARE COdd, CEven, ThisNum : INTEGER COdd ← 0 CEven ← 0 INPUT ThisNum WHILE ThisNum <> 99 IF ThisNum MOD 2 = 1 THEN COdd ← COdd + 1 ELSE CEven ← CEven + 1 ENDIF INPUT ThisNum ENDWHILE OUTPUT "Count of odd and even numbers: ", COdd, CEven ENDPROCEDURE </pre> Mark as follows Max 6 marks: 1 Procedure heading and ending 2 Local COdd, CEven and ThisNum declared as integers 3 Conditional loop while ThisNum <> 99 4 Input ThisNum in a loop 5 Check ThisNum is odd or even in a loop 6 Increment appropriate count in a loop, both counts must have been initialised before loop 7 After the loop output COdd and CEven with a suitable message following a reasonable attempt at counting	6
------	---	----------

54. O/N 23/P22/Q6/b(i,ii)

6(b)(i)	Function	1
6(b)(ii)	FUNCTION CheckFiles (NameRoot : STRING) RETURNS INTEGER	1

6(a)	Example Solution <pre> PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER) DECLARE FileName, Suffix : STRING DECLARE Count : INTEGER FOR Count ← 1 TO NumFiles Suffix ← NUM_TO_STR(Count) WHILE LENGTH(Suffix) <> 3 Suffix ← '0' & Suffix ENDWHILE FileName ← NameRoot & '.' & Suffix OPENFILE FileName FOR WRITE WRITEFILE FileName, "This is File " & FileName CLOSEFILE FileName NEXT Count ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, including parameters, and ending 2 Loop for NumFiles iterations 3 Attempt to create filename suffix using NUM_TO_STR() in a loop 4 Completely correct filename 5 OPENFILE in WRITE mode and subsequent CLOSE in a loop 6 WRITE initial first line to the file in a loop	6
------	---	----------

55. O/N 23/P22/Q8/b

8(b)	Example solution <pre> PROCEDURE ProcessMsg(ThisMsg : STRING) DECLARE ThisDest : STRING DECLARE Response : BOOLEAN DECLARE StackNum : INTEGER IF LENGTH(ThisMsg) >= 4 THEN ThisDest ← LEFT(ThisMsg, 3) IF ThisDest = MyID THEN // It's for this computer StackNum ← 1 ELSE StackNum ← 2 ENDIF Response ← StackMsg(ThisMsg, StackNum) IF Response = FALSE THEN OUTPUT "Message discarded - no room on stack" ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Ignore message if data field is empty 2. Extract <code>ThisDest</code> from <code>ThisMsg</code> 3. Test if destination is this computer 4. Attempt to use <code>StackMsg()</code> 5. Fully correct use of <code>StackMsg()</code> for both cases / stacks 6. Test <code>StackMsg()</code> return value for both cases 7. Following a reasonable attempt at MP6 output warning if <code>StackMsg()</code> returns <code>FALSE</code>	7
------	---	---

56. O/N 23/P23/Q1/b,c

1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			4
1(b)			

57. O/N 23/P23/Q4/a,b

4(a)	<pre> PROCEDURE RandList() DECLARE Count, BaseNum, ThisNum : INTEGER CONSTANT StepVal = 10 BaseNum ← 0 FOR Count ← 1 TO 25 ThisNum ← BaseNum + INT(RAND(StepVal)) OUTPUT ThisNum BaseNum ← BaseNum + StepVal NEXT Count ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 <u>Local</u> loop counter Count as integer MP3 Loop to iterate 25 times or more for each unique number MP4 'Attempt' to generate a random number including use of INT() in a loop MP5 Ensure that number generated is greater than previous and change 'previous' MP6 Output random number after an attempt at MP5 in a loop	6
4(b)	One mark for simplified logical expression: MP1 Result[x + 1] <= Result[x] ALTERNATIVE SOLUTION: Result[x] >= Result[x + 1]	1

58. O/N 23/P23/Q6

6	<pre> FUNCTION TestNum(ThisNum : STRING) RETURNS INTEGER IF LEFT(ThisNum,3) = RIGHT(ThisNum 3) THEN RETURN 3 ENDIF IF RIGHT(ThisNum, 3) = "000" THEN RETURN 2 ENDIF IF MID(ThisNum, 4, 1) = MID(ThisNum, 5, 1)___ AND MID(ThisNum, 5, 1) = MID(ThisNum, 6, 1) THEN RETURN 1 ENDIF RETURN 0 ENDFUNCTION </pre> MP1 Function heading and ending including parameter and return type MP2 Test for Condition 1 MP3 Test for Condition 2 MP4 Test for Condition 3 MP5 Return the highest value if more than one condition is satisfied MP6 Return zero if no condition matched	6
---	---	---

59. O/N 23/P23/Q7/b(ii)

7(b)(ii)	<pre> PROCEDURE Sub-9 (BYREF Param : MyType) </pre> MP 1 One mark for BYREF MP 2 One mark for the rest of the statement	2
----------	---	---

60. O/N 23/P23/Q8/c

8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 OUTPUT the data in a loop MP5 INPUT the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

61. M/J 23/P21/Q1/b,c

1(b)	One mark for correct description of: Error 1: Brackets mismatch // 2/value should be added after brackets/function // Addition between a string and a number is not valid // STR_TO_NUM / the function needs to be passed a string / not an integer Error 2: No Error Error 3: MONTH() returns an integer and this is being compared with a character/string // Integer cannot be compared to a string // 6 should not be in quotes	3
------	---	---

1(c)(i)			2
		Expression	
	1	(Points > 99) OR Active	
	2	(Points MOD 2 = 0) OR Exempt	
	3	(Points <= 75) AND (Active OR Exempt)	
	4	(Active OR NOT Active) AND NOT Exempt	
One mark for any two rows correct Two marks for all rows correct			
1(c)(ii)	NOT Exempt		1

62. M/J 23/P21/Q2/b

2(b)	One mark for declaration: <u>DECLARE StartDate</u> : <u>DATE</u> One mark for each underlined part of assignment: <u>StartDate</u> ← <u>SETDATE (15, 11, 2005)</u>	3
------	--	----------

63. M/J 23/P21/Q4

4	<pre>Function Replace(OldString : STRING, Char1, Char2 : CHAR) __ RETURNS : STRING DECLARE NewString : STRING DECLARE ThisChar : CHAR DECLARE Index : INTEGER NewString ← "" FOR Index ← 1 TO LENGTH(OldString) ThisChar ← MID(OldString, Index, 1) IF ThisChar = Char1 THEN ThisChar ← Char2 ENDIF NewString ← NewString & ThisChar NEXT Index RETURN NewString ENDFUNCTION</pre> Mark as follows: - Function heading and ending, including parameters and return type - Declaration of local variables used including loop counter - Loop for length of OldString - Extract char and test in a loop - Use of concatenate to build NewString replace char if necessary, in a loop - Return NewString after reasonable attempt	6
---	---	----------

64. M/J 23/P21/Q8/a

8(a)	<pre> FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z' THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Check <code>ThisString</code> is exactly 5 characters in length 2 Use of <code>SuppExists()</code> with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character in a loop 5 Test if char is alphabetic in a loop 6 ...catering for both upper and lower case 7 Return Boolean following a reasonable attempt	7
------	---	----------

65. M/J 23/P22/Q1/a

1(a)(i)	Use of constants	1
1(a)(ii)	One mark per bullet point (or equivalent to max 3): 1 Postal rates are entered once only 2 Avoids input error / changing the cost accidentally // avoids different values for postal rates at different points in the program 3 When required, the constant representing the postal rate value is changed once only // easier to maintain the program when the postal rates change 4 Makes the program easier to understand Note: Max 3 marks	3

66. M/J 23/P22/Q2/a,b

2(a)	<code>MyDOB ← SETDATE(17, 11, 2007)</code>	1
------	--	----------

2(b)	<u>NumMonths</u> $\leftarrow$ <u>12 - MONTH(MyDOB)</u>	2
	One mark per underlined part	

67. M/J 23/P22/Q4

4	<pre> Function GetNum(ThisString : STRING, ThisChar : CHAR) RETURNS INTEGER DECLARE Index, Count : INTEGER Count $\leftarrow$ 0 FOR Index $\leftarrow$ 1 TO LENGTH(ThisString) IF MID(ThisString, Index, 1) = ThisChar THEN Count $\leftarrow$ Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Function heading and end, including parameters and return type 2 Declare local Integers for <code>Index</code> and <code>Count</code> 3 Loop for length of <code>ThisString</code> 4 Extract a character and compare with <u>parameter</u> in a loop 5 Increment <code>Count</code> if match in a loop 6 Return <code>Count</code> after loop	6
---	--	----------

68. M/J 23/P22/Q6

6	Example of iterative solution: <pre> PROCEDURE Square(Dim : INTEGER) DECLARE Count : INTEGER DECLARE ThisChar : CHAR DECLARE StringA, StringB : STRING CONSTANT FILLER = '*' StringA ← "" ThisChar ← NUM_TO_STR(Dim) FOR Count ← 1 TO Dim StringA ← StringA & ThisChar //build up first & last line NEXT Count StringB ← ThisChar FOR Count ← 1 TO Dim - 2 StringB ← StringB & FILLER //build up intermediate line NEXT Count StringB ← StringB & ThisChar // add final digit OUTPUT StringA FOR Count ← 1 TO Dim - 2 OUTPUT StringB NEXT Count IF Dim <> 1 THEN OUTPUT StringA ENDIF ENDPROCEDURE </pre> For loop-based solutions, mark as follows: 1 Procedure heading and ending including parameter 2 Loop using parameter, containing attempt to construct first line / last line 3 Construct first line / last line 4 Attempt at loop to construct intermediate line 5 Output first / last line of square when <code>Dim > 2</code> 6 Output all intermediate lines in a loop 7 Correct output of first two squares Note: Max 6 marks	6
---	---	---

6

Example of selection-based solution:

```

PROCEDURE Square(Dim : INTEGER)
  DECLARE Count : INTEGER

  CASE OF Dim
    1 : OUTPUT "1"
    2 : OUTPUT "22"
      OUTPUT "22"
    3 : OUTPUT "333"
      OUTPUT "3*3"
      OUTPUT "333"
    4 : OUTPUT "4444"
      FOR Count ← 1 TO 2
        OUTPUT "4**4"
      NEXT Count
      OUTPUT "4444"
    5 : OUTPUT "55555"
      FOR Count ← 1 TO 3
        OUTPUT "5***5"
      NEXT Count
      OUTPUT "55555"
    6 : OUTPUT "666666"
      FOR Count ← 1 TO 4
        OUTPUT "6****6"
      NEXT Count
      OUTPUT "666666"
    7 : OUTPUT "7777777"
      FOR Count ← 1 TO 5
        OUTPUT "7*****7"
      NEXT Count
      OUTPUT "7777777"
    8 : OUTPUT "88888888"
      FOR Count ← 1 TO 6
        OUTPUT "8*****8"
      NEXT Count
      OUTPUT "88888888"
    9 : OUTPUT "999999999"
      FOR Count ← 1 TO 7
        OUTPUT "9*****9"
      NEXT Count
      OUTPUT "999999999"

  ENDCASE

ENDPROCEDURE

```

For in-line / selection-based solutions, mark as follows:

- 1 Procedure heading and ending including parameter
- 2 Correct use of parameter to select all required squares
- 3 Correct output of first two squares
- 4 At least six squares correctly output
- 5 At least one loop to output lines with multiple asterisks
- 6 All number squares output correctly

69. M/J 23/P22/Q8/a

8(a)	<pre> FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are all numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	7
------	--	----------

70. M/J 23/P23/Q2/a

2(a)	One mark for each underlined part <pre> IF <u>DAYINDEX(MyDOB)</u> <u>= 5</u> THEN </pre>	2
------	--	----------

71. M/J 23/P23/Q4

4	<pre> FUNCTION MakeString(Count : INTEGER, AChar : CHAR) RETURNS STRING DECLARE MyString : STRING DECLARE Index : INTEGER IF Count < 1 THEN MyString ← "ERROR" ELSE MyString ← "" FOR Index ← 1 TO Count MyString ← MyString & AChar NEXT Index ENDIF RETURN MyString ENDFUNCTION </pre> MP1 Function heading and end including parameters and return type MP2 Declaration of locals <code>Index</code> and <code>MyString</code> MP3 Test for <code>Count < 1</code> and if true, assign "ERROR" to <code>MyString</code> / Immediate RETURN MP4 Loop for <code>Count</code> iterations MP5 Use of concatenate – must have been initialised in a loop MP6 Return STRING (correctly in both cases)	6
---	--	---

72. M/J 23/P23/Q6/a,b

6(a)	Max 7 marks <pre>PROCEDURE Select(Start, End : INTEGER) DECLARE ThisNum, Total: INTEGER DECLARE ThisString : STRING DECLARE Char1, Char2 : CHAR FOR ThisNum ← Start+1 TO End-1 ThisString ← NUM_TO_STR(ThisNum) Char1 ← RIGHT(ThisString, 1) Char2 ← LEFT(RIGHT(ThisString, 2), 1) Total ← STR_TO_NUM(Char1) + STR_TO_NUM(Char2) IF Total = 6 THEN OUTPUT ThisString ENDIF NEXT ThisNum ENDPROCEDURE</pre> MP1 Procedure heading and ending MP2 (Count-controlled) Loop MP3 with correct range from Start+1 to End-1 MP4 Convert ThisNum (loop counter) to a string MP5 Extract the last two/first/second 'character digit(s)' required in a loop MP6 Extract the second individual 'character digit' required in a loop MP7 Calculate the sum of the last two digits MP8 If sum = 6 then OUTPUT the number (either string or integer) in a loop	7
6(b)	Max 4 marks MP1 The function will take two integer parameters - the number and the (required) total MP2 ... and return a Boolean OR: CheckNum(Number, Total : INTEGER) MP1 RETURNS BOOLEAN MP2 Two marks for the advantages: MP3 CheckNum() can be called repeatedly as and when required MP4 CheckNum() is designed and tested once (then used repeatedly) MP5 Any subsequent change to CheckNum() needs to be made once only // is easier to maintain/modify	4

73. O/N 22/P21/Q1/a,c

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example encryption) 3 They are readily available / speed up development time	3										
1(c)	One mark per row: <table><thead><tr><th>Statement</th><th>Error</th></tr></thead><tbody><tr><td>IF EMPTY ← "" THEN</td><td>Should be "=" not ←</td></tr><tr><td>Status ← IS_NUM(-23.4)</td><td>Parameter should be a string (or char) // should not be a real</td></tr><tr><td>X ← STR_TO_NUM("37") + 5</td><td>NO ERROR</td></tr><tr><td>Y ← STR_TO_NUM("37" + "5")</td><td>Wrong operator – should be & or Parameter is not a string</td></tr></tbody></table>	Statement	Error	IF EMPTY ← "" THEN	Should be "=" not ←	Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real	X ← STR_TO_NUM("37") + 5	NO ERROR	Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string	4
Statement	Error											
IF EMPTY ← "" THEN	Should be "=" not ←											
Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real											
X ← STR_TO_NUM("37") + 5	NO ERROR											
Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string											

74. O/N 22/P21/Q6/a

6(a)	One mark for any part correct (accept equivalent wording) (Max 1): • Condition evaluates to TRUE if bracket contents evaluate to FALSE: • Bracket contents evaluate to FALSE if: • Dots: zero / less than one or • Ats: not equal to one or • Others: less than nine	1
------	---	----------

75. O/N 22/P21/Q7/a

7(a)(i)	One mark per point (Max 7) as follows: 1 Declaration of local variables for Par1 Par2 and Par3 2 Loop to end of (parameter) string // until operator is found 3 Extract a character in a loop... 4 Attempt at extraction of three parts of expression using substring functions 5 Completely correct extraction of all three parts of expression 6. Convert string to Integer using STR_TO_NUM(<something sensible>) 7 Attempt to interpret at least two operators (Par2): + - * / 8 Corresponding correct calculation (all operators) and final Output of result <pre> PROCEDURE Calculate(Expression : STRING) DECLARE Val1, Val2, Index : INTEGER DECLARE Result : REAL DECLARE Par1, Par2, Par3 : STRING CONSTANT PLUS = '+' CONSTANT MINUS = '-' CONSTANT MULTIPLY = '*' CONSTANT DIVIDE = '/' FOR Index ← 1 TO LENGTH(Expression) //search for operator ThisChar ← MID(Expression, Index, 1) IF IS_NUM(ThisChar) = FALSE THEN Par1 ← LEFT(Expression, Index - 1) Par2 ← ThisChar Par3 ← RIGHT(Expression, LENGTH(Expression) - Index) ENDIF NEXT Index Val1 ← STR_TO_NUM(Par1) Val2 ← STR_TO_NUM(Par3) CASE OF Par2 PLUS : Result ← Val1 + Val2 MINUS : Result ← Val1 - Val2 MULTIPLY : Result ← Val1 * Val2 DIVIDE : Result ← Val1 / Val2 ENDCASE OUTPUT Result ENDPROCEDURE </pre>	7
7(a)(ii)	FUNCTION Calculate(Expression : STRING) RETURNS REAL	1

76. O/N 22/P21/Q8/b

8(b)	<code>CALL CountErrors("Jim01Prog.txt", 20)</code> One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	2
------	---	----------

77. O/N 22/P22/Q1/a,c

1(a)(i)	One mark for each point (Max 2): - When a task which is repeated / reused / performed in several places - When a part of an algorithm performs a specific task - Reduces complexity of program / program is simplified // subroutine already available - Testing / debugging / maintenance is easier	2
---------	---	----------

1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2
----------	---	----------

1(c)	One mark per row:		3
	Expression	Evaluation	
	MID(CharList, MONTH(FlagDay), 1)	'D'	
	INT(Count / LENGTH(CharList))	4	
	(Count >= 99) AND (DAY(FlagDay) > 23)	FALSE	

78. O/N 22/P22/Q4/b

4(b)	Pre-condition (loop) / count-controlled (loop)	1
------	--	----------

79. O/N 22/P22/Q6/a

6(a)	Example by repeated multiplication: Mark as follows (multiplication solution), (Max 7): 1 Function heading and ending including parameter and return type 2 Declaration and initialisation of local Num 3 Any conditional loop 4 Conditional loop until ThisValue found or Try out of range 5 Multiply Try by Num in a loop 6 Compare Try with ThisValue and set termination if the same in a loop 7 Increment Num and repeat in a loop 8 Attempt to Return Num if ThisValue is a factorial or -1 otherwise <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER DECLARE Found : BOOLEAN Num ← 0 Found ← FALSE Try ← 1 WHILE Try <= ThisValue AND Found = FALSE Num ← Num + 1 Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found Found ← TRUE ENDIF ENDWHILE IF Found = TRUE THEN RETURN Num ELSE RETURN -1 ENDIF ENDFUNCTION </pre>	7
------	---	----------

6(a)	Alternative FOR LOOP solution. Mark as follows: 1 Function heading and ending including parameter and return type 2 Declaration of local Integer value for Num and Try 3 Count-controlled Loop from 1 to ThisValue 4 Multiply Try by Num in a loop 5 Compare Try with ThisValue in a loop 6 ...Immediate return of Num if they are the same in a loop 7 Return -1 if ThisValue not found after loop <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER Try ← 1 FOR Num ← 1 TO ThisValue Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found RETURN Num ENDIF NEXT Num RETURN -1 ENDFUNCTION </pre>	
------	---	--

80. O/N 22/P23/Q5/a,b(ii)

5(a)(i)	One mark per point: 1 Procedure heading and ending including four parameters... 2 ...and use of BYREF for the three extracted values 3 Extract and assign SID 4 Extract and assign SDesc 5 Calculation of length of SCost (remainder of string) 6 Extract and assign SCost following an attempt at MP5 <pre> PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING) SID ← LEFT(TLine, 5) SDesc ← MID(TLine, 6, 32) SCost ← RIGHT(TLine, LENGTH(TLine) - 37) ENDPROCEDURE </pre>	6
5(a)(ii)	One mark each (Max 2): 1 Provides a mechanism to allow calling program to pass data 2 Defines the four parameters of Unpack() 3 ... giving their <u>data type and order</u>	2

5(b)(ii)	One mark per point (Max 2): • The new function will return an item of type <code>StockItem</code> • Need to declare/use a (local) variable of type <code>StockItem</code> • <code>Costfield</code> needs to be converted from a string to a real	2
----------	--	----------

81. O/N 22/P23/Q6/b

6(b)	One mark per point: 1 Function heading and ending including parameters 2 Declaration of local variable for Result / alt mechanism 3 Check for Reject 4 Check for Accept 5 Check for Recheck (or just default to third option) 6 Return Result following a reasonable attempt <pre> Function Status(Actual, Min, Max : INTEGER) RETURNS CHAR DECLARE Result : CHAR CONSTANT Accept = 'A' CONSTANT Reject = 'R' CONSTANT ReCheck = 'C' Result ← ReCheck //Check if reject IF Actual > Max + 2 OR Actual < Min - 2 THEN Result ← Reject ENDIF //Check if acceptable IF Actual < Max - 2 AND Actual > Min + 2 THEN Result ← Accept ENDIF RETURN Result ENDFUNCTION </pre>	6
------	--	----------

82. O/N 21/P21/Q6/c

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	----------

83. O/N 21/P22/Q1/c,d

1(c)

Expression	Evaluates to
ASC ('C')	67
2 * STR_TO_NUM ("27")	54
INT (27 / 2)	13
"Sub" & MID ("Abstraction" , 4 , 5)	"Subtract"

One mark per row

Function names must be exactly as shown

4

1(d)	Expression	Evaluates to	2
	PressureOK AND HiFlow	FALSE	
	PumpOn OR PressureOK	TRUE	
	NOT PumpOn OR (PressureOK AND NOT HiFlow)	TRUE	
	NOT (PumpOn OR PressureOK) AND NOT HiFlow	FALSE	
1 mark for any two rows correct 2 marks for all rows correct.			

84. O/N 21/P22/Q4/a,c

4(a)	Answer		5
	A line number containing a variable being incremented	19 / 21 / 23	
	The type of loop structure	pre-condition	
	The number of functions used	3	
	The number of parameters passed to function STR_TO_NUM()	1	
	The name of a procedure other than Check()	Result	
4(c)(i)	One mark per point: Structure: A count-controlled loop Justification: The number of iterations is known // repeats for the length of InString		2
4(c)(ii)	15, 23 One mark for both line numbers		1

85. O/N 21/P22/Q6/b,c(ii),d

6(b)	One mark for: (A flag is used to) exit the loops // iteration is terminated - as soon as a Screen element with value 1 is found	2
------	--	----------

6(c)(ii)	One mark for advantage and one for disadvantage: Advantage: (max 1) • Only have to change one module if specification changes • Less repetitive code / fewer lines of code • Aids re-usability Disadvantage: (max 1) • Single module more complex / more error prone / more difficult to debug ... • Single module cannot be split among programmers / teams Max 2	2
6(d)	<pre> PROCEDURE GetCentre () DECLARE StartRow, EndRow, StartCol, EndCol : INTEGER StartRow ← FirstRowSet() IF StartRow = -1 THEN CentreRow ← -1 // no 'touch' detected ELSE EndRow ← LastRowSet() StartCol ← FirstColSet() EndCol ← LastColSet() CentreRow ← INT((StartRow + EndRow)/2) CentreCol ← INT((StartCol + EndCol)/2) ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Call <any Set function> and check for -1 // check for no element set 2 ...and if so set CentreRow to -1 3 Call all 4 Set functions to get 'extremity' values 4 Calculate centre row and centre column 5 Use of INT() function or DIV operator on values from MP4 6 Assign calculated values to CentreRow and CentreCol Note: Max 5 if procedure heading and ending missing or incorrect (ignore array if passed as a parameter) or any local variables are undefined or of incorrect type	6

6(a)	<pre> FUNCTION FirstRowSet() RETURNS INTEGER DECLARE Row, Col : INTEGER DECLARE Found : BOOLEAN // array is 1280 x 800 Row ← 1 Found ← FALSE WHILE Row ≤ 800 AND Found = FALSE // top to bottom Col ← 1 WHILE Col ≤ 1280 AND Found = FALSE // left to right IF Screen[Row,Col] = 1 THEN Found ← TRUE // end function as soon as first // found ENDIF Col ← Col + 1 ENDWHILE Row ← Row + 1 ENDWHILE IF Found = FALSE THEN // nothing found Row ← 0 ENDIF RETURN Row - 1 ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending and return type 2 (Conditional) outer loop 1 to 800 (row) 3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column) 4 Reference <code>Screen</code> element and test for = 1 // <> 0 5 and if true save row number and exit loops 6 Increment index variables in both inner and outer loop 7 Return Row number or -1, following a reasonable attempt	7
------	---	---

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	---

87. M/J 21/P21/Q1/b

1(b)

Statement	Error
Result ← 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
SubString ← MID("pseudocode", 4, 1)	NO ERROR
IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect
Result ← Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator
Message ← "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number

One mark for each row

5

88. M/J 21/P21/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
	One mark for each underlined part	

89. M/J 21/P21/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	---	----------

90. M/J 21/P21/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	---

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	----------

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	--	--

91. M/J 21/P22/Q5/b

5(b)(i)	Answer		4
	Give a line number containing an example of an initialisation statement.	07	
	Give a line number containing the start of a repeating block of code.	09 / 10	
	Give a line number containing a logic statement.	12	
	Give the number of parameters of function MID () .	3	
One mark for each row			

5(b)(ii)	IF (NextChar >= 'a') AND (NextChar <= 'z') THEN	2
	One mark for IF ... AND ... One mark for both conditions	

92. M/J 21/P22/Q8/b

8(b)	<pre> Procedure GetInitials() DECLARE NewString, NextWord : STRING DECLARE ThisWordNum, Index : INTEGER ThisWordNum ← 0 NewString ← "" REPEAT ThisWordNum ← ThisWordNum + 1 Index ← GetStart(ThisWordNum) IF Index <> -1 THEN //if there is ThisWordNum NextWord ← GetWord(Index) IF IgnoreWord(NextWord) = FALSE THEN NewString ← NewString & UCASE(LEFT(NextWord, 1)) ENDIF ENDIF UNTIL Index = -1 OUTPUT NewString ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declare <code>NewString</code> and initialise to empty string 2 Conditional loop to pick out all words from <code>FNString</code> 3 Evaluate result of <code>GetStart()</code> in a loop 4 Test result <code><> -1</code> and if not: 5 Assign result of <code>GetWord()</code> to a variable in a loop 6 Test result of <code>IgnoreWord()</code> in a loop 7 If not ignored, add the next initial letter to <code>NewString</code> in a loop 8 Increment <code>ThisWordNum</code> (must have been initialised) in a loop 9 Output <code>NewString</code> (must be all upper case) outside loop Note: Max 8 marks	8
------	--	---

93. M/J 21/P23/Q1/b

1(b)	Statement	Error	5
	Result $\leftarrow$ 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING	
	SubString $\leftarrow$ MID("pseudocode", 4, 1)	NO ERROR	
	IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect	
	Result $\leftarrow$ Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator	
	Message $\leftarrow$ "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number	
One mark for each row			

94. M/J 21/P23/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
One mark for each underlined part		

95. M/J 21/P23/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	---	---

96. M/J 21/P23/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	---

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	---

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	--	--

Unit 12

SOFTWARE DEVELOPMENT

85 items · 427 marks

Syllabus sections in this unit

- 12.1 Program Development Life cycle
- 12.2 Program Design
- 12.3 Program Testing and Maintenance

Unit 12: Software Development

1. M/J 26/P21/Q3/a,b

- 3 A business operates five call centres. The software currently in use in each call centre is to be replaced; a new system will be developed by a software company.

(a) The analysis stage included a feasibility study.

State **three** other activities carried out during the analysis stage.

- 1
- 2
- 3

[3]

(b) The analysis stage has ended.

Complete the statements about the development of the new system.

Before any programming is started, the stage for the proposed new system must be completed.

For this large project, the tasks will be broken down into several ; each will be coded by a different programming team.

Once each team has completed its work, testing must be carried out.

Before the new system is delivered to the business, testing will be carried out in-house.

It is decided to install the software initially in only one of the call centres where testing will be carried out.

The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until maintenance is completed.

[6]

2. M/J 26/P21/Q8/a,b,c

- 8 Global array `MyData` contains the values shown.

Study the pseudocode algorithm and array `MyData`

```

FUNCTION MyTrace(Target : INTEGER) RETURNS INTEGER
  DECLARE LeftPointer, RightPointer, MidPointer : INTEGER
  LeftPointer ← 1
  RightPointer ← 15
  WHILE LeftPointer ≤ RightPointer
    MidPointer ← (LeftPointer + RightPointer) DIV 2
    IF MyData[MidPointer] = Target THEN
      OUTPUT "Found " & NUM_TO_STR(MidPointer)
      RETURN MidPointer
    ELSE
      IF Target > MyData[MidPointer] THEN
        LeftPointer ← MidPointer + 1
      ELSE
        RightPointer ← MidPointer - 1
      ENDIF
    ENDIF
  ENDWHILE
  OUTPUT "Not found"
  RETURN -1
ENDFUNCTION

```

	MyData
1	2
2	8
3	9
4	12
5	13
6	15
7	18
8	19
9	20
10	23
11	25
12	26
13	39
14	38
15	40

- (a) Complete the trace table for the function `MyTrace` when the following pseudocode statement is executed.

`Result ← MyTrace(15)`

LeftPointer	RightPointer	MidPointer

[4]

- (b) Give the output.

[1]

- (c) Give the value assigned to `Result`

[1]

3. M/J 26/P22/Q1/b,c,d,e,f

- 1** A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a)** When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (i)** Complete the pseudocode to declare the variable to be used to hold the password.

```
DECLARE Pass : .....
```

[1]

- (ii)** Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

```
IF ..... (Pass) < 6 OR ..... (Pass) > ..... THEN
```

```
    OUTPUT "Error"
```

```
    OUTPUT "The password has to contain between 6 and 8 characters."
```

```
ENDIF
```

[2]

- (iii)** Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```

DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ..... , ..... )
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF

```

[7]

- (b) A state-transition diagram is being created as part of the design process for the module, which will control the alarm system once it has been successfully installed. Two of the states already identified are that the alarm is in:

- standby mode
- on mode (alarm system is active).

- (i) Identify **two** other states that should be part of the state-transition diagram.

1

2

[2]

- (ii) Two events that will cause a change of state are:

- entering the password when the alarm system is in standby mode
- entering the password when the alarm system is in on mode.

Identify **three** other events that would cause a change of state in the state-transition diagram for the alarm system.

1

2

3

[3]

- (c) The validation rules from 1(a) are given again:

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

Complete the table with test data that could be used to test the validation rules.

Each example of test data given must:

- be invalid for the specific rule being tested
- be valid for the other **two** rules **not** being tested
- test a different aspect of the rule being tested.

Two of the tests have been completed for you.

rule being tested	example of test data	reason for invalid test data
rule 1	1A	password is too short
rule 1		
rule 2		
rule 2		
rule 3	PASSWRD2	letters are repeated
rule 3		
rule 3		

[3]

(d) Five outcomes that could be produced at the design stage in the program development lifecycle include:

- state-transition diagrams
- pseudocode
- test strategy
- test plan
- structured English.

Identify **three** other outcomes that could be produced at the design stage.

1

2

3

[3]

(e) The design and testing stages are two of the stages in the program development lifecycle used when developing the program to control the alarm system.

Identify **two** other stages in the program development lifecycle.

1

2

[2]

- (f) All the individual modules for the alarm system have now been written, tested and debugged as necessary. All these modules are combined to form the program, and further testing is now needed.

- (i) Identify this method of testing.

[1]

- (ii) One of the modules does not work as expected when added to the program.

Identify and describe a testing method that can be used to address this problem, so that the program can continue to be tested.

method

description

[3]

4. M/J 26/P23/Q3/a,b,c

- 3** A company wishes to replace the point-of-sale software that each shop uses at its checkout. A software company is developing a new system.

(a) Complete the summary about the development of the new system.

The initial stage is carried out by studying in detail the current system.

A design for the proposed new system is produced.

Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. testing is carried out.

The final stage of the project uses testing to test all programs on the company's hardware. After successful testing, the company approves the new system.

The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. maintenance has to be carried out.

[4]

(b) For this large project, the task was divided between two programming teams.

Team A were responsible for the screen designs and all outputs; they involved the company at the earliest possible stage and obtained feedback.

Team B used a more traditional approach to the program development life cycle; the stages in the development followed an agreed sequence.

Identify the program development life cycle model adopted by each team, and describe **one** additional feature of each approach.

team A model

feature

team B model

feature

[4]

(c) State **three** activities carried out during the design stage of the program development life cycle.

1

2

3

[3]

5. M/J 26/P23/Q7/a,b

7 Study the pseudocode algorithm. Numbers will be copied into the *Data* array.

```
// STAGE 1
Num1 ← 12
Index ← 1
FOR i ← 2 TO Num1 DIV 2
    IF Num1 MOD i = 0 THEN
        Data[Index] ← i
        Index ← Index + 1
    ENDIF
NEXT i
Index ← Index - 1
OUTPUT Index, " values were found"
```

(a) (i) Complete the trace table for STAGE 1

Num1	Index	i	Data					
			1	2	3	4	5	6

[4]

(ii) State how the numbers in the *Data* array are related.

[1]

(b) The complete algorithm has a second stage.


```

// STAGE 2

Num2 ← 8
Match ← FALSE

REPEAT
    IF Num2 MOD Data[Index] = 0 THEN
        Match ← TRUE
    ELSE
        Index ← Index - 1
    ENDIF
UNTIL Match = TRUE OR Index = 0

IF Match = TRUE THEN
    OUTPUT Data[Index]
ELSE
    OUTPUT "Nothing found"
ENDIF

```

One line of the STAGE 1 code is changed:

```
Num1 ← 20
```

The programmer creates a final program with the STAGE 1 code followed immediately by the STAGE 2 code.

The STAGE 1 trace table shows the relevant final variable values after the completion of STAGE 1.

STAGE 1				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

This is followed by running of the STAGE 2 code.

(i) Complete the trace table for STAGE 2

STAGE 2				Data					
Num1	Num2	Index	Match	1	2	3	4	5	6
20		4		2	4	5	10		

[3]

(ii) State the purpose of the STAGE 2 algorithm.

[1]

6. O/N 25/P21/Q1/b

1 A program is being developed to meet a particular customer requirement.

(b) Different test methods will be used at different stages of the program development.

Complete the table by identifying the test method that matches the test description.

The first row has been completed for you.

Test description	Test method
carried out as soon as a program module has been coded	alpha
carried out as program modules are being combined	
completed by the developers without referring to the code	
completed by the customer	

[3]

7. O/N 25/P21/Q7/a,b

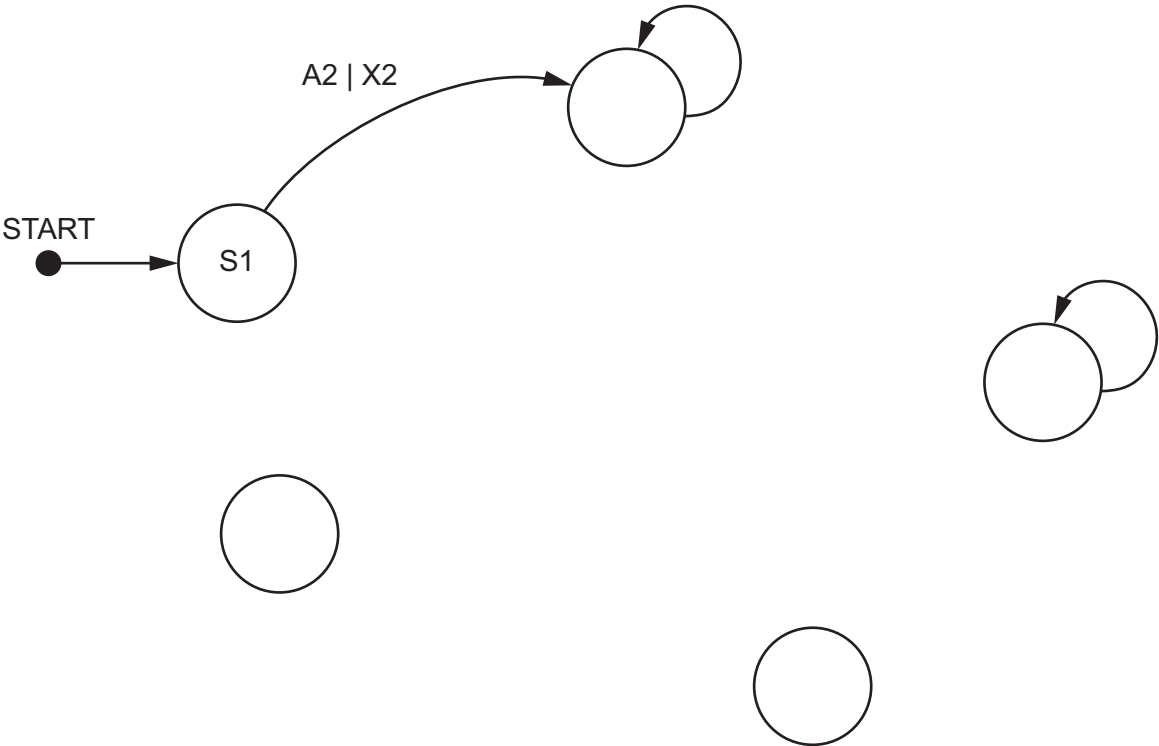
7 There are several different ways to express an algorithm during the design of a program.

(a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A2	X2	S3
S1	A1	X1	S2
S2	A4	X4	S5
S3	A1		S3
S3	A3	X3	S2
S3	A2	X4	S4
S4	A1	X1	S4
S4	A3		S2
S4	A4	X4	S5

Complete the state-transition diagram to represent the information given in the table.



(b) A structure chart is used to document a different part of the program.

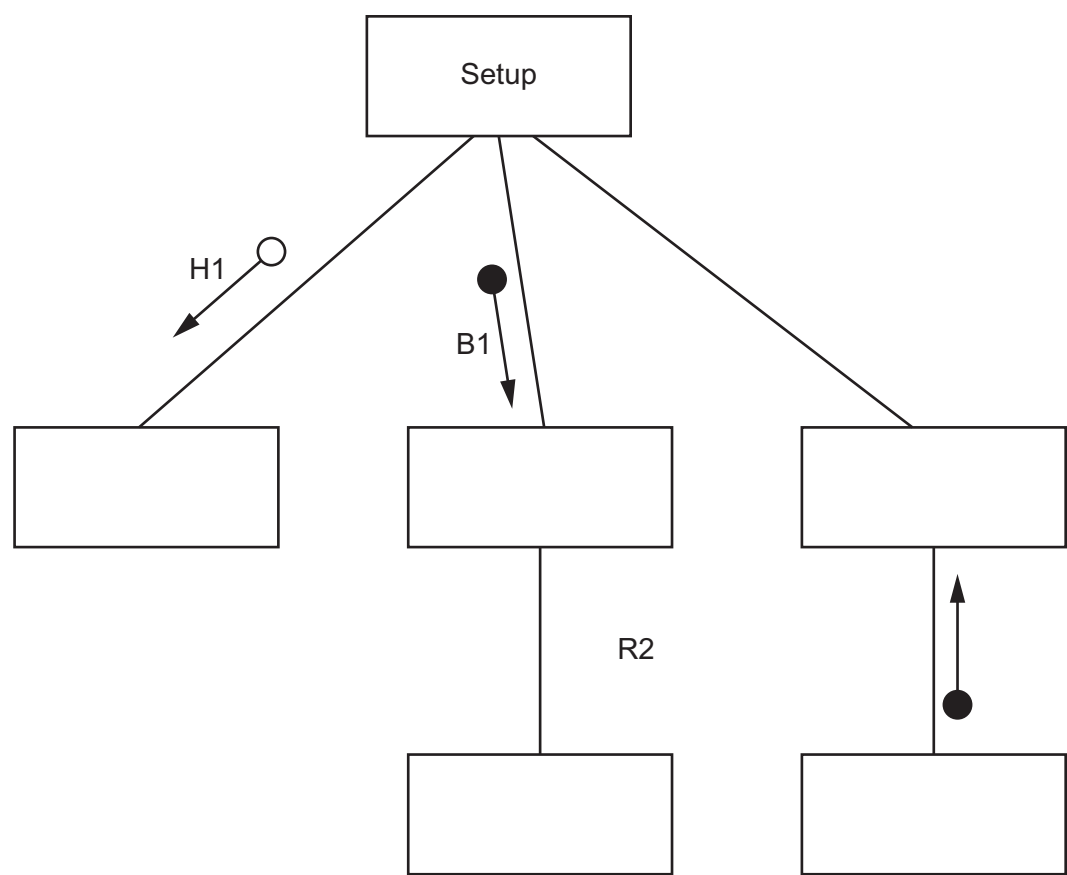
[5]

This part of the program contains six modules:

Pseudocode module header
PROCEDURE Setup()
PROCEDURE Restart(H1 : STRING, C1 : INTEGER)
FUNCTION Modify(B1 : BOOLEAN) RETURNS INTEGER
PROCEDURE Final(T1 : INTEGER)
PROCEDURE Update(BYREF R2 : STRING)
FUNCTION Confirm() RETURNS BOOLEAN

Module Setup will repeatedly call **three** of the modules.

Complete the structure chart to document the information given in the table.



8. O/N 25/P21/Q8/c

- 8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
<code>OKToBorrow()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - search the array for loan records for the specified student - output a suitable message to say whether the student may, or may not, borrow another book

- (c) It is decided to introduce a system of fines for books that have been borrowed for too long.

Two new requirements are defined:

1. Each loan has a maximum length, represented as a number of days.
2. Each book in the library will be assigned one of three categories. Each category has a different maximum loan length.

Record structure and program design changes are needed to meet these two requirements.

Outline the changes that are necessary to meet the **two** requirements.

[2]

9. O/N 25/P22/Q5/a,b

- 5** A program is developed to satisfy a specific customer requirement.

The project follows a program development life cycle model. This model divides the development process into several different stages.

- (a)** The table lists some of the development activities.

Complete the table by writing the name of the life cycle stage for each activity:

Activity	Name of life cycle stage
a structure chart is produced	
a program is modified to allow it to run on new hardware	
the programmer identifies the customer's requirements	
an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	

[4]

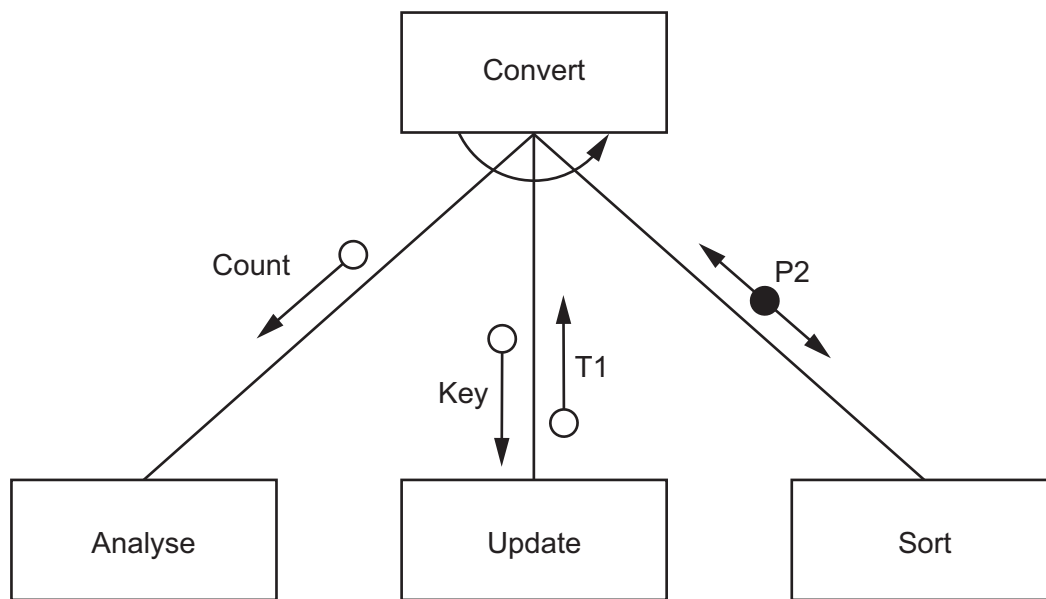
- (b)** Alpha and beta testing has been completed. The final testing stage is carried out by the customer.

Identify this final testing stage.

[1]

10. O/N 25/P22/Q7/a,b

7 The structure chart shows part of a program design:



(a) Explain the meaning of the curved arrow symbol in this structure chart.

[2]

(b) The program designer has noted that:

- Count is the number of elements in an array
- T1 is a value representing the number of elements in an array that have been modified
- Key is of type `STRING`

Write the pseudocode module headers for analyse, update and sort.

Analyse:

Update:

Sort:

[4]

11. O/N 25/P22/Q8/d

- 8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
StudentID	STRING	the unique ID of the student who has borrowed the book
BookID	STRING	the unique ID of the book being borrowed
OnLoan	BOOLEAN	TRUE if the book has not been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
<code>CountLoans()</code>	- called with a parameter of type <code>STRING</code> representing a <code>StudentID</code> - counts the number of books currently on loan to the specified student - counts the number of books that the student has already returned - output both counts together with a suitable message

- (d) It is decided to extend the program so that a new module `Reminder()` will send an email to the student three days before the book is due to be returned.

Outline the changes that will need to be made to the data stored and how this data will be used to generate the reminder email.

Data

Use

[2]

12. O/N 25/P23/Q5/a

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
Num[2] ← 2
Num[3] ← 5
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count <= 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called in the statement:

CALL Process(1)

[illegible]

[5]

13. O/N 25/P23/Q6/a

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (a) A function `Generate()` is written to take an original number as a parameter and to return the check digit.

Outline a test plan that could be used to fully test function `Generate()`

Assume that the parameter is valid.

[2]

14. O/N 25/P23/Q7/a,b

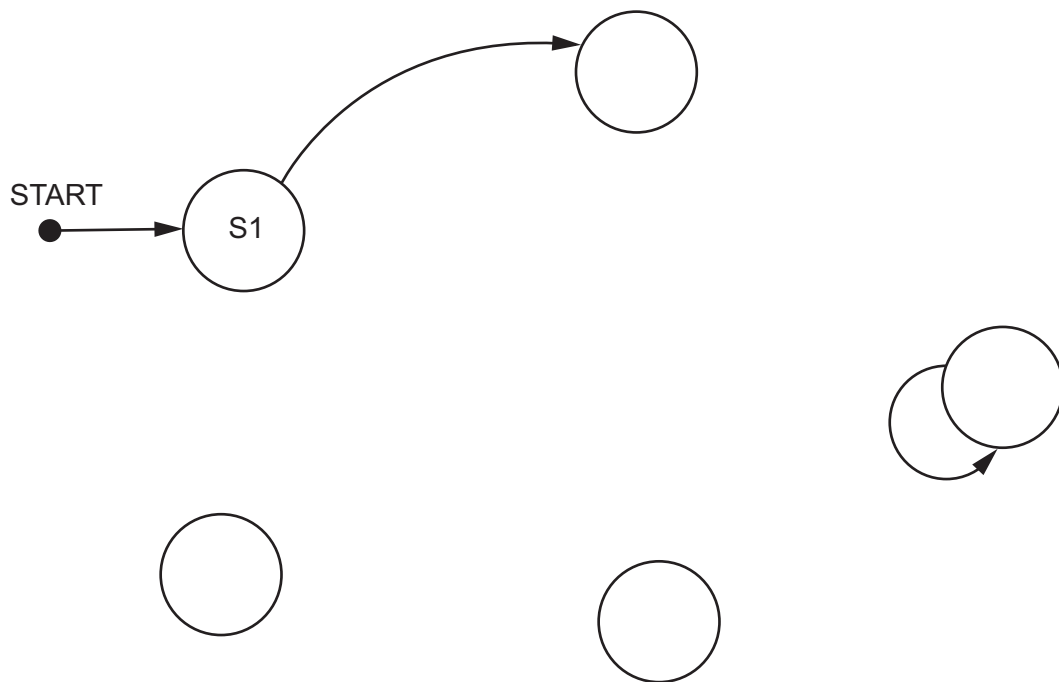
- 7 There are several different ways to express an algorithm during the design of a program.

- (a) One part of the program contains an algorithm which is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1		S2
S2	A2	X4	S3
S3	A1	X1	S3
S3	A2	X1	S3
S3	A3	X3	S4
S3	A4		S2
S4	A3	X4	S5
S4	A4	X4	S5
S4	A1		S2

Complete the state-transition diagram to represent the information given in the table:



[5]

- (b) A structure chart is used to document a different part of the program, made up of five modules.

Program notes:

- module Setup calls either module Restart, or module Confirm
- module Confirm takes a string as a parameter and returns an integer
- module Modify has no parameters and returns a Boolean
- module Update takes a string as a parameter
- module Restart repeatedly calls module Update followed by module Modify
- module Restart takes a string as a parameter that is passed by reference.

Draw a structure chart to represent the relationship between the **five** modules, including all parameters and return values.

[4]

15. M/J 25/P21/Q2/a,b,c

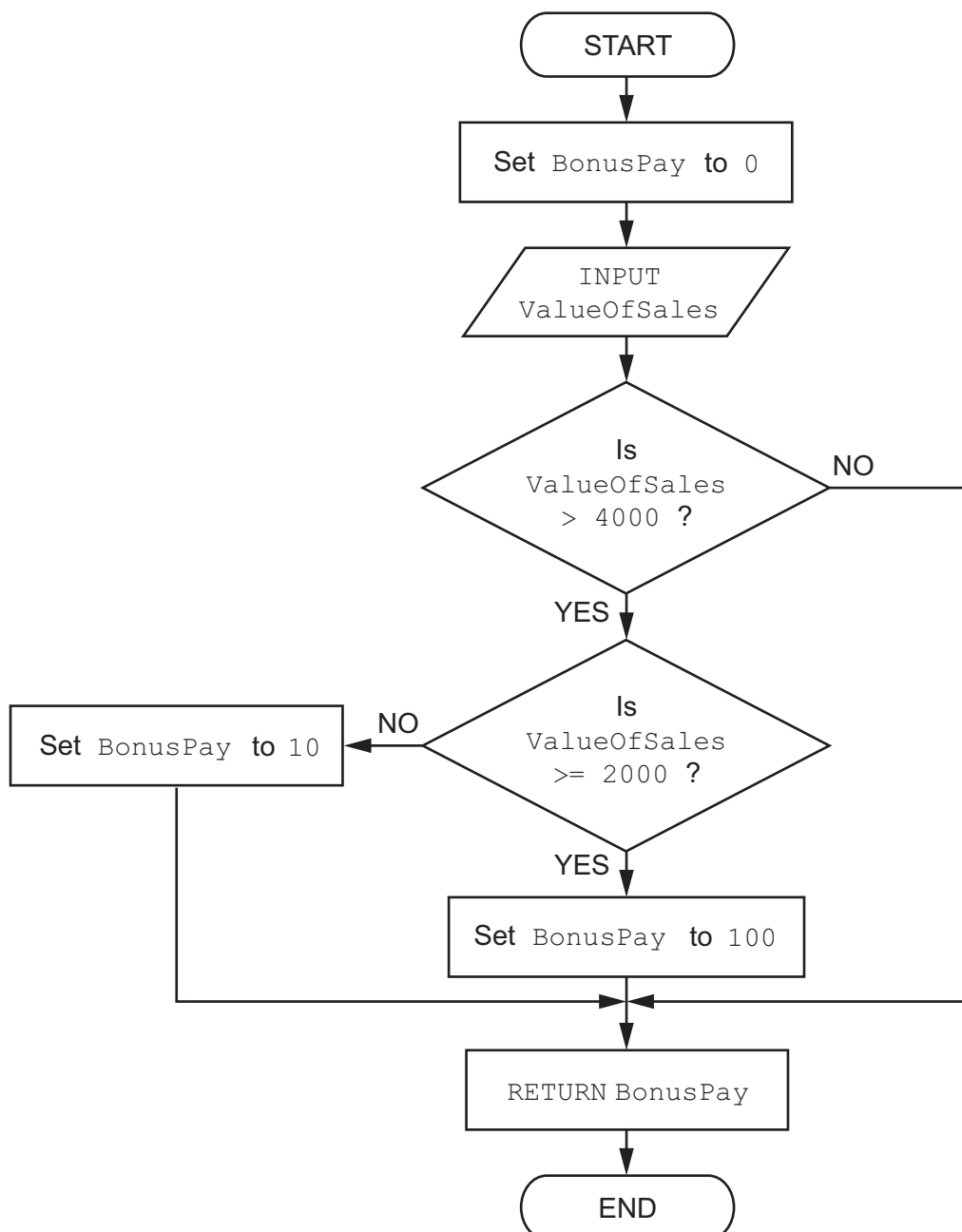
2 A program is being developed to calculate the pay of employees working for a company.

A function `CalculateBonus()` calculates bonus pay based on the value of sales.

Bonus pay is calculated as shown in the table.

Value of sales (in dollars)	Bonus pay (in dollars)
below 2000	0
between 2000 and 4000 inclusive	10
above 4000	100

A flowchart for the function `CalculateBonus()` has been designed.



(a) The flowchart contains logic errors.

One logic error is that `BonusPay` will **not** be set to 10 although the `ValueOfSales` input is between 2000 and 4000 inclusive.

(i) Explain why this error occurs.

[1]

(ii) Explain how this error could be corrected.

[1]

(b) There are different ways to reduce the risk of errors when developing the new program, such as the use of constants.

(i) State a value that could be replaced by a constant in the function `CalculateBonus()`

[1]

(ii) Explain how the use of constants helps to reduce the risk of programming errors.

[2]

(iii) One other way that can reduce the risk of errors when writing the program is the use of library routines.

Explain how library routines can reduce the risk of programming errors.

[1]

(c) All the logic errors have been corrected, and the program has been coded.

Identify and describe **two** other types of error that the program could contain.

Type of error

Description

Type of error

Description

[4]

16. M/J 25/P21/Q4/b

4 Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I <= 4                                //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J >= 0 AND Chars[J] > Key          //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(b) Complete the trace table by dry running the algorithm.

17. M/J 25/P21/Q6/a

- 6** A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

- (a)** A test plan is needed to fully test the module.

Complete the table. The first line has been completed for you.

Assume the sensors generate only integer values.

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted

[4]

18. M/J 25/P21/Q7/b(ii)

- 7** A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	- called with two integer parameters: - the number of coffees ordered - the current number of points - output a suitable message giving the number of free coffees - return a value for the new points total.

- (b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

`<CustomerID>,<Points>`

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at "100001" with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

`"100205,0"`

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
<code>AddNewCustomers()</code>	- called with an integer parameter representing the number of new customers to be added to the loyalty scheme - adds a new line, containing the required information, to the text file <code>Loyalty.txt</code> for each customer added to the loyalty scheme - outputs each new customer ID added to the loyalty scheme

- (ii) The requirements for `AddNewCustomers()` are changed. There may be no existing customers in the loyalty scheme.

Explain the changes that will need to be made to the module `AddNewCustomers()`

[4]

19. M/J 25/P22/Q1/a,b

1 A program is being developed to control the production line in a factory.

(a) A number of different program life cycles are available for the development of a program.

(i) Explain the need for different program development life cycles.

[1]

(ii) Coding is a stage in a program development life cycle.

State **one** consideration that would influence the choice of programming language.

[1]

(b) The program has been in use for a number of months and adaptive maintenance is required.

(i) Give **three** reasons why adaptive maintenance may be required.

1

2

3

[3]

(ii) As well as adaptive maintenance, other types of program maintenance may be needed.

Identify **one** other type of program maintenance.

[1]

20. M/J 25/P22/Q2/b

2 A program to calculate the pay of employees working for a company is being designed.

- (b)** One of the program modules used to calculate employees' weekly bonus pay has been completed. The amount of bonus pay they receive is based on the number of hours worked and the value of sales made as shown in the table.

Bonus pay and value of sales are both in dollars.

Hours worked	Value of sales	Bonus pay
between 1 and 40 inclusive	2000 or less	0
between 1 and 40 inclusive	above 2000	50
above 40	2000 or less	10
above 40	above 2000	100

- (i)** The module is tested using white-box testing.

State **two** tests, using valid data, that can be used to test different paths through the program.

Test one:

Hours worked

Value of sales

Bonus pay

Test two:

Hours worked

Value of sales

Bonus pay

[2]

- (ii)** The program to calculate pay uses a number of modules which are each called from different places in the program.

The program is to be tested using stub testing before all the program modules have been completed.

Describe stub testing.

[2]

- (iii)** The program has been completed and compiles successfully.

The program is tested using black-box testing.

Identify and describe **one** type of error that black-box testing could detect.

Type of error

Description

[2]

21. M/J 25/P22/Q5/a

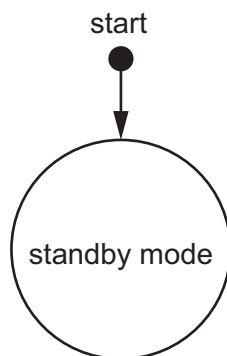
- 5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

- (a) Complete the state-transition diagram for the automated digital camera system.



[4]

22. M/J 25/P22/Q6/a

also Unit 10 (3 marks); this unit 4 marks

- 6 Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

Data		Pointer	
1	1018	1	7
2	1007	2	3
3	1010	3	1
4	1056	4	6
5	1092	5	-1
6	1062	6	5
7	1034	7	4
8	0	8	9
9	0	9	10
10	0	10	-1

Study the pseudocode:

```

PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
  DECLARE New, Current, Last : INTEGER
  CONSTANT NullPointer = -1
  New ← Unused
  Last ← NullPointer
  Current ← Start
  WHILE Current <> NullPointer AND Data[Current] < Value
    Last ← Current
    Current ← Pointer[Current]
  ENDWHILE
  Pointer[New] ← Pointer[Last]
  Pointer[Last] ← New
  Data[New] ← Value
ENDPROCEDURE

```

- (a) (i) Complete the trace table below by dry running the procedure `Place()` when it is called by the statement:

`CALL Place(1043, 2, 8)`

Value	Start	Unused	New	Last	Current

- (ii) Complete the diagram showing the contents of the global arrays `Data` and `Pointer` after the procedure `Place()` has run to completion when called as shown in part (a)(i).

	Data		Pointer
1	1018	1	
2	1007	2	
3	1010	3	
4	1056	4	
5	1092	5	
6	1062	6	
7	1034	7	
8		8	
9		9	
10		10	

[3]

23. M/J 25/P23/Q4/b

- 4 A programmer is developing a new stock control program for a shop owner to produce sales reports.

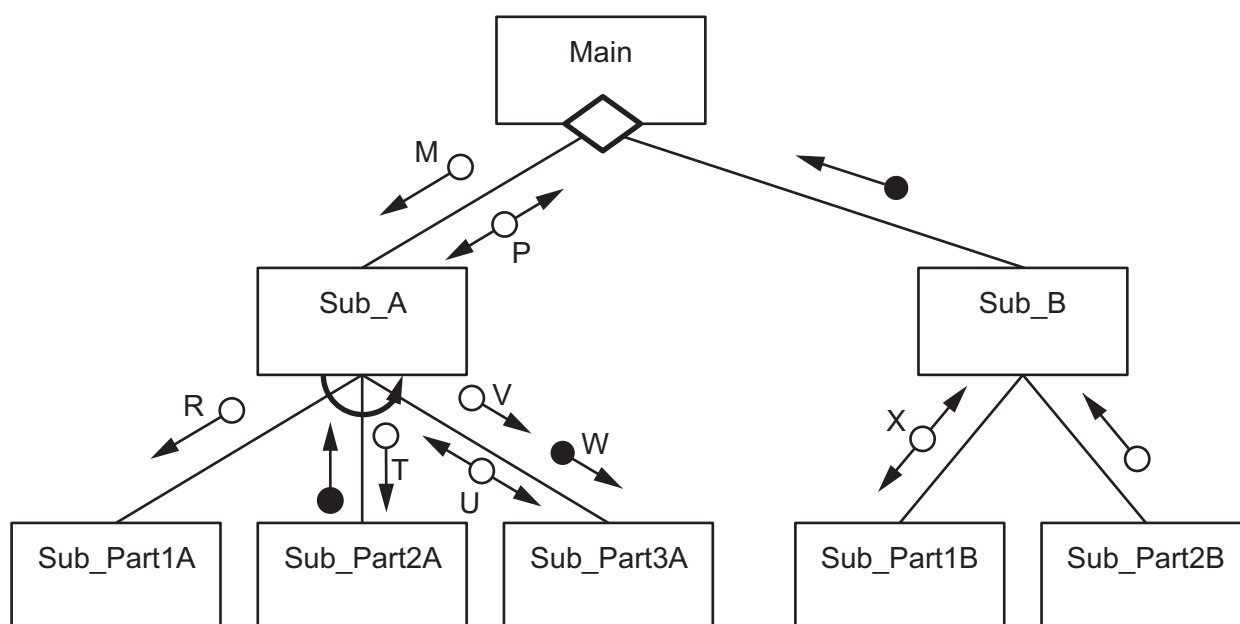
(b) Once the program has been completed, it is tested using the walkthrough method.

Describe the walkthrough method and explain how it can be used to identify errors.

[3]

24. M/J 25/P23/Q6/b,c

- 6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(b) Write the pseudocode to define the module headers:

Sub_Part1A
Sub_Part2A
Sub_Part3A

[5]

(c) The structure chart uses the two symbols shown.

Complete the table to explain what each symbol means and how the relevant modules in the structure chart are affected.

Symbol	Explanation
	
	

[3]

25. O/N 24/P21/Q1/a

- 1** A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25     // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

- (a)** The pseudocode contains a logical error.

Identify the error **and** suggest a correction.

Error

Correction

[2]

26. O/N 24/P21/Q3/c

- 3** An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

- (a)** The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file
2. Loop until
3. and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop,
8. Output `LineX`, `LineY`, `LineZ`

[4]

- (c)** The requirement changes, and the algorithm will now output three lines from the file, starting from a **given** line number.

The modified algorithm will be implemented as a function which will:

- be called with an integer parameter representing the given line number
- output three lines, starting at the given line
- return `TRUE` if the 3 lines are output, or `FALSE` if it was not possible to output the 3 lines.

Describe the changes that need to be made to **steps 2 to 8** of the algorithm given in part **(a)**.

[4]

27. O/N 24/P21/Q4/a

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (a) Identify the type of error that could occur if these checks are **not** carried out **and** state a cause of this error.

Type

Cause

[2]

28. O/N 24/P21/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

- (a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
The walkthrough method is used.	
An algorithm is implemented in a programming language.	
The client is interviewed about problems with the current system.	
The program is modified to run on new hardware.	
Records and file structures are defined.	

[5]

- (b) The program contains a validation function.

(i) The function will:

- take an integer value as a parameter
- return `TRUE` if the value is within the range 24 to 37, inclusive
- otherwise return `FALSE`.

Complete the table to define a test plan to thoroughly test the operation of the function.

Type of test data	Test data value	Expected result
Normal	30	TRUE

[4]

(ii) The function is to be tested on its own. When it is shown to work correctly the function will be combined with other modules and testing will continue.

Identify the type of testing that this represents.

[1]

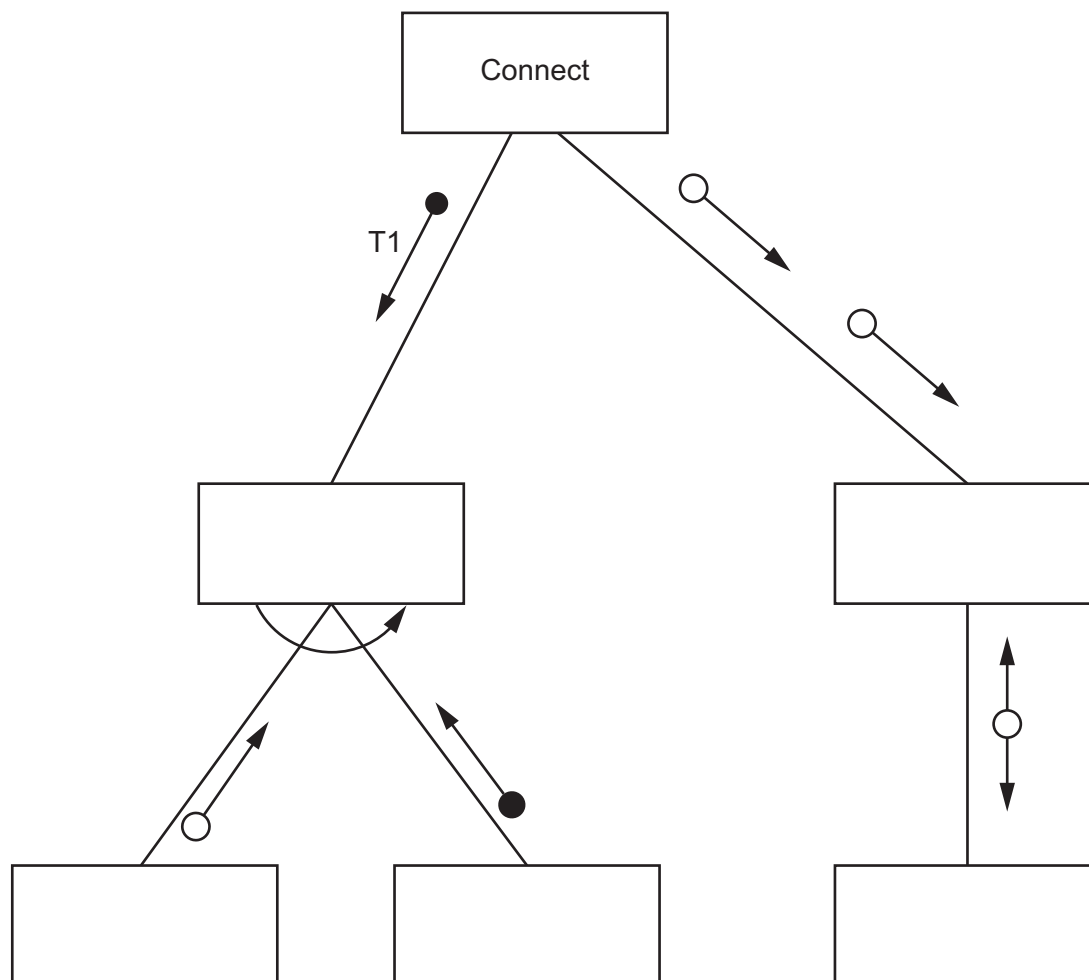
29. O/N 24/P21/Q7/a,b

7 A program contains six modules with headers as follows:

Pseudocode module header
PROCEDURE Connect ()
FUNCTION Activate (H1 : STRING, Code : INTEGER) RETURNS BOOLEAN
FUNCTION Sync (T1 : BOOLEAN, S2 : REAL) RETURNS INTEGER
PROCEDURE Initialise (BYREF ID : INTEGER, BYVAL CC : INTEGER)
FUNCTION Reset (RA : STRING) RETURNS INTEGER
FUNCTION Enable (SA : INTEGER) RETURNS BOOLEAN

Module Connect () will call either Activate () or Sync (). This is decided at run-time.

(a) Complete the structure chart for these modules.



[5]

(b) Explain the meaning of the curved arrow symbol used in the diagram in part (a).

[2]

30. O/N 24/P22/Q1/a

- 1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

(a) The error in the program needs to be corrected.

Identify the stage of the program development life cycle that this correction is made in.

[1]

31. O/N 24/P22/Q5/a,b

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN RetVal
ENDFUNCTION
```

- (a) Run-time errors can be generated in different ways. For example, a run-time error will be generated if a function is called with invalid parameters.

The pseudocode contains three statements that could generate a run-time error.

Write the **three** statements **and** explain how each could generate a run-time error.

Statement 1

Explanation

Statement 2

Explanation

Statement 3

Explanation

[3]

- (b) One type of run-time error can cause a program to stop responding ('freezing').

Identify a particular type of programming construct that can generate this type of error **and** explain why it occurs.

Construct

Explanation

[2]

32. O/N 24/P22/Q6/b

- 6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	"Cheese"
2	"Onion"
3	"Salmon"
4	"Anchovies"
5	"Peanut Butter"

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	"White"
2	"Brown"
3	"Pitta"

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

- (b) The owner decides that some combinations of fillings do not go well together. For example, anchovies and peanut butter.

Describe how the design could be changed to prevent certain combinations being selected.

33. O/N 24/P22/Q7/c,d

- 7** A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

- (c)** It is decided to adopt a formal program development life cycle model for the development of the new module.

The analysis of the new module is complete and the project moves on to the design stage. During this stage all the necessary algorithms and module designs will be defined.

State **three other** items that will be defined for the new module during the design stage.

1

2

3

[3]

- (d)** Part of the coffee shop program contains three program modules as follows:

- Module `Init()` has no parameters and returns a Boolean.
- Module `Reset()` takes a string as a parameter and returns an Integer.
- Module `Check()` repeatedly calls `Init()` followed by `Reset()`.

Draw a structure chart to represent the relationship between the **three** modules, including all parameters and return values.

[3]

34. O/N 24/P23/Q5/a,b

- 5 A software developer follows a program development life cycle. The life cycle divides the development process into various stages.

(a) The following table lists some development activities.

Complete the table by writing the name of the life cycle stage for each activity.

Activity	Name of life cycle stage
A compiler is used.	
A program that has been released for general use is modified.	
The dry run method is used.	
The program structure is specified.	

[4]

- (b) A software developer has written modules `Test_A()` and `Test_B()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has to stop.

Identify a method that can be used to continue testing the main program **before** the errors in these modules have been corrected **and** describe how this would work.

Method

Description

[3]

35. O/N 24/P23/Q7/a

- 7 A coffee shop owner wants to introduce a computerised loyalty card system.

A programmer discusses the details of the system with the shop owner.

- (a) Identify the stage of the program development life cycle that this discussion is part of **and** give a document that will be produced during this stage.

Stage

Document

[2]

36. M/J 24/P21/Q5/a,b

- 5 A global 1D array of strings contains three elements which are assigned values as shown:

```
Data[1] ← "aaaaaa"
Data[2] ← "bbbbbb"
Data[3] ← "cccccc"
```

Procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```

PROCEDURE Process(Format : STRING)
    DECLARE Count, Index, L : INTEGER
    DECLARE Result : STRING
    DECLARE C : CHAR

    Result ← "*****"

    FOR Count ← 1 TO LENGTH(Format) STEP 2
        C ← MID(Format, Count, 1)
        L ← STR_TO_NUM(MID(Format, Count + 1, 1))

        Index ← (Count + 1) DIV 2

        CASE OF C
            'X' : Result ← TO_UPPER(Data[Index])
            'Y' : Result ← TO_LOWER(Data[Index])
            'Z' : Result ← "***" & Data[Index]
        ENDCASE

        Data[Index] ← LEFT(Result, L)
    NEXT Count

ENDPROCEDURE

```

(a) Complete the trace table by dry running the procedure when it is called as follows:

CALL Process("X3Y2W4")

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]

- (b) The procedure is to be modified. If variable `C` is assigned a value other than 'X', 'Y' or 'Z', then procedure `Error()` is called and passed the value of variable `C` as a parameter.

This modification can be implemented by adding a **single line** of pseudocode.

- (i) Write the single line of pseudocode.

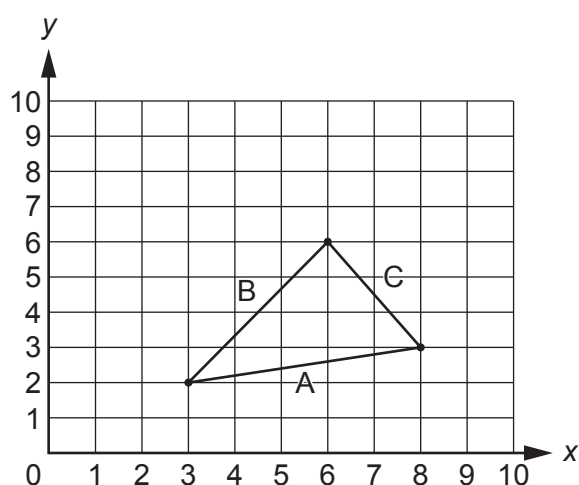
[1]

- (ii) State where this new line should be placed.

[1]

37. M/J 24/P21/Q6/b

- 6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



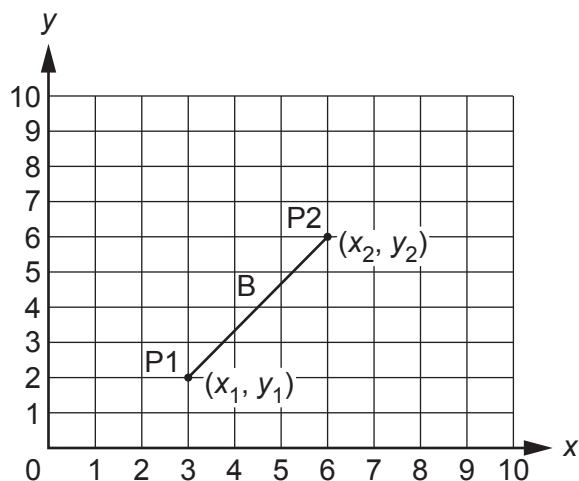
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator '^' represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
    ENDFUNCTION
```

[6]

(b) The test used to check if a triangle is right-angled can be written in two ways:

$$A^2 = B^2 + C^2$$

or

$$A = \sqrt{B^2 + C^2}$$

The symbol $\sqrt{}$ represents the square root operation. For example, $\sqrt{81} = 9$

A new function `SQRT()` is written to perform the square root operation. The function takes an integer number as a parameter and returns a positive real value representing the square root of the number.

During testing it is found that the `SQRT()` function returns a value that is only accurate to 4 decimal places.

For example, `SQRT(25)` returns 5.0000125 rather than the correct value of 5.0

The function `IsRA()` from part (a) is modified to use the new `SQRT()` function to test if a triangle is right-angled.

Describe a problem that might occur when using the modified `IsRA()` function **and** suggest a solution that still allows the `SQRT()` function to be used.

Problem

Solution

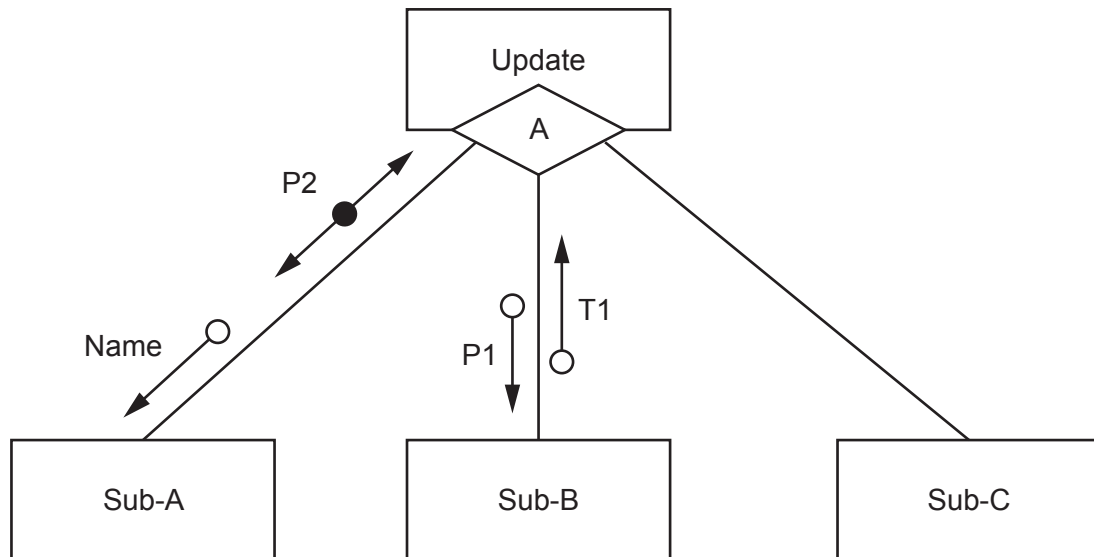
[2]

38. M/J 24/P21/Q7/b

- 7 A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

(b) The structure chart illustrates part of the membership program:



Data item notes:

- Name contains the name of a club member
- P1 and T1 are of type real.

(i) Explain the meaning of the diamond symbol (labelled with the letter A) in the chart.

[2]

(ii) Write the pseudocode module headers for Sub-A and Sub-B.

Sub-A

Sub-B

[4]

39. M/J 24/P22/Q5/a(i),b

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(a) (i) The pseudocode contains **two** syntax errors and **one** other error.

Identify the errors.

Syntax error 1

Syntax error 2

Other error

[3]

(b) After correcting all syntax errors, the pseudocode is translated into program code which compiles without generating any errors.

When the program is executed it unexpectedly stops responding.

Identify the type of error that has occurred.

[1]

40. M/J 24/P22/Q7/b

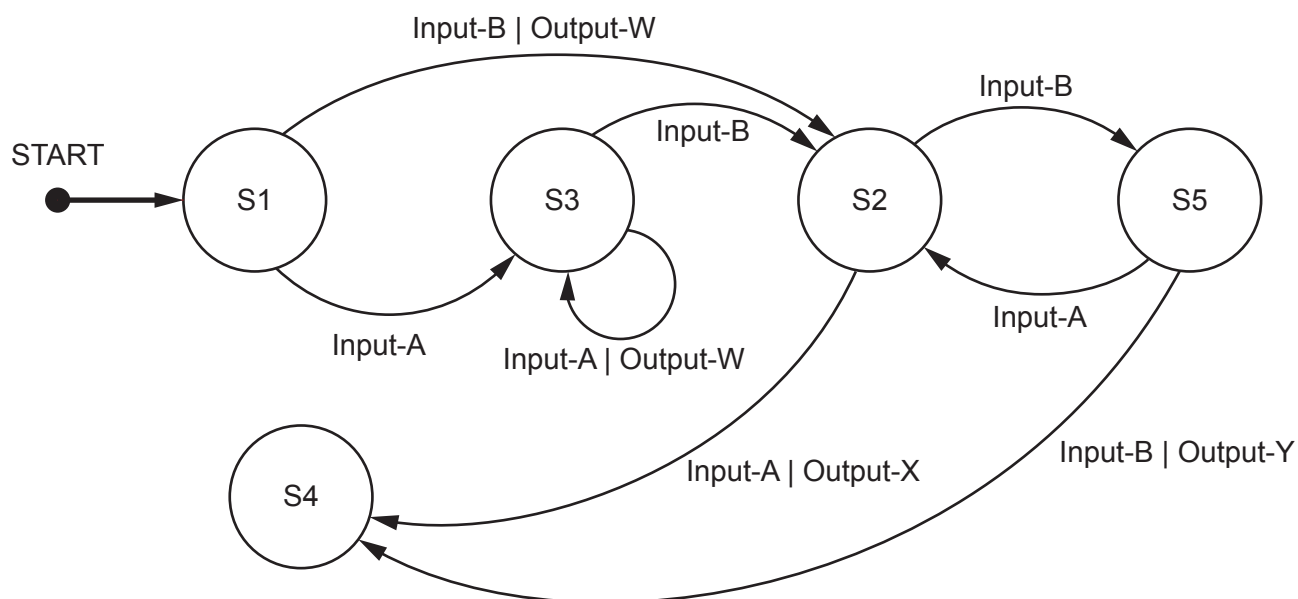
7 A fitness club has a computerised membership system.

The system stores information for each club member: name, home address, email address, mobile phone number, date of birth and exercise preferences.

Many classes are full, and the club creates a waiting list for each class. The club adds details of members who want to join a class that is full to the waiting list for that class.

When the system identifies that a space is available in one of the classes, a new module will send a text message to each member who is on the waiting list.

(b) A different part of the program is represented by the following state-transition diagram.



(i) Complete the table to show the inputs, outputs and next states.

Assume that the current state for each row is given by the 'Next state' on the previous row. For example, the first Input-A is made when in state S1.

If there is no output for a given transition, then the output cell should contain 'none'.

The first two rows have been completed.

Input	Output	Next state
		S1
Input-A	none	S3
	Output-W	
	none	
Input-B		
Input-A		
		S4

[5]

(ii) Identify the input sequence that will cause the minimum number of state changes in the transition from S1 to S4.

[1]

41. M/J 24/P23/Q6/b(ii)

- 6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.



(b) The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

(ii) State **one** benefit of increasing the number of steps.

[1]



42. M/J 24/P23/Q7/a(i),b

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (a) (i) Identify the modules that would be implemented as functions.

[1]

- (b) Draw a structure chart to show the relationship between the seven modules and the parameters passed between them.

[6]

**43. O/N 23/P21/Q5/a,b**

- 5 An algorithm is designed to find the smallest numeric value from an input sequence and count how many numeric values have been input.

An example of an input sequence is:

23, AB56, 17, 23ZW, 4, 10, END

Numeric input values are all integers and non-numeric input is ignored, except for the string "END" which is used to terminate the sequence.

The algorithm is expressed in pseudocode as shown:

```
DECLARE NextInput : STRING
DECLARE Min, Count, Num : INTEGER

Min ← 999
Count ← 0

REPEAT
    INPUT NextInput
    IF IS_NUM(NextInput) = TRUE THEN
        Num ← STR_TO_NUM(NextInput)
        IF Num > Min THEN
            Min ← Num
        ENDIF
        Count ← Count & 1
    ENDIF
UNTIL NextInput ← "END"

OUTPUT "The minimum value is ", Min, " and the count was ", Count
```

(a) The pseudocode contains three errors due to the incorrect use of **operators**.

Identify each error **and** state the correction required.

1

2

3

[3]

(b) The operator errors are corrected and the algorithm is tested as follows:

The input sequence:

18, 4, ONE, 27, 189, ERIC, 3, 65, END

produces the output:

The minimum value is 3 and the count was 6

The algorithm is tested with a different test data sequence. The sequence contains a mix of integer and non-numeric values. It is terminated correctly but the algorithm produces unexpected results.

(i) Explain the problem with the algorithm.

[2]

(ii) Give a sequence of **four** test data values that could be input to demonstrate the problem.

Value 1

Value 2

Value 3

Value 4

[2]

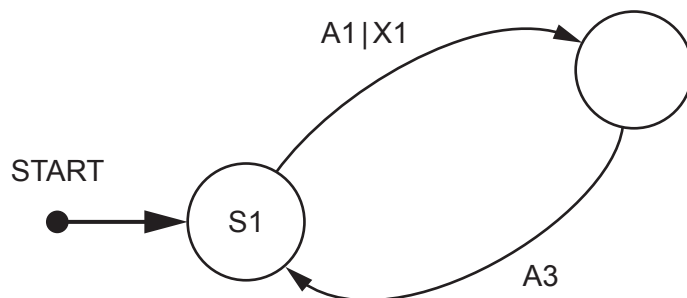
44. O/N 23/P21/Q7

7 An algorithm is represented by a state-transition diagram.

The table shows the inputs, outputs and states for the algorithm:

Current state	Input	Output	Next state
S1	A1	X1	S2
S2	A3	none	S1
S2	A2	X4	S5
S5	A1	X1	S5
S5	A4	X2	S2
S5	A3	none	S3
S1	A9	X9	S3
S3	A9	X9	S4

Complete the state-transition diagram to represent the information given in the table.



[5]

45. O/N 23/P21/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "****"

(c) One of the text files to be sent contains several blank lines (lines that do not contain any text).

(i) Explain why this is a problem.

[2]

(ii) Explain how the message format could be changed to allow a blank line to be sent.

[2]

46. O/N 23/P22/Q4/b

4 A procedure `Count ()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(b) The procedure `Count ()` is to be tested.

Typical test data would consist of odd and even values, for example:

23, 5, 64, 100, 2002, 1, 8, 900, 99

The purpose of this test would be to test a typical mix of even and odd values and check the totals.

Give **three** test data sequences that may be used to test **different** aspects of the procedure.

Do **not** include invalid data.

Sequence 1:

Test data

Purpose of test.

Sequence 2:

Test data

Purpose of test.

Sequence 3:

Test data

Purpose of test.

[3]

47. O/N 23/P22/Q5

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 1  
Mix[2] ← 3  
Mix[3] ← 4  
Mix[4] ← 2
```

A procedure `Process ()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE Value, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  REPEAT
    Value ← Mix[Index]
    Mix[Index] ← Mix[Index] - 1
    Index ← Value
    Count ← Count + 1
  UNTIL Count = 5

  Mix[4] ← Count * Index

ENDPROCEDURE
```

Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(3)
```

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]

48. O/N 23/P22/Q7/a,b

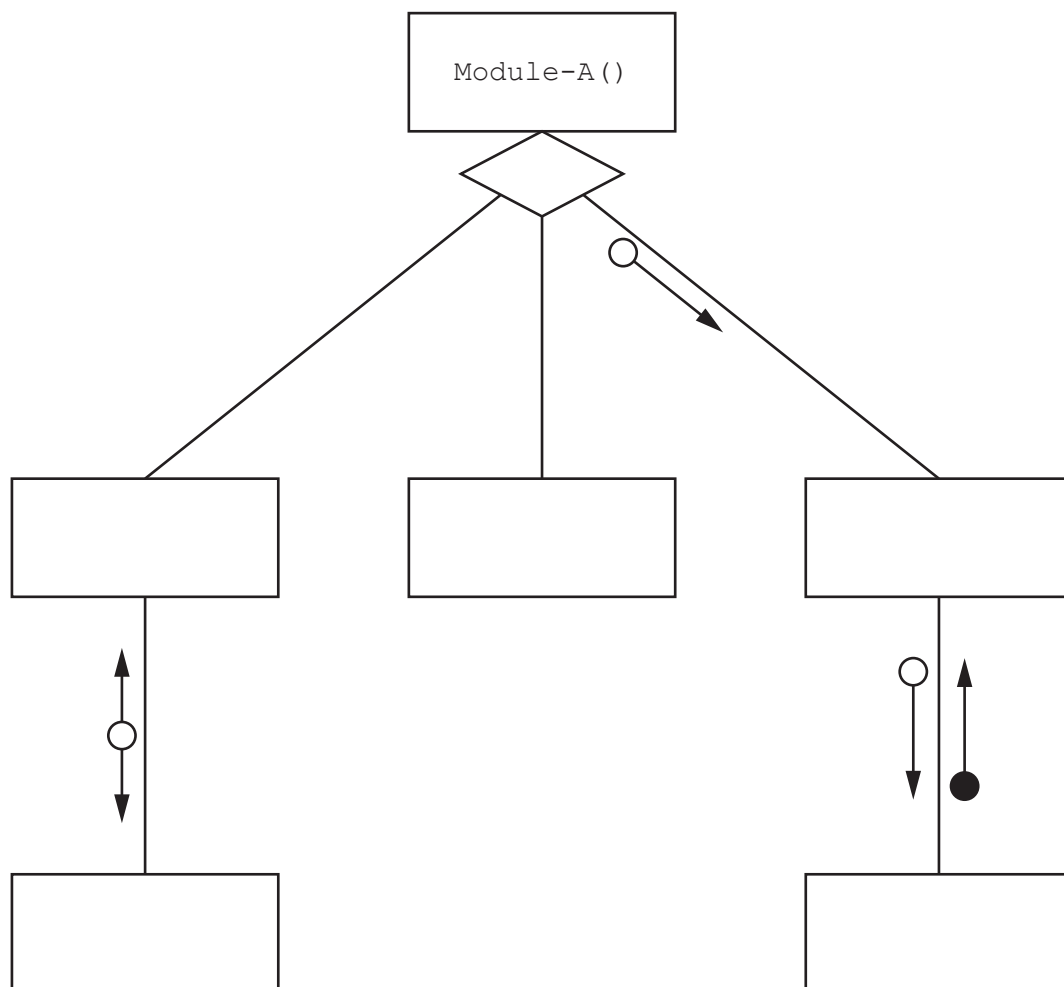
7 A program contains six modules:

Pseudocode module header
PROCEDURE Module-A()
PROCEDURE Module-X(T1 : INTEGER, S2 : REAL)
PROCEDURE Reset(BYREF Code : INTEGER)
FUNCTION Restore(OldCode : INTEGER) RETURNS BOOLEAN
FUNCTION Module-Y(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Module-Z(SA : INTEGER) RETURNS INTEGER

Module-X() **calls** Reset()

Module-Y() **calls** Restore()

(a) Complete the structure chart for these modules.



[4]

(b) Explain the meaning of the diamond symbol as used in the diagram in part (a).

[2]

49. O/N 23/P23/Q1/d

- 1 A program is being developed in pseudocode before being converted into a programming language.
- (d) The hardware that runs the program is changed and the program needs to be modified so that it works with the new hardware.

Identify the type of maintenance that this represents **and** give **one other** reason why this type of maintenance may be needed.

Type

Reason

[2]

50. O/N 23/P23/Q5

- 5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 4
Mix[2] ← 2
Mix[3] ← 3
Mix[4] ← 5
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Start : INTEGER)
  DECLARE Value, Index, Total : INTEGER

  Index ← Start
  Total ← 0

  WHILE Total < 20
    Value ← Mix[Index]
    Total ← Total + Value

    IF Index < 4 THEN
      Mix[Index] ← Mix[Index] + Mix[Index+1]
    ELSE
      Mix[Index] ← Mix[Index] + Mix[1]
    ENDIF
    Index ← (Value MOD 4) + 1

  ENDWHILE

  Mix[1] ← Total * Index

ENDPROCEDURE
```

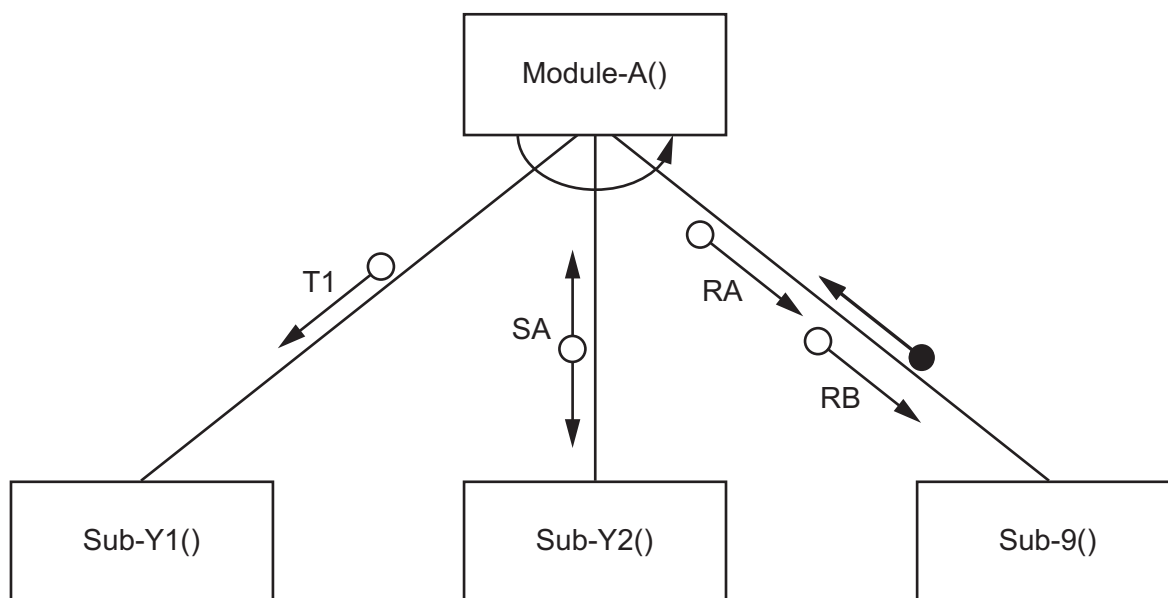
Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(2)
```

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]

51. O/N 23/P23/Q7/a

7 A structure chart shows the modular structure of a program:



(a) Explain the meaning of the curved arrow symbol which begins and ends at Module-A().

[2]

52. O/N 23/P23/Q8/b,d

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by <code>GetData()</code> to the file • repeats until the data field is "*****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

- (b) The use of the string "*****" as explained in the module description for `ReceiveFile()` may cause a problem.

Explain the problem and suggest a solution.

Problem

Solution

[3]

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing a port number - transmits the message using the given port
<code>Chat()</code>	- takes two parameters: - a string representing a Destination ID - an integer representing a port number - extracts data from a received message using <code>GetData()</code> and outputs it - forms a message using data input by the user and sends it using <code>Transmit()</code> - repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

`<STX><DestinationID><SourceID><Data><ETX>`

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

[7]

(d) Module `GetData()` returns the data field from a message that has been received. If no message is available, the module waits until one has been received.

Explain the limitation of this on module `Chat()` from part (c).

Describe a modification to `GetData()` to address this limitation.

Limitation

Modification

[3]

53. M/J 23/P21/Q5/a,b

- 5** Several companies are developing websites to market a new type of games console. The company that is first to create a website that can demonstrate the interactive features of the games console will have an advantage over the others. The requirements for the website are likely to change as more information about the features of the console are made available.

One company has decided to develop their website using a program development life cycle based on the waterfall model.

- (a) (i)** Give **two** reasons why this may **not** be the most appropriate model to use in this case.

Reason 1

Reason 2

[2]

- (ii)** Identify a **more appropriate** program development life cycle model for this scenario.

[1]

- (b)** The website has been running in test mode for several weeks.

Identify **and** describe a final stage of testing that should take place before the website is made available to all customers.

Stage

Description

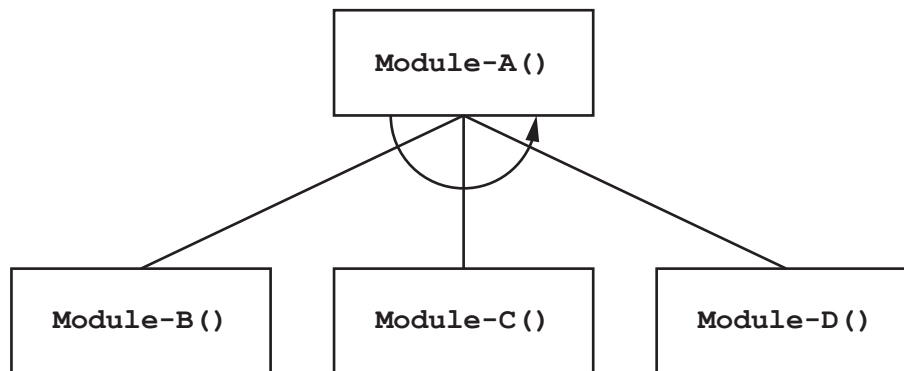
[3]

54. M/J 23/P21/Q7/b

- 7** A school has a computerised library system that allows students to borrow books for a fixed length of time. The system uses text files to store details of students, books and loans.

A new module is to be written which will generate emails to each student who has an overdue book.

- (b)** The program designer produces a structure chart for the new module. Part of the structure chart is shown:



- (i)** Explain the relationship between the four modules shown.

[2]

- (ii)** Two new modules are added: `Module-X()` and `Module-Y()`.

- `Module-X()` has no parameters.
- `Module-Y()` will take a string and a real number as parameters and return a Boolean value.
- `Module-D()` will call either `Module-X()` or `Module-Y()`.

Draw **only** the part of the structure chart that represents the relationship between `Module-X()`, `Module-Y()` and `Module-D()`.

[3]

55. M/J 23/P21/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

- (b) A new module has been defined:

Module	Description
<code>CheckNewItem()</code>	- called with a parameter of type string representing a line of item information - checks to see whether an item with the same <code>ItemNum</code> already exists in the file - returns <code>TRUE</code> if the <code>ItemNum</code> is not already in the file, otherwise returns <code>FALSE</code>

Write **efficient** pseudocode for module `CheckNewItem()`.

[7]

- (c) The program modules `SuppExists()`, `IsNewSupp()` and `CheckNewItem()` are part of a group of modules that are combined to create a complete stock control program.

Each module in the program is tested individually during development and is debugged as necessary. It is then added to the program and further testing performed.

(i) Identify this method of testing.

[1]

(ii) One of the modules does not work properly when it is added to the program.

Describe a testing method that can be used to address this problem so that testing can continue and other modules can be added.

[2]

56. M/J 23/P22/Q5/a,b,d

- 5** A programmer has produced the following pseudocode to output the square root of the numbers from 1 to 10.

Line numbers are for reference only.

```
10 DECLARE Num : REAL
11 Num ← 1.0
...
40 REPEAT
41     CALL DisplaySqrt(Num)
42     Num ← Num + 1.0
43 UNTIL Num > 10
...
50 PROCEDURE DisplaySqrt(BYREF ThisNum : REAL)
51     OUTPUT ThisNum
52     ThisNum ← Sqrt(ThisNum)           // Sqrt returns the square root
53     OUTPUT " has a square root of ", ThisNum
54 ENDPROCEDURE
```

The pseudocode is correctly converted into program code.

Function `Sqrt()` is a library function and contains no errors.

The program code compiles without errors, but the program gives unexpected results. These are caused by a design error.

- (a)** Explain why the program gives unexpected results.

[3]

- (b)** Explain why the compiler does **not** identify this error.

[1]

- (d)** The pseudocode is converted into program code as part of a larger program.

During compilation, a complex statement generates an error.

The programmer does not want to delete the complex statement but wants to change the statement so that it is ignored by the compiler.

State how this may be achieved.

[1]

57. M/J 23/P22/Q7/b

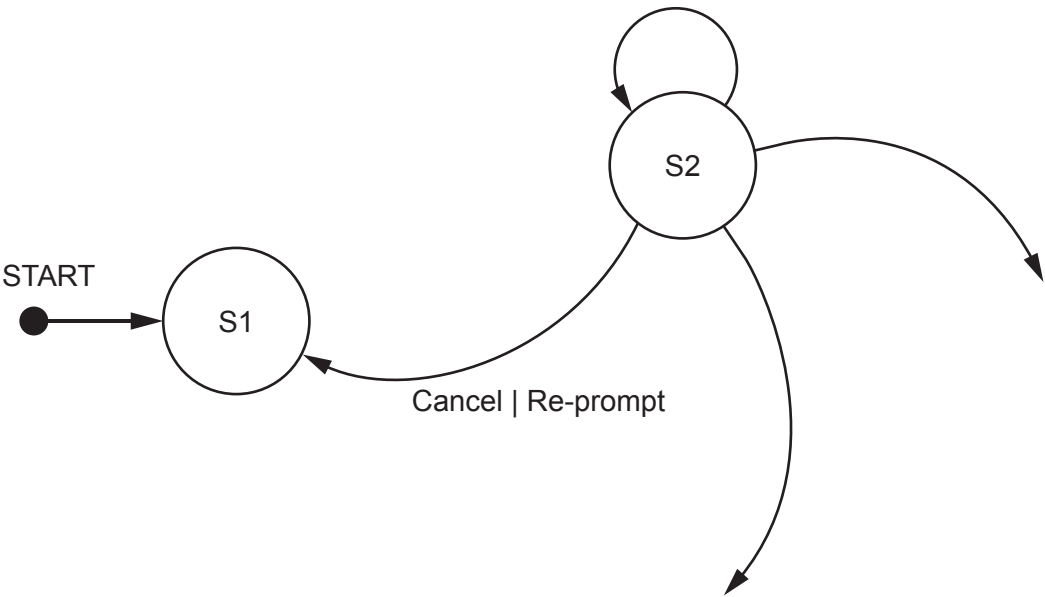
- 7 A computer system for a shop stores information about each customer. The items of information include name and address (both postal and email) together with payment details and order history. The system also stores the product categories they are interested in and how they would like to be contacted.
- (b) The program includes a module to validate a Personal Identification Number (PIN). This is used when customers pay for goods using a bank card.

A state-transition diagram has been produced for this module.

The table show the inputs, outputs and states for this part of the program:

Current state	Input	Output	Next state
S1	Input PIN		S2
S2	Re-input PIN	Display error	S2
S2	Cancel	Re-prompt	S1
S2	Valid PIN	Enable payment	S4
S2	Too many tries	Block Account	S3

Complete the state-transition diagram to represent the information given in the table.



58. M/J 23/P22/Q8/c

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (c) The program contains modules `SuppExists()` and `CheckSupplier()`. These have been written but contain errors. These modules are called from several places in the main program and testing of the main program (integration testing) has had to stop.

Identify a method that can be used to continue testing the main program before the errors in these modules have been corrected **and** describe how this would work.

Method

Description

59. M/J 23/P23/Q1/b

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```
10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100 FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103 NEXT PCount
```

- (b) The pseudocode contains two errors. One error is that variable `PCount` has not been declared.

Identify the **other** error **and** state the line number where it occurs.

Error

Line number

[2]

60. M/J 23/P23/Q5/a,b

- 5 A program is designed, coded and compiled without errors. The compiled code is sent for testing.

- (a) The program will be tested using the walkthrough method.

Additional information will be needed before this method can be used.

Identify this additional information **and** explain why it is needed.

Additional information

Explanation

[3]

- (b) Testing is completed and the program is made available to users.

Some time later, changes are made to the program to improve the speed of response.

State the type of maintenance that has been applied to the program.

[1]

61. M/J 23/P23/Q7/b

- 7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

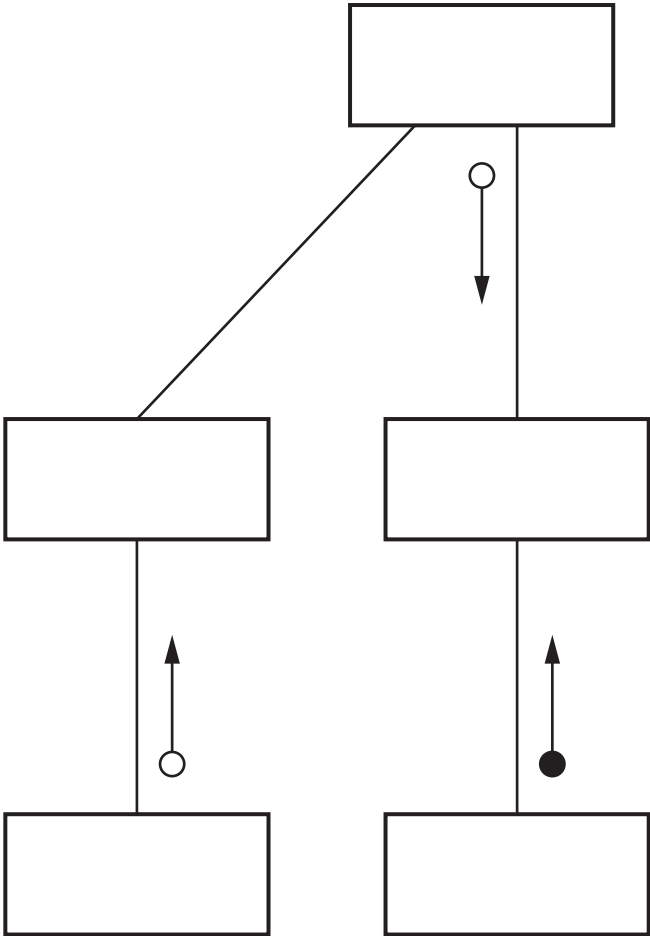
A program helps the staff to manage the borrowing of books.

(b) Part of the library program contains program modules with headers as follows:

Pseudocode module header
PROCEDURE Module-X()
PROCEDURE Module-Y(BYREF RA : INTEGER, SA : REAL)
PROCEDURE Overlay()
FUNCTION Replace(RA : INTEGER, RB : BOOLEAN) RETURNS BOOLEAN
FUNCTION Reset(TA : STRING) RETURNS INTEGER

Module-X() and Module-Y() are both called from module Overlay().

Complete the structure chart.



62. M/J 23/P23/Q8/c

- 8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
SupplierCode	3 alphabetic characters	code to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
<code>ChangeSupp()</code>	- called with two parameters <code>Code1</code> and <code>Code2</code> of type string that represent valid supplier codes - creates a new file <code>NewStock.txt</code> from the contents of the file <code>Stock.txt</code> where any reference to <code>Code1</code> is replaced by <code>Code2</code> - returns a count of the number of items that have had their supplier code changed

(b) A new module is required:

Module	Description
<code>Report_1()</code>	- takes a parameter of type string that represents a <code>SupplierCode</code> - searches the <code>Stock.txt</code> file for each line of item information that contains the given <code>SupplierCode</code> - produces a formatted report of items for the given <code>SupplierCode</code>, for example, for supplier DRG, the output could be: <pre>Report for Supplier: DRG Item Description 1234 USB Printer Cable 3m 1273 32GB USB Flash Drive 1350 Mouse Mat 320 x 240mm Number of items listed: 3</pre>

Write pseudocode for module `Report_1()`.

- (c) The format of the output from module `Report_1()` from **part (b)** is changed. The number of items listed is moved to the top of the report as shown in the example:

```
Report for Supplier: DRG
Number of items listed: 3

Item          Description

1234          USB Printer Cable 3m
1273          32GB USB Flash Drive
1350          Mouse Mat 320 x 240mm
```

- (i) Explain why this new layout would increase the complexity of the algorithm.

[2]

- (ii) The algorithm will be modified to produce the report in the new format. The modified algorithm will be implemented so that the file `Stock.txt` is only read once.

Describe the modified algorithm.

[3]

63. O/N 22/P21/Q6/b

- 6 The following pseudocode algorithm attempts to check whether a string is a valid email address.

```
FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
           IF Ats > 1 THEN
             Valid ← FALSE
           ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF
```



```

UNTIL Index > LENGTH(InString) OR Valid = FALSE

IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
ENDIF

RETURN Valid

ENDFUNCTION

```

(b) (i) Complete the trace table by dry running the function when it is called as follows:

Result ← IsValid("Liz.123@big@net")

Index	NextChar	Dots	Ats	Others	Valid

[5]

(ii) State the value returned when IsValid() is called using the expression shown in part (b)(i).

[1]

64. O/N 22/P21/Q7/b

7 A simple arithmetic expression is stored as a string in the format:

`<Value1><Operator><Value2>`

An operator character is one of the following: '+' '-' '*' '/'

Example arithmetic expression strings:

`"803+1904"`

`"34/7"`

(b) A string representing an arithmetic expression could be in the correct format but be impossible to evaluate.

Give an example of a correctly formatted string and explain why evaluation would be impossible.

Example string

Explanation

[2]

65. O/N 22/P21/Q8/d

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	- takes the name of an existing project file as a parameter of type string - returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	- takes a line from a project file as a parameter of type string - returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	- takes two parameters: - the name of a project file as a string - the maximum number of errors as an integer - uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid - calls <code>CheckLine()</code> for each line in the project file - counts the number of errors - outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (d) Module `CheckLine()` includes a check for syntax errors.

Two examples of syntax error that **cannot** be detected from examining a **single** line are those involving selection and iteration.

Give **two other** examples.

1

2

[2]

66. O/N 22/P22/Q1/b

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc (Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers `Count` and `Message` **and** explain their use.

Term

Use

[2]

- (b) The algorithm in **part (a)** is part of a program that will be sold to the public.
All the software errors that were identified during in-house testing have been corrected.

Identify **and** describe the additional test stage that may be carried out before the program is sold to the public.

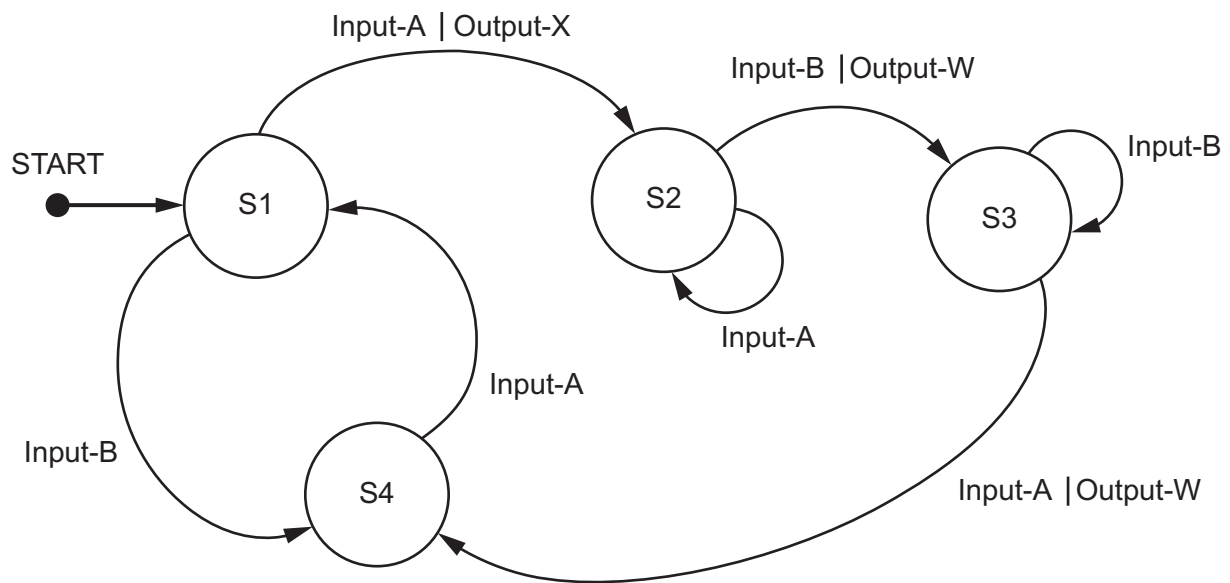
Test stage

Description

[4]

67. O/N 22/P22/Q2/b

(b) Examine the following state-transition diagram.



Complete the table to show the inputs, outputs and next states.

Input	Output	Next state
		S1
Input-A		
		S2
	Output-W	
	Output-W	

[4]

68. O/N 22/P22/Q6/b

- 6 (a) The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter

- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

[7]

- (b) A program is written to allow a user to input a sequence of values to be checked using the function `FindBaseNumber()`.

The user will input one value at a time. The variable used to store the user input has to be of type string because the user will input 'End' to end the program.

Valid input will be converted to an integer and passed to `FindBaseNumber()` and the return value will be output.

Complete the table by giving **four** invalid strings that may be used to test distinct aspects of the required validation. Give the reason for your choice in each case.

Input	Reason for choice

69. O/N 22/P23/Q1/b,c,d

1 A program is required for a shopping website.

(b) The designer considers the use of a development life cycle to split the development of the website into several stages.

(i) State **one** benefit of a development life cycle when developing the website.

[1]

(ii) Analysis is one stage of a development life cycle.

State **one** document that may be produced from the analysis stage of the website project.

[1]

(c) The program will be developed using the Rapid Application Development (RAD) life cycle.

(i) State **one** principle of this life cycle.

[1]

(ii) Give **two** benefits and **one** drawback of its use compared to the waterfall life cycle.

Benefit 1

Benefit 2

Drawback

[3]

(d) Adaptive maintenance needs to be carried out on the website program.

Give **two** reasons why adaptive maintenance may be required.

1

2

[2]

70. O/N 22/P23/Q5/d

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (d) The program that includes `Unpack()` is tested using the walkthrough method.

Describe this method **and** explain how it can be used to identify an error.

[3]

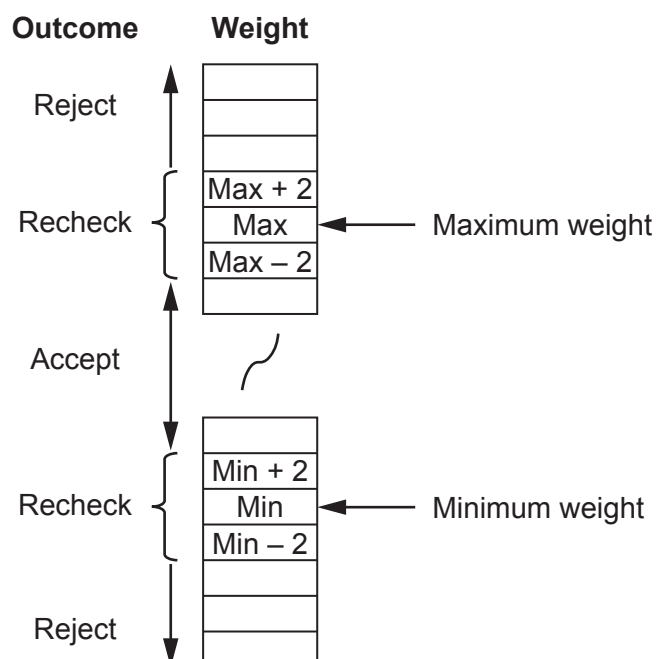
71. O/N 22/P23/Q6/a

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (a) Complete the following test plan for **five** tests that could be performed on function `Status()`. The tests should address all possible outcomes.

Test number	Component weight	Min	Max	Expected return value
1				'A'
2				
3				
4				
5				

[5]

72. O/N 21/P21/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

- (a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

- (ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

73. O/N 21/P21/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

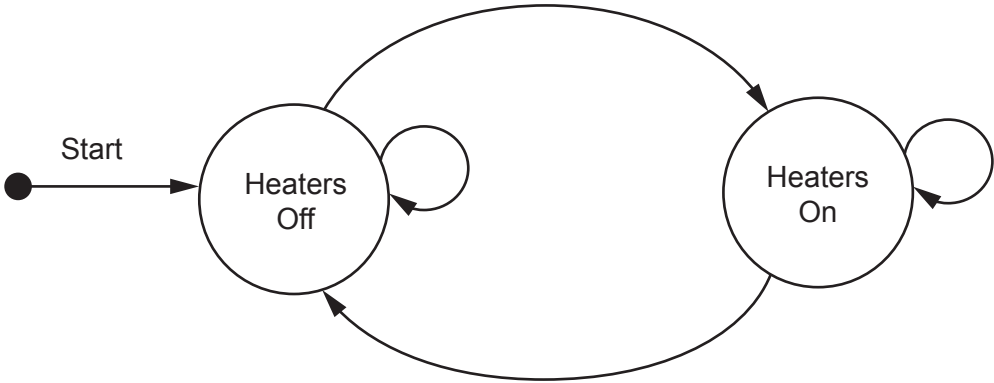
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

74. O/N 21/P21/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

75. O/N 21/P22/Q2/b

- (b) A software company is working on a project to develop a website for a school.

The school principal has some ideas about the appearance of the website but is unclear about all the details of the solution. The principal would like to see an initial version of the website.

- (i) Identify a life cycle method that would be appropriate in this case.

Give a reason for your choice.

Life cycle method

Reason

[2]

- (ii) The website project has progressed to the design stage.

State **three** activities that will take place during the design stage of the program development life cycle.

1

2

3

[3]

76. O/N 21/P22/Q6/c(i)

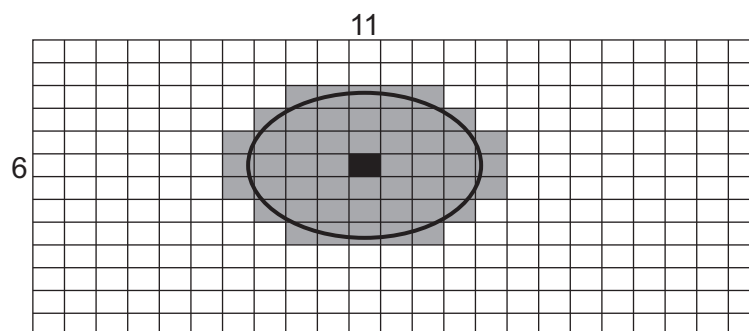
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will be set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	• Searches for the first row that has an array element set to 1 • Returns the index of that row (1 is the first row) • Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	• Searches for the last row that has an array element set to 1 • Returns the index of that row • Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	• Searches for the first column that has an array element set to 1 • Returns the index of that column (1 is the first column) • Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	• Searches for the last column that has an array element set to 1 • Returns the index of that column • Returns -1 if there are no elements set to 1

(c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (i) Outline the changes needed to convert one of the existing modules into this single module.

[2]

77. O/N 21/P23/Q1/a

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(a) (i) State what her colleague means by “does not contain any syntax errors”.

[1]

(ii) Identify **and** describe **one** other type of error that the program may contain.

Type of error

Description

[2]

78. O/N 21/P23/Q4/a,b

4 A program controls the heating system in a sports hall.

Part of the program involves reading a value from a sensor. The sensor produces a numeric value that represents the temperature. The value is an integer, which should be in the range 0 to 40 inclusive.

A program function has been written to validate the values from the sensor.

(a) A test plan is needed to test the function.

Complete the table. The first line has been completed for you.

You can assume that the sensor will generate only integer data values.

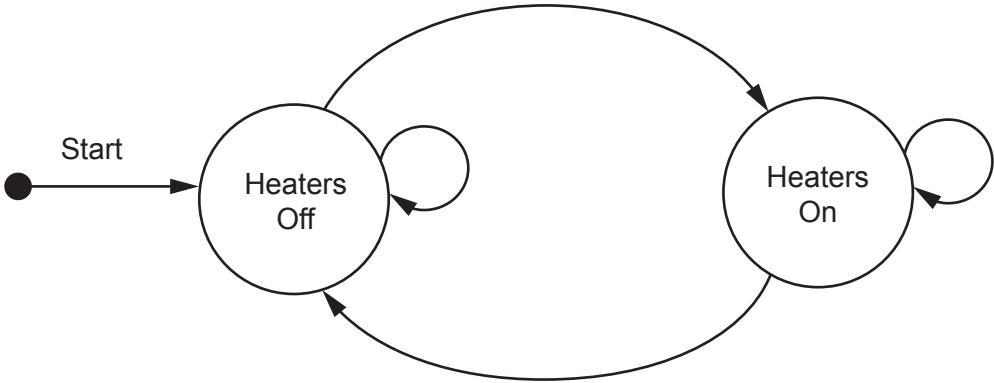
Test	Test data value	Explanation	Expected outcome
1	23	Normal data	Data is accepted
2			
3			
4			
5			

[4]

(b) A program module controls the heaters. This module operates as follows:

- If the temperature is below 10, switch the heaters on.
- If the temperature is above 20, switch the heaters off.

Complete the following state-transition diagram for the heating system:



[3]

79. O/N 21/P23/Q5/b

- 5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	"CJL404"
Host ID	"Lib01"
Time and date	"08:30, June 01, 2021"

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

"CJL404Lib01<separator>08:30, June 01, 2021"

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

- (b) The choice of data structure was made during one stage of the program development life cycle.

Identify this stage.

[1]

80. M/J 21/P21/Q2/a

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
<code>UpdateLoan()</code>	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
<code>LoanExtend()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
<code>CheckReserve()</code>	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
<code>LoanReturn()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.



[5]

81. M/J 21/P21/Q4/a,c

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert("VinVa∇∇Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

TO_LOWER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case

Example: TO_LOWER("JIM 803") returns "jim 803"

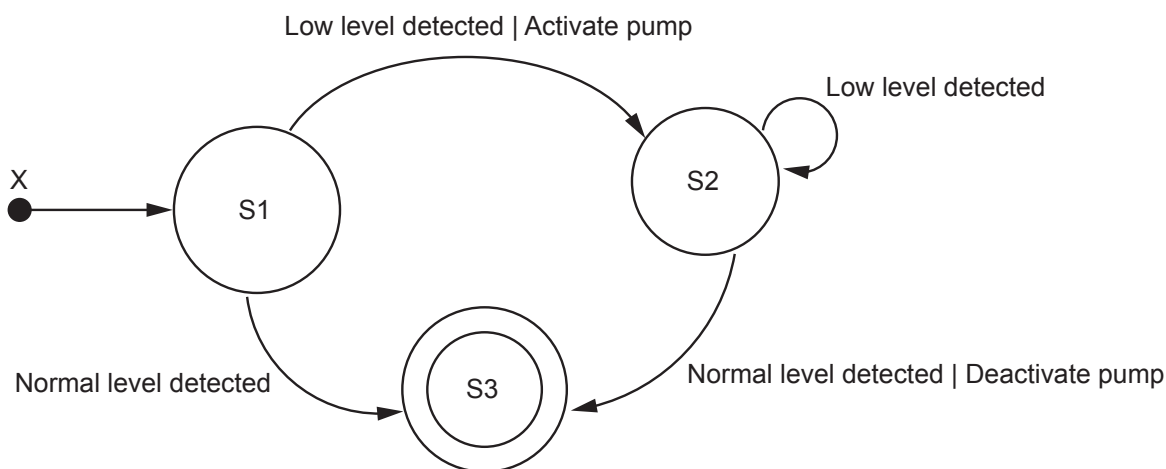
NUM_TO_STR(x : <data type>) RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: NUM_TO_STR(87.5) returns "87.5"

82. M/J 21/P22/Q2/a

2 (a) Examine the following state-transition diagram.



(i) Complete the table with reference to the diagram.

Answer

The number of transitions that result in a different state	
The number of transitions with associated outputs	
The label that should replace 'X'	
The final or halting state	

[4]

(ii) The current state is S1. The following inputs occur.

1. Low level detected
2. Low level detected
3. Low level detected
4. Low level detected

Give the number of outputs and the current state.

Number of outputs

Current state

[2]

83. M/J 21/P22/Q7/a,b

7 A procedure, `FormatName()`:

- is called with a string containing words and spaces as its parameter. In this context, a word is any sequence of characters that does not contain a space character.
- creates a new formatted string from this string with the following requirements:
 1. Any leading spaces removed (spaces before the first word).
 2. Any trailing spaces removed (spaces after the last word).
 3. Any multiple spaces between words converted to a single space.
 4. All characters converted to lower case.

The `FormatName()` procedure has been written in a programming language and is to be tested using the black-box method.

- (a) Give a test string that could be used to show that all **four** formatting requirements have been applied correctly.

Use the symbol '∇' to represent a space character.

[3]

- (b) The `FormatName()` procedure should assign a value to the global variable `FString`.

There is a fault in the program, which means that the assignment does not always take place.

Explain **two** ways of exposing the fault.

[2]

84. M/J 21/P23/Q2/a

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
<code>UpdateLoan()</code>	• Calls either <code>LoanExtend()</code> or <code>LoanReturn()</code>
<code>LoanExtend()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Calls <code>CheckReserve()</code> to see whether the book has been reserved for another library user • Returns <code>TRUE</code> if the loan has been extended, otherwise returns <code>FALSE</code>
<code>CheckReserve()</code>	• Called with <code>BookID</code> • Returns <code>TRUE</code> if the book has been reserved, otherwise returns <code>FALSE</code>
<code>LoanReturn()</code>	• Called with parameters <code>LoanID</code> and <code>BookID</code> • Returns a <code>REAL</code> (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.



[5]

85. M/J 21/P23/Q4/a,c

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(a) Complete the trace table below by dry running the function when it is called as follows:

Result ← Convert("VinVa∇∇Cup")

Note: The symbol '∇' has been used to represent a space character.
Use this symbol for any space characters in the trace table.

[1]

- (ii) Change 2 may be implemented by moving one line of the pseudocode.

Write the number of the line to be moved and state its new position.

Line number

New position

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Characters that are not upper case alphabetic are returned unchanged

Example: LCASE('W') returns 'w'

UCASE(ThisChar : CHAR) RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Characters that are not lower case alphabetic are returned unchanged

Example: UCASE('a') returns 'A'

TO_UPPER(ThisString : STRING) RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case

Example: TO_UPPER("Error 803") returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value

Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

Answers Section

1. M/J 26/P21/Q3/a,b

3(a)	MP1 Investigate the current system /any example // investigate any other similar systems MP2 Discussion/interviews with managers MP3 Discussion/interviews with staff who operate the system MP4 Carry out a survey of employees/customers // carry out a focus group of employees/customers MP5 Observe the current system in use MP6 Produce the <u>requirements specification</u> MP7 Decide on the (program development) life cycle to be used Max 3	3
3(b)	Before any programming is started, the DESIGN stage for the proposed new system must be completed. For this large project, the tasks will be broken down into several MODULES // SUB-TASKS each will be coded by a different programming team. Once each team has completed its work, INTEGRATION // UNIT testing must be carried out. Before the new system is delivered to the business, ALPHA testing is carried out in-house. It is decided to install the software initially in only one of the call centres where BETA testing is carried out. The new software is finally rolled out to all five call centres. In the first week of its use, staff report that the software still requires extra features which were not originally agreed. The software is withdrawn until ADAPTIVE maintenance is completed. 1 mark per gap	6

2. M/J 26/P21/Q8/a,b,c

8(a)	<table><tr><th>LeftPointer</th><th>RightPointer</th><th>MidPointer</th></tr><tr><td>1</td><td>15</td><td>8</td></tr><tr><td></td><td>7</td><td>4</td></tr><tr><td>5</td><td></td><td>6</td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td></tr></table>	LeftPointer	RightPointer	MidPointer	1	15	8		7	4	5		6																4
	LeftPointer	RightPointer	MidPointer																										
	1	15	8																										
		7	4																										
	5		6																										
8(b)	Found 6	1																											
8(c)	6	1																											

3. M/J 26/P22/Q1/b,c,d,e,f

1(b)(i)	One mark per state Example answers: 1 Sensor activated (mode) // Intruder/movement detected (mode) // Alert Mode 2 Alarm sounding // Light flashing (mode) // Alarm activated 3 Error mode // Power loss detected (mode) // Battery too low mode 4 Tamper detected (mode) // Lockdown (mode) 5 Disarm Mode // Maintenance Mode // Password <u>Change</u> Mode // Service Mode Max 2	2	
1(b)(ii)	One mark per event Example answers: 1 Movement / pressure (detected) 2 Window opened / broken (detected) 3 Door open (detected) 4 Entering wrong password (repeatedly) 5 Entering password to enter / exit maintenance mode / silence alarm 6 Tampering / damaging alarm (system) 7 Time delay // Time passed after alarm triggered / before alarm activated 8 Battery charge too low 9 Pressing panic button	3	Do not accept repeating examples in stem. Reject Turning system on. Reject reference to creating password.

1(c)

Example Solution

rule being tested	example of test data	reason for invalid test data
rule 1	1A	password is too short
rule 1	PHRASE345678	Password too long
rule 2	ABCDEF	Does not contain a digit
rule 2	1234567	Does not contain a letter
rule 3	PASSWRD2	letters are repeated
rule 3	ABCDEF11	Digits / Numbers are repeated
rule 3	AAA111	Both digits and letters are repeated Accept Characters repeated

Mark as follows:

Rule 1 test:

1 mark for **one** example of test data for rule 1 **and** appropriate reason

Test data	A string of more than 8 characters that contains both letters and digits. The letters must be upper case Each character only occurs once
Reason	Password is too long

3

Do not accept once if lower-case letters or characters that are not letters or digits used.

Reject if test data breaks Rule 2 or/and Rule 3

1(c)	Rule 2 test (1 mark): 1 mark for two examples of test data that only breaks rule 2 and appropriate reasons. <table><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no digits</td></tr><tr><td>Reason</td><td>Does not contain a digit</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long that contains no letters</td></tr><tr><td>Reason</td><td>Does not contain a letter</td></tr></table> Rule 3: 1 mark for two examples of test data for rule 3 and appropriate reasons <table><tr><td>Test data</td><td>A string between 6 and 8 characters long where only digits are repeated</td></tr><tr><td>Reason</td><td>Digits are repeated</td></tr><tr><td>Test data</td><td>A string between 6 and 8 characters long where both characters and digits are repeated</td></tr><tr><td>Reason</td><td>Both digits and letters are repeated</td></tr></table>	Test data	A string between 6 and 8 characters long that contains no digits	Reason	Does not contain a digit	Test data	A string between 6 and 8 characters long that contains no letters	Reason	Does not contain a letter	Test data	A string between 6 and 8 characters long where only digits are repeated	Reason	Digits are repeated	Test data	A string between 6 and 8 characters long where both characters and digits are repeated	Reason	Both digits and letters are repeated		Reject if letters repeated Reject if digits repeated Reject characters repeated Reject if letters repeated Reject if no letters
Test data	A string between 6 and 8 characters long that contains no digits																		
Reason	Does not contain a digit																		
Test data	A string between 6 and 8 characters long that contains no letters																		
Reason	Does not contain a letter																		
Test data	A string between 6 and 8 characters long where only digits are repeated																		
Reason	Digits are repeated																		
Test data	A string between 6 and 8 characters long where both characters and digits are repeated																		
Reason	Both digits and letters are repeated																		
1(d)	One mark per example Example answers: 1 Structure chart // Problem / Program broken into modules // Modular Diagram 2 Flowchart 3 Data Flow Diagram 4 Identifier / Variable table // Data structure / Record structure (designs) 5 File structure // Database Design 6 E-R Diagram 7 HCI Design // Report Design 8 Class Diagram // UML Diagram 9 Programming language to use Max 3	3	Reject 'Decomposition' by itself Reject Module Overview by itself																
1(e)	One mark per stage 1 Analysis 2 Coding // Implementation 3 Maintenance Max 2 marks	2	NE Development / Programming																
1(f)(i)	Integration	1																	

1(f)(ii)	One mark for the name, one for the description Method: Stub (testing) Description: 1 A simple / dummy module is written to replace the module (that is not working) 2 The simple / dummy module will return an expected value / will output a message (to show the module has been called)	3	Do not mark description if method incorrect
1(a)(i)	STRING	1	
1(a)(ii)	IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for BOOLEAN 2 LetterFlag ← <u>FALSE</u> 1 mark for FALSE 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for LENGTH 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for MID, Index and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for IS_NUM 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for NOT	7	

4. M/J 26/P23/Q3/a,b,c

3(a)	The initial ANALYSIS stage is carried out by studying in detail the current system. A design for the proposed new system is produced. Following the design and coding stages, it is decided to install the completed new software initially in only two of the shops. BETA testing is carried out. The final stage of the project uses ACCEPTANCE testing to test all programs on the company's hardware. After successful testing, the company approves the new system. The company then installs the new software in all shops. In the first week of its use, unexpected bugs are reported. CORRECTIVE maintenance has to be carried out.	4	
3(b)	Team A MP1 Rapid Application Development /RAD // Iterative MP2 Large project can be broken down into smaller steps MP3 All stakeholders/users are involved (at all stages) MP4 There is constant discussion between the client (and Team A) MP5 Prioritises the client needs and end-user experience MP6 A <u>prototype</u> is available at an early stage MP7 More readily allows for requirement changes to be made MP8 Minimal planning is required MP9 Programs/modules can be developed in parallel Max 2 Team B MP10 Waterfall (method/model) MP11 Next stage does not start until the current stage is successfully completed MP12 More predictable for budget MP13 More predictable for timeline/completion MP14 All requirements are identified before coding MP15 Late changes cannot be accommodated Max 2	4	Reject 'early version'.
3(c)	MP1 Production of flowcharts // structure <u>charts</u> // STDs // Structured English // E-R diagrams MP2 Consider data storage (hardware / database / files) MP3 Design inputs screens // Design screen outputs // reports to be produced MP4 Select (Programming) language to be used MP5 Produce <u>pseudocode algorithms</u> / data structures / procedures/functions/modules are produced / Identifier table MP6 Produce a testing strategy statement // (detailed) test plan MP7 Consider – is there relevant code available from program libraries? Max 3	3	Reject 'program structure'.

5. M/J 26/P23/Q7/a,b

7(a)(i)	<table><tr><th>Num1</th><th>Index</th><th>i</th><th colspan="6">Data</th></tr><tr><th></th><th></th><th></th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th></tr><tr><td>12</td><td>1</td><td>2</td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>2</td><td>3</td><td></td><td>3</td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>3</td><td>4</td><td></td><td></td><td>4</td><td></td><td></td><td></td></tr><tr><td></td><td>4</td><td>5</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>5</td><td>6</td><td></td><td></td><td></td><td>6</td><td></td><td></td></tr><tr><td></td><td>4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> Mark by column(s)	Num1	Index	i	Data									1	2	3	4	5	6	12	1	2	2							2	3		3						3	4			4					4	5								5	6				6				4																																			4	
Num1	Index	i	Data																																																																																																			
			1	2	3	4	5	6																																																																																														
12	1	2	2																																																																																																			
	2	3		3																																																																																																		
	3	4			4																																																																																																	
	4	5																																																																																																				
	5	6				6																																																																																																
	4																																																																																																					
7(a)(ii)	Factors / the divisors of Num1 / 12 // all the numbers which divide exactly into Num1	1																																																																																																				
7(b)(i)	<table><tr><th>Num1</th><th>Num2</th><th>Index</th><th>Match</th><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th></tr><tr><td>20</td><td>8</td><td>4</td><td>FALSE</td><td>2</td><td>4</td><td>5</td><td>10</td><td></td><td></td></tr><tr><td></td><td></td><td>3</td><td>FALSE</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td>2</td><td>TRUE</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> Max 2 for any additional incorrect entries	Num1	Num2	Index	Match	1	2	3	4	5	6	20	8	4	FALSE	2	4	5	10					3	FALSE									2	TRUE																											3																																								
Num1	Num2	Index	Match	1	2	3	4	5	6																																																																																													
20	8	4	FALSE	2	4	5	10																																																																																															
		3	FALSE																																																																																																			
		2	TRUE																																																																																																			
7(b)(ii)	Highest common factor // Greatest common divisor of Num1 and Num2 // by good explanation i.e. 4 is the highest number that divides exactly into both 8 and 20	1																																																																																																				

6. O/N 25/P21/Q1/b

1(b)	Test description	Test method	3
	carried out as soon as a program module has been coded	alpha	
	carried out as program modules are being combined	integration	
	completed by the developers without referring to the code	black-box	
	completed by the customer	acceptance / beta	
One mark per row (2 to 4)			

7. O/N 25/P21/Q7/a,b

7(a)	<pre> graph TD START((START)) --> S1((S1)) S1 -- "A1 X1" --> S2((S2)) S1 -- "A2 X2" --> S3((S3)) S2 -- "A3 X3" --> S3 S2 -- "A4 X4" --> S5((S5)) S3 -- "A2 X4" --> S4((S4)) S3 -- "A1" --> S3 S4 -- "A3" --> S2 S4 -- "A4 X4" --> S5 S4 -- "A1 X1" --> S4 </pre> One mark for each: 1 Label A1 X1 added on line from S1 to state labelled S2 2 States S3, S4 and S5 labelled 3 Line from S2 to S5 with event label A4 X4 4 Lines from S3 with correct event labels including 'loop' label 5 Lines from S4 with correct event labels including 'loop'	5
------	---	----------

7(b)	<pre> graph TD Setup[Setup] Restart[Restart] Modify[Modify] Final[Final] Update[Update] Confirm[Confirm] Setup -- H1 --> Restart Setup -- B1 --> Modify Setup -- T1 --> Final Restart -- C1 --> Setup Modify -- R2 --> Update Update --> Modify Final --> Confirm Confirm --> Final Confirm --> Setup </pre> One mark per point: 1 All boxes correctly labelled 2 Iteration arrow 3 Parameter to Restart 4 Return value from Modify and parameter to Final 5 BYREF parameter to Update	5
------	--	---

8. O/N 25/P21/Q8/c

8(c)	Award marks for reference to following design features: Change to existing Record: 1 Store date due back as a new data item / in the loan record 2 Store date of loan and the loan length category as new data items / in the loan record Max 1 New Record 3 New Records with Category (and Loan length field) Change to Program: 4 (Use the loan length category of a book to) calculate the date due date 5 Compare due date with today's date (to check if overdue / to calculate fine) 6 Map loan length category to length of loan / date due back Max 1 Max 2 marks	2
------	--	---

9. O/N 25/P22/Q5/a,b

5(a)	Activity	Name of life cycle stage	4
	a structure chart is produced	Design	
	a program is modified to allow it to run on new hardware	Maintenance	
	the programmer identifies the customer's requirements	Analysis	
	an Integrated Development Environment (IDE) provides context-sensitive help such as 'auto-complete'	Coding	
One mark per row			
5(b)	Acceptance testing		1

10. O/N 25/P22/Q7/a,b

7(a)	Example explanation: The arrow means that <code>Convert</code> repeatedly calls <code>Analyse</code> then <code>Update</code> and then <code>Sort</code> (in that order)¹⁰ MP1 Reference to 'repetition'... MP2 Naming all <u>four</u> modules correctly and the sequence is stated or implied	2
7(b)	MP1 <u>PROCEDURE Analyse (Count : INTEGER)</u> <u>FUNCTION Update (Key : STRING) RETURNS INTEGER</u> MP2 MP3 MP4 <u>PROCEDURE Sort (BYREF P2 : BOOLEAN)</u> Mark as follows: One mark per underlined part	4

11. O/N 25/P22/Q8/d

8(d)	Additional date(s) solution MP1 Store the return date of each loan // the start date <u>and</u> the loan period MP2 Algorithm will do a calculation involving the MP1 data item(s) OR The student email address solution MP3 Store the student <u>email address</u> MP4 In order to send the email // the student is alerted to return the book Max 2	2
------	---	----------

12. O/N 25/P23/Q5/a

5(a)

5

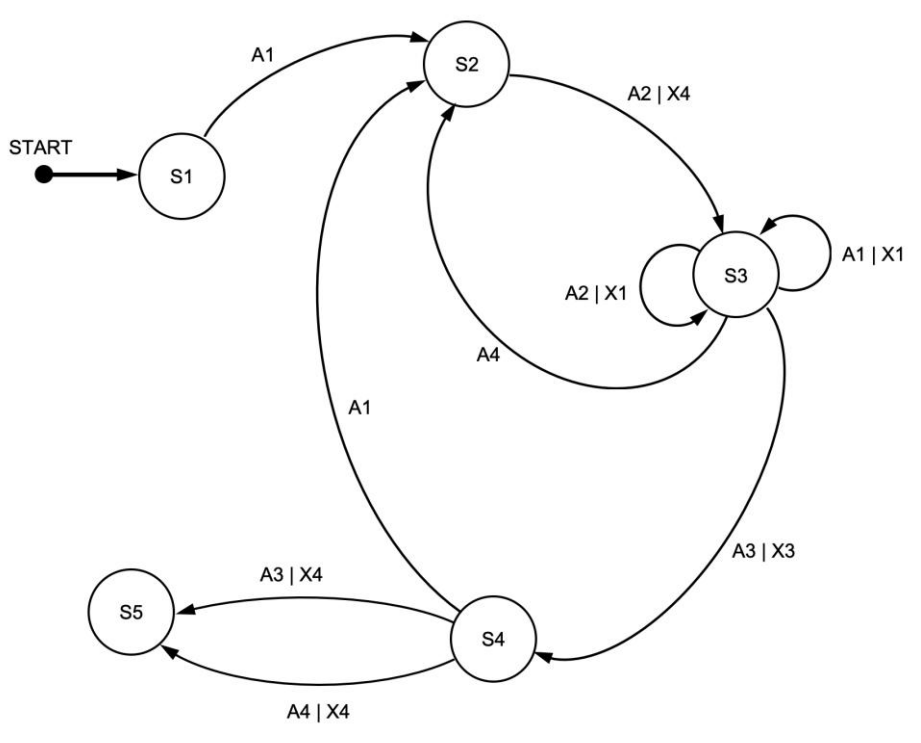
Index	CaseVar	Count	Num [1]	Num [2]	Num [3]	Num [4]
1		0	1	2	5	3
	1					
2		1	2			Zone 1
3						
	5					
		20				
4						
	3					
7		19				6
4						
	6					
		20				
1						
	2					
3		22	3			
4						
Zone 2	Zone 3	Zone 4	Zone 5			

Zones 1 to 4 : One mark for each set of values as shown
Zone 5: Num[4] set to 6 when Index is 7
and
Num[1] set to 3 when index is 3 for **second** time

13. O/N 25/P23/Q6/a

6(a)	One mark per point:	2
	1 Create a list of numbers that should generate all of the possible check digit values	
	2 and check that each / the / a generated value is as expected	

14. O/N 25/P23/Q7/a,b

7(a)	 One mark for each: 1 S2 to S5 labelled 2 Label A1 added to event from S1 to S2 3 Event lines from S2 to S3 and S3 to S2 correctly labelled 4 Events lines from S3 to S3 and event line from S3 to S4 all labelled correctly 5 Event lines from S4 to S5 correctly and event line from S4 to S2 labelled correctly Max 4 for any additional events/lines	5
------	---	----------

7(b)	<pre> graph TD Setup{Setup} --> Restart[Restart] Restart --> Confirm[Confirm] Confirm --> Setup Restart --> Update[Update] Update --> Restart Restart --> Modify[Modify] Modify --> Restart </pre> One mark for each: 1 All boxes correctly labelled and connected in correct hierarchy 2 BYREF parameter to Restart, parameter to Confirm and return 3 Diamond and loop symbol 4 Parameter to Update and return from Modify	4
------	--	---

15. M/J 25/P21/Q2/a,b,c

2(a)(i)	MP1 The two decision boxes are in the wrong order // The first decision symbol is wrong MP2 The first decision should check for values greater than or equal to 2000 Max 1	1
2(a)(ii)	Explaining how to correct the flowchart. MP1 Swap around the two decision boxes MP2 The first decision box should check for values greater than or equal to 2000 and the second decision box should check for values above 4000 Max 1	1
2(b)(i)	2000 / 4000 / 10 / 100	1
2(b)(ii)	MP1 The value cannot be changed // avoids being different in two places MP2 Makes the program easier to understand MP3 Avoids repeatedly writing the same value throughout the program MP4 A change to the value requires a change in only one statement Max 2	2

2(b)(iii)	MP1 (The code is) tried and tested so error free MP2 Provides functionality / code that the programmer may find difficult to code themselves Max 1	1
2(c)	MP1 Syntax (error) MP2 Rules of programming language // the language grammar was not followed MP3 Run-time (error) MP4 The program performs an illegal operation Mark as follows: One mark for naming each type of error and one mark for the description	4

16. M/J 25/P21/Q4/b

4(b)

6

I	Key	J	Chars [J]	Chars			
				[1]	[2]	[3]	[4]
2				'D'	'T'	'H'	'R'
	'T'	1	'D'				
3							
	'H'	2	'T'			'T'	
		1	'D'				
					'H'		
4							
	'R'	3	'T'				'T'
		2	'H'			('R')	
						'R'	
5							

MP1

MP2

MP3

MP4

MP5

MP6

Chars array final values

'D'	'H'	'R'	'T'
-----	-----	-----	-----

Mark as follows:

MP1, MP2, MP3, MP4, MP5

1 mark per enclosure

17. M/J 25/P21/Q6/a

6(a)

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted
boundary/ extreme/ normal	0 / 1	data item is accepted
boundary/ extreme/ normal	59 / 60	data item is accepted
abnormal	>= 61	data item is rejected
normal	15	data item is accepted

4

Mark as follows:
1 mark for each row with (Type **and** Value **and** Outcome)

18. M/J 25/P21/Q7/b(ii)

7(b)(ii)	MP1 Check if the text file <code>loyalty.txt</code> exists / is empty MP2 If file does not exist it must be created (using write mode) MP3 If the file has to be created / is empty then set the first <code>CustomerID</code> to 100001 MP4 Write "100001,0" to <code>loyalty.txt</code> (using write mode) MP5 For all but the first customer (to be added to the empty file) use the existing code / module Max 4	4
----------	--	----------

19. M/J 25/P22/Q1/a,b

1(a)(i)	One mark for Choice of program development cycle depends on the approach/method required to produce the program Or One mark for a factor that would influence the choice of program development cycle, e.g. • Rapid development of program/prototypes required • Need for prototypes (at an early stage) • Complexity of problem • Skills / experience of development team / programmer(s) • Budget / Time / Resources available • Size of development team • Developer / programmer can return to earlier stages / any stage • Avoidance of repeating previous stages in development of program • How much involvement clients will have • Allows the (program) requirements to be changed during program development • (Program) requirements agreed at start of development of program Max 1	1
1(a)(ii)	Which programming language would best be suited to control the production line // Which programming language would best suit the problem being solved	1
1(b)(i)	Examples include: • Change to (production line) requirements • New technology / hardware available (to control production line) • Changes made to library modules used • Change in relevant legislation Max 3	3
1(b)(ii)	Perfective // Corrective	1

20. M/J 25/P22/Q2/b

2(b)(i)	One mark for each of set test values Hours worked between 1 and 40 inclusive Sales value ≤ 2000 Bonus Pay: 0 Hours worked above 40 Sales value ≤ 2000 Bonus Pay: 10 Hours worked between 1 and 40 inclusive Sales value > 2000 Bonus Pay: 50 Hours worked above 40 Sales value above 2000 Bonus Pay: 100 max 2	2
2(b)(ii)	One mark per point • Simple modules are written to replace each of the unfinished modules. • Each simple module will return an expected value / will output a message to show it has been called.	2
2(b)(iii)	One mark for naming type of error and one mark for corresponding description Type of error: Logic (error) Description: Where the program does not behave as expected / Does not give expected result / An error in the logic of the algorithm Or Type of error: Run-time (error) Description: The program performs an illegal operation Max 2	2

21. M/J 25/P22/Q5/a

5(a)

```
graph TD; Start((Start)) --> Standby((standby mode)); Standby -- "(turn) on" --> Detect((detect mode)); Detect -- "(turn) off" --> Standby; Active((active mode)) -- "(turn) off" --> Standby; Detect -- "movement detected" --> Active; Active -- "20 second elapsed" --> SeqComp((sequence complete)); SeqComp -- "time saved" --> Detect;
```

Mark as follows:

1 Mark: The three new states drawn and labelled.

1 Mark: Two of the events drawn with labels connecting correct states and correct line direction

1 Mark: Four of the events drawn with labels connecting correct states and correct line direction.

1 Mark: All and only six events drawn with labels connecting correct states and correct line direction.

4

22. M/J 25/P22/Q6/a

6(a)(i)

Value	Start	Unused	New	Last	Current
1043	2	8	8	-1	2
				2	3
				3	1
				1	7
				7	4

Region 1

Region 2

Region 3

Region 4

Award 1 mark per region

4

6(a)(ii)			3

Data	Pointer
11018	17
21007	23
31010	31
41056	46
51092	5-1
61062	65
71034	78
81043	84
90	910
100	10-1

Mark as follows:

1 mark for global array `Data` row 8 containing the value 1043

1 mark for global array `Pointer` row 7 containing the value 8 and row 8 containing the value 4

1 mark for all other rows in both arrays

23. M/J 25/P23/Q4/b

4(b)	MP1 Going through the program a line / statement at a time // doing a dry run // single-stepping (at run-time with an IDE) // Peer testing/review is carried out MP2 Creating a trace table MP3 Draw up test data // draw up list of inputs with expected output // boundary, normal, abnormal data is used MP4 Errors will be indicated when a variable / output gives an unexpected value MP5 Faults in the logic of the program can be detected // Errors may be indicated by an unexpected path through the program Max 3	3
------	---	---

24. M/J 25/P23/Q6/b,c

6(b)	Sub_Part1A: <u>PROCEDURE Sub Part1A(R: INTEGER)</u> 1 Mark Sub_Part2A: <u>FUNCTION Sub Part2A(T : REAL) RETURNS BOOLEAN</u> 2 Marks Sub_Part3A: <u>PROCEDURE Sub Part3A(V: INTEGER, W: BOOLEAN, BYREF U : STRING)</u> 2 Marks	5						
6(c)	<table><tr><th>Symbol</th><th>Explanation</th></tr><tr><td></td><td>The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)</td></tr><tr><td></td><td>The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A</td></tr></table> MP1 Shape1: reference to <u>selection</u> MP2 Shape2: reference to <u>repetition</u> // iteration MP3 Descption (×2) including correct use of all module names in both parts	Symbol	Explanation		The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)		The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A	3
Symbol	Explanation							
	The module Main calls either Sub_A or Sub_B (which one is called is determined at run time)							
	The module Sub_A will repeatedly call Sub_Part1A followed by Sub_Part2A then by Sub_Part3A							

25. O/N 24/P21/Q1/a

1(a)	One mark per point: Error: The calculation of TaxPayable is incorrect Correction: TaxPayable ← (ItemCost * TaxRate) / 100	2
------	---	----------

26. O/N 24/P21/Q3/c

3(c)	Two loop solution One mark per point: 1 Loop until given line number (-1) / <parameter> (-1) (lines have been read) 2 ... if end of file is reached then return FALSE 3 Loop for three lines // Read three lines // Repeat of 4 and 5 for three lines 4 Read a line and output it 5 ... if end of file is reached then return FALSE 6 After outputting the (required) lines return TRUE 7 Ordering of lines no longer needed Max 4 marks Alternative solution One loop solution mark scheme that reads to given line number + 2 One mark per point 1 Loop until given line number + 2 / <parameter> + 2 (lines have been read) 2 OR end of file is reached 3 Return FALSE if EOF reached 4 Read a line from the file in the loop 5 Continue to order the last three lines read in the loop // Steps 4 to 6 stay the same 6 (If FALSE has not been returned) output the required three lines (in the correct order) 7 ... and Return TRUE Max 4 marks	4
3(a)	One mark per emboldened part: 1 Open the file (Result.txt) in read mode 2 Loop until EOF(Result.txt) // EOF // end of file 3 Read a / the / next line (from the file) and store in ThisLine 4 Assign LineY to LineX 5 Assign LineZ to LineY 6 Assign ThisLine to LineZ 7 After the loop, close the file (Result.txt) 8 Output LineX, LineY, LineZ	4

27. O/N 24/P21/Q4/a

4(a)	One mark per point: Type: Run-time error Cause: A divide by zero operation is attempted // Attempt to convert a non-numeric string to a number	2
------	---	----------

28. O/N 24/P21/Q5/a,b

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>The walkthrough method is used.</td><td>Testing</td></tr><tr><td>An algorithm is implemented in a programming language.</td><td>Coding</td></tr><tr><td>The client is interviewed about problems with the current system.</td><td>Analysis</td></tr><tr><td>The program is modified to run on new hardware.</td><td>Maintenance</td></tr><tr><td>Records and file structures are defined.</td><td>Design</td></tr></table> One mark per row	Activity	Name of life cycle stage	The walkthrough method is used.	Testing	An algorithm is implemented in a programming language.	Coding	The client is interviewed about problems with the current system.	Analysis	The program is modified to run on new hardware.	Maintenance	Records and file structures are defined.	Design	5															
Activity	Name of life cycle stage																												
The walkthrough method is used.	Testing																												
An algorithm is implemented in a programming language.	Coding																												
The client is interviewed about problems with the current system.	Analysis																												
The program is modified to run on new hardware.	Maintenance																												
Records and file structures are defined.	Design																												
5(b)(i)	One mark per row <table><tr><th>Type of test data</th><th>Test data value</th><th>Expected result</th></tr><tr><td>Abnormal</td><td>12 (< 23)</td><td>FALSE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>23</td><td>FALSE</td></tr><tr><td>Boundary / Extreme</td><td>24</td><td>TRUE</td></tr><tr><td>Boundary</td><td>25</td><td>TRUE</td></tr><tr><td>Boundary</td><td>36</td><td>TRUE</td></tr><tr><td>Boundary /Extreme</td><td>37</td><td>TRUE</td></tr><tr><td>Abnormal / Boundary / Extreme</td><td>38</td><td>FALSE</td></tr><tr><td>Abnormal</td><td>99 (> 38)</td><td>FALSE</td></tr></table> Max 4 marks	Type of test data	Test data value	Expected result	Abnormal	12 (< 23)	FALSE	Abnormal / Boundary / Extreme	23	FALSE	Boundary / Extreme	24	TRUE	Boundary	25	TRUE	Boundary	36	TRUE	Boundary /Extreme	37	TRUE	Abnormal / Boundary / Extreme	38	FALSE	Abnormal	99 (> 38)	FALSE	4
Type of test data	Test data value	Expected result																											
Abnormal	12 (< 23)	FALSE																											
Abnormal / Boundary / Extreme	23	FALSE																											
Boundary / Extreme	24	TRUE																											
Boundary	25	TRUE																											
Boundary	36	TRUE																											
Boundary /Extreme	37	TRUE																											
Abnormal / Boundary / Extreme	38	FALSE																											
Abnormal	99 (> 38)	FALSE																											
5(b)(ii)	Integration (testing)	1																											

29. O/N 24/P21/Q7/a,b

7(a)	One mark per point: 1 All module identified in correct hierarchy 2 Parameters to both Sync and Activate and return values 3 Parameters to both Reset and Enable 4 Parameters to Initialise 5 Decision diamond	5
7(b)	Explanation: Means that Sync repeatedly calls Reset and (then) Enable One mark for each point: MP1: reference to iteration MP2: naming all three modules correctly including correct sequence of calls	2

30. O/N 24/P22/Q1/a

1(a)	(Corrective) Maintenance	1
------	--------------------------	----------

31. O/N 24/P22/Q5/a,b

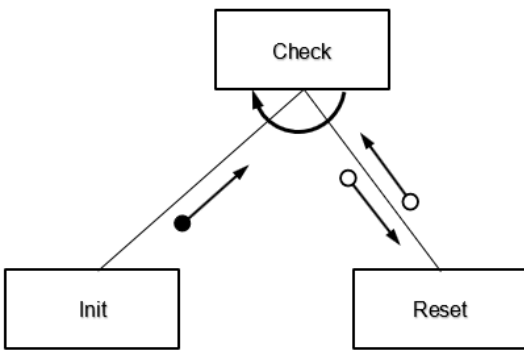
5(a)	MP1 <code>Count ← INT(100 / Number)</code> Number could be zero (giving a divide by zero) MP2 <code>Index ← Data[Number]</code> Potential error: Value <code>Number</code> could be outside the range of array indices MP3 <code>ReturnValue ← TO_UPPER(RIGHT(Label, Count))</code> Potential Error: Number to extract may be too big / negative / out of range for use in the <code>RIGHT</code> function // <code>Label</code> has insufficient characters MP4 <code>RETURN RetVal</code> Potential Error: There is no value to be returned // there is no variable named <code>RetVal</code> Mark as follows: 1 mark for each statement and description Max 3 marks	3
5(b)	MP1 Construct: A (pre/post) <u>conditional</u> loop MP2 Explanation: The terminating condition is never satisfied	2

32. O/N 24/P22/Q6/b

6(b)	Answers include: MP1 For each filling, create a list of <u>acceptable</u> / <u>incompatible</u> fillings/indexes MP2 When selecting the second filling, (as well as checking for an unused element) <u>check</u> that the filling / index is / is not on the list ALTERNATIVE: MP1 Create a list of 'good' combinations MP2 <u>Randomly select</u> from this list	2
------	---	----------

33. O/N 24/P22/Q7/c,d

7(c)	MP1 Data structures // data dictionary // identifier table(s) // validation rules MP2 Data-flow diagram // state-transition diagram MP3 User interface // Format for the email MP4 Testing method / Test plan / Test data / Trace tables MP5 Choice of email protocol to be used // Programming language to be used // Development environment MP6 Use of library routines // program to send the email Max 3 marks	3
------	---	----------

7(d)	 MP1 Three boxes correctly labelled and correct hierarchy MP2 Parameter and return values MP3 Iteration arrow	3
------	--	----------

34. O/N 24/P23/Q5/a,b

5(a)	<table><tr><th>Activity</th><th>Name of life cycle stage</th></tr><tr><td>A compiler is be used.</td><td>Coding</td></tr><tr><td>A program that has been released for general use is modified</td><td>Maintenance</td></tr><tr><td>The dry run method is used.</td><td>Testing</td></tr><tr><td>The program structure is specified.</td><td>Design</td></tr></table> One mark per row	Activity	Name of life cycle stage	A compiler is be used.	Coding	A program that has been released for general use is modified	Maintenance	The dry run method is used.	Testing	The program structure is specified.	Design	4
Activity	Name of life cycle stage											
A compiler is be used.	Coding											
A program that has been released for general use is modified	Maintenance											
The dry run method is used.	Testing											
The program structure is specified.	Design											
5(b)	One mark for method – two marks for the description MP1 Stub testing MP2 The modules <u>Test A()</u> and <u>Test B()</u> are replaced by dummy modules MP3 ... which return a known result // An output statement is displayed when called (to check it works) // gives expected output	3										

35. O/N 24/P23/Q7/a

7(a)	One mark per answer: MP1 Analysis MP2 Named document, examples include: • A <u>requirement specification</u> • Definition of System Objectives • List of problems with existing system • Survey results • Feasibility study • Interview notes Max 2 marks	2
------	---	----------

36. M/J 24/P21/Q5/a,b

5(a)	<table><tr><th>Count</th><th>C</th><th>L</th><th>Index</th><th>Result</th><th>Data[1]</th><th>Data[2]</th><th>Data[3]</th></tr><tr><td></td><td></td><td> </td><td></td><td>"*****"</td><td>"aaaaaaa"</td><td>"bbbbbbb"</td><td>"cccccc"</td></tr><tr><td>1</td><td>'X'</td><td>3</td><td>1</td><td>"AAAAA A"</td><td>"AAA"</td><td></td><td></td></tr><tr><td>3</td><td>'Y'</td><td>2</td><td>2</td><td>"bbbbbb b"</td><td></td><td>"bb"</td><td></td></tr><tr><td>5</td><td>'W'</td><td>4</td><td>3</td><td></td><td></td><td></td><td>"bbbb"</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table> One mark per zone.	Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]					"*****"	"aaaaaaa"	"bbbbbbb"	"cccccc"	1	'X'	3	1	"AAAAA A"	"AAA"			3	'Y'	2	2	"bbbbbb b"		"bb"		5	'W'	4	3				"bbbb"																																																	6
Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]																																																																																			
				"*****"	"aaaaaaa"	"bbbbbbb"	"cccccc"																																																																																			
1	'X'	3	1	"AAAAA A"	"AAA"																																																																																					
3	'Y'	2	2	"bbbbbb b"		"bb"																																																																																				
5	'W'	4	3				"bbbb"																																																																																			
5(b)(i)	<u>OTHERWISE : CALL Error(C)</u>	1																																																																																								
5(b)(ii)	After the 'z' clause in the CASE construct // before the ENDCASE	1																																																																																								

37. M/J 24/P21/Q6/b

6(b)	1 mark for statement of problem: Problem: - The function will return an incorrect value // the test will fail 1 mark for solution: Solution: - Round the calculated values (to a known number of decimal places) - Define a threshold below which any difference can be ignored	2
6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR___ (Len2 = Len1 + Len3) OR___ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: - Calculate the square of the length of at least one side - Calculation of all three lengths squared correctly One correct comparison of square of three lengths All three comparisons... - combined using logical operators / nested IF // completely correct selection - Return result correctly in both cases	6

38. M/J 24/P21/Q7/b

7(b)(i)	Means that Update calls (one of) either Sub-A, Sub-B or Sub-C One mark for each point: - reference to selection / decision / if - naming all four modules correctly	2
7(b)(ii)	<pre> PROCEDURE Sub-A (Name : STRING, BYREF P2 : BOOLEAN) FUNCTION Sub-B (P1 : REAL) RETURNS REAL </pre> One mark per underlined part in each case	4

39. M/J 24/P22/Q5/a(i),b

5(a)(i)	One mark per error: Syntax: 1. <code>NEXT Index</code> (should be <code>ENDWHILE</code>) 2. <code>'&'</code> used to concatenate an integer (in <code>OUTPUT</code> statement) Other: 3. <code>Accesses element outside range</code> // <code>Accesses element 0</code>	3
5(b)	Run-time	1

40. M/J 24/P22/Q7/b

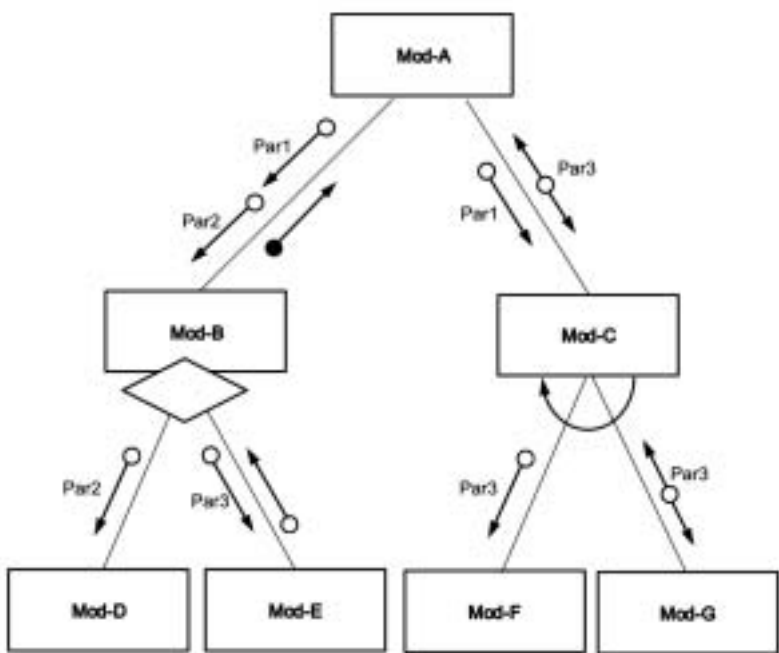
7(b)(i)	<table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>none</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-B</td><td>none</td><td>S2</td></tr> <tr> <td>Input-B</td><td>none</td><td>S5</td></tr> <tr> <td>Input-A</td><td>none</td><td>S2</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S4</td></tr> </tbody> </table> One mark per row 3 to 7	Input	Output	Next state			S1	Input-A	none	S3	Input-A	Output-W	S3	Input-B	none	S2	Input-B	none	S5	Input-A	none	S2	Input-A	Output-X	S4	5
Input	Output	Next state																								
		S1																								
Input-A	none	S3																								
Input-A	Output-W	S3																								
Input-B	none	S2																								
Input-B	none	S5																								
Input-A	none	S2																								
Input-A	Output-X	S4																								
7(b)(ii)	Input-B, Input-A	1																								

41. M/J 24/P23/Q6/b(ii)

6(b)(ii)	The progress display will more accurately show the progress of the task	1
----------	---	----------

42. M/J 24/P23/Q7/a(i),b

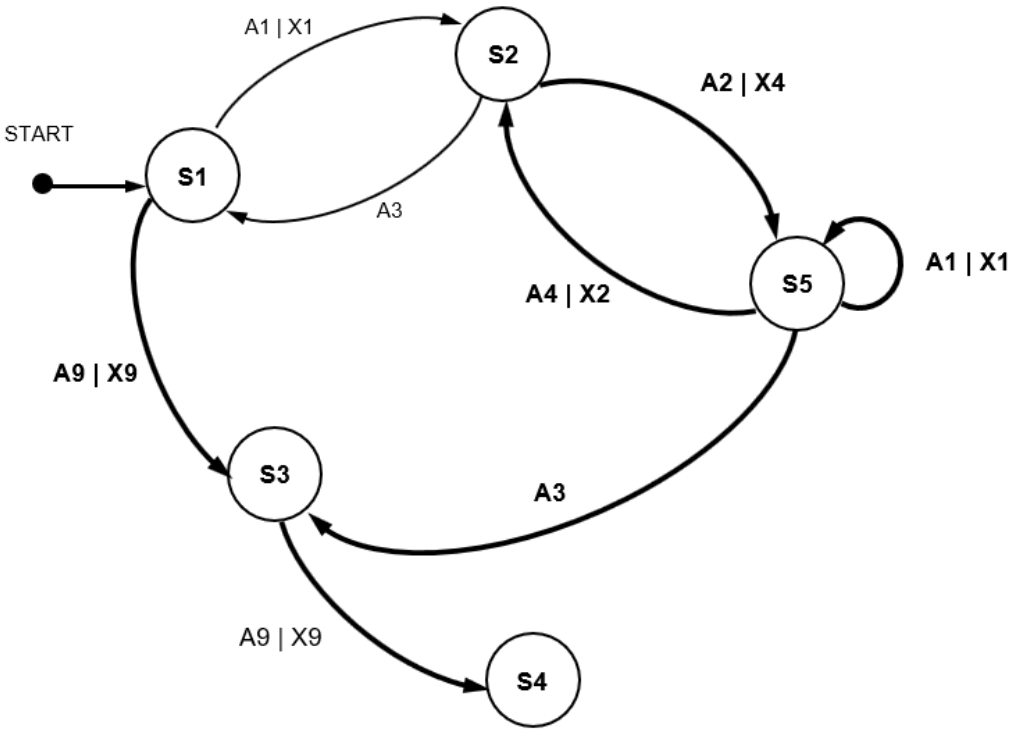
7(a)(i)	Mod-B() and Mod-E()	1
---------	---------------------	----------

7(b)	 One mark per bullet: 1 All modules correctly labelled and interconnected 2 Parameters between Mod-A and Mod-B and return value from Mod-B 3 Parameters between Mod-A and Mod-C 4 Diamond applied to Mod-B only 5 Iteration arrow applied to Mod-C only 6 All parameters at lower level and return value	6
------	--	---

43. O/N 23/P21/Q5/a,b

5(a)	MP1 Num > Min should be Num < Min MP2 Count & 1 should be Count + 1 MP3 NextInput ← "END" should be NextInput = "END"	3
5(b)(i)	MP1 If all the numeric input values are greater than 999 // If there are no numeric values in the sequence MP2 then the minimum will be given as <u>999</u> (and not one of the input values)	2
5(b)(ii)	Many possible correct answers, for example: MP1 Mixture non-numeric and numeric with 3 or 4 values - with all numerics greater than 999 Examples: 1325, DOG, 7868, 7615 // SNAKE, 3478, SPIDER MP2 Final value: END	2

44. O/N 23/P21/Q7

7	 MP1 S2 labelled MP2 S3, S4 and S5 added MP3 Line from S1 to S3 MP4 All three lines between S2 and S5 (including S5 to S5) MP5 Line from S5 to S3 AND from S3 to S4	5
---	--	---

45. O/N 23/P21/Q8/c

8(c)(i)	MP1 A message cannot contain a zero-length data field MP2 ... so a blank line cannot be sent // there is no way to send a blank line	2
8(c)(ii)	MP1 Append a (special) character to the start of the message text MP2 interpret the new field data as a blank line ALTERNATIVE MP1 Change the message protocol and use an additional field to act as an indicator MP2 Interpret the new field data	2

46. O/N 23/P22/Q4/b

4(b)	One mark per set, including stated purpose. Max 3 marks Example answers: 1 data set with (only) odd values, terminated with 99 2 data set with (only) even values, terminated with 99 3 data sets with same number of odd and even values, terminated with 99 4 data sets with all even / all odd with just one odd/even value, terminated with 99 5 data sets with no values just final 99 6 data sets without (terminating) 99 // missing or incorrectly placed 99	3
------	---	---

47. O/N 23/P22/Q5

5

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]
3		0	1	3	4	2
	4				3	
4		1				
	2					1
2		2				
	3			2		
3		3				
	3				2	
3		4				
	2				1	
2		5				
						10

6

One mark for:

- Initialisation row
- Each iteration of Count 1 to 4 with correct Index, Count and array Mix as shown
- Final iteration, Count = 5, correct Index, Count and array Mix as shown plus Mix[4] assignment

48. O/N 23/P22/Q7/a,b

7(a)	One mark per point: 1 Module names 2 Parameters with labels to Module-X and between Module-X and Reset 3 Parameter (with label) to Module-Z and return from Module-Z 4 Parameters with labels to Module-Y and Restore and return values from Module-Y	4
7(b)	Means that Module-A calls either one of Module-X, Module-Y or Module-Z (which one is called is decided at runtime). One mark for reference to selection One mark for naming all four modules correctly	2

49. O/N 23/P23/Q1/d

1(d)	MP1 Type: Adaptive MP2 Reason: The (user) requirement(s) changes // to accommodate legislative changes	2
------	--	---

50. O/N 23/P23/Q5

5

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]
2		0	4	2	3	5
	2	2				
				5		
3						
	3	5				
					8	
4						
	5	10				
						9
2						
	5	15				
				13		
2						
	13	28				
				21		
2						
			56			

6

MP1 Row 1 (initialisation)

Each iteration (1 – 5):

MP2 1 – Total 2

MP3 2 – Total 5

MP4 3 – Total 10

MP5 4 – Total 15

MP6 5 – Total 28 **and** final Mix[1] = 56

51. O/N 23/P23/Q7/a

7(a)	MP1 iteration / looping MP2 naming all four modules correctly in the correct sequence //_e.g. Module-A repeatedly calls Sub-Y1, then SubY2 then Sub-9	2
------	---	---

52. O/N 23/P23/Q8/b,d

8(b)	Max 3 marks Problem: • If the file being sent contains a line of the string "****" • then the file being written by <code>ReceiveFile()</code> will end at this point // subsequent file lines will be lost Solution: • Read the file (at the sending end) to find the number of lines it contains • Send an initial message which defines the number of lines in the file ALTERNATIVE SOLUTION: • (Transmitter program) chooses a different terminator string / character that doesn't occur in the file • Transmitter program sends the terminator string / character before first line of file / before the transfer begins	3
8(d)	Note: Max 3 marks (from either limitation or modification list) Limitation: 1 <code>GetData()</code> does not return a value until a message has been received 2 So once a message has been sent the user has to wait for a reply // chat is half-duplex Modification: 3 If no response allow the receiver to exit chat at any time ... 4 <code>GetData()</code> should immediately return a suitable message // set a time limit 5 ... which <code>Chat()</code> can detect and respond by allowing the conversation to continue	3

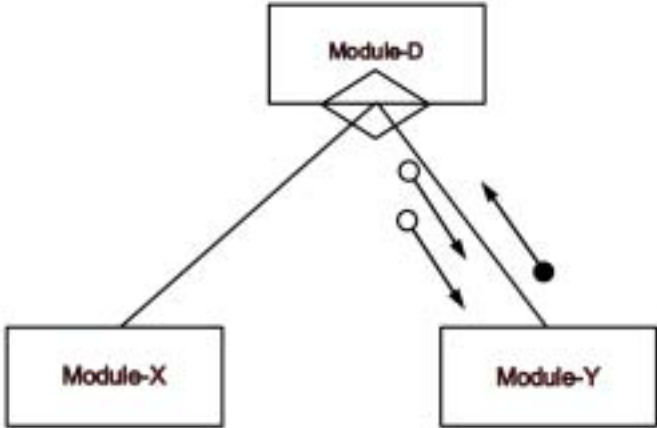
8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 <code>OUTPUT</code> the data in a loop MP5 <code>INPUT</code> the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

53. M/J 23/P21/Q5/a,b

5(a)(i)	Reasons include: 1 No working software until late in the life cycle so slower to market than competitors // Does not allow the creation of early versions/prototypes (which can be updated later) 2 More difficult/slower to cope with changes to the requirements // website slower to be updated to reflect new requirements 3 Needs high involvement/feedback of the stake holders /customer / client One mark per point Max 2 marks	2
5(a)(ii)	Iterative / Rapid Application Development / RAD	1

5(b)	One mark for Stage Stage: Beta testing Max 2 marks for Description Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the website/software works as required / works in the real world // User will identify errors in the website/software 3 Users will feedback (problems) / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	3
------	---	----------

54. M/J 23/P21/Q7/b

7(b)(i)	One mark per point: • Module-A calls the other three modules • The process is repeated	2
7(b)(ii)	 One mark per bullet point: • All rectangles correctly labelled and interconnected • All and only parameters as shown • Selection diamond	3

55. M/J 23/P21/Q8/c

8(c)(i)	Integration testing	1
8(c)(ii)	Two marks for the description: • A dummy/simple module is written to replace the module that does not work properly • The dummy/simple module will return an expected value // will output a message to show it has been called	2

56. M/J 23/P22/Q5/a,b,d

5(a)	One mark per point: - parameter / Num has been passed by reference // should have been passed by value - so when the value / ThisNum is modified (in procedure DisplaySqrt()) - the new value will be used in the loop (lines 40–43) // Num will be changed to modified value	3
5(b)	The rules of the language have not been broken // there are no syntax errors	1
5(d)	Answers include: - Change the statement into a comment - Change the statement to a string representing a literal value and assign it to a variable / output it Note: max 1 mark	1

57. M/J 23/P22/Q7/b

7(b)	<pre> graph LR START((START)) --> S1((S1)) S1 -- "Input PIN" --> S2((S2)) S2 -- "Re-input PIN Display error" --> S2 S2 -- "Too many tries Block Account" --> S3((S3)) S2 -- "Valid PIN Enable payment" --> S4((S4)) S2 -- "Cancel Re-prompt" --> S1 </pre> One mark for each: - Line from S1 to S2 and label - S2 loop label - S3 circle and label on incoming - S4 circle and label on incoming	4
------	---	----------

58. M/J 23/P22/Q8/c

8(c)	One mark for method: Method: Stub testing Two marks for description: • The modules <code>SuppExists()</code> and <code>CheckSupplier()</code> are replaced by dummy modules • ...which return a known result / contain an output statement to show they have been called	3
------	--	----------

59. M/J 23/P23/Q1/b

1(b)	One mark per point: • The (second dimension/index of the) array is declared from 1 to 50 but the loop runs from 0 to 49 • Line number: 10 / 100 / 101 / 102	2
------	---	----------

60. M/J 23/P23/Q5/a,b

5(a)	Max 3 marks Additional Information: MP1 The (program/source) code/specification MP2 test plan // inputs/test data <u>and</u> expected outputs Explanation: MP3 The structure / design / algorithm of the program of the program needs to be known MP4 so that all paths through the program can be tested	3
5(b)	Perfective	1

61. M/J 23/P23/Q7/b

7(b)	Max 3 marks MP1 All modules correctly labelled and interconnected. MP2 Correct parameters and return type to Module-X and Reset MP3 Correct parameters and return type to Module-Y and Replace MP4 Correct parameters and return type to Overlay and Module-Y	3
------	---	----------

62. M/J 23/P23/Q8/c

8(c)(i)	Max 2 marks MP1 Must 'calculate' the count before any item + description output / after the file is read once MP2 Lines to be output have to be <u>stored</u> ... MP3 The file has to be read twice	2
8(c)(ii)	One mark per point: MP1 Loop through the file calculating the count MP2 Save 'selected' items in <u>an array</u> MP3 (After all lines have been read), output the header lines / count MP4 Loop through <u>the array</u> to output each array element	3

8(b)	<pre> PROCEDURE Report_1(Supp : STRING) DECLARE Count : INTEGER DECLARE ThisItemNum, ThisDesc, ThisLine, ThisCode : STRING Count ← 0 OPENFILE "Stock.txt" FOR READ OUTPUT "Report for Supplier:" & Supp OUTPUT "" //Blank line as per example OUTPUT "Item Description" OUTPUT "" //Blank line as per example WHILE NOT EOF("Stock.txt") READFILE("Stock.txt", ThisLine) ThisCode ← Mid(ThisLine, 5, 3) IF ThisCode = Supp THEN ThisItemNum ← LEFT(ThisLine, 4) ThisDesc ← RIGHT(ThisLine, LENGTH(ThisLine) - 7) OUTPUT ThisItem & " " & ThisDesc Count ← Count + 1 ENDIF ENDWHILE CLOSEFILE "Stock.txt" OUTPUT "" //Blank line as per example OUTPUT "Number of items listed: ", Count ENDPROCEDURE </pre> MP1 Output report header (blank lines optional) – Must contain the parameter code MP2 Conditional loop until EOF("Stock.txt") MP3 Read a line from Stock.txt AND extract SupplierCode in a loop MP4 Test if SupplierCode = Supp then increment count (must have been Initialised) MP5 Extract AND output item and description in a loop MP6 Output the final line with count	6
------	--	---

63. O/N 22/P21/Q6/b

6(b)(i)	One mark for each area as outlined:	5																																																																																										
	<table><tr><th>Index</th><th>NextChar</th><th>Dots</th><th>Ats</th><th>Others</th><th>Valid</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>0</td><td>TRUE</td></tr><tr><td>1</td><td>'L'</td><td></td><td></td><td>1</td><td></td></tr><tr><td>2</td><td>'i'</td><td></td><td></td><td>2</td><td></td></tr><tr><td>3</td><td>'z'</td><td></td><td></td><td>3</td><td></td></tr><tr><td>4</td><td>'.'</td><td>1</td><td></td><td></td><td></td></tr><tr><td>5</td><td>'1'</td><td></td><td></td><td>4</td><td></td></tr><tr><td>6</td><td>'2'</td><td></td><td></td><td>5</td><td></td></tr><tr><td>7</td><td>'3'</td><td></td><td></td><td>6</td><td></td></tr><tr><td>8</td><td>'@'</td><td></td><td>1</td><td></td><td></td></tr><tr><td>9</td><td>'b'</td><td></td><td></td><td>7</td><td></td></tr><tr><td>10</td><td>'i'</td><td></td><td></td><td>8</td><td></td></tr><tr><td>11</td><td>'g'</td><td></td><td></td><td>9</td><td></td></tr><tr><td>12</td><td>'@'</td><td></td><td>2</td><td></td><td>FALSE</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Index	NextChar	Dots	Ats	Others	Valid			0	0	0	TRUE	1	'L'			1		2	'i'			2		3	'z'			3		4	'.'	1				5	'1'			4		6	'2'			5		7	'3'			6		8	'@'		1			9	'b'			7		10	'i'			8		11	'g'			9		12	'@'		2		FALSE							
Index	NextChar	Dots	Ats	Others	Valid																																																																																							
		0	0	0	TRUE																																																																																							
1	'L'			1																																																																																								
2	'i'			2																																																																																								
3	'z'			3																																																																																								
4	'.'	1																																																																																										
5	'1'			4																																																																																								
6	'2'			5																																																																																								
7	'3'			6																																																																																								
8	'@'		1																																																																																									
9	'b'			7																																																																																								
10	'i'			8																																																																																								
11	'g'			9																																																																																								
12	'@'		2		FALSE																																																																																							
6(b)(ii)	FALSE	1																																																																																										

64. O/N 22/P21/Q7/b

7(b)	Example string: "23/0" (Any divide by zero example) Reason: The result is infinity / cannot be represented / is undefined // will cause the program to crash	2
------	---	----------

65. O/N 22/P21/Q8/d

8(d)	One mark for each (Max 2): Examples: 1 Incorrect block structure. Missing keyword denoting part of block (for example <code>ENDPROCEDURE</code> , <code>ENDFUNCTION</code> , <code>ENDTYPE</code>) 2 Data type errors, for example, assigning an integer value to a string 3 Identifier used before it is declared 4 Incorrect parameter use	2
------	---	----------

66. O/N 22/P22/Q1/b

1(b)	One mark for test stage, one mark for each description point (Max 3 for Description) Test stage: Beta testing Description: 1 Testing carried out by a small group of (potential) users 2 Users will check that the software works as required / works in the real world / does not contain errors 3 Users will feedback problems / suggestions for improvement 4 Problems / suggestions identified are addressed (before the program is sold)	4
1(a)(i)	One mark for each point (Max 2): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	2
1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2

67. O/N 22/P22/Q2/b

2(b)	One mark per row (row 2 to 5): <table border="1"> <thead> <tr> <th>Input</th><th>Output</th><th>Next state</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>S1</td></tr> <tr> <td>Input-A</td><td>Output-X</td><td>S2</td></tr> <tr> <td>Input-A</td><td>(none)</td><td>S2</td></tr> <tr> <td>Input-B</td><td>Output-W</td><td>S3</td></tr> <tr> <td>Input-A</td><td>Output-W</td><td>S4</td></tr> </tbody> </table>	Input	Output	Next state			S1	Input-A	Output-X	S2	Input-A	(none)	S2	Input-B	Output-W	S3	Input-A	Output-W	S4	4
Input	Output	Next state																		
		S1																		
Input-A	Output-X	S2																		
Input-A	(none)	S2																		
Input-B	Output-W	S3																		
Input-A	Output-W	S4																		

68. O/N 22/P22/Q6/b

6(b)	One mark per row. Examples of invalid strings: 1 Non-numeric but not "End" // contains space or other non-numeric characters 2 Real number 3 Integer value out of range (i.e. ≤ 0) 4 Empty string 5 Correct word but wrong case e.g. "end" rather than "End" <table><tr><th>Input</th><th>Reason for choice</th></tr><tr><td>"Aardvark"</td><td>Non-numeric (and not "End")</td></tr><tr><td>"27.3"</td><td>Numeric but not an integer</td></tr><tr><td>"-3" // "0"</td><td>A non-positive integer</td></tr><tr><td>""</td><td>An empty string</td></tr></table>	Input	Reason for choice	"Aardvark"	Non-numeric (and not "End")	"27.3"	Numeric but not an integer	"-3" // "0"	A non-positive integer	""	An empty string	4
Input	Reason for choice											
"Aardvark"	Non-numeric (and not "End")											
"27.3"	Numeric but not an integer											
"-3" // "0"	A non-positive integer											
""	An empty string											

69. O/N 22/P23/Q1/b,c,d

1(b)(i)	Easier to manage/plan/cost // Clear deliverables produced at (end of) each stage	1
1(b)(ii)	One mark per point (Max 1): • The problem definition • Requirements specification // Client requirements • Documentation related to current system (e.g. ER diagram of current system, DFD of current system, feasibility study)	1
1(c)(i)	One mark per point (Max 1): • Modules are developed in parallel / as prototypes • Minimal / no detailed planning is carried out // Allows for changes to requirements • Flexible development process • Small incremental releases are made, each adding functionality • Used for time critical development • Client involved during (all stages) of development	1

1(c)(ii)	Examples include: Benefits: (Max 2 marks) • Quicker development possible / Multiple areas can be worked on at same time • Prototype produced (at early stage in process) • Easier to change requirements / quicker delivery of usable modules • Early review possible / closer cooperation between client and developers Drawback: (Max 1 mark) • Difficult to estimate cost / time to complete project • Documentation often omitted • Lack of client availability throughout life cycle // too easy for client to keep changing their mind	3
1(d)	One mark for each point (Max 2) Examples include: 1 Change to website requirements 2 New technologies available to host website // changes made to library modules used 3 Change in relevant legislation	2

70. O/N 22/P23/Q5/d

5(d)	One mark per point (Max 3) 1 The program is checked by creating a trace table / going through the program a line at a time 2to record/check variable (values) as they change 3 Error may be indicated when variable given an unexpected value 4 Error may be indicated by an unexpected path through the program // Faults in the logic of the program can be detected	3
------	--	----------

71. O/N 22/P23/Q6/a

6(a)	One mark per row	5																														
	Examples:																															
	<table><tr><th>Test number</th><th>Component weight</th><th>Min</th><th>Max</th><th>Expected return value</th></tr><tr><td>1</td><td>300</td><td>290</td><td>315</td><td>'A'</td></tr><tr><td>2</td><td>> 317</td><td>290</td><td>315</td><td>'R'</td></tr><tr><td>3</td><td>317</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>4</td><td>288</td><td>290</td><td>315</td><td>'C'</td></tr><tr><td>5</td><td>< 288</td><td>290</td><td>315</td><td>'R'</td></tr></table>	Test number	Component weight	Min	Max	Expected return value	1	300	290	315	'A'	2	> 317	290	315	'R'	3	317	290	315	'C'	4	288	290	315	'C'	5	< 288	290	315	'R'	
Test number	Component weight	Min	Max	Expected return value																												
1	300	290	315	'A'																												
2	> 317	290	315	'R'																												
3	317	290	315	'C'																												
4	288	290	315	'C'																												
5	< 288	290	315	'R'																												

72. O/N 21/P21/Q1/a

1(a)(i)	One from: - The program obeys the rules / grammar of the programming language used - The program will run // it can be compiled / interpreted - Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: - An error in the algorithm / design of the solution - the program does not behave as expected / give the expected output. - Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: - The program performs an illegal instruction / invalid operation - Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

73. O/N 21/P21/Q4/a,b

4(a)

Test	Test data value	Explanation	Expected Outcome
1	23	Normal Data	Data is accepted
2	0	Boundary Data	Data is accepted
3	40	Boundary Data	Data is accepted
4	>= 41	Abnormal Data	Data is rejected
5	<= -1	Abnormal Data	Data is rejected

One mark per row for rows 2 to 5.

4

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	---

74. O/N 21/P21/Q5/b

5(b)	Design	1
------	--------	---

75. O/N 21/P22/Q2/b

2(b)(i)	One mark for each: Life cycle method: Iterative // Rapid Application Development (RAD) Reason: Provides a working model / prototype at an early stage for the principal to approve / review	2
2(b)(ii)	Decisions will be made regarding: • Data structures • Algorithms / flowcharts / pseudocode • Program structure (modules) / use of library routines / module - team allocation • User interface // Web-page layout / content (for given scenario) • Testing method / plan • Choice of programming language / program environment Max 3	3

76. O/N 21/P22/Q6/c(i)

6(c)(i)	One mark for: • Parameter(s) need to be passed to the module to identify the type of search • Search algorithm is controlled by (global) variables / parameters Alternative: • The search algorithms from the original modules are included in the new module • The new module needs to return / store the four values (the results of the four searches)	2
---------	---	----------

77. O/N 21/P23/Q1/a

1(a)(i)	One from: • The program obeys the rules / grammar of the programming language used • The program will run // it can be compiled / interpreted • Accept by example. e.g. 'no mis-spelt keywords' / 'all brackets match'	1
1(a)(ii)	One mark for type plus one for corresponding description Type of error: A logic error Description: • An error in the algorithm / design of the solution • the program does not behave as expected / give the expected output. • Accept by example e.g. wrong arithmetic operator used / wrong loop count OR Type of error: Run-time error Description: • The program performs an illegal instruction / invalid operation • Accept by example: divide by zero or endless loop or simply 'crashes' / freezes	2

78. O/N 21/P23/Q4/a,b

4(a)	Test	Test data value	Explanation	Expected Outcome	4
	1	23	Normal Data	Data is accepted	
	2	0	Boundary Data	Data is accepted	
	3	40	Boundary Data	Data is accepted	
	4	>= 41	Abnormal Data	Data is rejected	
	5	<= -1	Abnormal Data	Data is rejected	
One mark per row for rows 2 to 5.					

4(b)	<pre> graph LR Start((Start)) --> Off((Heaters Off)) Off -- "Temp >= 10" --> Off Off -- "Temp < 10" --> On((Heaters On)) On -- "Temp <= 20" --> On On -- "Temp > 20" --> Off </pre> One mark for each label: • Temp < 10 (Heaters Off to Heaters On) • Temp > 20 (Heaters On to Heaters Off) • on BOTH loops (non-contradictory values)	3
------	--	----------

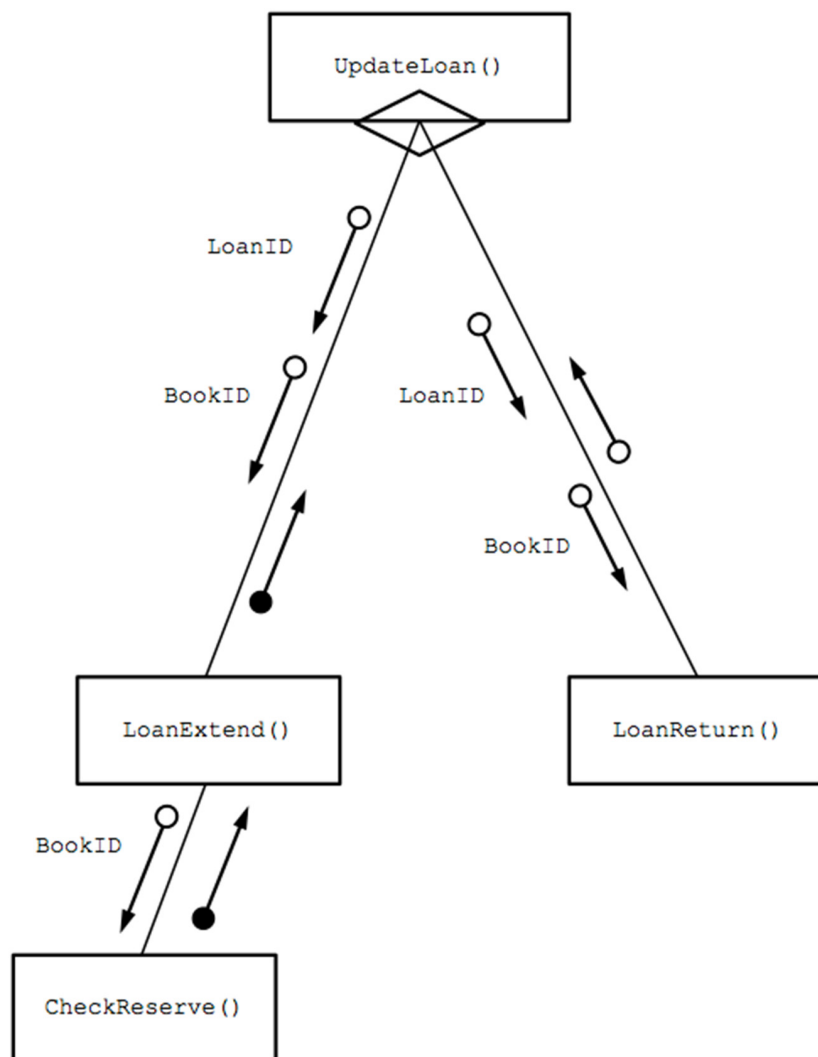
79. O/N 21/P23/Q5/b

5(b)	Design	1
------	--------	----------

80. M/J 21/P21/Q2/a

2(a)

5



One mark for:

- 1 All four boxes correctly labelled and positioned
- 2 Selection diamond (only on `UpdateLoan` and no iteration arrows)
- 3 Parameters to and from `LoanExtend()`
- 4 Parameters to and from `CheckReserve()`
- 5 Parameters to and from `LoanReturn()`

81. M/J 21/P21/Q4/a,c

4(a)

5

Name	Flag	Index	NewName	ThisChar
"VinVaVVcup"				
	TRUE	1	""	
				'V'
			"V"	
		2		'i'
	FALSE		"VI"	
		3		'n'
			"VIn"	
		4		'V'
	TRUE		"VInV"	
		5		'a'
	FALSE		"VInVA"	
		6		'V'
	TRUE		"VInVAV"	
		7		'V'
			"VInVAVV"	
		8		'C'
	FALSE		"VInVAVVC"	
		9		'u'
			"VInVAVVCu"	
		10		'p'
			"VInVAVVCup"	
	FALSE	11	"VInVAVVCup"	'p'

Mark as follows:

One mark for each of columns 2 to 5 (condone missing final 11)

One mark for final row all correct (including final 11)

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE (MID (Name, Index, 1)</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE (ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2

82. M/J 21/P22/Q2/a

2(a)(i)	The number of transitions that result in a different state	3	4
	The number of transitions with associated outputs	2	
	The label that should replace 'X'	Start	
	The final or halting state	S3	
	One mark per row		
2(a)(ii)	Number of outputs: 1 Current state: S2		2

83. M/J 21/P22/Q7/a,b

7(a)	Test string should contain: • At least one leading space (before the first word) • at least one trailing space (after the last word) • at least one instance of multiple spaces between words • At least one upper case character Mark as follows: One mark for one correct Two marks for three correct Three marks for all correct	3
------	---	----------

7(b)	One mark for each: • Dry run / produce a trace table / walk through the code • Add output statements to allow the code to be tracked • Insert a Breakpoint into the program // use single-stepping to execute instructions // monitor variables using a watch window • Try different test values to see which ones fail Note: Max 2 marks	2
------	---	----------

84. M/J 21/P23/Q2/a

2(a)	<pre> graph TD UpdateLoan[UpdateLoan()] LoanExtend[LoanExtend()] CheckReserve[CheckReserve()] LoanReturn[LoanReturn()] UpdateLoan -- LoanID --> LoanExtend LoanExtend -- BookID --> UpdateLoan UpdateLoan -- LoanID --> LoanReturn LoanReturn -- BookID --> UpdateLoan LoanExtend -- BookID --> CheckReserve CheckReserve --> LoanExtend </pre> The diagram shows four activity boxes: <code>UpdateLoan()</code> at the top, <code>LoanExtend()</code> at the bottom left, <code>CheckReserve()</code> at the bottom left below <code>LoanExtend()</code>, and <code>LoanReturn()</code> at the bottom right. A selection diamond is located at the bottom of the <code>UpdateLoan()</code> box. Two outgoing arrows from the diamond lead to <code>LoanExtend()</code> and <code>LoanReturn()</code>. The arrow to <code>LoanExtend()</code> is labeled <code>LoanID</code> and has a white circle at its start. The arrow to <code>LoanReturn()</code> is also labeled <code>LoanID</code> and has a white circle at its start. From <code>LoanExtend()</code>, an arrow labeled <code>BookID</code> points back to the selection diamond in <code>UpdateLoan()</code>, with a black circle at its start. From <code>LoanReturn()</code>, an arrow labeled <code>BookID</code> points back to the selection diamond in <code>UpdateLoan()</code>, with a white circle at its start. From <code>LoanExtend()</code>, another arrow labeled <code>BookID</code> points down to <code>CheckReserve()</code>, with a white circle at its start. An arrow points from <code>CheckReserve()</code> back to <code>LoanExtend()</code>, with a black circle at its start. One mark for: 1 All four boxes correctly labelled and positioned 2 Selection diamond (only on <code>UpdateLoan()</code> and no iteration arrows) 3 Parameters to and from <code>LoanExtend()</code> 4 Parameters to and from <code>CheckReserve()</code> 5 Parameters to and from <code>LoanReturn()</code>	5
------	--	----------

4(c)(i)	A couple of solutions: <pre>24 ThisChar ← LCASE (MID (Name, Index, 1)</pre> ALTERNATIVE: <pre>31 NewName ← NewName & LCASE (ThisChar)</pre> Ignore line number	1
4(c)(ii)	One mark for each: Line number: 26 New position: Move to after line 27 / line 28	2

Topic index: where each part was filed

Reference	Unit	Marks	Page	Also / context
M/J 26/P21/Q1/a,b	11	2	351	
M/J 26/P21/Q2/a(i)	10	1	116	
M/J 26/P21/Q2/a(ii,iii)	9	4	4	
M/J 26/P21/Q3/a,b	12	9	525	
M/J 26/P21/Q4	9	4	5	
M/J 26/P21/Q5/a,b	11	5	351	
M/J 26/P21/Q6/a,b	11	6	352	
M/J 26/P21/Q6/c	10	6	117	ctx (a), (b)
M/J 26/P21/Q7/a,b,c	10	12	117	
M/J 26/P21/Q8/a,b,c	12	6	525	
M/J 26/P21/Q9/a,b,c	10	18	119	
M/J 26/P22/Q1/a(i)	10	1	122	
M/J 26/P22/Q1/a(ii,iii)	11	9	352	
M/J 26/P22/Q1/b,c,d,e,f	12	17	527	ctx (a)
M/J 26/P22/Q2/a,b,c	9	5	5	
M/J 26/P22/Q3	9	6	6	
M/J 26/P22/Q4/a,b	10	9	122	
M/J 26/P22/Q5/a,b	10	6	124	
M/J 26/P22/Q5/c	11	4	354	
M/J 26/P22/Q6/a	11	8	355	
M/J 26/P22/Q6/b	10	10	125	
M/J 26/P23/Q1/a,b	11	2	356	
M/J 26/P23/Q2/a	10	1	127	
M/J 26/P23/Q2/b,c	9	3	6	
M/J 26/P23/Q3/a,b,c	12	11	531	
M/J 26/P23/Q4	9	4	7	
M/J 26/P23/Q5/a	11	4	356	
M/J 26/P23/Q5/b	10	7	128	
M/J 26/P23/Q6/a,b,c,d	10	13	129	
M/J 26/P23/Q7/a,b	12	9	532	
M/J 26/P23/Q8/a,b	10	19	130	also U12:3
O/N 25/P21/Q1/a	11	4	357	insert (Appendix)
O/N 25/P21/Q1/b	12	3	534	
O/N 25/P21/Q2	9	5	7	
O/N 25/P21/Q3/a,b	10	9	133	
O/N 25/P21/Q4/a,b	10	9	134	
O/N 25/P21/Q5/a	10	3	135	
O/N 25/P21/Q5/b	11	3	358	
O/N 25/P21/Q6/b	11	7	359	
O/N 25/P21/Q7/a,b	12	10	534	
O/N 25/P21/Q8/a,b	10	15	136	
O/N 25/P21/Q8/c	12	2	537	
O/N 25/P22/Q1/a	9	4	8	
O/N 25/P22/Q1/b	10	3	137	
O/N 25/P22/Q1/c	11	4	359	ctx (b)
O/N 25/P22/Q2	9	5	8	
O/N 25/P22/Q3/a	9	6	9	
O/N 25/P22/Q3/b,c	10	4	137	
O/N 25/P22/Q4/a,b	10	9	139	
O/N 25/P22/Q4/c	9	4	10	
O/N 25/P22/Q5/a,b	12	5	538	
O/N 25/P22/Q6	11	7	360	
O/N 25/P22/Q7/a,b	12	6	539	
O/N 25/P22/Q8/a,b,c	10	16	140	
O/N 25/P22/Q8/d	12	2	540	
O/N 25/P23/Q1/a,b,c	11	10	360	
O/N 25/P23/Q2/a,b	9	8	11	
O/N 25/P23/Q3	10	5	141	
O/N 25/P23/Q4	10	6	142	
O/N 25/P23/Q5/a	12	5	540	
O/N 25/P23/Q5/b	11	3	361	
O/N 25/P23/Q6/a	12	2	543	

Reference	Unit	Marks	Page	Also / context
O/N 25/P23/Q6/b	11	7	363	
O/N 25/P23/Q7/a,b	12	9	543	
O/N 25/P23/Q8/a,b,c	10	20	142	
M/J 25/P21/Q1/a,b,c	9	12	12	
M/J 25/P21/Q2/a,b,c	12	10	545	
M/J 25/P21/Q3	9	8	13	
M/J 25/P21/Q4/a	11	2	364	
M/J 25/P21/Q4/b	12	6	546	
M/J 25/P21/Q5	10	7	145	
M/J 25/P21/Q6/a	12	4	549	
M/J 25/P21/Q6/b	10	8	146	
M/J 25/P21/Q7/a	11	6	365	
M/J 25/P21/Q7/b(i)	10	8	146	
M/J 25/P21/Q7/b(ii)	12	4	549	
M/J 25/P22/Q1/a,b	12	6	551	
M/J 25/P22/Q1/c,d	10	8	148	
M/J 25/P22/Q2/a	9	2	13	
M/J 25/P22/Q2/b	12	6	552	
M/J 25/P22/Q3	9	8	13	
M/J 25/P22/Q4	9	6	14	
M/J 25/P22/Q5/a	12	4	553	
M/J 25/P22/Q5/b	10	8	149	
M/J 25/P22/Q6/a	12	7	553	also U10:3
M/J 25/P22/Q6/b	10	2	150	
M/J 25/P22/Q7/a,b,c	10	18	150	
M/J 25/P23/Q1/a,b	11	10	366	insert (Appendix)
M/J 25/P23/Q1/c	10	4	152	
M/J 25/P23/Q2/a,b,c	10	11	152	
M/J 25/P23/Q3	9	7	14	
M/J 25/P23/Q4/a	9	5	15	
M/J 25/P23/Q4/b	12	3	555	
M/J 25/P23/Q5	11	8	367	
M/J 25/P23/Q6/a	11	1	368	
M/J 25/P23/Q6/b,c	12	8	555	
M/J 25/P23/Q7/a,b(ii)	11	10	368	
M/J 25/P23/Q7/b(i)	10	8	155	
O/N 24/P21/Q1/a	12	2	557	
O/N 24/P21/Q1/b,d	11	5	371	
O/N 24/P21/Q1/c	10	2	156	
O/N 24/P21/Q2	11	6	371	
O/N 24/P21/Q3/a,b	9	5	16	
O/N 24/P21/Q3/c	12	4	558	ctx (a)
O/N 24/P21/Q4/a	12	2	559	
O/N 24/P21/Q4/b,c	11	8	372	
O/N 24/P21/Q5/a,b	12	10	559	
O/N 24/P21/Q6/a	10	2	157	
O/N 24/P21/Q6/b	11	7	372	insert inline
O/N 24/P21/Q7/a,b	12	7	561	
O/N 24/P21/Q8/a	11	6	377	
O/N 24/P21/Q8/b,c	10	9	157	
O/N 24/P22/Q1/a	12	1	562	
O/N 24/P22/Q1/d	10	3	159	ctx (c)
O/N 24/P22/Q2/a,b(ii)	11	7	378	
O/N 24/P22/Q2/b(i)	10	2	160	
O/N 24/P22/Q3/a,b	10	7	161	
O/N 24/P22/Q4	11	6	379	
O/N 24/P22/Q5/a,b	12	5	562	
O/N 24/P22/Q5/c	11	2	380	
O/N 24/P22/Q6/a	10	7	161	
O/N 24/P22/Q6/b	12	2	563	
O/N 24/P22/Q7/a,b	9	4	17	
O/N 24/P22/Q7/c,d	12	6	564	
O/N 24/P22/Q8/a,b,c,d	10	18	162	
O/N 24/P23/Q1/a	9	4	17	
O/N 24/P23/Q1/b	10	3	164	
O/N 24/P23/Q1/c	11	3	380	ctx (b)

Reference	Unit	Marks	Page	Also / context
O/N 24/P23/Q2/a,b	9	7	18	
O/N 24/P23/Q3/a	10	2	165	
O/N 24/P23/Q3/b,c	9	8	19	
O/N 24/P23/Q4/a,b	10	12	165	
O/N 24/P23/Q5/a,b	12	7	565	
O/N 24/P23/Q6	11	7	380	insert inline
O/N 24/P23/Q7/a	12	2	565	
O/N 24/P23/Q7/b	9	4	20	
O/N 24/P23/Q8/a,b,c	10	16	166	
M/J 24/P21/Q1/a	10	4	168	
M/J 24/P21/Q1/b	11	4	384	
M/J 24/P21/Q1/c	9	3	21	ctx (a)
M/J 24/P21/Q2/a	9	5	21	
M/J 24/P21/Q2/b	11	2	384	
M/J 24/P21/Q3/a,b	10	8	169	
M/J 24/P21/Q4	10	6	170	
M/J 24/P21/Q5/a,b	12	8	565	
M/J 24/P21/Q6/a	11	6	384	
M/J 24/P21/Q6/b	12	2	567	ctx (a)
M/J 24/P21/Q7/a	9	6	23	
M/J 24/P21/Q7/b	12	6	569	
M/J 24/P21/Q8/a	11	8	386	
M/J 24/P21/Q8/b	10	7	171	
M/J 24/P22/Q1/a,c	9	6	23	ctx (b)
M/J 24/P22/Q1/b	11	4	387	insert (Appendix)
M/J 24/P22/Q2/a,b	9	10	24	
M/J 24/P22/Q3/a,b	10	9	172	
M/J 24/P22/Q4	11	5	387	
M/J 24/P22/Q5/a(i),b	12	4	570	
M/J 24/P22/Q5/a(ii)	11	2	388	
M/J 24/P22/Q6/a	11	7	389	
M/J 24/P22/Q6/b	10	3	173	
M/J 24/P22/Q7/a	9	3	28	
M/J 24/P22/Q7/b	12	6	570	
M/J 24/P22/Q8/a	9	2	28	
M/J 24/P22/Q8/b	11	6	390	
M/J 24/P22/Q8/c	10	8	173	ctx (b)
M/J 24/P23/Q1/a,b	11	9	391	insert (Appendix)
M/J 24/P23/Q2/a,b	9	7	29	
M/J 24/P23/Q3/a	10	4	175	
M/J 24/P23/Q3/b	9	5	29	
M/J 24/P23/Q4/a	11	6	392	
M/J 24/P23/Q4/b	10	5	175	
M/J 24/P23/Q5/a,b	11	6	392	
M/J 24/P23/Q6/a,b(i)	11	8	393	
M/J 24/P23/Q6/b(ii)	12	1	571	
M/J 24/P23/Q7/a(i),b	12	7	573	
M/J 24/P23/Q7/a(ii)	11	2	394	
M/J 24/P23/Q8/a	11	7	395	
M/J 24/P23/Q8/b	10	8	176	
O/N 23/P21/Q1/a,b,c	11	8	396	
O/N 23/P21/Q1/d	10	3	177	
O/N 23/P21/Q2/a,b	9	7	32	
O/N 23/P21/Q3/a,b	10	8	177	
O/N 23/P21/Q4/a,b	10	9	179	
O/N 23/P21/Q5/a,b	12	7	573	
O/N 23/P21/Q6/a,b	11	9	397	
O/N 23/P21/Q7	12	5	575	
O/N 23/P21/Q8/a,b	10	9	179	
O/N 23/P21/Q8/c	12	4	575	
O/N 23/P21/Q8/d	11	6	397	
O/N 23/P22/Q1/a	9	4	33	
O/N 23/P22/Q1/b	11	4	399	
O/N 23/P22/Q1/c	10	3	181	ctx (a)
O/N 23/P22/Q2/a	9	5	33	
O/N 23/P22/Q2/b	11	2	400	

Reference	Unit	Marks	Page	Also / context
O/N 23/P22/Q3/a,b	10	9	181	
O/N 23/P22/Q4/a	11	6	400	
O/N 23/P22/Q4/b	12	3	577	
O/N 23/P22/Q5	12	6	577	
O/N 23/P22/Q6/a,b(iii)	10	7	183	
O/N 23/P22/Q6/b(i,ii)	11	2	401	ctx (a)
O/N 23/P22/Q7/a,b	12	6	580	
O/N 23/P22/Q8/a,c	10	11	183	ctx (b)
O/N 23/P22/Q8/b	11	7	401	
O/N 23/P23/Q1/a	10	4	186	
O/N 23/P23/Q1/b,c	11	5	404	
O/N 23/P23/Q1/d	12	2	581	
O/N 23/P23/Q2/a	9	5	34	
O/N 23/P23/Q2/b,c	10	4	186	ctx (a)
O/N 23/P23/Q3/a,b	10	9	188	
O/N 23/P23/Q4/a,b	11	7	405	
O/N 23/P23/Q5	12	6	581	
O/N 23/P23/Q6	11	6	405	
O/N 23/P23/Q7/a	12	2	583	
O/N 23/P23/Q7/b(i)	10	3	189	
O/N 23/P23/Q7/b(ii)	11	2	406	
O/N 23/P23/Q8/a	10	7	190	
O/N 23/P23/Q8/b,d	12	6	583	ctx (c)
O/N 23/P23/Q8/c	11	7	406	
M/J 23/P21/Q1/b,c	11	6	409	
M/J 23/P21/Q2/a	9	4	36	
M/J 23/P21/Q2/b	11	3	410	
M/J 23/P21/Q3/a	10	6	191	
M/J 23/P21/Q3/b	9	5	37	
M/J 23/P21/Q4	11	6	410	
M/J 23/P21/Q5/a,b	12	6	586	
M/J 23/P21/Q6	10	6	192	
M/J 23/P21/Q7/a	9	3	38	
M/J 23/P21/Q7/b	12	5	587	
M/J 23/P21/Q8/a	11	7	411	
M/J 23/P21/Q8/b,d,e	10	11	192	
M/J 23/P21/Q8/c	12	3	588	ctx (b)
M/J 23/P22/Q1/a	11	4	412	
M/J 23/P22/Q1/b	9	3	38	
M/J 23/P22/Q1/c	10	3	194	
M/J 23/P22/Q2/a,b	11	3	412	insert inline
M/J 23/P22/Q2/c	9	6	39	
M/J 23/P22/Q3/a	10	5	195	
M/J 23/P22/Q3/b	9	5	39	
M/J 23/P22/Q4	11	6	415	
M/J 23/P22/Q5/a,b,d	12	5	590	
M/J 23/P22/Q6	11	6	416	
M/J 23/P22/Q7/a	9	5	40	
M/J 23/P22/Q7/b	12	4	591	
M/J 23/P22/Q8/a	11	7	417	
M/J 23/P22/Q8/b	10	7	196	
M/J 23/P22/Q8/c	12	3	592	
M/J 23/P23/Q1/a,c	10	4	197	
M/J 23/P23/Q1/b	12	2	593	
M/J 23/P23/Q1/d	9	4	41	
M/J 23/P23/Q2/a	11	2	417	
M/J 23/P23/Q3/a,b(i)	10	6	197	
M/J 23/P23/Q3/b(ii)	9	5	41	
M/J 23/P23/Q4	11	6	418	
M/J 23/P23/Q5/a,b	12	4	593	
M/J 23/P23/Q6/a,b	11	11	418	
M/J 23/P23/Q7/a	9	5	42	
M/J 23/P23/Q7/b	12	3	594	
M/J 23/P23/Q8/a,b	10	14	199	
M/J 23/P23/Q8/c	12	5	595	ctx (b)
O/N 22/P21/Q1/a,c	11	7	419	

Reference	Unit	Marks	Page	Also / context
O/N 22/P21/Q1/b	10	5	200	
O/N 22/P21/Q2/a,b	9	5	43	
O/N 22/P21/Q3	9	5	43	
O/N 22/P21/Q4/a,b	10	12	201	
O/N 22/P21/Q5	10	5	202	
O/N 22/P21/Q6/a	11	1	420	
O/N 22/P21/Q6/b	12	6	596	
O/N 22/P21/Q7/a	11	8	421	
O/N 22/P21/Q7/b	12	2	598	
O/N 22/P21/Q8/a,c	10	15	203	
O/N 22/P21/Q8/b	11	2	422	ctx (a)
O/N 22/P21/Q8/d	12	2	599	ctx (a)
O/N 22/P22/Q1/a,c	11	7	423	
O/N 22/P22/Q1/b	12	4	600	ctx (a)
O/N 22/P22/Q2/a	9	7	44	also U10:2
O/N 22/P22/Q2/b	12	4	601	
O/N 22/P22/Q3/a,b,c	10	13	203	
O/N 22/P22/Q4/a	9	6	44	
O/N 22/P22/Q4/b	11	1	423	
O/N 22/P22/Q5	9	4	46	
O/N 22/P22/Q6/a	11	7	425	
O/N 22/P22/Q6/b	12	4	601	ctx (a)
O/N 22/P22/Q7/a,b,c	10	18	204	
O/N 22/P23/Q1/a	10	4	206	
O/N 22/P23/Q1/b,c,d	12	8	604	
O/N 22/P23/Q2	9	3	47	
O/N 22/P23/Q3	9	5	47	
O/N 22/P23/Q4/a,b	10	10	207	
O/N 22/P23/Q5/a,b(ii)	11	10	426	
O/N 22/P23/Q5/b(i)	10	1	208	ctx (a) intro
O/N 22/P23/Q5/c	9	3	48	ctx (a) intro
O/N 22/P23/Q5/d	12	3	605	ctx (a) intro
O/N 22/P23/Q6/a	12	5	605	
O/N 22/P23/Q6/b	11	6	427	
O/N 22/P23/Q7/a,b(i)	10	14	208	
O/N 22/P23/Q7/b(ii)	9	3	48	
O/N 21/P21/Q1/a	12	3	606	
O/N 21/P21/Q1/b	10	4	210	
O/N 21/P21/Q1/c	9	7	50	
O/N 21/P21/Q2/a,b	9	12	50	
O/N 21/P21/Q3/a,b,c	10	13	210	
O/N 21/P21/Q4/a,b	12	7	607	
O/N 21/P21/Q5/a,c	10	9	213	
O/N 21/P21/Q5/b	12	1	608	
O/N 21/P21/Q6/a,b	10	13	213	
O/N 21/P21/Q6/c	11	6	427	
O/N 21/P22/Q1/a	9	2	52	
O/N 21/P22/Q1/b	10	3	215	
O/N 21/P22/Q1/c,d	11	6	429	insert inline
O/N 21/P22/Q2/a	9	5	52	
O/N 21/P22/Q2/b	12	5	609	
O/N 21/P22/Q3/a,b	10	14	216	
O/N 21/P22/Q4/a,c	11	8	433	
O/N 21/P22/Q4/b	9	3	54	
O/N 21/P22/Q5/a,b	10	10	217	
O/N 21/P22/Q6/a	10	7	218	
O/N 21/P22/Q6/b,c(ii),d	11	10	434	ctx (a)
O/N 21/P22/Q6/c(i)	12	2	609	
O/N 21/P23/Q1/a	12	3	610	
O/N 21/P23/Q1/b	10	4	219	
O/N 21/P23/Q1/c	9	7	55	
O/N 21/P23/Q2/a,b	9	12	55	
O/N 21/P23/Q3/a,b,c	10	13	219	
O/N 21/P23/Q4/a,b	12	7	611	
O/N 21/P23/Q5/a,c	10	9	221	
O/N 21/P23/Q5/b	12	1	612	

Reference	Unit	Marks	Page	Also / context
O/N 21/P23/Q6/a,b	10	13	222	
O/N 21/P23/Q6/c	11	6	435	
M/J 21/P21/Q1/a	9	4	57	
M/J 21/P21/Q1/b	11	5	437	
M/J 21/P21/Q1/c	10	3	223	
M/J 21/P21/Q2/a	12	5	612	
M/J 21/P21/Q2/b	11	2	437	ctx (a)
M/J 21/P21/Q2/c	9	6	57	
M/J 21/P21/Q3/a,b	10	10	224	
M/J 21/P21/Q4/a,c	12	8	613	insert inline
M/J 21/P21/Q4/b	11	2	438	insert inline
M/J 21/P21/Q5	10	8	224	
M/J 21/P21/Q6/a,b	10	6	225	
M/J 21/P21/Q7/a,c	11	12	440	
M/J 21/P21/Q7/b	9	4	58	
M/J 21/P22/Q1/a	10	8	225	also U11:4
M/J 21/P22/Q1/b	9	3	60	ctx (a)(i), (a)(ii)
M/J 21/P22/Q2/a	12	6	617	
M/J 21/P22/Q2/b	9	5	61	
M/J 21/P22/Q3/a,b,c,d	10	6	226	
M/J 21/P22/Q4	9	6	61	
M/J 21/P22/Q5/a	10	9	226	also U12:3
M/J 21/P22/Q5/b	11	6	442	insert inline
M/J 21/P22/Q6	10	8	227	
M/J 21/P22/Q7/a,b	12	5	618	
M/J 21/P22/Q8/a	10	5	228	
M/J 21/P22/Q8/b	11	8	444	
M/J 21/P23/Q1/a	9	4	62	
M/J 21/P23/Q1/b	11	5	445	
M/J 21/P23/Q1/c	10	3	229	
M/J 21/P23/Q2/a	12	5	618	
M/J 21/P23/Q2/b	11	2	445	ctx (a)
M/J 21/P23/Q2/c	9	6	62	
M/J 21/P23/Q3/a,b	10	10	229	
M/J 21/P23/Q4/a,c	12	8	619	insert inline
M/J 21/P23/Q4/b	11	2	446	insert inline
M/J 21/P23/Q5	10	8	230	
M/J 21/P23/Q6/a,b	10	6	230	
M/J 21/P23/Q7/a,c	11	12	448	
M/J 21/P23/Q7/b	9	4	63	

Appendix: Insert: pseudocode functions and operators (from M/J 26/P23)

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type CHAR or STRING
- A CHAR may be assigned to, or concatenated with, a STRING
- A STRING of length greater than 1 cannot be assigned to a CHAR

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the '-' character, the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN returns TRUE if ThisString represents a valid numeric value. <datatype> may be CHAR or STRING Example: IS_NUM("-12.36") returns TRUE
ASC(ThisChar : CHAR) RETURNS INTEGER returns an integer value (the ASCII value) of character ThisChar Example: ASC('A') returns 65, ASC('B') returns 66
CHR(x : INTEGER) RETURNS CHAR returns the character whose integer value (the ASCII value) is x Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER returns the integer part of x Example: INT(27.5415) returns 27
RAND(x : INTEGER) RETURNS REAL returns a real number in the range 0 to x (not inclusive of x). Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER returns the day number from ThisDate Example: DAY(04/10/2003) returns 4
MONTH(ThisDate : DATE) RETURNS INTEGER returns the month number from ThisDate Example: MONTH(04/10/2003) returns 10
YEAR(ThisDate : DATE) RETURNS INTEGER returns the year number from ThisDate Example: YEAR(04/10/2003) returns 2003
DAYINDEX(ThisDate : DATE) RETURNS INTEGER returns the day index number from ThisDate where Sunday = 1, Monday = 2 etc. Example: DAYINDEX(09/05/2023) returns 3
SETDATE(Day, Month, Year : INTEGER) RETURNS DATE returns a value of type DATE with the value of <Day>/<Month>/<Year> Example: SETDATE(26, 10, 2003) returns a date corresponding to 26/10/2003
TODAY() RETURNS DATE returns a value of type DATE corresponding to the current date.

Text file functions

EOF(FileName : STRING) RETURNS BOOLEAN

returns TRUE if there are no more lines to be read from file FileName
will generate an error if the file is not already open in READ mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example: 10 DIV 3 evaluates to 3

Comparison operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes: • may be used to compare types REAL and INTEGER • may be used to compare types CHAR and STRING • case sensitive when used to compare types CHAR and/or STRING • cannot be used to compare two records.
<	
>=	
<=	
<>	
	Examples: • "Program" = "program" evaluates to FALSE • Count = 4 evaluates to TRUE when Count contains the value 4